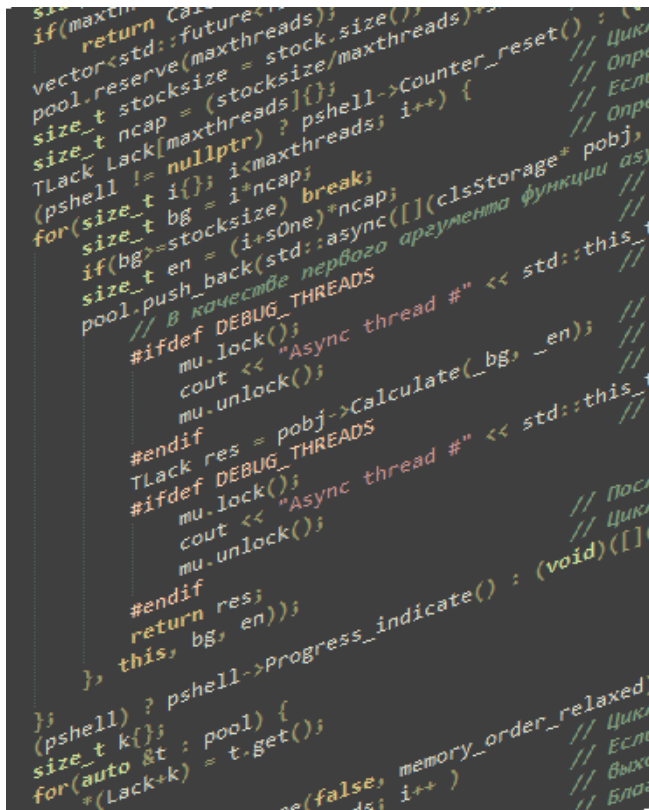




ОПИСАНИЕ И РУКОВОДСТВО ПО ПРИМЕНЕНИЮ

Версия библиотеки 1.3.5

Александр Пидкасистый

2026

FROMA2

Библиотека программных компонентов для
финансово-экономического моделирования
частичных затрат предприятия

ОГЛАВЛЕНИЕ

Расчет частичных затрат и возможности FROMA2	7
Расчет частичных затрат и его место в управлении предприятием	7
Использование в ценовой политике предприятия	8
Использование в ассортиментной политике предприятия	8
Использование в политике снабжения предприятия	8
Использование РЧЗ на основе переменных затрат для анализа рентабельности.....	8
Возможности FROMA2.....	8
Пример 1	9
Пример 2	9
Пример 3	11
Лицензия на библиотеку FROMA2	11
Состав и структура библиотеки FROMA2	12
Модуль warehouse.....	12
Модуль manufact	13
Модуль common_values	14
Модуль LongReal	14
Модуль Impex.....	15
Модуль serialization	16
Модуль baseproject.....	16
Модуль agregate	17
Модуль progress.....	17
Типы данных для информационных потоков	17
Структура strItem	18
Структура strNameMeas.....	19
Область имен nmBPTypes.....	19
Тип вещественных чисел.....	20
Тип индикатора прогресса	21
Склад ресурсов	21
Класс clsSKU	22
Поля класса clsSKU	22
Конструкторы, деструктор, перегруженные операторы	24
Алгоритмы учета. Секция Private.....	25
Алгоритмы учета. Секция Public	31
Get-методы.....	32
Set-методы	33
Методы сохранения состояния объекта в файл и восстановления из файла.....	35

Методы визуального контроля	35
Как пользоваться классом clsSKU	36
Пример программы с классом clsSKU	36
Класс clsStorage	38
Поля класса clsStorage	39
Конструкторы, деструктор, перегруженные операторы	40
Get-методы. Секция Private	41
Get-методы. Секция Public	41
Set-методы. Секция Private	44
Set-методы. Секция Public	44
Расчетные методы. Секция Private	50
Расчетные методы. Секция Public	52
Методы сохранения состояния объекта в файл и восстановления из файла	54
Методы визуального контроля	54
Как пользоваться классом clsStorage	55
Пример программы с классом clsStorage	56
Производство	59
Вспомогательные методы	59
Класс clsRecipeItem	60
Поля класса clsRecipeItem	60
Конструкторы, деструктор, перегруженные операторы	61
Алгоритмы учета. Секция Private	62
Алгоритмы учета. Секция Public	66
Get-методы	67
Методы сохранения состояния объекта в файл и восстановления из файла	68
Методы визуального контроля	69
Класс clsManufactItem	71
Поля класса clsManufactItem	72
Конструкторы, деструктор, перегруженные операторы	73
Алгоритмы учета. Секция private	75
Алгоритмы учета. Секция public	75
Get-методы	75
Set – методы	76
Методы сохранения состояния объекта в файл и восстановления из файла	77
Методы визуального контроля	78
Пример программы с классом clsManufactItem	79
Класс clsManufactory	81
Поля класса clsManufactory	82

Конструкторы, деструктор, перегруженные операторы	82
Set-методы.	84
Сервисные методы	87
Вычислительные методы	87
Get-методы. Секция Private	89
Get-методы. Секция Public	90
Методы сохранения состояния объекта в файл и восстановления из файла	92
Методы визуального контроля	93
Пример программы с классом clsManufactory	94
Как пользоваться классами модуля manufact	97
Создание проекта	98
Добавление производства для конкретного продукта	99
Расчет потребности в сырье и материалах для производства продукции	99
Получение информации о потребности в ресурсах для производства продукции	100
Расчет учетных цен на сырье и материалы	101
Ввод учетных цен на сырье и материалы	101
Расчет себестоимости незавершенного производства и готовой продукции	103
Получение информации о себестоимости незавершенного производства и готовой продукции	103
Моделирование с помощью классов clsStorage и clsManufactory	104
Создание модели с одним центром затрат	104
Создание модели с несколькими центрами затрат	105
Импорт и экспорт данных	107
Класс clsImpex	108
Поля класса clsImpex	108
Конструкторы, перегруженные операторы, сервисные функции	109
Методы импорта	110
Методы преобразования	111
Методы экспорта	111
Get-методы	111
Методы визуального контроля	114
Пример программы с классом clsImpex	114
Внеклассовые функции	115
Арифметика длинных вещественных чисел	116
Класс LongReal	118
Поля класса LongReal	118
Сервисные функции. Секция Private	119
Конструкторы, деструктор, метод обмена	121
Перегрузка операторов присваивания	122

Get – методы	122
Методы масштабирования числа.....	123
Перегрузка логических и арифметических операторов	124
Перегрузка операторов ввода и вывода	126
Методы сохранения состояния объекта в файл и восстановления из файла.....	126
Методы визуального контроля	128
Структура lstream и методы для работы с ней.....	129
Поля структуры lstream	129
Методы структуры lstream.....	129
Методы для работы со структурой lstream	130
Сериализация и десериализация данных	131
Родительский класс для модели	133
Класс clsBaseProject	133
Поля класса clsBaseProject	133
Конструкторы, деструктор, метод обмена, метод сброса состояния; Перегрузка операторов присваивания	134
Set – методы.....	135
Функции вывода отчета. Секция protected.....	136
Функции вывода отчета. Секция Public.....	136
Методы сериализации и десериализации. Секция protected	136
Методы сериализации и десериализации. Секция Public	137
Агрегатор для модели предприятия	137
Посетители для вариантного типа T_agregate	138
struct Getter_exporting_data	138
struct Getter_exporting_names	138
struct Getter_exporting_count	139
struct Calculater_back	139
struct Calculater_fwrd	140
struct Setter_ship	140
struct Getter_ship	140
struct Setter_purch	141
struct Getter_purch.....	141
Getter_NameMeas	141
struct Setter_progress_message	142
Класс clsAgregate.....	142
Поля класса clsAgregate.....	142
Конструкторы, деструктор, метод обмена; Перегрузка операторов присваивания.....	143
Get – методы	144

Set - методы.....	145
Методы визуального контроля	146
Расчётные методы	147
Методы сериализации и десериализации	147
Отображение прогресса при выполнении расчетов	148
класс clsProgress_shell	148
Поля класса clsProgress_shell	148
Методы КЛАССА clsProgress_shell	148
Класс clsprogress_bar	150
Поля класса clsprogress_bar	150
Методы класса clsprogress_bar	150
Как включить и отключить индикацию прогресса	152
Инструменты для отображения результатов расчетов	153
Константы и классы для методов визуального контроля и отчетов	153
Класс class clsTextField	154
Класс class clsDataField.....	154
Методы визуального контроля	155
Классы и функции для вывода отчета	156
Внеклассовые функции	157
Класс class clsRePrint	159
Репозиторий	164
Состав и структура репозитория	164
Сборка проекта с помощью CMake	165
Краткое описание примеров	166
Программа для расчета полных и удельных переменных затрат производственного предприятия Model_1	169
Целевая аудитория для программы	169
Возможности программы	169
Папки и файлы	170
Структура модели и логика работы	171
Первые шаги работы с программой.....	172
Повторный запуск программы	178
Запуск программы с файлом конфигурации в качестве аргумента.....	180
Программные компоненты.....	181
Класс clsCostCenterStorage - склад с двумя центрами затрат	193
Состав модуля clsCostCenterStorage.....	194
Поля класса clsCostCenterStorage	195
Конструкторы, деструктор, перегруженные операторы	196

Методы сериализации и десериализации	197
Set-методы	198
Расчетные методы	198
Get – методы	199
Методы экспорта	200
Удаляемые методы предка	200
Скрываемые методы предка	201
Описание Демонстрационной программы для класса clsCostCenterStorage	201
Программа для расчёта полных и удельных переменных затрат склада Model_2	205
Целевая аудитория для программы	206
Возможности программы	206
Папки и файлы	207
Структура модели и логика работы	209
Первые шаги работы с программой.....	210
Повторный запуск программы	215
Программные компоненты.....	216

РАСЧЕТ ЧАСТИЧНЫХ ЗАТРАТ И ВОЗМОЖНОСТИ FROMA2

РАСЧЕТ ЧАСТИЧНЫХ ЗАТРАТ И ЕГО МЕСТО В УПРАВЛЕНИИ ПРЕДПРИЯТИЕМ

Предпринимательские решения, принятые на основе полученной из расчета [совокупных затрат](#) информации, могут быть ошибочными. Основная причина этого – распределение [постоянных затрат](#) с помощью коэффициентов и наценок по местам возникновения и по объектам затрат, что не всегда оправдано. В связи с этим достоверный анализ и контроль затрат возможен лишь условно.

Исходя из расчета совокупных затрат большая часть постоянных затрат через решения, действующие в течении длительного времени, ложится на производственные мощности предприятия. В краткосрочном аспекте эти постоянные затраты неизменяемы, поэтому для краткосрочных решений значимы только переменные затраты. Таким образом, расчет совокупных затрат дает искажённую информацию.

Чтобы избежать этого недостатка расчета совокупных затрат, используются системы [расчета частичных затрат](#) (далее по тексту - РЧЗ). В этих системах, в противоположность расчету совокупных затрат, за отчетный период только определенная часть совокупных затрат распределяется по местам возникновения и по объектам затрат.

Библиотека FROMA2 версии 1.3.5 предназначена для РЧЗ на основе переменных затрат. В этой системе все затраты за расчетный период разделяются на [постоянные](#) и [переменные](#). При этом только переменные затраты распределяются по местам возникновения и объектам затрат.

Разработанный в США американским экономистом Д. Харрисом в 1936 году РЧЗ на основе переменных затрат называется Директ-костинг ([Direct Costing](#)). Центральным понятием для РЧЗ на основе переменных затрат является понятие «покрытие издержек».

Покрытие издержек определяется как разница между ценой на рынке и переменными затратами. Рассчитываются могут следующие виды покрытия издержек, связанные друг с другом иерархически:

- Покрытие затрат на единицу продукции (штучное покрытие издержек);
- Покрытие затрат за период для одного вида продукции;
- Покрытие затрат за период для одной категории/группы продуктов;
- Покрытие затрат за период для всего предприятия.

Исходя из штучного покрытия издержек, рассчитываются все последующие виды покрытия издержек, как сумма предыдущих ступеней.

Результат хозяйственной деятельности предприятия за период определяется как разница между покрытием издержек всего предприятия и его постоянными затратами. Каждый продукт с положительным покрытием издержек участвует тем самым, как минимум частично, в покрытии постоянных затрат предприятия.

Продукт, который по совокупному расчету затрат на рынке не покрыл себестоимости, тем не менее дает положительный экономический результат, пока он участвует в покрытии издержек. Поэтому оперирование категориями покрытия затрат и является базой РЧЗ на основе переменных затрат.

РЧЗ на основе переменных затрат способен принести существенную пользу при принятии решений руководством предприятия, которую невозможно получить от расчета совокупных затрат. Разложение затрат на постоянные и переменные соответствует разделению решений предприятия на краткосрочные и долгосрочные.

Краткосрочными могут считаться те области решений, которые не оказывают влияния на изменение производственных мощностей предприятия, т.е. на оснащенность машинами, персоналом и т.д. Средне- и долгосрочные решения, напротив, могут влиять на изменение мощностей предприятия и тем самым на

постоянные затраты. РЧЗ на основе переменных затрат наиболее подходит поэтому для области краткосрочных решений.

ИСПОЛЬЗОВАНИЕ В ЦЕНОВОЙ ПОЛИТИКЕ ПРЕДПРИЯТИЯ

РЧЗ на основе переменных затрат определяет в качестве краткосрочной нижней границы цены переменные затраты на единицу продукции;

ИСПОЛЬЗОВАНИЕ В АССОРТИМЕНТНОЙ ПОЛИТИКЕ ПРЕДПРИЯТИЯ

РЧЗ на основе переменных затрат позволяет провести ранжирование продуктов по величине покрытия затрат:

- для удаления из ассортимента продуктов с отрицательной величиной покрытия;
- для определения очередности и объемов загрузки дефицитных мощностей при ограничениях производственных мощностей (конкуренция между продуктами за машину, на которой они производятся, «узкое место») путем ранжирования продуктов по величине покрытия затрат с учетом востребованных рынком объемов этих продуктов и времени их обработки на этих дефицитных мощностях;

ИСПОЛЬЗОВАНИЕ В ПОЛИТИКЕ СНАБЖЕНИЯ ПРЕДПРИЯТИЯ

При необходимости принятия решения о сокращении глубины собственного производства (концепция [бережливого производства](#), подразумевающая отказ от производства отдельных компонентов в пользу закупки их на стороне), с помощью РЧЗ на основе переменных затрат могут быть определены верхние границы закупочных цен;

ИСПОЛЬЗОВАНИЕ РЧЗ НА ОСНОВЕ ПЕРЕМЕННЫХ ЗАТРАТ ДЛЯ АНАЛИЗА РЕНТАБЕЛЬНОСТИ.

РЧЗ на основе переменных затрат позволяет оценить уровень сбыта продукции, при котором достигается компенсация всех затрат и образуется прибыль (точка безубыточности или [break-even point](#)).

ВОЗМОЖНОСТИ FROMA2

Библиотека FROMA2¹ – это библиотека программного обеспечения для экономического моделирования, финансового анализа и планирования операционной деятельности предприятия. Библиотека FROMA2 написана на языке [C++](#) и представляет собой набор программных модулей, предназначенных для использования программистами при создании дружественных потребителю программ для РЧЗ на основе переменных затрат. Библиотека FROMA2 является [статической библиотекой](#).

Библиотека FROMA2 позволяет моделировать формирование частичных затрат и производить их расчет для:

- Склада ресурсов (склада сырья и материалов, склада готовой продукции, склада промежуточных изделий);
- Производства (места обработки ресурсов и превращения их в окончательные или промежуточные изделия в соответствии с рецептурами готовой продукции и/или технологическими картами).

В качестве ресурсов, затраты которых могут считаться поштучно, могут выступать:

- Сырье, материалы, комплектующие;
- Продукты, используемые в производстве других продуктов;
- Удельные (штучные) затраты энергоносителей (электричество, газ, вода, тепло);

¹ FROMA2 – аббревиатура, сокращение от "Free Operation Manager 2".

- Удельные человеческие ресурсы (сдельный труд);
- Удельные амортизационные отчисления.

Библиотека FROMA2 может быть использована как для планирования, так и для учета фактических переменных затрат.

ПРИМЕР 1

Есть предприятие, состоящее из трех подразделений: *Склада сырья и материалов, Производства и Склада готовой продукции*. Для этого предприятия типична картина:

В разные периоды времени предприятием закупаются разные по объему и закупочным ценам партии сырья и материалов. Часть сырья и материалов формирует переходящий запас на Складе сырья и материалов, другая часть - направляется на Производство продукции. Готовая продукция направляется на другой склад – Склад готовой продукции. Часть продукции формирует переходящий запас на Складе готовой продукции, другая часть - отгружается покупателям.

Руководству предприятия для принятия управленческих решений требуются данные об учетной себестоимости сырья и материалов, поступающих в производство, о материальной себестоимости произведенной продукции и о материальной себестоимости продаваемой продукции.

Библиотека FROMA2 позволяет сотрудникам IT-подразделения этого предприятия или его подрядчикам собрать финансово-экономическую модель из трех указанных выше подразделений, установить принцип учета запасов: ФИФО (англ. аббревиатура FIFO, - "первым вошел - первым вышел"), ЛИФО (англ. аббревиатура LIFO, - "последним вошел - первым вышел"²), или ПО СРЕДНЕЙ СЕБЕСТОИМОСТИ (англ. аббревиатура AVERAGE, - "по-среднему") и рассчитать плановые или фактические показатели:

- объем производства продукции, необходимый для отгрузки потребителям и поддержания требуемого запаса на Складе готовой продукции;
- объем закупок сырья и материалов, необходимый для производства продукции и поддержания требуемого запаса на Складе ресурсов;
- учетную себестоимость сырья и материалов, направляемых в производство;
- учетную себестоимость остатков сырья и материалов на Складе ресурсов;
- материальную себестоимость продукции, направляемой на склад готовой продукции;
- материальную себестоимость готовой продукции, отгружаемой покупателям;
- материальную себестоимость остатков продукции на Складе готовой продукции.

Сборка модели предполагает написание программы на языке C++, в которой каждое указанное выше подразделение соответствует определенному специализированному классу из библиотеки, а структура данных, передаваемых между пользователем и/или между экземплярами этих классов, соответствует стандартам информации программного интерфейса библиотеки.

О том, как сконструировать модель для данного примера можно прочитать в разделе «СОЗДАНИЕ МОДЕЛИ С ОДНИМ ЦЕНТРОМ ЗАТРАТ».

ПРИМЕР 2

Руководитель предприятия из Примера 1 поставил своему IT-подразделению новую задачу: на еженедельной основе предоставлять данные о производственной себестоимости производимой и продаваемой продукции, включающей в себя:

² В России метод LIFO не используется с 1 января 2008 года согласно Приказу Минфина России от 26.03.2007 N 26н «О внесении изменений в нормативные правовые акты по бухгалтерскому учету».

- материальную себестоимость продукции (см. предыдущий пример);
- прямые затраты на оплату труда рабочих, осуществляющих производство продукции, с начислениями на неё;
- затраты на электричество, учитываемое при изготовлении каждой единицы продукции.

Решение поставленной задачи с помощью библиотеки FROMA2 может быть достигнута несколькими путями.

Первый путь. Если руководству предприятия необходимы данные о производственной себестоимости продукции без разбивки на материальную себестоимость, прямые затраты труда и прямые затраты на электроэнергию, то модель из трех объектов, сформированная в прошлом примере, вполне подойдет для этой задачи. Изменяется структура данных: в нее добавляются данные о трудозатратах на единицу продукции и стоимости этого труда, о потреблении электроэнергии на каждый вид продукта и ее стоимости. В результате рассчитываем плановые или фактические показатели:

- объем производства продукции, необходимый для отгрузки потребителям и поддержания требуемого запаса на Складе готовой продукции;
- объем закупок сырья и материалов, необходимый для производства продукции и поддержания требуемого запаса на Складе ресурсов;
- трудозатраты, необходимые для производства продукции;
- электропотребление, необходимое для производства продукции;
- учетную себестоимость сырья и материалов, направляемых в производство;
- учетную себестоимость остатков сырья и материалов на Складе ресурсов;
- производственную себестоимость продукции, направляемой на склад готовой продукции;
- производственную себестоимость готовой продукции, отгружаемой покупателям;
- производственную себестоимость остатков продукции на Складе готовой продукции.

Второй путь. Руководству предприятия необходимы данные о производственной себестоимости продукции наряду с данными о материальной себестоимости продукции, т.к. для управленческих решений ему требуются оба типа данных. В этом случае к трем объектам из предыдущего примера возможно добавить четвертый так, чтобы производственное подразделение предприятия описывалось не одним, а двумя экземплярами специализированного класса из библиотеки: один объект отвечает за расчет материальной себестоимости продукции, другой – за затраты, включающие в себя трудозатраты и затраты на электричество. К существующей структуре данных добавляется новая структура данных, включающая в себя данные о трудозатратах на единицу продукции и стоимости этого труда и о потреблении электроэнергии на каждый вид продукта и ее стоимости. В результате рассчитываем плановые или фактические показатели:

- объем производства продукции, необходимый для отгрузки потребителям и поддержания требуемого запаса на Складе готовой продукции;
- объем закупок сырья и материалов, необходимый для производства продукции и поддержания требуемого запаса на Складе ресурсов;
- учетную себестоимость сырья и материалов, направляемых в производство;
- учетную себестоимость остатков сырья и материалов на Складе ресурсов;
- материальную себестоимость продукции, направляемой на склад готовой продукции;
- материальную себестоимость готовой продукции, отгружаемой покупателям;
- материальную себестоимость остатков продукции на Складе готовой продукции;
- трудозатраты, необходимые для производства продукции;
- электропотребление, необходимое для производства продукции;
- полные и удельные затраты на труд и электроэнергию;
- производственную себестоимость продукции, направляемой на склад готовой продукции;
- производственную себестоимость готовой продукции, отгружаемой покупателям;
- производственную себестоимость остатков продукции на Складе готовой продукции.

Методические рекомендации из раздела «[СОЗДАНИЕ МОДЕЛИ С ОДНИМ ЦЕНТРОМ ЗАТРАТ](#)» также подходят для решения этой задачи.

ПРИМЕР 3

На предприятии из *Примеров 1 и 2* принято решение *обособить* Склад сырья и материалов, Производство и Склад готовой продукции, выделив их в отдельные [центры затрат](#). Это связано, например с тем, что часть сырья и материалов предприятие продает другим компаниям, и руководитель нашего предприятия хочет, чтобы затраты на труд сотрудников этого склада, электроэнергию и другие затраты покрывались отпускной ценой продаваемых ресурсов.

На складе готовой продукции, например, тоже новшества, - ввели дополнительную операцию: групповую упаковку и маркировку заказываемых одним заказчиком товаров (комплектация продукции по заказчикам). Это потребовало дополнительный расход упаковочного материала и трудозатраты на упаковывание, и руководитель нашего предприятия также хочет, чтобы затраты, возникающие на СГП, можно было бы контролировать и удерживать на плановом уровне.

Таким образом возникла новая задача: на еженедельной основе получать информацию об

- удельных и полных затратах склада сырья и материалов,
- удельных и полных затратах производственного подразделения и
- удельных и полных затратах склада готовой продукции.

Как создать модель для решения подобной задачи описано в разделе «[СОЗДАНИЕ МОДЕЛИ С НЕСКОЛЬКИМИ ЦЕНТРАМИ ЗАТРАТ](#)».

ЛИЦЕНЗИЯ НА БИБЛИОТЕКУ FROMA2

Библиотека FROMA2 распространяется под лицензией, использующей в качестве шаблона стандартную лицензию [MIT \(X11 License\)](#) в соответствии со [Статьей 1286.1](#) Гражданского кодекса Российской Федерации (часть четвертая) от 18.12.2006 N 230-ФЗ (в ред. от 22.07.2024 и любой более поздней редакции).

Copyright © 2025 Пидкасистый Александр Павлович

Данная лицензия разрешает лицам, получившим копию данного программного обеспечения и сопутствующей документации (далее — Программное обеспечение), безвозмездно использовать Программное обеспечение без ограничений, включая неограниченное право на использование, копирование, изменение, слияние, публикацию, распространение, сублицензирование и/или продажу копий Программного обеспечения, а также лицам, которым предоставляется данное Программное обеспечение, при соблюдении следующих условий:

Указанное выше уведомление об авторском праве и данные условия должны быть включены во все копии или значимые части данного Программного обеспечения.

ДАННОЕ ПРОГРАММНОЕ ОБЕСПЕЧЕНИЕ ПРЕДОСТАВЛЯЕТСЯ «КАК ЕСТЬ», БЕЗ КАКИХ-ЛИБО ГАРАНТИЙ, ЯВНО ВЫРАЖЕННЫХ ИЛИ ПОДРАЗУМЕВАЕМЫХ, ВКЛЮЧАЯ ГАРАНТИИ ТОВАРНОЙ ПРИГОДНОСТИ, СООТВЕТСТВИЯ ПО ЕГО КОНКРЕТНОМУ НАЗНАЧЕНИЮ И ОТСУТСТВИЯ НАРУШЕНИЙ, НО НЕ ОГРАНИЧИВАЯСЬ ИМИ. НИ В КАКОМ СЛУЧАЕ АВТОРЫ ИЛИ ПРАВООБЛАДАТЕЛИ НЕ НЕСУТ ОТВЕТСТВЕННОСТИ ПО КАКИМ-ЛИБО ИСКАМ, ЗА УЩЕРБ ИЛИ ПО ИНЫМ ТРЕБОВАНИЯМ, В ТОМ ЧИСЛЕ, ПРИ ДЕЙСТВИИ КОНТРАКТА, ДЕЛИКТЕ ИЛИ ИНОЙ СИТУАЦИИ, ВОЗНИКШИМ ИЗ-ЗА ИСПОЛЬЗОВАНИЯ ПРОГРАММНОГО ОБЕСПЕЧЕНИЯ ИЛИ ИНЫХ ДЕЙСТВИЙ С ПРОГРАММНЫМ ОБЕСПЕЧЕНИЕМ.

Copyright (c) 2025 Alexandr Pidkasisty

Permission is hereby granted, free of charge, to any person obtaining a copy of this software and associated documentation files (the "Software"), to deal in the Software without restriction, including without limitation the rights to use, copy, modify, merge, publish, distribute, sublicense, and/or sell copies of the Software, and to permit persons to whom the Software is furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM, OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE SOFTWARE.

В случае возникновения коллизий, вызванных различными толкованиями русскоязычной версии настоящей лицензии и англоязычной версии лицензии, русскоязычная версия принимается в качестве основной.

In case of collisions caused by different interpretations of the Russian-language version of the license and the English-language version of the license, the Russian-language version is accepted as the main one.

СОСТАВ И СТРУКТУРА БИБЛИОТЕКИ FROMA2

Библиотека FROMA2 написана на языке программирования C++ и включает в себя несколько модулей. Ключевыми модулями библиотеки являются модуль *warehouse* и модуль *manufact*, реализующих наборы инструментов для моделирования центра учета запасов и центра учета затрат соответственно.

МОДУЛЬ WAREHOUSE

Модуль *warehouse* представлен заголовочным файлом *warehouse_module.h* и файлом реализации *warehouse_lib.cpp*. В модуле содержатся классы и методы, реализующие алгоритмы учета запасов на *складе ресурсов*.

Ядром модуля является класс *clsSKU*, описывающий склад ресурсов для одной номенклатурной позиции (одного SKU, Stock Keeping Unit - единицы складского учета) и включающий в себя основные алгоритмы учета. Экземпляр этого класса содержит информацию о количестве периодов проекта, наименовании номенклатурной позиции, единице измерения объемных показателей; информацию о закупках, отгрузках, остатков по периодам в натуральном, ценовом и денежном выражении. Класс содержит методы учета запасов FIFO, LIFO или «по-среднему», метод автоматического расчета закупок с учетом плана отгрузок и плана остатков в каждом периоде и метод расчета балансовых остатков товаров в каждом периоде проекта. В классе также присутствуют сервисные методы, позволяющие корректно изменять длительность проекта и другие параметры, а также имеются методы сериализации/ десериализации данных (запись состояния экземпляра класса в файл и восстановления этого состояния из файла) и методы визуального контроля.

Основным классом модуля *warehouse*, с которым чаще всего придется иметь дело программистам, использующим библиотеку FROMA2, является класс *clsStorage*. Этот класс описывает склад ресурсов для множества номенклатурных позиций (множества *SKU*). Экземпляр класса представляет собой контейнер и интерфейс для нескольких объектов типа *clsSKU* и позволяет моделировать движение по складу множества

номенклатурных единиц. Экземпляр этого класса содержит информацию о количестве периодов проекта, домашней валюте проекта (валюте основных расчетов), применяемом принципе учета запасов (FIFO, LIFO или «по-среднему»), количестве номенклатурных позиций, учитываемых на складе, контейнер для множества экземпляров класса `clsSKU` (однотоварных складов) и другие служебные параметры. Класс содержит методы сериализации и десериализации, расчетные методы и методы визуального контроля.

Детальное описание инструментов модуля содержится в разделе «[СКЛАД РЕСУРСОВ](#)».

МОДУЛЬ MANUFACT

Модуль *manufact* представлен заголовочным файлом *manufact_module.h* и файлом реализации *manufact_lib.cpp*. В модуле содержатся классы и методы, реализующие алгоритмы для производства³.

Ядром модуля является [класс `clsRecipeItem`](#), описывающий рецептуру и/или технологическую карту отдельного продукта/услуги и содержащий информацию о длительности производственного цикла и расходе ресурсов (сырья, материалов, компонентов, энергоносителей, сдельного труда, штучной амортизации и проч.) на производство единицы данного продукта/услуги в натуральном выражении в каждый период производственного цикла. Класс также содержит расчетные методы, позволяющие скалькулировать объем потребления ресурсов в натуральном выражении для всего плана выпуска данного продукта на протяжении всего проекта, рассчитать объем, удельную и полную себестоимость⁴ готового продукта и незавершенного производства в каждый период проекта. Класс содержит методы сериализации и десериализации и методы визуального контроля.

[Класс `clsManufactItem`](#) описывает производство отдельного продукта/услуги на протяжении всего проекта и содержит информацию о потребности в ресурсах для выпуска продукта/услуги в заданных объемах по заданному графику, учетные цены на ресурсы, объемы, полную и удельную себестоимость изготовленного продукта/услуги, балансовую полную и удельную стоимость незаконченного производства. Класс *clsManufactItem* является контейнером и интерфейсом для класса *clsRecipeItem*. Класс также содержит методы сериализации и десериализации и методы визуального контроля.

Основным классом модуля *manufact*, с которым чаще всего придется иметь дело программистам, использующим библиотеку FROMA2, является [класс `clsManufactory`](#). Этот класс описывает полноценное производство *ассортимента* продуктов/услуг. Экземпляр класса представляет собой контейнер и интерфейс для нескольких объектов типа *clsManufactItem* и позволяет моделировать производство *множества* номенклатурных единиц продукции/услуг. Экземпляр этого класса содержит информацию о количестве периодов проекта, полной номенклатуре и потребности в ресурсах, используемых в производстве. Методы класса позволяют добавлять учетные цены на все ресурсы, получать объединенную информацию об объемах, удельной и полной себестоимости готовой продукции и незавершенного производства каждого продукта. Класс также содержит методы сериализации и десериализации и методы визуального контроля.

Детальное описание инструментов модуля содержится в разделе «[ПРОИЗВОДСТВО](#)».

³ «Производство» следует понимать в широком смысле слова: это и производство товарной продукции, и производство услуг. В частности, услуг хранения на складе ресурсов и других центрах учета затрат (см. раздел «[МОДЕЛИРОВАНИЕ С ПОМОЩЬЮ КЛАССОВ `clsStorage` и `clsManufactory`](#)»).

⁴ Под себестоимостью в этом разделе будем понимать себестоимость используемых ресурсов: удельная себестоимость – себестоимость ресурсов в единице продукции, полная себестоимость – себестоимость ресурсов в партии продукта. Это может быть материальная себестоимость, если ресурсами являются сырье и материалы, производственная себестоимость, если в дополнении к сырью и материалам используются сдельный труд, затраты на энергоресурсы и штучная амортизация или только часть производственной себестоимости, если в качестве ресурсов использовать только труд, энергоресурсы и проч., без сырья и материалов.

МОДУЛЬ COMMON_VALUES

Модуль *common_values* представлен единственным файлом *common_values.hpp*, в котором объединены заголовки классов и методы и их реализация.

В модуле определен стандарт данных для информационных потоков внутри экземпляров классов библиотеки, между экземплярами классов и пользователем (ввод, хранение, обработка и вывод информации), см. раздел «[ТИПЫ ДАННЫХ ДЛЯ ИНФОРМАЦИОННЫХ ПОТОКОВ](#)».

Данные о натуральных, ценовых и стоимостных величинах, а также долях и процентах этих величин представлены в виде вещественных чисел. Для вещественных чисел используется либо встроенный в C++ тип *double*, либо собственный вещественный тип библиотеки *LongReal*. В модуле можно выбрать тип представления вещественных чисел. Для типа *LongReal* можно также настроить количество десятичных знаков для вещественного числа и диапазон значений его экспоненты. Скорость вычислений при использовании типа *LongReal* ниже, а объем потребления вычислительных ресурсов больше, чем при использовании типа *double*, но в некоторых случаях использование типа *LongReal* позволяет избежать *ошибок потери точности* при расчетах, см. раздел «[ТИП ВЕЩЕСТВЕННЫХ ЧИСЕЛ](#)».

В модуле *common_values* также представлены [инструменты для отображения результатов расчётов](#):

- наиболее употребимые константы и типы, используемые во всех классах;
- классы и функции визуального контроля работоспособности методов из модулей *warehouse* и *manufact* (для удобства «отлова ошибок» при модернизации последних);
- классы и методы для вывода исходных данных и результатов расчетов на терминал и/или в текстовый файл.

МОДУЛЬ LONGREAL

Модуль *LongReal* представлен заголовочным файлом *LongReal_module.h* и файлом реализации *LongReal_lib.cpp*. В модуле содержатся классы и методы, реализующие вещественные числа и основные арифметические и логические операции над ними. [Тип LongReal](#) является альтернативой встроенному в C++ типу *double* и предназначен для проведения расчетов с высокой точностью.

Если точность вычислений с типом *double* программисту покажется неудовлетворительной⁵ и ему будет мало 15 десятичных разрядов (например, в ситуации, когда себестоимость партий продуктов измеряется миллиардами рублей, а расход сырья на единицу продукции измеряется величинами с пятью знаками после запятой и более, могут возникать ошибки потери точности, особенно заметные на больших объемах данных и длительных сроках проекта), он может использовать тип *LongReal*. В типе *LongReal* по умолчанию установлено 64 десятичных разряда для мантиссы числа, а его экспонента лежит в диапазоне значений от минус 32768 до плюс 32768. При желании программиста и наличии больших вычислительных ресурсов можно изменить требуемое число десятичных разрядов и диапазон значений экспоненты: число десятичных разрядов ограничивается значением $(\text{numeric_limits}<\text{size_t}>::\text{max}()/2-1)$, а значения экспоненты могут лежать в пределах от $(\text{numeric_limits}<\text{int}>::\text{min}()/2+1)$ до $(\text{numeric_limits}<\text{int}>::\text{max}()/2-1)$, где $\text{max}()$ и $\text{min}()$ – функции из стандартного для C++ модуля *limits*⁶.

Вещественные числа типа *LongReal* могут быть инициализированы и им могут быть присвоены вещественные числа, представленные в виде:

- чисел типа *LongReal*,

⁵ Тип *double* представляет вещественное число двойной точности с плавающей точкой в диапазоне +/- 1.7E-308 до 1.7E+308. В памяти занимает 8 байт (64 бита). В числах *double*: 1 знаковый бит, 11 бит для экспоненты и 52 бит для мантиссы, то есть в сумме 64 бита. 52-разрядная мантисса позволяет определить точность до 16 десятичных знаков (<https://metanit.com/cpp/tutorial/2.3.php>).

⁶ Использование крайних значений для мантиссы и экспоненты чисел типа *LongReal* не тестировалось.

- чисел типа *double* (в фиксированном и научном форматах),
- чисел, представленных строкой.

Вещественные числа типа *LongReal* могут быть возвращены для присваивания и для вывода в виде:

- числа, представленного строкой цифр, целая и дробная часть которого разделена запятой,
- числа типа *long double*,
- числа типа *double*,
- числа типа *float*.

Вещественное число типа *LongReal* может быть выведено также в виде строки вида «*.ddddEn*» (мантисса *dddd* со степенью *n*).

Основные арифметические и логические операции над вещественными числами типа *LongReal*:

- Проверка числа на ноль,
- Проверка на положительную бесконечность,
- Проверка на отрицательную бесконечность,
- Проверка на неопределенность (*nan*),
- Оператор сравнения равно,
- Оператор сравнения не равно,
- Оператор умножения,
- Унарный минус,
- Оператор больше,
- Оператор меньше,
- Оператор больше или равно,
- Оператор меньше или равно,
- Оператор сложения,
- Оператор вычитания,
- Оператор декремента на заданную величину,
- Оператор инкремента на заданную величину,
- Вычисление обратного числа,
- Оператор деления,
- Оператор взятия модуля числа.

Методы сериализации и десериализации модуля позволяют сохранять в файл и читать из файла:

- Единичные вещественные числа типа *LongReal*;
- Массивы вещественных чисел типа *LongReal*.

Помимо арифметических, логических операций и операций сериализации и десериализации, в классе предусмотрены методы ввода и вывода вещественного числа и методы визуального контроля внутреннего представления вещественного числа.

Детальное описание инструментов модуля содержится в разделе «[АРИФМЕТИКА ДЛИННЫХ ВЕЩЕСТВЕННЫХ ЧИСЕЛ](#)».

МОДУЛЬ IMPEX

Модуль *Impex* представлен заголовочным файлом *Impex_module.h* и файлом реализации *Impex_lib.cpp*. В модуле содержатся классы и методы, реализующие обмен данными между объектами, относящимися к библиотеке FROMA2 и программами сторонних разработчиков с помощью [CSV-файлов](#).

Основой модуля *Impex* является класс [clsImpex](#), представляющий собой буфер обмена информацией и его методы, позволяющие:

- Прочитать информацию из *CSV-файла* в буфер;
- Вернуть всё или часть содержимого буфера в стандартном формате представления данных библиотеки FROMA2, пригодном для использования в финансово-экономической модели, сконструированной из объектов библиотеки FROMA2;
- Прочитать информацию из финансово-экономической модели, сконструированной из объектов библиотеки FROMA2 в буфер;
- Транспонировать буфер (по правилам [транспонирования](#) матрицы);
- Выгрузить всю или часть информации из буфера в *CSV-файл*.

Модуль содержит и другие вспомогательные методы.

Данный модуль, в частности, использовался для оценки корректности расчетов финансово-экономических моделей, построенных с помощью библиотеки *FROMA2*. Для сравнения результатов расчётов строились одинаковые модели: одна с помощью библиотеки *FROMA2*, другая с помощью программы [Project Expert 7](#), хорошо зарекомендовавшей себя на российском рынке. Исходные данные из модели, построенной в *Project Expert 7*, экспортировались в *CSV-файлы*, а затем импортировались в модель, построенную с помощью *FROMA2*. Результаты расчётов в обоих моделях были экспортированы в итоговые *CSV-файлы*, содержимое которых сравнивалось между собой. Сравнения показали идентичность результатов расчётов и корректность моделей, построенных с помощью библиотеки *FROMA2*.

Детальное описание инструментов модуля содержится в разделе «[ИМПОРТ И ЭКСПОРТ ДАННЫХ](#)».

МОДУЛЬ SERIALIZATION

Модуль *serialization* представлен единственным файлом *serialization_module.hpp*, в котором объединены заголовки функций и их реализация. В нем представлены шаблоны и нешаблонные перегруженные функции для сериализации и десериализации данных. Сериализация и десериализация предусмотрена для данных ограниченного набора типов, достаточных для функционирования библиотеки FROMA2:

- Строки символов типа *string* стандартного модуля [string](#);
- Единичного объекта [тривиально копируемого типа](#);
- *Массива* объектов тривиально копируемого типа.

В совокупности с методами сериализации/ десериализации вещественных чисел типа [LongReal](#), представленных в одноименном модуле, эти функции полностью покрывают потребности любой финансово-экономической модели, сконструированной из объектов библиотеки FROMA2 в сохранении своего состояния в файл с заданным именем и последующего восстановления своего состояния из этого файла.

Детальное описание инструментов модуля содержится в разделе «[СЕРИАЛИЗАЦИЯ И ДЕСЕРИАЛИЗАЦИЯ ДАННЫХ](#)».

МОДУЛЬ BASEPROJECT

Модуль *baseproject* представлен заголовочным файлом *baseproject_module.h* и файлом реализации *baseproject_lib.cpp*. В модуле описывается класс [clsBaseProject](#), единственное назначение которого – представлять удобство родительского класса (интерфейса) для пользовательского класса финансово-экономической модели, сконструированной из объектов библиотеки FROMA2. Предполагается, что любая финансово-экономическая модель, конструируемая из объектов библиотеки FROMA2, может быть представлена единственным классом, являющимся потомком *clsBaseProject*, а в качестве полей этого нового

класса будут выступать экземпляры классов *clsStorage*, *clsManufactory* и других классов библиотеки *FROMA2*. Это не является обязательным условием моделирования, но может быть полезным для программистов.

Основными удобствами использования класса *clsBaseProject* в качестве «родителя» для классов-потомков является:

- Средства управления названием и описанием проекта;
- Средства управления выбора устройства и вывода отчета (пустое устройство, терминал или файл);
- Типовые методы класса, ответственные за инкапсуляцию, наследование и полиморфизм (конструктор по умолчанию и виртуальный деструктор, конструктор копирования и перемещения, функция обмена значениями между объектами, операторы присваивания копированием и перемещением, метод сброса состояния объекта и др.);
- Средства сериализации и десериализации.

Детальное описание инструментов модуля содержится в разделе «[РОДИТЕЛЬСКИЙ КЛАСС ДЛЯ МОДЕЛИ](#)».

МОДУЛЬ AGREGATE

Модуль *agregate* представлен заголовочным файлом *agregate_module.h* и файлом реализации *agregate_lib.cpp*. Модуль предназначен для моделирования предприятия с несколькими *центрами учета* и несколькими *центрами затрат* *clsStorage* и *clsManufactory*, объединенных в последовательную цепочку объектов указанных типов. Модуль содержит класс *clsAgregate*, являющийся потомком класса *clsBaseProject* и реализующий ряд методов, позволяющих производить действия сразу с несколькими объектами *clsStorage* и *clsManufactory*, независимо от их количества и последовательности в цепочке объектов. К таким методам, в частности, относятся:

- Общие расчётные методы, объединяющие передачу данных между объектами, входящими в состав цепочки и реализующие корректный последовательный вызов расчётных методов этих объектов;
- Методы экспорта данных в *cvs*-файлы;
- Методы вывода отчётов;
- Методы ввода/ вывода данных.

Детальное описание инструментов модуля содержится в разделе «[АГРЕГАТОР ДЛЯ МОДЕЛИ ПРЕДПРИЯТИЯ](#)».

МОДУЛЬ PROGRESS

Отображение прогресса или хода выполнения расчетов призвано поддержать комфортное состояние пользователя при длительной загрузке процессора вычислительными действиями. В модулях *WAREHOUSE* и *MANUFACT* присутствуют разные вычислительные методы, осуществляющие свою работу как в однопоточном, так и в многопоточном режимах. Объекты библиотеки FROMA2 могут использоваться как в консольных, так и оконных приложениях. Инструменты данного модуля *progress* предоставляют возможность обеспечивать визуализацию проводимых вычислений во всех таких случаях.

Модуль *progress* представлен единственным файлом *progress_module.hpp*, в котором объединены заголовки классов и методы и их реализация. Классы и методы модуля предназначены для вывода на экран индикатора прогресса (хода вычислений) при выполнении длительных расчётов.

Детальное описание инструментов модуля содержится в разделе «[ОТОБРАЖЕНИЕ ПРОГРЕССА ПРИ ВЫПОЛНЕНИИ РАСЧЕТОВ](#)».

ТИПЫ ДАННЫХ ДЛЯ ИНФОРМАЦИОННЫХ ПОТОКОВ

Количественные данные, такие как объемные (килограммы, штуки, партии и т.п.), стоимостные (рубли и т.п.) и ценовые величины (рублей за штуку и т.п.), представляются во всех модулях библиотеки FROMA2 вещественными числами.

Описательные данные, такие как название ресурса (например, «изделие номер 5»), единица измерения его натурального объема (например, «шт.»), представляются во всех модулях объектами типа [std::string](#) стандартного модуля C++ [<string>](#).

СТРУКТУРА STRITEM

Количественные данные, относящиеся к какому-либо ресурсу, обладающие сразу тремя количественными характеристиками (объемной, стоимостной и ценовой) представлены структурой **strItem** модуля [COMMON VALUES](#). Эта структура предназначена для хранения информации об объеме, цене и стоимости единственной партии ресурсов (например, сырья или готовой продукции), поступающей, находящейся или убывающей со склада ресурсов и/или производства.

Таблица 1 Поля структуры *strItem*

Тип поля структуры	Описание
volume	Тип <i>decimal</i> - алиас, псевдоним для вещественного типа. В зависимости от выбора программиста принимает значение <i>double</i> или <i>LongReal</i> . Поле задаёт размер партии сырья и материалов в натуральном измерении (например, в кг)
price	Тип <i>decimal</i> . Поле задаёт цену ресурса за единицу (например, руб./кг)
value	Тип <i>decimal</i> . Поле задаёт стоимость партии для конкретного ресурса (например, руб)

Таблица 2 Функции структуры *strItem*

Функции структуры	Описание
StF	Метод сериализации состояния объекта в поток типа ofstream
RfF	Метод десериализации состояния объекта из потока типа ifstream

Все поля и функции структуры являются публичными.

bool strItem::StF(ofstream &_outF);

Метод записывает состояние объекта типа *strItem* в поток.

Параметры:

&_outF – файловый поток, открытый для записи, типа *ofstream*.

Возвращает значение *true*, если операция прошла успешно, в противном случае возвращает *false*. Использует метод [SEF](#) из модуля *SERIALIZATION* или перегруженный метод [SEF](#) из модуля *LongReal* для каждого из полей (*volume*, *price* и *value*) в зависимости от типа вещественных чисел, используемых в структуре *strItem*.

bool strItem::RfF(ifstream &_inF);

Метод восстанавливает состояние объекта типа *strItem* из потока.

Параметры:

&_inF - файловый поток, открытый для чтения типа *ifstream*.

Возвращает значение *true*, если операция прошла успешно, в противном случае возвращает *false*. Использует метод [DSF](#) из модуля *SERIALIZATION* или перегруженный метод [DSF](#) из модуля *LongReal* для каждого из полей (*volume*, *price* и *value*) в зависимости от типа вещественных чисел, используемых в структуре *strItem*.

Таблица 3 Перегрузки функций, не являющихся членами

Перегрузки функций, не являющихся членами	Описание
operator<<	Перегрузка оператора вывода информации в поток ostream .

friend ostream& operator<<(ostream &stream, strItem st);

Перегруженный оператор вывода информации из объекта типа *strItem* в поток.

Параметры:

&stream - поток с именем *stream* типа *ostream*;

st - экземпляр структуры типа *strItem* с именем st.

Возвращает поток *stream* с записанными туда значениями *volume*, *price* и *value*, разделенных строкой символов ", ". После значения *value* идет комбинация символов перевода строки "\n". Использует стандартный оператор вывода в поток "<<". Перегруженный оператор используется в методах визуального контроля модулей [WAREHOUSE](#) и [MANUFACT](#).

СТРУКТУРА strNameMeas

Описательные данные, относящиеся к какому-либо ресурсу, имеющему свой объект типа *strItem*, представлены структурой **strNameMeas** модуля [COMMON_VALUES](#). Эта структура предназначена для хранения информации о названии ресурса (например, «Изделие номер 5») и названия единицы его натурального измерения в партии (например, «шт.»).

Таблица 4 Поля структуры strNameMeas

Тип поля структуры	Описание
name	Тип std::string. Наименование номенклатурной позиции
measure	Тип std::string. Наименование единицы измерения номенклатурной позиции

Структура strNameMeas не имеет собственных методов.

ОБЛАСТЬ ИМЕН nmBPTypes

Оба стандартных для библиотеки FROMA2 типа *strItem* и *strNameMeas* включены в область имен nmBPTypes модуля [COMMON_VALUES](#). Помимо них в эту область имен включены ряд наиболее употребимых констант, типов и функций.

Таблица 5 Другие наиболее важные константы, типы и методы области имен nmBPTypes

Субъекты области имен nmBPTypes	Описание
const decimal epsln	«Бесконечно малая величина», являющаяся «условным нулем». Применяется для корректного сравнения вещественных чисел друг с другом и с нулем.
ReportData	Тип данных перечисления <i>enum</i> ; состоит из конечного набора именованных констант типа <i>size_t</i> . Назначение: выбор нужного поля при получении данных из объекта типа <i>strItem</i> : «volume» или 0 – поле volume, «price» или 1 – поле price и «value» или 2 – поле value объекта типа <i>strItem</i>
Currency	Тип данных перечисления <i>enum</i> ; состоит из конечного набора именованных констант типа <i>size_t</i> . Каждая константа представляет собой индекс валюты, в которой

	представлены исходные данные и производятся расчеты денежных величин: «RUR» или 0, «USD» или 1, «EUR» или 2, «CNY» или 3.
CurrencyTXT	Массив строковых констант с наименованиями валюты в виде текста.
constexpr bool is_real_v	Функция проверки допустимости типов. Выполняется на этапе компиляции. Допустимые типы: арифметический и LongReal .
constexpr bool is_tbl_d_v	Функция проверки допустимости типов. Выполняется на этапе компиляции. Допустимые типы: арифметический , LongReal и strItem .

Для решения [проблемы сравнения вещественных чисел](#) друг с другом и/или с нулем, применяется константа **epsIn**. Эта константа представляет собой «бесконечно малую величину», являющуюся «условным нулем» для корректного сравнения вещественных чисел друг с другом и нулем. По умолчанию она равна 10^{-9} . Это значение можно изменить⁷.

Значение константы в типе *Currency* должно соответствовать индексу элемента с наименованием валюты в массиве *CurrencyTXT*. (Например, значение константы EUR типа *Currency* равно 2, и значение элемента массива *CurrencyTXT* [2] с индексом 2 должно быть равным «EUR»). При добавлении в тип *Currency* новых констант и в массив *CurrencyTXT* новых названий валют, необходимо следить, чтобы наименование добавляемой константы и название валюты корреспондировали по смыслу друг с другом, а также следить за тем, чтобы значение добавляемой константы равнялось индексу добавляемого элемента массива.

ТИП ВЕЩЕСТВЕННЫХ ЧИСЕЛ

Вещественные числа в библиотеке FROMA2 по желанию программиста могут быть представлены либо стандартным для C++ типом [double](#), либо собственным типом библиотеки [LongReal](#), реализация которого представлена в модуле [LONGREAL](#). За установку типа вещественного числа отвечает строка кода в модуле [COMMON VALUES](#):

Таблица 6 Выбор типа вещественного числа

Альтернативная строка кода	Результат
typedef double decimal;	Вещественные числа представлены типом <i>double</i> ; decimal - псевдоним для вещественного типа
typedef LongReal decimal;	Вещественные числа представлены типом <i>LongReal</i> ; decimal - псевдоним для вещественного типа

Вместо *double* может быть установлен иной [тип вещественного числа](#), например, *long double* или *float*. Однако такая подстановка не рекомендуется: она не гарантирует корректность расчетов и может привести к непредсказуемому результату⁸.

Для корректного копирования *массивов* вещественных чисел, будь то числа типа *double* или типа *LongReal*, в модуле [COMMON VALUES](#) представлена шаблонная функция

```
template<typename T>
```

```
constexpr void var_cpy(T* destin, const T* source, const size_t Num)
```

Метод копирует данные из массива, на который ссылается указатель *source* в массив, на который ссылается указатель *destin*. Массивы должны иметь одинаковую размерность. Инстанцирование шаблона происходит на этапе компиляции.

Параметры:

⁷ Значение 10^{-9} равно значению макроса DBL_EPSILON из стандартного заголовочного файла [<float> \(float.h\)](#). Это разница между единицей и наименьшим значением, превышающим единицу, которое можно представить для вещественных чисел типа [double](#) в C++.

⁸ Например, при выборе типа вещественного числа *float*, значение *epsIn* из пространства имен *nmBTypes* (Таблица 5), должно быть равно 10^{-5} или больше по модулю.

`destin` – указатель на массив, *куда* копируются данные; тип элементов массива определяется шаблоном *T*;

`source` – указатель на массив, *откуда* копируются данные; тип элементов массива определяется шаблоном *T*;

`Num` – размер (число элементов) массивов.

Если элементы массивов имеют [тривиально копируемый тип](#), копирование осуществляется блоками памяти с помощью функции [memcpy](#). В противном случае копирование поэлементное с помощью функции [copy_n](#). Тип *double* относится к *тривиально копируемому типу*, элементы этого типа копируются с помощью функции [memcpy](#). Для элементов типа *LongReal* используется функция [copy_n](#).

Функция позволяет абстрагироваться от конкретного типа используемых в расчетах вещественных чисел и широко применяется в модулях [warehouse](#) и [manufact](#).

ТИП ИНДИКАТОРА ПРОГРЕССА

Для визуализации процесса вычислений может быть использован индикатор прогресса практически любого стороннего класса. Основное требование к такому классу – наличие в своем составе метода [Update\(int\)](#). Этому требованию соответствует как входящий в библиотеку *FROMA2* класс индикатора прогресса для консольных приложений [clsprogress_bar](#), так и класс [wxProgressDialog](#) библиотеки [wxWidgets](#) для приложений с графическим интерфейсом. При этом возможные кандидаты на роль индикатора прогресса не ограничиваются перечисленными выше классами. За установку типа индикатора отвечает строка кода в модуле [COMMON VALUES](#):

Альтернативная строка кода	Результат
<code>typedef clsprogress_bar type_progress</code>	Выбран индикатор класса <i>clsprogress_bar</i> ; <i>type_progress</i> – псевдоним для типа класса индикатора
<code>typedef wxProgressDialog type_progress</code>	Выбран индикатор класса <i>wxProgressDialog</i> из библиотеки <i>wxWidgets</i> ; <i>type_progress</i> – псевдоним для типа класса индикатора

Вместо `clsprogress_bar` может быть выбран иной класс индикатора прогресса, имеющий в своем составе метод [Update\(int\)](#).

Детальное описание инструментов индикатора прогресса содержится в разделе «[ОТОБРАЖЕНИЕ ПРОГРЕССА ПРИ ВЫПОЛНЕНИИ РАСЧЕТОВ](#)».

СКЛАД РЕСУРСОВ

Основные типы, константы, классы и методы, реализующие алгоритмы для склада ресурсов, находятся в модуле [WAREHOUSE](#). Модуль *warehouse* представляет собой набор инструментов для моделирования [центра учета запасов](#). Инструменты модуля позволяют реализовать [партионный учет](#) ресурсов⁹. Состав модуля приведен в таблице ниже.

Таблица 7 Состав модуля *WAREHOUSE*

Состав модуля	Описание	Заметка
<code>enum AccountingMethod</code>	Тип перечисление <i>enum</i> , состоящий из значений FIFO , LIFO , AVG	Предназначен для выбора принципа учета запасов: FIFO, LIFO или По-среднему
<code>const string AccountTXT</code>	Массив строковых констант тип <i>string</i> с наименованиями принципов учета запасов в виде строки символов	Предназначен для вывода информации о выбранном типе учета в виде строки символов

⁹ Инструменты модуля *warehouse* не предназначены для учёта добавленной стоимости, формируемой на складе ресурсов. Для этих целей применяются инструменты модуля [MANUFACT](#).

enum ChoiseData	Тип перечисление <i>enum</i> , состоящий из значений <i>purchase</i> , <i>balance</i> , <i>shipment</i>	Предназначен для выбора данных: поступления ресурсов на склад, баланс остатков на складе и отгрузки со склада
enum PurchaseCalc	Тип перечисление <i>enum</i> , состоящий из значений <i>calc</i> и <i>nocalc</i>	Флаг разрешающий/ запрещающий автоматический расчет закупок под требуемые объемы отгрузок. В случае запрещающего флага <i>nocalc</i> , объем закупок вводится вручную
const string PurchaseCalcTXT[]	Массив строковых констант тип <i>string</i> с наименованиями флага авторасчета закупок в виде строки символов	Предназначен для вывода информации об установленном флаге в виде строки символов
const string ProhibitedTXT[]	Массив строковых констант тип <i>string</i> с наименованиями флага разрешения/ запрещения закупок и отгрузок в одном и том же периоде в виде строки символов	Предназначен для вывода информации об установленном флаге clsSKU::indr в виде строки символов
struct TLack	Тип структуры для передачи информации о величине дефицита и наименования ресурса, где возник дефицит. Поля для данных: - decimal <i>lack</i> (величина дефицита), - string <i>Name</i> (наименование ресурса, где выявлен дефицит)	В случае ручного ввода объема поступления ресурсов на склад возможна ситуация, когда для требуемых отгрузок и остатков поступающих на склад ресурсов недостаточно; в этом случае переменная <i>TLack</i> возвращает величину дефицита и название ресурса, где образовался этот дефицит
class clsSKU	Класс Склада для одной номенклатурной позиции (моносклад)	Экземпляр класса позволяет моделировать учетные процессы при движении через склад одной единственной номенклатурной единицы; содержит методы учета запасов
class clsStorage	Класс Склада для множества номенклатурных позиций	Экземпляр класса представляет собой контейнер для нескольких объектов типа <i>clsSKU</i> и позволяет моделировать учетные процессы при движении через склад множества номенклатурных единиц

Помимо указанных в таблице членов, в заголовочном файле `warehouse_module.h` содержится ряд строковых констант, используемых при выводе информации на экран/файл.

КЛАСС *clsSKU*

Класс *clsSKU* предназначен для *моделирования учётных процессов* на складе комплектующих, сырья и материалов, а также учётных процессов на складе готовой продукции¹⁰. Экземпляр класса *clsSKU* описывает склад ресурсов и позволяет вести *партионный* учет для *одной* номенклатурной позиции (*одного* SKU, [Stock Keeping Unit](#) - единицы складского учета).

Основные *задачи*, которые можно решить применением класса *clsSKU*:

1. Рассчитать объем и график закупок/ поступлений ресурсов на склад, обеспечивающих заданный объем и график отгрузки ресурсов со склада и выполнение норматива неснижаемого остатка на складе.
2. Рассчитать учётную себестоимость ресурсов, отгружаемых со склада в каждый период отгрузки в соответствии с известными объемами партий и ценами ресурсов на входе склада в каждый период закупки/ поступления и выбранным [принципом учета запасов](#).

ПОЛЯ КЛАССА *clsSKU*

¹⁰ И сырье, и материалы, и комплектующие, и полуфабрикаты, и готовую продукцию будем называть одним словом «ресурсы».

Таблица 8 Поля класса *clsSKU*. Секция *private*

Поле класса	Описание	Заметка
size_t PrCount	Тип size_t . Количество периодов проекта. Нулевой период – это период перед началом проекта	Значение устанавливается при создании объекта класса. Может корректироваться.
string name	Тип string . Название номенклатурной единицы складского учета	Значение устанавливается при создании объекта класса. Может корректироваться.
string measure	Тип string . Наименование единицы измерения объемов ресурсов (кг, штук и т.п.)	Значение устанавливается при создании объекта класса. Может корректироваться
strItem *Pur	(Pur – от слова Purchase.) Тип указатель на массив из партий ресурсов, поступивших на склад. Элемент массива имеет тип <i>strItem</i>	Динамический массив. При создании экземпляра класса создается и заполняется нулями
strItem *Rem	(Rem – от слова Remaining.) Тип указатель на вспомогательный массив остатков от партий ресурсов на складе на конец проекта (партии отличаются друг от друга датами поступления на склад). Элемент массива имеет тип <i>strItem</i>	Динамический массив. При создании экземпляра класса создается и заполняется нулями. Носит вспомогательный характер
strItem *Bal	(Bal – от слова Balance) Тип указатель на массив остатков ресурсов на складе в каждый период проекта (независимо от того, в какие периоды были получены ресурсы). Элемент массива имеет тип <i>strItem</i>	Динамический массив. При создании экземпляра класса создается и заполняется нулями
strItem *Ship	(Ship – от слова Shipment) Тип указатель на массив партий ресурсов, отгружаемых со склада по плану (план отгрузок)	Динамический массив. При создании экземпляра класса создается и заполняется нулями
decimal lack	Тип определяется псевдонимом decimal . Дефицит ресурсов для отгрузки со склада; если <i>lack</i> >0, то отгрузка по плану невозможна	Расчётная величина. При создании экземпляра класса получает значение ноль.
bool indr	Тип bool. Переменная, хранящая установленное пользователем разрешение использовать закупаемые в каком-либо i-м периоде ресурсы для отгрузки в этом же периоде: <i>true</i> - можно, <i>false</i> - нельзя	Значение устанавливается при создании объекта класса. Может корректироваться. Для случаев, когда приход на склад осуществляется в конце периода, а отгрузка – в начале следующего, - флаг устанавливается в значение <i>false</i> . По умолчанию устанавливается <i>false</i> .
AccountingMethod acct	Переменная типа <i>AccountingMethod</i> , хранящая установленный пользователем принцип учета запасов: <i>FIFO</i> , <i>LIFO</i> или <i>AVG</i>	Значение устанавливается при создании объекта класса. Корректировке без уничтожения экземпляра класса не подлежит
PurchaseCalc pcalc	Переменная типа <i>PurchaseCalc</i> , хранящая установленный пользователем флаг разрешения/ запрещения (<i>calc</i> / <i>nocalc</i>) автоматического расчета закупок под требуемые объемы отгрузок.	Значение устанавливается при создании объекта класса. Может корректироваться. По умолчанию устанавливается автоматический расчет <i>pcalc</i> = <i>calc</i> .
decimal share	Запас ресурсов на складе в каждый период, выраженный в доле от объема отгрузок за этот период	Значение устанавливается при создании объекта класса. Может корректироваться

Методы класса *clsSKU* можно условно разделить на следующие группы:

- Конструкторы, деструктор, перегруженные операторы;
- Алгоритмы учета (методы в секции Private и методы в секции Public);
- Get-методы;
- Set-методы;

- Методы сохранения состояния объекта в файл и восстановления из файла;
- Методы визуального контроля

Ниже подробно описываются все методы.

КОНСТРУКТОРЫ, ДЕКТРУКТОР, ПЕРЕГРУЖЕННЫЕ ОПЕРАТОРЫ

clsSKU()

Конструктор по умолчанию (без параметров). Создает и инициализирует переменные: `PrCount` = `sZero`; `name` = `measure` = `EmpStr`; `lack` = `dZero`; `indr` = `false`; `acct` = `FIFO`; `pccalc` = `calc`; `share` = `dZero`. Динамические массивы не создаются, указатели на них устанавливаются в `nullptr`.

clsSKU(const size_t _PrCount, const string &_name, const string &_measure, AccountingMethod _ac, bool _ind, PurchaseCalc _flag, const decimal& _share, const stritem _Ship[])

Конструктор с вводом объемов отгрузок. Конструктор инициализирует переменные, создает и заполняет нулями динамические массивы `Pur`, `Rem`, `Bal` и `Ship`.

Параметры:

`_PrCount` - количество периодов проекта; устанавливает значение поля класса `PrCount`;

`&_name` – ссылка на название номенклатурной единицы складского учета; устанавливает значение поля класса `name`;

`&_measure` – ссылка на название единицы измерения номенклатурной единицы складского учета; устанавливает значение поля класса `measure`;

`_ac` – принцип учета запасов: `FIFO`, `LIFO` или `AVG`; устанавливает значение поля класса `acct`;

`_ind` – флаг разрешения/ запрещения использовать закупаемые в каком-либо i-м периоде ресурсы для отгрузки в этом же периоде; устанавливает значение поля класса `indr`;

`_flag` – флаг разрешения/ запрещения автоматического расчета закупок под требуемые объемы отгрузок; устанавливает значение поля класса `pccalc`;

`&_share` – ссылка на запас ресурсов на складе в каждый период, выраженный в доле от объема отгрузок за этот период; устанавливает значение поля класса `share`;

`_Ship[]` – указатель на массив с объемом отгрузок (в каждом элементе массива заполнены только поля `volume`, поля `price` и `value` заполнены нулями); устанавливает значение поля класса `Ship`.

clsSKU(const size_t _PrCount, const string &_name, const string &_measure, AccountingMethod _ac, bool _ind, PurchaseCalc _flag, const decimal& _share)

Конструктор с параметрами. Аналогичен предыдущему конструктору, но без ввода объемов отгрузок в виде массива `stritem _Ship[]`.

clsSKU(const clsSKU &obj)

Конструктор копирования.

Параметры:

`&obj` – ссылка на копируемый константный экземпляр класса `clsSKU`.

clsSKU(clsSKU &&obj)

Конструктор перемещения.

Параметры:

&obj - перемещаемый экземпляр класса clsSKU.

clsSKU& operator=(const clsSKU &obj)

Перегрузка оператора присваивания копированием.

Параметры:

&obj – ссылка на копируемый константный экземпляр класса clsSKU.

clsSKU& operator=(clsSKU &&obj)

Перегрузка оператора присваивания перемещением

Параметры:

&&obj - перемещаемый экземпляр класса clsSKU.

~clsSKU()

Деструктор.

void swap(clsSKU &obj) noexcept

Функция обмена значениями между экземплярами класса.

Параметры:

&obj – ссылка на обмениваемый экземпляр класса clsSKU.

bool operator == (const string &Rightname) const

Перегрузка оператора сравнения «равно» для поиска экземпляра класса по имени.

Параметры:

&Rightname – ссылка на строку символов, с которой сравнивается поле [name](#) экземпляра класса.

АЛГОРИТМЫ УЧЕТА. СЕКЦИЯ PRIVATE**decimal AVGcalc(const strItem P[], strItem R[], strItem S[], size_t N, bool ip)**

Функция рассчитывает цены и стоимость ресурсов, отгружаемых со склада в каждом периоде проекта по принципу [AVG](#) (*по среднему*) формирует вспомогательный массив остатков от партий ресурсов на складе на конец проекта ([Rem](#)) с учетом плановых отгрузок и возвращает дефицит сырья для отгрузок, если таковой имеет место быть. Тип возвращаемого значения определяется псевдонимом [decimal](#).

Функция не оптимизирует план отгрузок, если возникает дефицит сырья и материалов; в периоды, где возникает дефицит, расчет цен и стоимости отгружаемых партий НЕ КОРРЕКТЕН!

Блок схема алгоритма приведена на рисунке ниже (Рисунок 1).

Параметры:

P[] - указатель на массив закупок/поступлений на склад (массив изменять запрещено, const); тип элемента массива [strItem](#).

R[] - указатель на вспомогательный массив, - после всех проведенных расчетов показывает остатки от каждой партии на конец проекта; тип элемента массива [strItem](#).

`S[]` – указатель на массив отгрузок, у которого заполнены только поля *volume* (массив изменяется в процессе работы функции: рассчитываются поля *price* и *value*); тип элемента массива `strItem`.

`N` - размерность массивов закупок, остатков и отгрузок, совпадает с количеством периодов проекта; стандартный тип *size_t*.

`ip` - индикатор, признак того, можно ли использовать закупаемые в каком-либо периоде ресурсы для отгрузки в этом же периоде: *"true"* означает, что закупка осуществляется в начале каждого периода и закупаемые ресурсы могут быть использованы в отгрузках того же периода, *"false"* означает, что закупки производятся в конце каждого периода, поэтому отгрузка этих ресурсов возможна только в следующих периодах; стандартный тип *bool*.

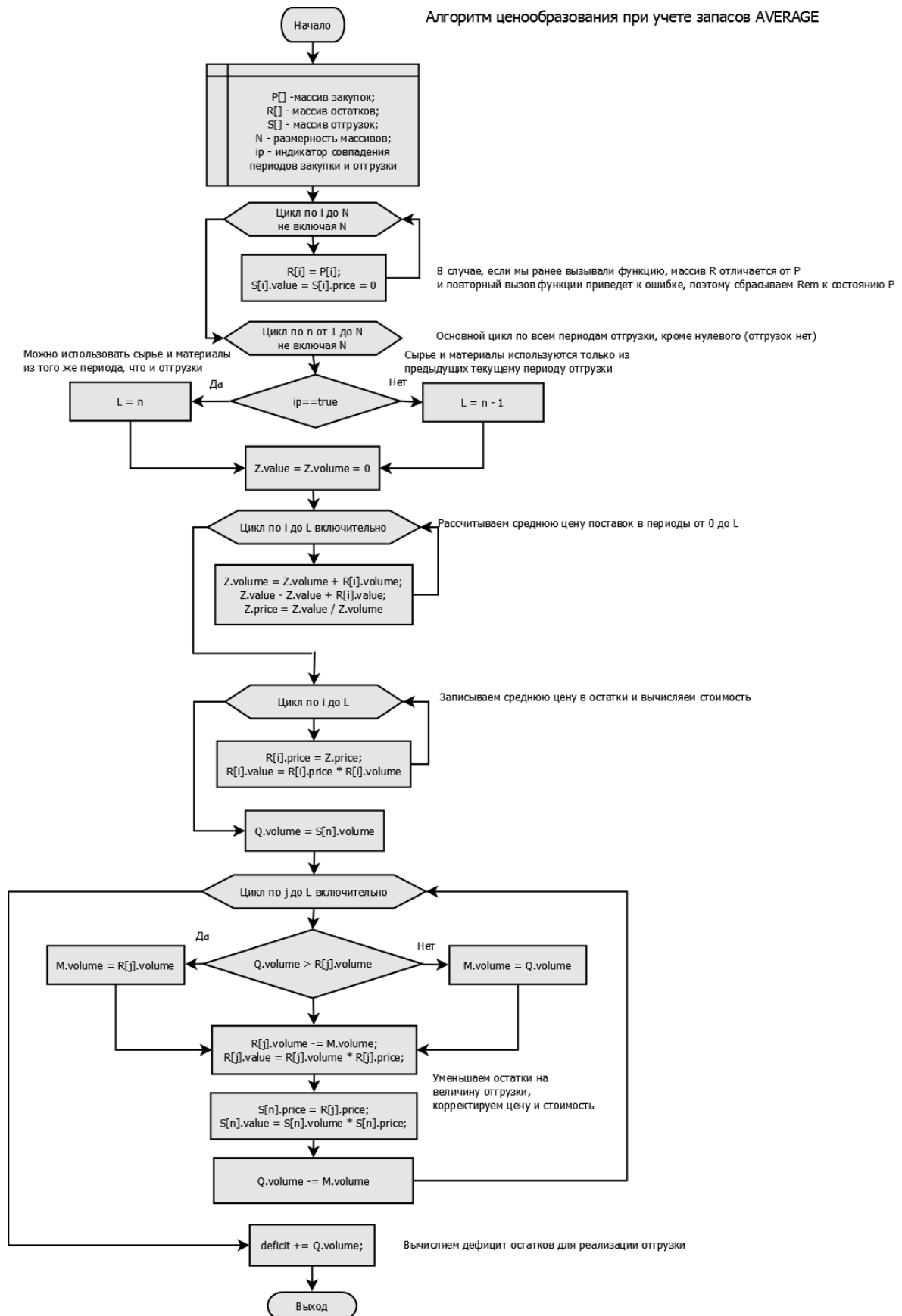


Рисунок 1 Блок-схема алгоритма учета запасов по принципу среднего (AVG)

decimal FLFcalc(const strItem P[], strItem R[], strItem S[], size_t N, bool ip, AccountingMethod ind)

Функция рассчитывает цены и стоимость сырья и материалов, отгружаемых со склада в каждом периоде проекта. по принципу [FIFO](#) или [LIFO](#) в зависимости от индикатора метода учета [ind](#), формирует вспомогательный массив остатков от партий ресурсов на складе на конец проекта ([Rem](#)) с учетом плановых отгрузок и возвращает дефицит сырья для отгрузок, если таковой имеет место быть. Тип возвращаемого значения определяется псевдонимом [decimal](#).

Функция не оптимизирует план отгрузок, если возникает дефицит сырья и материалов; в периоды, где возникает дефицит, расчет цен и стоимости отгружаемых партий НЕ КОРРЕКТЕН!

Блок схема алгоритма приведена на рисунке ниже (Рисунок 2).

Параметры:

P[] - указатель на массив закупок/поступлений на склад (массив изменять запрещено, const); тип элемента массива [strItem](#).

R[] - указатель на вспомогательный массив, - после всех проведенных расчетов показывает остатки от каждой партии на конец проекта; тип элемента массива [strItem](#).

S[] – указатель на массив отгрузок, у которого заполнены только поля *volume* (массив изменяется в процессе работы функции: рассчитываются поля *price* и *value*); тип элемента массива [strItem](#).

N - размерность массивов закупок, остатков и отгрузок, совпадает с количеством [периодов проекта](#); стандартный тип [size_t](#).

[ip](#) - индикатор, признак того, можно ли использовать закупаемые в каком-либо периоде ресурсы для отгрузки в этом же периоде: "*true*" означает, что закупка осуществляется в начале каждого периода и закупаемые ресурсы могут быть использованы в отгрузках того же периода, "*false*" означает, что закупки производятся в конце каждого периода, поэтому отгрузка этих ресурсов возможна только в следующих периодах; стандартный тип *bool*.

[ind](#) – флаг, устанавливающий принцип учета запасов: FIFO или LIFO.

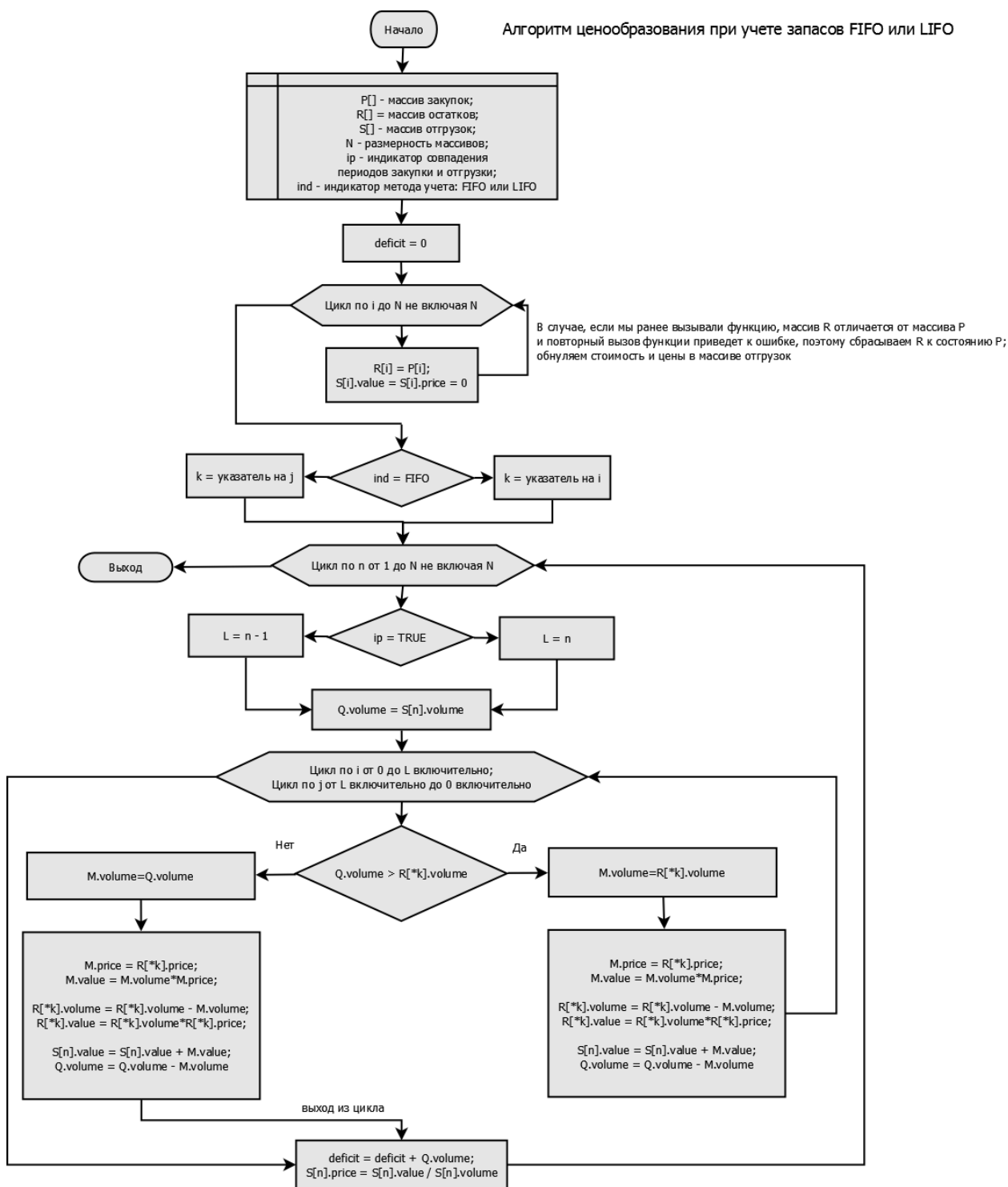


Рисунок 2 Блок-схема алгоритма учета запасов по принципам FIFO и LIFO

decimal PURcalc(strItem P[], const strItem S[], size_t N, bool ip, decimal Shr)

Функция рассчитывает объемы закупок, необходимые для выполнения плана отгрузок и обеспечения заданного остатка ресурсов (остаток задается как доля от объема отгрузок за период) и возвращает дефицит ресурсов в предплановом (нулевом) периоде. Рассчитываются закупки в плановых периодах - с первого до последнего (N-1). Закупки в нулевой период задаются вручную и характеризуют остатки ресурсов к началу проекта (элемент стартового баланса). Тип возвращаемого значения определяется псевдонимом [decimal](#).

Блок-схема алгоритма приведена на рисунке ниже (Рисунок 3).

Параметры:

P[] - указатель на массив закупок/ поступлений на склад (изменяется в процессе работы функции); тип элемента массива [strItem](#).

S[] - указатель на массив отгрузок, у которого заполнены только поля *volume* (массив изменять запрещено, const); тип элемента массива [strItem](#).

N - размерность массивов закупок/ поступлений на склад, остатков и отгрузок, совпадает с количеством [периодов проекта](#); стандартный тип *size_t*.

ip - индикатор, признак того, можно ли использовать закупаемые в каком-либо периоде ресурсы для отгрузки в этом же периоде: "true" означает, что закупка осуществляется в начале каждого периода и закупаемые ресурсы могут быть использованы в отгрузках того же периода, "false" означает, что закупки производятся в конце каждого периода, поэтому отгрузка этих ресурсов возможна только в следующих периодах; стандартный тип *bool*.

Shr - плановый остаток сырья, заданный, как доля отгрузок за период (минимальное число ноль – на складе поддерживаются нулевые остатки).

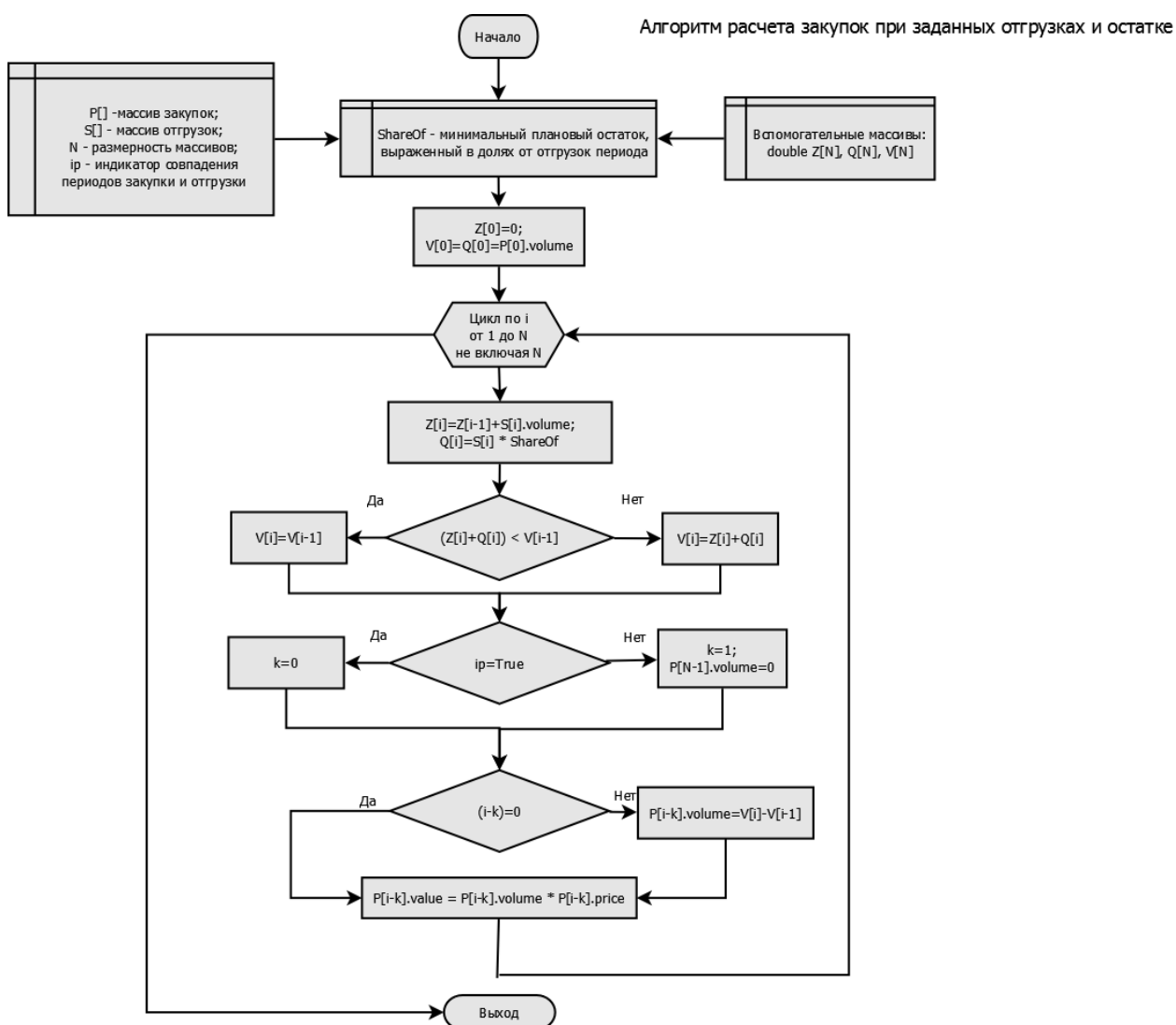


Рисунок 3 Блок-схема алгоритма расчета закупок по периодам

void BALcalc(const strItem P[], strItem B[], const strItem S[], size_t N)

Функция формирует массив балансовых остатков ресурсов на складе по периодам. Функцию следует вызывать только после корректного расчета объемов закупок (функцией [PURcalc](#)) и цен отгрузок (функцией [FLFcalc](#) или [AVGcalc](#)).

Параметры:

P[] - указатель на массив корректно рассчитанных закупок/ поступлений на склад (неизменен); тип элемента массива [strItem](#).

B[] - указатель на массив балансовых остатков (изменяется в процессе работы функции); тип элемента массива [strItem](#).

S[] - указатель на массив корректно рассчитанных отгрузок (неизменен); тип элемента массива [strItem](#).

N - размерность массивов закупок/ поступлений на склад, остатков и отгрузок, совпадает с количеством [периодов проекта](#); стандартный тип [size_t](#).

АЛГОРИТМЫ УЧЕТА. СЕКЦИЯ PUBLIC**const decimal& CalcPrice()**

Функция рассчитывает цены и стоимость отгружаемых ресурсов в каждом периоде на основе установленного пользователем принципа учета запасов: [FIFO](#), [LIFO](#) или [AVG](#), записывает результаты расчетов в массив [Ship](#), возвращает величину дефицита сырья и материалов для обеспечения плановых отгрузок. Если дефицит [lack](#) не равен нулю, рассчитанные цены и стоимость могут нести некорректные значения. Функция вызывает методы [AVGcalc](#), [FLFcalc](#), в зависимости от установленного принципа учета запасов; если дефицит меньше заранее заданной «бесконечно малой величины» [epsln](#), то дефицит обнуляется и вызывается функция [BALcalc](#). Тип возвращаемого значения определяется псевдонимом [decimal](#).

const decimal& CalcPurchase()

Функция проверяет флаг [pcalc](#) и, в случае установленного значения [calc](#), вызывает метод [PURcalc](#) и рассчитывает объемы закупок, необходимые для выполнения плана отгрузок и обеспечения заданного остатка ресурсов, записывает результаты расчетов в массив [Pur](#), возвращает величину дефицита ресурсов для обеспечения плановых отгрузок в нулевом периоде. Тип возвращаемого значения определяется псевдонимом [decimal](#). Если же флаг [pcalc](#) установлен в значение [nocalc](#), функция прекращает работу и возвращает ноль.

bool Resize(size_t _n)

Функция изменяет размеры массивов [Pur](#), [Rem](#), [Bal](#), [Ship](#) типа [strItem](#), устанавливая новое число периодов проекта, равное [_n](#). При [_n](#) < [PrCount](#), данные обрезаются, при [_n](#) > [PrCount](#), - добавляются новые элементы массивов с нулевыми значениями. Функция возвращает [true](#) при удачном изменении размеров массивов, [false](#) - в противном случае.

Параметры:

[_n](#) – новое значение продолжительности проекта (новое количество периодов); стандартный тип [size_t](#).

TLack Calculate()

Функция вызывает метод [CalcPurchase](#) и рассчитывает требуемый объем закупок/ поступлений на склад, если выставлен флаг [pcalc](#), разрешающий автоматический расчет. Проверяет дефицит ресурсов для выполнения заданного плана отгрузок и остатков. Если дефицит [lack](#) больше заранее заданной «бесконечно малой величины» [epsln](#), то функция возвращает величину этого дефицита и на этом работа функции заканчивается.

Если же величина дефицита меньше [epsln](#), то функция вызывает метод [CalcPrice](#) и рассчитывает учетную удельную и полную себестоимости отгружаемой партии в соответствии с выбранным принципом учета запасов [FIFO](#), [LIFO](#) или [AVG](#). Функция повторно проверяет дефицит ресурсов для отгрузки со склада [lack](#). Если дефицит [lack](#) больше заранее заданной «бесконечно малой величины» [epsln](#), то функция возвращает величину этого дефицита и на этом работа функции заканчивается. Иначе величина дефицита обнуляется, и функция возвращает ноль. Информация возвращается в виде структуры типа [TLack](#).

Надо помнить, что если [lack](#) > [epsln](#), сделанные ранее расчеты объемных и стоимостных показателей могут быть некорректными.

GET-МЕТОДЫ

const size_t& Count() const

Возвращает const-ссылку на длительность проекта, выраженную в [количестве периодов](#).

const string& Name() const

Возвращает const-ссылку на [название](#) номенклатурной единицы складского учета.

const string& Measure() const

Возвращает const-ссылку на название [единицы натурального измерения](#) номенклатурной единицы складского учета.

string AccMethod() const

Возвращает [принцип учета запасов](#) в виде текстовой строки.

string Permission() const

Возвращает [флаг разрешения/запрещения отгрузок](#) в одном периоде в виде текстовой строки.

bool PermissionBool() const

Возвращает [флаг разрешения/запрещения отгрузок](#) в одном периоде в виде значения типа bool.

string AutoPurchase() const

Возвращает [флаг авторасчета/ ручного расчета](#) закупок в виде текстовой строки.

const PurchaseCalc& GetAutoPurchase() const

Возвращает const-ссылку на [флаг авторасчета/ ручного расчета закупок](#); тип возвращаемой величины [PurchaseCalc](#).

const decimal& Share() const

Возвращает const-ссылку на [норматив запаса ресурсов](#) в долях от объема отгрузки. Тип возвращаемого значения определяется псевдонимом [decimal](#).

const strItem* GetDataItem(const ChoiseData _cd, size_t _N) const

Функция возвращает информацию, содержащуюся в элементе массива с индексом _N в виде const-указателя на структуру типа [strItem](#).

Параметры:

_cd – именованное целое число из перечисленных в типе [ChoiseData](#), указывающее на массив, из которого нужно выбрать данные: “*purchase*” – из массива закупок/ поступлений ([Pur](#)), “*balance*” – из массива остатков ([Bal](#)), *shipment* – из массива отгрузок ([Ship](#));

_N – номер периода, совпадающий с индексом элемента выбранного массива.

const strItem* GetShipment() const

Метод возвращает константный указатель на массив отгрузок со склада [Ship](#).

const strItem* GetPurchase() const

Метод возвращает константный указатель на массив закупок/поступлений на склад [Pur](#).

const strItem* GetBalance() const

Метод возвращает константный указатель на массив остатков на складе [Bal](#).

const decimal& GetLack() const

Метод возвращает величину дефицита ресурсов, хранящуюся в поле [lack](#) для отгрузки со склада. Тип возвращаемого значения определяется псевдонимом decimal.

SET-МЕТОДЫ**void SetName(const string& _name)**

Меняет название номенклатурной единицы складского учета поле [name](#).

Параметры:

_name – новое название номенклатурной единицы складского учета.

void SetMeasure(const string& _measure)

Меняет название единицы измерения номенклатурной единицы складского учета поле [measure](#).

Параметры:

_measure – новое название единицы измерения номенклатурной единицы складского учета.

void SetPermission(bool _indr)

Устанавливает флаг разрешения/ запрещения отгрузок и закупок в одном периоде поле [indr](#).

Параметры:

_indr – новый флаг разрешения/ запрещения отгрузок в одном периоде: *true* - можно, *false* – нельзя.

void SetAutoPurchase(PurchaseCalc _pcalc)

Устанавливает флаг автоматического/ ручного расчета закупок/ поступлений на склад поле [pcalc](#).

Параметры:

_pcalc – новый флаг разрешения/ запрещения (*calc/ nocalc*) автоматического расчета закупок/поступлений на склад под требуемые объемы отгрузок.

void SetShare(decimal _share)

Устанавливает новый норматив запаса ресурсов на складе в долях от объема отгрузок поле [share](#).

Параметры:

_share – новое значение величины запаса, выраженное в долях от объема закупок. Тип вещественного числа определяется псевдонимом [decimal](#).

void SetAccount(AccountingMethod _acct)

Устанавливает новый принцип учета запасов поле [acct](#).

Параметры:

_acct – новое значение принципа учета запасов: *FIFO*, *LIFO* или *AVG*.

bool SetDataItem(const ChoiseData _cd, const strItem _U, size_t _N)

Функция записывает в элемент массива с индексом `_N` (период проекта с номером `_N`) новые значения *volume*, *price* и *value* из переменной `_U`; выбор массива производится значением переменной `_cd`. Возвращает *true*, если запись удалась.

Параметры:

`_cd` – признак редактируемого массива: “purchase” – массив *Pur*, “balance” – массив *Bal*, “shipment” – массив *Ship*;

`_U` – переменная с новыми данными, которые записываются в элемент массива;

`_N` – индекс редактируемого элемента массива (номер периода проекта).

bool SetPurchase(const strItem _unit[], size_t _count)

Функция загружает данные о закупке ресурсов на склад (объемы, цены, стоимость). Данные загружаются копированием.

Параметры:

`_unit` – указатель на массив загружаемых данных типа *strItem*;

`_count` – размерность массива (может не совпадать с размерностью внутреннего массива закупок *Pur*, равной *PrCount*. Если размер переданного в функцию массива больше массива закупок, то данные обрезаются, иначе используется размер полученного массива).

bool SetPurchase(strItem* &_unit, size_t _count)

Функция загружает данные о закупке ресурсов на склад (объемы, цены, стоимость). Данные загружаются перемещением во внутренний массив *Pur* и копированием массива *Pur* во внутренний массив *Rem*.

Параметры:

`_unit` – ссылка на указатель на массив загружаемых данных типа *strItem*;

`_count` – размерность массива (может не совпадать с размерностью внутреннего массива закупок *Pur*, равной *PrCount*. Если размер переданного в функцию массива больше массива закупок, то данные обрезаются, иначе используется размер полученного массива).

bool SetPurPrice(const strItem _unit[], size_t _count)

Функция загружает данные о ценах закупаемых/поставляемых на склад ресурсов из полей “price” массива `_unit[]` в поля “price” внутренних массивов *Pur* и *Rem* и заполняет поля “value” массивов *Pur* и *Rem* произведением значений из их же полей “volume” и “price”.

Параметры:

`_unit[]` – указатель на массив загружаемых данных с ценами ресурсов типа *strItem*;

`_count` – размерность массива (может не совпадать с размерностью внутреннего массива закупок *Pur*, равной *PrCount*. Если размер переданного в функцию массива больше массива закупок, то данные обрезаются, иначе используется размер полученного массива).

bool SetShipment(const decimal _unit[], size_t _count)

Функция загружает данные об объемах плановых отгрузок ресурсов со склада. Тип вещественного числа загружаемого массива определяется псевдонимом *decimal*. Метод возвращает *true* в случае удачной загрузки данных.

Параметры:

`_unit[]` – указатель на массив загружаемых данных с ценами ресурсов типа [decimal](#);

`_count` - размерность массива (может не совпадать с размерностью внутреннего массива закупок [Ship](#), равной [PrCount](#). Если размер переданного в функцию массива больше массива закупок, то данные обрезаются, иначе используется размер полученного массива).

bool SetShipment(const strItem _unit[], size_t _count)

Функция загружает данные о плановых отгрузках ресурсов со склада. Значимыми значениями являются только объемы в натуральном выражении (поля volume). Данные загружаются копированием во внутренний массив Ship. Метод возвращает true в случае удачной загрузки данных.

Параметры:

`_unit[]` – указатель на массив типа [strItem](#), содержащий объемы отгрузок;

`_count` - размерность массива (может не совпадать с размерностью внутреннего массива закупок [Ship](#), равной [PrCount](#). Если размер переданного в функцию массива больше массива закупок, то данные обрезаются, иначе используется размер полученного массива).

bool SetShipment(strItem* &_unit, size_t _count)

Функция загружает данные о плановых отгрузках ресурсов со склада. Значимыми значениями являются только объемы в натуральном выражении (поля volume). Данные загружаются перемещением во внутренний массив Ship. Метод возвращает true в случае удачной загрузки данных.

Параметры:

`_unit[]` - ссылка на указатель на массив типа [strItem](#), содержащий объемы отгрузок;

`_count` - размерность массива (может не совпадать с размерностью массива закупок [PrCount](#)). Внимание!!! После перемещения массив `_unit` не пуст и содержит неопределенные данные! Ответственность за его удаление лежит на пользователе.

МЕТОДЫ СОХРАНЕНИЯ СОСТОЯНИЯ ОБЪЕКТА В ФАЙЛ И ВОССТАНОВЛЕНИЯ ИЗ ФАЙЛА

bool StF(ofstream &_outF) const

Метод имплементации записи в файловую переменную текущего экземпляра класса (запись в файл, метод сериализации). Название метода является аббревиатурой, расшифровывается “StF” как “Save to File”.

Параметры:

`&_outF` - экземпляр класса [ofstream](#) для записи данных. При инициализации переменной `_outF` необходимо установить флаг бинарного режима открытия потока (не текстового!) [ios::binary](#).

bool RfF(ifstream &_inF)

Метод имплементации чтения из файловой переменной текущего экземпляра класса (чтение из файла, метод десериализации). Название метода является аббревиатурой, расшифровывается “RfF” как “Read from File”.

Параметры:

`&_inF` - экземпляр класса [ifstream](#) для чтения данных. При инициализации переменной `_inF` необходимо установить флаг бинарного режима открытия потока (не текстового!) [ios::binary](#).

МЕТОДЫ ВИЗУАЛЬНОГО КОНТРОЛЯ

void View() const

Метод выводит на экран состояние закупок [Pur](#), остатков от партий на конец проекта [Rem](#), балансовых остатков ресурсов в каждом периоде проекта [Bal](#), отгрузок [Ship](#). Метод использовался, как вспомогательный при разработке и отладки кода.

void ViewData(const ChoiseData _cd) const

Метод выводит на экран состояние конкретного выбранного массива [Pur](#), [Bal](#) или [Ship](#). выбор массива производится значением переменной `_cd` типа [ChoiseData](#).

Параметры:

`_cd` – флаг выбора массива для вывода значений его элементов на экран: “purchase” – массив закупок [Pur](#), “balance” – массив остатков [Bal](#), “shipment” – массив отгрузок [Ship](#).

КАК ПОЛЬЗОВАТЬСЯ КЛАССОМ `clsSKU`

Класс `clsSKU` может быть пригоден для самостоятельного применения в редких случаях *моноресурсного* склада (склада для *одной* номенклатурной позиции, *одного* SKU). Обычно класс `clsSKU` используется в составе класса [clsStorage](#) – класса для нескольких номенклатурных позиций. Тем не менее при желании использовать объект класса `clsSKU` в качестве самодостаточного, можно рекомендовать следующую последовательность действий.

1. Создаем новый экземпляр класса `clsSKU` в динамической памяти с помощью [конструктора с параметрами](#).
2. Если использовался конструктор без ввода объемов отгрузок, вводим отгрузки с помощью метода [SetShipment](#).
3. Вводим цены на ресурс в каждом периоде проекта с помощью метода [SetPurPrice](#). Если подразумевается остаток ресурса на начало проекта (в нулевом периоде) или есть желание ввести объемы закупок/ поступлений на склад вручную, не используя автоматический расчет ([nocalc](#)), то ввод объемов, цен и стоимости можно объединить в методе [SetPurchase](#).
4. При вводе некорректных данных, их изменение в любом из внутренних массивов класса в любом периоде проекта можно осуществить с помощью метода [SetDataItem](#).
5. Рассчитываем *потребность в ресурсах* (при наличии флага [calc](#)), рассчитываем *учетную удельную и полную себестоимости* отгружаемой партии в соответствии с выбранным принципом учета запасов и проверяем [дефицит](#) с помощью метода [Calculate](#).
6. При наличии дефицита, удаляем созданный объект, изменяем исходные данные и снова возвращаемся к п. 1, либо редактируем данные, как указано в п.4. Добиваемся отсутствия дефицита и, соответственно, корректности расчетных величин.
7. «Достаем» из экземпляра класса массивы с данными, содержащими в т.ч. и искомые расчетные величины: массив с *отгрузками* в натуральном, удельном и полном стоимостном выражении с помощью метода [GetShipment](#), массив с *остатками* в натуральном, удельном и полном стоимостном выражении с помощью метода [GetBalance](#), массив с *закупками/ поступлениями* на склад в натуральном, удельном и полном стоимостном выражении с помощью метода [GetPurchase](#). Если нет необходимости в получении массивов и достаточно получить данные из конкретного массива в каком-либо заданном периоде проекта, можно воспользоваться функцией [GetDataItem](#).
8. При желании вывести содержание массивов в стандартный поток `std::cout`, можно воспользоваться одним из двух [методов визуального контроля](#): `View` или `ViewData`.
9. После того, как созданный в п.1 экземпляр класса `clsSKU` не нужен, его удаляем.
10. На любом этапе между п.п. 1 и 9 состояние объекта класса `clsSKU` можно сохранить в файл с последующим восстановлением из этого файла с помощью [методов сериализации и десериализации](#).

ПРИМЕР ПРОГРАММЫ С КЛАССОМ `clsSKU`

Ниже представлен код программы, демонстрирующей работу класса clsSKU.

```
#include <iostream>
#include <warehouse_module.h>           // Подключаем модуль склада

using namespace std;

int main() {
    // Исходные данные
    size_t PrCount = 13;                // Количество периодов проекта
    AccountingMethod Amethod = AVG;     // Принцип учета запасов
    string Name_01 = "Pork Meat";       // Наименование позиции сырья
    string Meas_01 = "kg";              // Единица измерения
    bool PurchPerm = false;              // Разрешение закупок и отгрузок в одном периоде
    PurchaseCalc flag = calc;           // Разрешение на автоматический расчет закупок
    decimal Share = 0.5;                // Запас сырья в каждый период, выраженный в доле
                                        // от объема отгрузок за этот период

    strItem ship[PrCount] = {
        0, 0, 0,
        40, 5, 0, // Отгрузки по периодам, начиная с 1-го
        40, 0, 0,
        45, 0, 0,
        50, 0, 0,
        55, 0, 0,
        55, 0, 0,
        55, 0, 0,
        60, 0, 0,
        60, 0, 0,
        60, 0, 0,
        60, 0, 0,
        60, 0, 0 };

    strItem purc[PrCount] = { 100, 5, 500, // Запас на начало проекта
        0, 6, 0, // Закупочные цены по периодам
        0, 7, 0,
        0, 8, 0,
        0, 9, 0,
        0, 10, 0,
        0, 11, 0,
        0, 12, 0,
        0, 13, 0,
        0, 14, 0,
        0, 15, 0,
        0, 16, 0,
        0, 17, 0 };
}
```

```

// Создаем склад для ресурса
clsSKU* SKU_A = new clsSKU(PrCount, Name_01, Meas_01, Amethod, PurchPerm, flag, Share,
ship);
SKU_A->SetPurchase(purc, PrCount); // Загружаем остаток на начало проекта
                                   // и закупочные цены ресурса
TLack lack = SKU_A->Calculate();    // Рассчитываем потребность в ресурсах
                                   // и проверяем дефицит
cout << "Checking deficit. Deficit is " << lack.lack << endl;
cout << "**** Purchase viewing ****" << endl;
SKU_A->ViewData(purchase);          // Вывод данных закупок
cout << "**** Shipment viewing ****" << endl;
SKU_A->ViewData(shipment);          // Вывод данных отгрузок

delete SKU_A;
return 0;
}
```

Программа выводит на экран информацию о закупках и об отгрузках.

Checking deficit. Deficit is 0
 *** Purchase viewing ***

By volume										
Name	Measure	0	1	2	3	4	5	6	7	8
Pork Meat	kg	100.00	0.00	47.50	52.50	57.50	55.00	55.00	62.50	60.00
By volume										
Name	Measure	9	10	11	12					
Pork Meat	kg	60.00	60.00	60.00	0.00					
By price										
Name	Measure	0	1	2	3	4	5	6	7	8
Pork Meat	RUR/kg	5.00	6.00	7.00	8.00	9.00	10.00	11.00	12.00	13.00
By price										
Name	Measure	9	10	11	12					
Pork Meat	RUR/kg	14.00	15.00	16.00	17.00					
By value										
Name	Measure	0	1	2	3	4	5	6	7	8
Pork Meat	RUR	500.00	0.00	332.50	420.00	517.50	550.00	605.00	750.00	780.00
By value										
Name	Measure	9	10	11	12					
Pork Meat	RUR	840.00	900.00	960.00	0.00					

Красной линией выделены: на первом рисунке расчетные величины закупок ресурсов, во втором – расчетные значения учетной себестоимости ресурсов при отгрузке.

*** Shipment viewing ***

By volume										
Name	Measure	0	1	2	3	4	5	6	7	8
Pork Meat	kg	0.00	40.00	40.00	45.00	50.00	55.00	55.00	55.00	60.00
By volume										
Name	Measure	9	10	11	12					
Pork Meat	kg	60.00	60.00	60.00	60.00					
By price										
Name	Measure	0	1	2	3	4	5	6	7	8
Pork Meat	RUR/kg	0.00	5.00	5.00	6.41	7.52	8.55	9.52	10.51	11.54
By price										
Name	Measure	9	10	11	12					
Pork Meat	RUR/kg	12.51	13.50	14.50	15.50					
By value										
Name	Measure	0	1	2	3	4	5	6	7	8
Pork Meat	RUR	0.00	200.00	200.00	288.33	376.11	470.37	523.46	577.82	692.61
By value										
Name	Measure	9	10	11	12					
Pork Meat	RUR	750.87	810.29	870.10	930.03					

Исходный код программы приведен в [репозитории](#) в папке [froma2/examples/SKU_example](#); файл *main.cpp*. Содержание файла может незначительно отличаться от приведенного выше.

КЛАСС `clsStorage`

Класс *clsStorage* предназначен для *моделирования учётных процессов* на складе ресурсов: комплектующих, сырья и материалов, готовой продукции и т.п. Экземпляр класса *clsStorage* описывает склад ресурсов и позволяет вести *партионный учет* для *множества* номенклатурных позиций (множества SKU). Класс представляет собой контейнер для нескольких экземпляров класса `clsSKU` (однотоварных складов).

Также как для класса [clsSKU](#), применением класса *clsStorage* решаются задачи:

1. Расчёта объема и графика закупок/ поступлений ресурсов на склад, обеспечивающих заданный объем и график отгрузки ресурсов со склада и выполнение норматива неснижаемых остатков на складе многономенклатурных позиций
2. Расчёта учётной себестоимости ресурсов, отгружаемых со склада в каждый период отгрузки в соответствии с известными объемами партий и ценами ресурсов на входе склада в каждый период закупки/ поступления и выбранным [принципом учета запасов](#).

ПОЛЯ КЛАССА `clsStorage`Таблица 9 Поля класса `clsStorage`. Секция `Private`

Поле класса	Описание	Заметка
<code>size_t PrCount</code>	Тип <code>size_t</code> . Количество периодов проекта. Нулевой период – это период перед началом проекта	Значение устанавливается при создании объекта класса. Может корректироваться.
<code>Currency hmcure</code>	Тип <code>Currency</code> . Домашняя (основная) валюта проекта	Значение устанавливается при создании объекта класса. Может корректироваться. Домашняя валюта распространяется на все номенклатурные позиции (элементы типа <code>clsSKU</code>).
<code>AccountingMethod acct</code>	Переменная типа <code>AccountingMethod</code> , хранящая установленный пользователем принцип учета запасов: FIFO, LIFO или AVG.	Значение устанавливается при создании объекта класса. Корректировке без уничтожения экземпляра класса не подлежит. Принцип учета действует на все номенклатурные позиции (элементы типа <code>clsSKU</code>).
<code>vector<clsSKU> stock</code>	Тип <code>vector<clsSKU></code> . Контейнер для множества экземпляров класса <code>clsSKU</code> (однотоварных складов)	Статическая переменная
<code>atomic<bool> Calculation_Exit</code>	Тип <code>atomic<bool></code> . Флаг завершения работы метода <code>Calculate(size_t, size_t)</code>	Значение <code>false</code> устанавливается при создании объекта класса. При копировании и перемещении объектов класса не копируется, не перемещается, а инициализируется значением <code>false</code> . Не участвует в обмене состояниями между объектами с помощью метода <code>swap</code> . Не сериализуется и не десериализуется. Используется в методе <code>Calculate(size_t, size_t)</code> как флаг остановки дальнейших расчетов при обнаружении дефицита; позволяет остановить расчеты в других потоках при многопоточных вычислениях
<code>clsProgress_shell<type_progress>* pshell{nullptr}</code>	Тип указатель на класс <code>clsProgress_shell</code> . Адрес внешнего объекта-оболочки для индикатора прогресса. Тип самого индикатора определяется псевдонимом <code>type_progress</code> .	При создании объекта класса устанавливается значение <code>nullptr</code> : по умолчанию индикация прогресса не осуществляется

Методы класса `clsStorage` можно условно разделить на следующие группы:

- Конструкторы, деструктор, перегруженные операторы;
- Get-методы (методы в секции `Private` и методы в секции `Public`);
- Set-методы (методы в секции `Private` и методы в секции `Public`);
- Расчетные методы;
- Методы сохранения состояния объекта в файл и восстановления из файла;
- Методы визуального контроля

Ниже подробно описываются все методы.

КОНСТРУКТОРЫ, ДЕКТРУКТОР, ПЕРЕГРУЖЕННЫЕ ОПЕРАТОРЫ

clsStorage()

Конструктор по умолчанию (без параметров). Создает и инициализирует переменные: `PrCount` = 0; `hmcure` = `RUR`; `acct` = `FIFO`; создает на стеке пустой контейнер `stock`. Инициализирует флаг `Calculation Exit` значением *false*.

clsStorage(size_t _pcnt, Currency _cur, AccountingMethod _ac)

Конструктор с вводом длительности проекта, валюты и принципа учета запасов. Инициализирует флаг `Calculation Exit` значением *false*.

Параметры:

`_pcnt` – количество периодов проекта; устанавливает значение поля `PrCount`; тип *size_t*;

`_cur` – домашняя валюта проекта; устанавливает значение поля класса `hmcure`; тип `Currency`;

`_ac` – принцип учета запасов: `FIFO`, `LIFO` или `AVG`; устанавливает значение поля класса `acct`.

clsStorage(size_t _pcnt, Currency _cur, AccountingMethod _ac, size_t stocksize)

Конструктор с параметрами. Аналогичен предыдущему конструктору, дополнительно резервирует память контейнера `stock` для нескольких номенклатурных позиций¹¹. Инициализирует флаг `Calculation Exit` значением *false*.

Параметры:

`_pcnt` – количество периодов проекта; устанавливает значение поля класса `PrCount`;

`_cur` – домашняя валюта проекта; устанавливает значение поля класса `hmcure`;

`_ac` – принцип учета запасов: `FIFO`, `LIFO` или `AVG`; устанавливает значение поля класса `acct`;

`stocksize` – количество номенклатурных позиций (экземпляров класса `clsSKU`), планируемых к добавлению в контейнер. Тип *size_t*.

clsStorage(const size_t _PrCount, const Currency _cur, const AccountingMethod _ac, const size_t stocksize, const strNameMeas RMNames[]);

Конструктор с созданием индивидуальных складов с заданными параметрами.

Параметры:

`_PrCount` - количество периодов проекта. Тип *size_t*;

`_cur` - домашняя валюта проекта. Тип `Currency`;

`_ac` – принцип учета запасов: `FIFO`, `LIFO` или `AVG`;

`stocksize` – количество номенклатурных позиций (экземпляров класса `clsSKU`), планируемых к добавлению в контейнер. Тип *size_t*;

`RMNames` - указатель на массив с наименованиями ресурсов и ед.измерения; тип `strNameMeas`.

¹¹ Резервирование памяти (<https://cplusplus.com/reference/vector/vector/reserve/>) существенно снижает нагрузку на компьютер при последующем добавлении новых элементов в контейнер.

clsStorage(const clsStorage &obj)

Конструктор копирования. Инициализирует флаг [Calculation_Exit](#) значением *false*.

Параметры:

&obj – ссылка на копируемый константный экземпляр класса clsStorage.

clsStorage(clsStorage &&obj)

Конструктор перемещения. Инициализирует флаг [Calculation_Exit](#) значением *false*.

Параметры:

&&obj - перемещаемый экземпляр класса clsStorage.

clsStorage::swap(clsStorage &obj) noexcept

Функция обмена значениями между экземплярами класса.

Параметры:

&obj – ссылка на обмениваемый экземпляр класса clsStorage.

clsStorage::operator=(const clsStorage &obj)

Перегрузка оператора присваивания копированием.

Параметры:

&obj – ссылка на копируемый константный экземпляр класса clsStorage.

clsStorage::operator=(clsStorage &&obj)

Перегрузка оператора присваивания перемещением

Параметры:

&&obj - перемещаемый экземпляр класса clsStorage.

virtual ~clsStorage()

Деструктор. Виртуальный.

GET-МЕТОДЫ. СЕКЦИЯ PRIVATE**strItem* getresult(const strItem* (clsSKU::*f)() const) const**

Метод создает динамический массив типа [strItem](#) и возвращает на него указатель. Массив содержит информацию об *объемах, учетной удельной и полной себестоимости*. В зависимости от функции, указатель на которую передается в качестве параметра, информация в массиве относится либо к закупкам/ поставкам, либо к остаткам, либо к отгрузкам. Используется в методах класса clsStorage: [GetPure](#), [GetBal](#) и [GetShip](#).

Параметры:

const strItem* (clsSKU::*f)() const – указатель на один из методов класса [clsSKU](#): [GetPurchase](#) – функция возврата информации о закупках/ поставках, [GetBalance](#) – функция возврата информации об остатках на складе, [GetShipment](#) - функция возврата информации об отгрузках.

GET-МЕТОДЫ. СЕКЦИЯ PUBLIC

const size_t& ProjectCount() const

Возвращает const-ссылку на длительность проекта, выраженную в [количестве периодов](#). Тип возвращаемой ссылки *size_t*.

size_t Size() const

Возвращает размер контейнера [stock](#), равный количеству номенклатурных позиций (количеству SKU). Тип возвращаемой величины *size_t*.

const string& Name(size_t i) const

Возвращает const-ссылку на наименование номенклатурной позиции для элемента контейнера с индексом *i*. Тип возвращаемой величины – ссылка на тип *string*.

Параметры:

i – Индекс элемента в контейнере [stock](#), тип параметра *size_t*.

const string& Measure(size_t i) const

Возвращает const-ссылку на наименование единицы измерения номенклатурной позиции для элемента контейнера с индексом *i*. Тип возвращаемой величины – ссылка на тип *string*.

Параметры:

i – Индекс элемента в контейнере [stock](#), тип параметра *size_t*.

strNameMeas* GetNameMeas() const

Метод создает динамический массив типа [strNameMeas](#) и возвращает на него указатель. Массив содержит информацию о наименованиях номенклатурных позиций и наименованиях их единиц измерения для всех ресурсов, проходящих через склад.

string Permission(size_t i) const

Метод возвращает флаг разрешения/запрещения закупок и отгрузок в одном и том же периоде в виде текстовой строки для элемента контейнера с индексом *i*. Тип возвращаемой величины *string*.

Параметры:

i – Индекс элемента в контейнере [stock](#), тип параметра *size_t*.

bool PermissionBool(size_t i) const

Метод возвращает флаг разрешения/запрещения закупок и отгрузок в одном и том же периоде в виде логического значения для элемента контейнера с индексом *i*. Тип возвращаемой величины *bool*.

Параметры:

i – Индекс элемента в контейнере [stock](#), тип параметра *size_t*.

string AutoPurchase(size_t i) const

Метод возвращает признак автоматического/ ручного расчета закупок в виде текстовой строки для элемента контейнера с индексом *i*. Тип возвращаемой величины *string*.

Параметры:

i – Индекс элемента в контейнере [stock](#), тип параметра *size_t*.

const size_t GetAutoPurchase(size_t i) const

Метод возвращает признак автоматического/ ручного расчета закупок в виде целого числа для элемента контейнера с индексом *i*. Тип возвращаемой величины *size_t*.

string HomeCurrency() const

Метод возвращает основную валюту проекта в виде текстовой строки. Тип возвращаемой величины *string*.

const size_t GetHomeCurrency() const

Возвращает основную валюту проекта в виде значения поля *hmcure*; тип *Currency*.

string Accounting() const

Метод возвращает *принцип учета* в виде текстовой строки. Тип возвращаемой величины *string*.

const decimal& Share(size_t i) const

Метод возвращает *норматив запаса* ресурсов в долях от объема отгрузки для элемента контейнера с индексом *i*. Тип возвращаемой величины определяется псевдонимом *decimal*.

Параметры:

i – индекс элемента в контейнере *stock*, тип параметра *size_t*.

decimal* GetShipPrice()

Метод создает массив с учетной удельной себестоимостью отгружаемых ресурсов для всей номенклатуры и возвращает указатель на него. Массив является одномерным аналогом матрицы размером *stock.size()*PrCount*. Тип возвращаемого указателя определяется псевдонимом *decimal*.

Метод обеспечивает совместимость с форматом входных данных для экземпляра *класса производство*, когда под ресурсами подразумеваются сырье, материалы, компоненты для производства продукции.

strItem* GetShip() const

Метод создает массив с объемами, учетной удельной и полной себестоимостями для всей номенклатуры отгрузок и возвращает указатель на него. Массив является одномерным аналогом матрицы размером *stock.size()*PrCount*¹². Тип возвращаемого указателя *strItem*.

В коде метода используется вызов функции *getresult* с параметром *&clsSKU::GetShipment*.

strItem* GetPure() const

Метод создает массив с объемами, учетной удельной и полной себестоимостями для всей номенклатуры закупок/ поступлений и возвращает указатель на него. Массив является одномерным аналогом матрицы размером *stock.size()*PrCount*. Тип возвращаемого указателя *strItem*.

В коде метода используется вызов функции *getresult* с параметром *&clsSKU::GetPurchase*.

strItem* GetBal() const

Метод создает массив с объемами, учетной удельной и полной себестоимостями для всей номенклатуры остатков на складе и возвращает указатель на него. Массив является одномерным аналогом матрицы размером *stock.size()*PrCount*. Тип возвращаемого указателя *strItem*.

¹² Обращение к (i,j)-элементу такой «матрицы» с помощью арифметики указателей производится так: **(указатель_на_массив + количество_столбцов * i + j)*, где *i* - номер строки, *j* - номер столбца знак "*" перед открывающей скобкой – оператор разыменовывания.

В коде метода используется вызов функции [getResult](#) с параметром [&clsSKU::GetPurchase](#).

const strItem* GetDataItem(const ChoiseData _cd, size_t index, size_t _N) const

Метод возвращает данные для элемента контейнера (номенклатурной позиции, SKU) с индексом `index` и периодом `_N`. Данные возвращаются в виде `const`-указателя на структуру типа [strItem](#), содержащую информацию об объемах, ценах и стоимости закупки/ поступления, балансовых остатках или отгрузки; выбор массива с данными производится значением переменной `_cd`.

В коде метода используется вызов одноименной функции [clsSKU::GetDataItem](#), которой передаются параметры `_cd` и `_N`.

Параметры:

`_cd` – именованное целое число из перечисленных в типе [ChoiseData](#), указывающее на массив, из которого нужно выбрать данные: *“purchase”* – из массива закупок/ поступлений ([Pur](#)), *“balance”* – из массива остатков ([Bal](#)), *shipment* – из массива отгрузок ([Ship](#));

`Index` – индекс элемента в контейнере [stock](#), тип параметра `size_t`;

`_N` – номер периода, совпадающий с индексом элемента выбранного массива

SET-МЕТОДЫ. СЕКЦИЯ PRIVATE

```
template <typename T, typename U = T>
```

```
void _setdataitem(size_t i, const T val, void (clsSKU::*f)(const U val))
```

Для SKU с индексом `i` шаблонный метод устанавливает новые: наименование, единицу измерения, разрешение на отгрузку и закупку в одном и том же периоде, флаг автоматического/ ручного расчета закупок, норматив запаса сырья. Используется в методах класса [clsStorage](#): [SetName](#), [SetMeasure](#), [SetPermission](#), [SetAutoPurchase](#), [SetShare](#).

Параметры:

`i` – индекс элемента в контейнере [stock](#), тип параметра `size_t`;

`val` – новый устанавливаемый параметр; для наименования и единицы измерения параметр должен иметь тип `const string&`, для разрешения на отгрузку и закупку – тип `const bool`, для флага автоматического/ ручного расчета – тип `const PurchaseCalc`, для норматива остатков – тип `const decimal` (вещественное число);

`void (clsSKU::*f)(const auto val)` – указатель на один из методов класса [clsSKU](#): [SetName](#), [SetMeasure](#), [SetPermission](#), [SetAutoPurchase](#), [SetShare](#);

SET-МЕТОДЫ. СЕКЦИЯ PUBLIC

```
void Set_progress_shell(clsProgress_shell<type_progress>* val)
```

Метод присваивает указателю [pshell](#) адрес внешнего объекта-оболочки индикатора прогресса. Тип оболочки [clsProgress_shell](#), тип индикатора прогресса определяется псевдонимом [type_progress](#).

Параметры:

`*val` – указатель на внешний объект-оболочку для класса индикатора прогресса.

По окончании использования индикатора, необходимо вернуть указатель [pshell](#) в `nullptr` командой `Set_progress_shell(nullptr)`.

```
void Set_progress_message(string&& _message)
```

Метод устанавливает новое значение сообщения во время вывода индикатора. Вызывает метод [clsProgress_shell::SetMessage](#).

Параметры:

_message – ссылка на сообщение, являющееся началом индикатора прогресса; тип указатель на *string*.

void Set_progress_maxcount(const int _mx)

Метод устанавливает новое значение максимального числа итераций для индикатора прогресса.

Параметры:

_mx – новое значение максимального числа итераций; тип *int*.

void SetCount(const size_t _n)

Метод устанавливает новое количество периодов проекта – изменяет значение поля [clsStorage::PrCount](#). Изменение одноименных полей элементов контейнера [clsSKU::PrCount](#) происходит позднее, в теле другого метода [clsStorage::Calculate\(size_t, size_t\)](#). До вызова метода [Calculate\(...\)](#) значения поля [clsStorage::PrCount](#) и одноименных полей элементов контейнера могут быть разными.

Параметры:

_n – новое значение количества периодов (длительности) проекта. Тип *size_t*.

void SetName(size_t i, const string &_name)

Метод меняет наименование номенклатурной позиции (SKU) для элемента контейнера [stock](#) с индексом *i*. Для смены наименования метод вызывает функцию [clsStorage:: setdataitem](#).

Параметры:

i – индекс элемента в контейнере [stock](#), тип параметра *size_t*;

&_name – ссылка на новое наименование номенклатурной позиции.

void SetMeasure(size_t i, const string &_measure)

Метод меняет наименование единицы измерения номенклатурной единицы (SKU) для элемента контейнера [stock](#) с индексом *i*. Для смены единицы измерения метод вызывает функцию [clsStorage:: setdataitem](#).

Параметры:

i – индекс элемента в контейнере [stock](#), тип параметра *size_t*;

&_measure – ссылка на новое наименование единицы измерения номенклатурной позиции.

void SetCurrency(const Currency _cur)

Метод меняет основную валюту проекта – изменяет значение поля [clsStorage::hmcu](#).

Параметры:

_cur – новая валюта проекта. Тип [Currency](#).

void SetPermission(size_t i, const bool _indr)

Устанавливает флаг разрешения/ запрещения отгрузок и закупок в одном и том же периоде для элемента контейнера с индексом *i*. Для смены флага метод вызывает функцию [clsStorage:: setdataitem](#).

Параметры:

i – индекс элемента в контейнере [stock](#), тип параметра *size_t*;

`_indr` – новый флаг разрешения/ запрещения отгрузок в одном периоде: *true* - можно, *false* – нельзя.

void SetAutoPurchase(size_t i, const PurchaseCalc _pcalc)

Устанавливает флаг автоматического/ ручного расчета закупок/ поступлений на склад для элемента контейнера с индексом *i*. Для смены флага метод вызывает функцию [clsStorage::setdataitem](#).

Параметры:

i – индекс элемента в контейнере [stock](#), тип параметра *size_t*;

`_pcalc` – новый флаг автоматического/ ручного расчета закупок/ поступлений на склад: *calc* – автоматический расчет, *nocalc* – ручной.

void SetShare(size_t i, const decimal _share)

Устанавливает новый норматив запаса ресурсов на складе в долях от объема отгрузок для элемента контейнера с индексом *i*. Для изменения норматива запаса метод вызывает функцию [clsStorage:: setdataitem](#).

Параметры:

i – индекс элемента в контейнере [stock](#), тип параметра *size_t*;

`_share` – новое значение величины запаса, выраженное в долях от объема закупок. Тип вещественного числа определяется псевдонимом [decimal](#).

void SetAccounting(const AccountingMethod _acct)

Устанавливает новый принцип учета запасов для всех элементов контейнера – изменяет значение поля [clsStorage::acct](#).

Параметры:

`_acct` – новое значение принципа учета запасов: *FIFO*, *LIFO* или *AVG*.

bool SetShipment(const strItem _unit[])

Функция копирует данные о плановых отгрузках ресурсов со склада. Значимыми значениями являются только объемы в натуральном выражении (поля *volume*). В случае успешной загрузки, функция возвращает *true*.

Параметры:

`_unit` - указатель на одномерный массив, являющийся аналогом двумерной матрицы размером [stock.size\(\)](#)**PrCount*, содержащий объемы отгрузок. Тип элемента массива [strItem](#). Данные загружаются копированием.

bool SetShipment(strItem* &_unit)

Функция перемещает данные о плановых отгрузках ресурсов со склада. Значимыми значениями являются только объемы в натуральном выражении (поля *volume*). В случае успешной загрузки, функция возвращает *true*.

Параметры:

`_unit` - ссылка на указатель на одномерный массив, являющийся аналогом двумерной матрицы размером [stock.size\(\)](#)**PrCount*, содержащий объемы отгрузок. Тип элемента массива *strItem*. Данные загружаются перемещением. После завершения работы метода указатель `_unit` становится равным *nullptr*.

bool SetDataItem(const ChoiseData _cd, const strItem _U, size_t index, size_t _N)

Функция записывает в элемент контейнера с индексом `index` и период проекта с номером `_N` новые значения `volume`, `price` и `value` из переменной `_U`; выбор массива элемента контейнера производится значением переменной `_cd`. Метод работает только с непустым контейнером. В случае успешной записи, метод возвращает `true`.

Параметры:

`_cd` – признак редактируемого массива: “purchase” – массив `clsSKU::Pur`, “balance” – массив `clsSKU::Bal`, “shipment” – массив `clsSKU::Ship`;

`_U` – переменная с новыми данными, которые записываются в элемент массива; тип `strItem`;

`Index` – индекс элемента в контейнере `stock`, тип параметра `size_t`;

`_N` – номер периода проекта (индекс редактируемого элемента в выбранном массиве `clsSKU::Pur`, `clsSKU::Bal` или `clsSKU::Ship` в индивидуальном складе с индексом `Index`); тип `size_t`.

bool SetSKU(const string &Name, const string &Measure, PurchaseCalc _flag, decimal _share, bool _perm, strItem _ship[], strItem _pur[])

Метод создает моноресурсный склад для новой номенклатурной позиции и добавляет его в контейнер (создает новый элемент непосредственно в контейнере – экземпляр класса `clsSKU`). Метод работает как с пустым, так и не пустым контейнером `stock`. Метод проверяет наличие в контейнере номенклатурной позиции с тем же наименованием. Если таковой нет, то вызывается [конструктор с параметрами clsSKU и с вводом объемов отгрузок](#). В созданном элементе вызывается метод `clsSKU::SetPurchase` для ввода объемов закупок/поступлений на склад. Предварительно предполагается, что поля `PrCount` и `acct` заполнены корректно.

Параметры:

`&Name` – ссылка на наименование новой номенклатурной позиции; тип ссылка на `string`;

`&Measure` – ссылка на название единицы измерения новой номенклатурной позиции; тип ссылка на `string`;

`_flag` – флаг разрешения/ запрещения автоматического расчета закупок под требуемые объемы отгрузок; тип [PurchaseCalc](#);

`_share` – запас ресурсов на складе в каждый период, выраженный в доле от объема отгрузок за этот период; тип вещественного числа определяется псевдонимом [decimal](#);

`_perm` – флаг разрешения/ запрещения использовать закупаемые в каком-либо i-м периоде ресурсы для отгрузки в этом же периоде; тип `bool`;

`_ship[]` – указатель на массив с объемом отгрузок (в каждом элементе массива заполнены только поля `volume`, поля `price` и `value` заполнены нулями); тип указатель на массив [strItem](#);

`_pur[]` – указатель на массив с объемами закупок/поступлений на склад; тип указатель на массив [strItem](#).

bool SetPurchase(size_t isku, const strItem _unit[], size_t _count)

Метод загружает массив закупок `_unit` размером `_count` в индивидуальный склад с номером `isku` (для элемента контейнера с индексом `isku` вызывается метод `clsSKU::SetPurchase`, которому передаются параметры `_unit[]` и `_count`). Возвращает значение `true`, если операция прошла успешно, в противном случае возвращает `false`.

Параметры:

`Isku` – индекс элемента в контейнере `stock`, тип параметра `size_t`;

`_unit[]` указатель на массив с объемами закупок/поступлений на склад; тип указатель на массив [strItem](#);

`_count` – размер массива `_unit[]`; может не совпадать со значением из поля `PrCount`; тип параметра `size_t`.

bool SetPurchase(const strItem _unit[])

Метод загружает данные о поставках ресурсов на склад (объемы, цены, стоимость). Функция возвращает *true* при удачном завершении операции.

Параметры:

_unit – указатель на одномерный массив, являющийся аналогом двумерной матрицы размером *stock.size()*PrCount*, содержащий данные о поставках. Тип элемента массива *strItem*. Данные загружаются копированием.

bool SetPurchase(strItem* &_unit)

Метод загружает данные о поставках ресурсов на склад (объемы, цены, стоимость). Функция возвращает *true* при удачном завершении операции.

Параметры:

_unit - ссылка на указатель на одномерный массив, являющийся аналогом двумерной матрицы размером *stock.size()*PrCount*, содержащий данные о поставках. Тип элемента массива *strItem*. Данные загружаются перемещением. После завершения работы метода указатель *_unit* становится равным *nullptr*.

bool SetPurPrice(const strItem _unit[])

Функция загружает данные о ценах закупаемых/поставляемых на склад ресурсов (только поля "price"). Поля с данными об объемах ("volume") и стоимости ("value") игнорируются и могут содержать любые значения. Одномерный массив *_unit* является аналогом двумерной матрицы размером *stock.size()*PrCount*¹³. Для каждого элемента контейнера вызывается метод *clsSKU::SetPurPrice*, которому передается указатель на часть массива, имеющей отношение к этому элементу: (*_unit+PrCount*i*), где *i* – номер элемента контейнера и количество периодов *PrCount*. Возвращает значение *true*, если операция прошла успешно, в противном случае возвращает *false*.

Параметры:

_unit – указатель на массив загружаемых данных с ценами ресурсов типа *strItem*.

bool CheckPurchase(const strItem _pur[]) const

При флаге, запрещающем автоматический расчет закупок/ поступлений на склад *clsSKU::pcalc*, массив закупок должен содержать корректные цифры, т.е произведение *volume * price* должно быть равным *value*. Если это условие не соблюдается, расчетная себестоимость ресурса может считаться некорректно. Данный метод проверяет каждый элемент массива, переданного функции в качестве параметра. Возвращает значение *true*, если для каждого элемента массива соотношение между *volume*, *price* и *value* корректное, в противном случае возвращает *false*.

Параметры:

_pur[] – указатель на проверяемый массив типа *strItem*.

bool SetStorage(size_t RMCount, const strNameMeas RMNames[], decimal ShipPlan[], decimal PricePur[])

Метод создает склад сразу для нескольких номенклатурных позиций ресурсов. После создания "пустого" экземпляра класса *clsStorage* с помощью одного из двух [конструкторов с параметрами](#), данный метод создает в контейнере моноресурсные склады для отдельных ресурсов в количестве, равном величине *RMCount*,

¹³ Обращение к (i,j)-элементу такой «матрицы» с помощью арифметики указателей производится так: **(указатель_на_массив + количество_столбцов * i + j)*, где *i* - номер строки, *j* - номер столбца, знак "*" перед открывающей скобкой – оператор разыменовывания.

копирует названия этим позициям и единицы измерения из массива `RMNames`, копирует объемы отгрузок в полях `"volume"` массива `clsSKU::Ship` и цены закупаемых/получаемых ресурсов в полях `"price"` массива `clsSKU::Pur`.

Этот метод устанавливает для всех частных моноресурсных складов флаг *разрешающий* автоматический расчет закупок (`pcalc = calc`), *нулевой* запас ресурсов на складе в каждый период (`share = 0`) и флаг *разрешающий* закупки и отгрузки в одном и том же периоде (`indr = true`); в дальнейшем эти параметры можно скорректировать для каждого склада отдельно другими методами класса.

Возвращает значение *true*, если операция прошла успешно, в противном случае возвращает *false*.

Параметры:

`RMCount` – число номенклатурных позиций ресурсов; тип `size_t`; при `RMCount=0`, метод ничего не делает и возвращает *false*;

`RMNames` – указатель на массив с названиями ресурсов и названиями единиц их натурального измерения, размерностью `RMCount`; тип `strNameMeas`; при указатели равном `nullptr`, метод ничего не делает и возвращает *false*;

`ShipPlan` – указатель на массив отгрузок всех ресурсов в каждом периоде проекта; массив представляет собой одномерный аналог двумерной матрицы размером `RMCount*PrCount`; тип вещественного числа, определяемый псевдонимом `decimal`; при указатели равном `nullptr`, метод ничего не делает и возвращает *false*;

`PricePur` – указатель на массив цен всех закупаемых ресурсов в каждом периоде проекта; массив представляет собой одномерный аналог двумерной матрицы размером `RMCount*PrCount`; тип вещественного числа, определяемый псевдонимом `decimal`; при указатели равном `nullptr`, метод ничего не делает и возвращает *false*.

bool SetStorage(size_t RMCount, const strNameMeas RMNames[], strItem ShipPlan[], strItem Purchase[])

Метод, похожий на предыдущий, но массивы с отгрузками и ценами имеют тип `strItem`. Помимо цен на ресурсы, в массиве `Purchase` находятся данные об закупках в натуральном выражении. Возвращает значение *true*, если операция прошла успешно, в противном случае возвращает *false*.

Метод разрешает принятие в качестве указателя на массив закупок/поставок `PricePur` значение `nullptr`. В этом случае ввод закупок может быть осуществлен позже другими методами (`SetPurchase` – для ввода объемов, удельной и полной учетной себестоимости закупок в выбранный моноресурсный склад; `SetPurPrice` – для ввода цен/удельной учетной себестоимости сразу во все моноресурсные склады).

Параметры:

`RMCount` – число номенклатурных позиций ресурсов; тип `size_t`; при `RMCount=0`, метод ничего не делает и возвращает *false*;

`RMNames` – указатель на массив с названиями ресурсов и названиями единиц их натурального измерения, размерностью `RMCount`; тип `strNameMeas`; при указатели равном `nullptr`, метод ничего не делает и возвращает *false*;

`ShipPlan` – указатель на массив отгрузок всех ресурсов в каждом периоде проекта; массив представляет собой одномерный аналог двумерной матрицы размером `RMCount*PrCount`; тип `strItem`, используются только поля `"volume"`, другие поля не используются и должны быть заполнены нулями; при указатели равном `nullptr`, метод ничего не делает и возвращает *false*;

`Purchase` – указатель на массив объемов и цен всех закупаемых ресурсов в каждом периоде проекта; массив представляет собой одномерный аналог двумерной матрицы размером `RMCount*PrCount`; тип `strItem`, используются все поля. Метод допускает использование указателя `Purchase = nullptr`.

bool SetData(const clsSKU &obj)

Метод добавляет в контейнер новый моноресурсный склад путем *копирования* отдельно созданного экземпляра класса [clsSKU](#) в контейнер. Перед копированием в контейнер нового элемента проверяется наличие в контейнере ресурса с таким же именем. Если совпадение не найдено, и операция копирования прошла успешно, возвращает значение *true*, в противном случае возвращает *false*.

Параметры:

&obj – ссылка на копируемый моноресурсный склад; тип [clsSKU](#).

bool SetData(clsSKU &&obj)

Метод добавляет в контейнер новый моноресурсный склад путем *перемещения* отдельно созданного экземпляра класса [clsSKU](#) в контейнер. Перед перемещением в контейнер нового элемента проверяется наличие в контейнере ресурса с таким же именем. Если совпадение не найдено, и операция перемещения прошла успешно, возвращает значение *true*, в противном случае возвращает *false*.

Параметры:

&&obj – ссылка на перемещаемый моноресурсный склад; тип [clsSKU](#).

void EraseSKU(size_t _i)

Метод удаляет номенклатурную позицию под номером *_i* и все связанные с ней данные.

Параметры:

_i – индекс элемента в контейнере; тип *size_t*.

РАСЧЕТНЫЕ МЕТОДЫ. СЕКЦИЯ PRIVATE

TLack Calculate(size_t bg, size_t en)

Многоцелевая функция, выполняющая следующие действия для каждого моноресурсного склада типа [clsSKU](#), являющегося элементом контейнера с индексом от *bg* до (*en-1*) включительно:

- Проверяет флаг остановки расчетов [Calculation Exit](#). Если флаг установлен в *true*, завершает работу метода и возвращает нулевой дефицит.
- Вызывает у каждого элемента контейнера из заданного диапазона индексов метод [clsSKU::SetAccount](#) и устанавливает принцип учета запасов из поля [clsStorage::acct](#) контейнера в поле [clsSKU::acct](#) элемента.
- Проверяет количество периодов проекта [clsSKU::PrCount](#) у каждого элемента контейнера из заданного диапазона индексов: если оно отличается от количества, установленного в переменной [clsStorage::PrCount](#), то для этого элемента вызывается функция [clsSKU::Resize](#) с параметром [clsStorage::PrCount](#), которая приводит количество периодов у элемента контейнера к заданному.
- Поочередно вызывает у каждого элемента контейнера из заданного диапазона индексов метод [clsSKU::Calculate](#), рассчитывая для этого элемента необходимый объем закупок (поля "volume" у массива [clsSKU::Pur](#), если у элемента выставлен флаг [calc](#)), удельные и полные себестоимости отгружаемых партий (поля "price" и "value" массива [clsSKU::Ship](#)), а также дефицит ресурсов.
- После каждого вычисления элемента контейнера посылает сигнал счётчику операций экземпляра класса [clsProgress_shell](#) для обработки индикатором прогресса.

Функция завершает работу при нахождении первого дефицита и сообщает о размере этого дефицита и наименовании *SKU*, где найден дефицит. При обнаружении дефицита, флаг [Calculation Exit](#) устанавливается в *true*. При многопоточных вычислениях установленный в *true* флаг обеспечивает остановку вычислений в других потоках. Дальнейшие расчеты не производятся. Информация возвращается в виде структуры типа [TLack](#).

В случае успешного завершения всех расчетов, функция возвращает переменную типа [TLack](#), у которой поле `lack` равно нулю, а поле `Name` равно пустой строке.

Все расчеты над элементами контейнера производятся последовательно. Функция предназначена для работы в многопоточных вычислениях; в каждом потоке вычисляется свой диапазон элементов контейнера.

Параметры:

`bg` – индекс первого элемента из диапазона обрабатываемых элементов контейнера; тип `size_t`;

`en` – индекс элемента, следующего за последней позицией из диапазона обрабатываемых элементов контейнера; тип `size_t`.

TLack CalcPurchase(size_t bg, size_t en)

Многоцелевая функция, выполняющая для каждого моноресурсного склада типа [clsSKU](#), являющегося элементом контейнера с индексом от `bg` до `(en-1)` действия, аналогичные действиям функции [TLack Calculate\(size_t bg, size_t en\)](#), за исключением расчёта удельных и полных себестоимостей отгружаемых партий. Таким образом, данный метод применим только для расчёта необходимых объемов закупок у элементов контейнера.

Функция завершает работу при нахождении первого дефицита и сообщает о размере этого дефицита и наименовании `SKU`, где найден дефицит. При обнаружении дефицита, флаг `Calculation_Exit` устанавливается в `true`. При многопоточных вычислениях установленный в `true` флаг обеспечивает остановку вычислений в других потоках. Дальнейшие расчеты не производятся. Информация возвращается в виде структуры типа [TLack](#).

В случае успешного завершения всех расчетов, функция возвращает переменную типа [TLack](#), у которой поле `lack` равно нулю, а поле `Name` равно пустой строке.

Все расчеты над элементами контейнера производятся последовательно. Функция предназначена для работы в многопоточных вычислениях; в каждом потоке вычисляется свой диапазон элементов контейнера.

Данную функцию целесообразно использовать вместо функции [TLack Calculate\(size_t bg, size_t en\)](#) в последовательных цепочках объектов типа [clsStorage](#), [clsManufactory](#), когда между расчётом необходимых объемов закупок и расчётом стоимостных показателей производятся промежуточные вычисления у соседних звеньев такой цепочки (например, в цепочке объектов из примера [Model 1](#)). Это позволяет экономить на вычислениях, результаты которых на данный момент времени неактуальны.

Параметры:

`bg` – индекс первого элемента из диапазона обрабатываемых элементов контейнера; тип `size_t`;

`en` – индекс элемента, следующего за последней позицией из диапазона обрабатываемых элементов контейнера; тип `size_t`.

TLack CalcPrice(size_t bg, size_t en)

Многоцелевая функция, выполняющая для каждого моноресурсного склада типа [clsSKU](#), являющегося элементом контейнера с индексом от `bg` до `(en-1)` действия, аналогичные действиям функции [TLack Calculate\(size_t bg, size_t en\)](#), за исключением расчёта необходимых объемов закупок у элементов контейнера. Для применения данной функции расчёт объемов в натуральном выражении должен быть произведен заранее, перед ее вызовом. Таким образом, данный метод применим только для расчёта удельных и полных себестоимостей отгружаемых партий.

Функция завершает работу при нахождении первого дефицита и сообщает о размере этого дефицита и наименовании `SKU`, где найден дефицит. При обнаружении дефицита, флаг `Calculation_Exit` устанавливается в `true`. При многопоточных вычислениях установленный в `true` флаг обеспечивает остановку вычислений в других потоках. Дальнейшие расчеты не производятся. Информация возвращается в виде структуры типа [TLack](#).

В случае успешного завершения всех расчетов, функция возвращает переменную типа [TLack](#), у которой поле `lack` равно нулю, а поле `Name` равно пустой строке.

Все расчеты над элементами контейнера производятся последовательно. Функция предназначена для работы в многопоточных вычислениях; в каждом потоке вычисляется свой диапазон элементов контейнера.

Данную функцию целесообразно использовать вместо функции [TLack Calculate\(size_t bg, size_t en\)](#) в последовательных цепочках объектов типа [clsStorage](#), [clsManufactory](#), когда между расчётом необходимых объемов закупок и расчётом стоимостных показателей производятся промежуточные вычисления у соседних звеньев такой цепочки (например, в цепочке объектов из примера [Model 1](#)). Это позволяет экономить на вычислениях, результаты которых на данный момент времени неактуальны.

Параметры:

`bg` – индекс первого элемента из диапазона обрабатываемых элементов контейнера; тип [size_t](#);

`en` – индекс элемента, следующего за последней позицией из диапазона обрабатываемых элементов контейнера; тип [size_t](#).

РАСЧЕТНЫЕ МЕТОДЫ. СЕКЦИЯ PUBLIC

TLack Calculate()

Многоцелевой метод, который обрабатывает *все* элементы контейнера. Аналогичен методу [Calculate\(size_t bg, size_t en\)](#) с параметрами `bg = 0`, `en = (stock.size()-1)`. Метод рассчитывает объемы закупок в натуральном выражении и удельные и полные себестоимости отгружаемых партий у каждого элемента контейнера, возвращает структуру типа [TLack](#). Метод *не* предназначен для работы в многопоточных вычислениях: все расчеты над всеми элементами контейнера производятся последовательно в одном основном потоке.

Функция не взаимодействует со счётчиком операций экземпляра класса [clsProgress_shell](#), а вызывает непосредственно метод отрисовки индикатора прогресса этого класса.

TLack CalcPurchase()

Многоцелевой метод, который обрабатывает *все* элементы контейнера. Аналогичен методу [CalcPurchase\(size_t bg, size_t en\)](#) с параметрами `bg = 0`, `en = (stock.size()-1)`. Метод рассчитывает объемы закупок в натуральном выражении и возвращает структуру типа [TLack](#). Метод *не* предназначен для работы в многопоточных вычислениях: все расчеты над всеми элементами контейнера производятся последовательно в одном основном потоке.

Функция не взаимодействует со счётчиком операций экземпляра класса [clsProgress_shell](#), а вызывает непосредственно метод отрисовки индикатора прогресса этого класса.

TLack CalcPrice()

Многоцелевой метод, который обрабатывает *все* элементы контейнера. Аналогичен методу [CalcPrice\(size_t bg, size_t en\)](#) с параметрами `bg = 0`, `en = (stock.size()-1)`. Метод рассчитывает удельные и полные себестоимости отгружаемых партий у каждого элемента контейнера и возвращает структуру типа [TLack](#). Метод *не* предназначен для работы в многопоточных вычислениях: все расчеты над всеми элементами контейнера производятся последовательно в одном основном потоке.

Функция не взаимодействует со счётчиком операций экземпляра класса [clsProgress_shell](#), а вызывает непосредственно метод отрисовки индикатора прогресса этого класса.

TLack Calculate(size_t icalc)

Функция-интерфейс для вызова одного из возможных расчетных методов: [Calculate\(\)](#), [CalcPurchase\(\)](#) или [CalcPrice\(\)](#).

Параметры:

Icalc – признак выбранного для вычисления метода: *Icalc = 0* для вызова *Calculate()*, *Icalc = 1* для вызова *CalcPurchase()*, *Icalc = 2* (или любое другое значение, кроме 0 и 1) для вызова *CalcPrice()*. Тип *size_t*.

TLack Calculate_future()

Метод является альтернативой методу *Calculate*. В методе *Calculate_future* расчеты производятся в параллельных потоках, число которых на единицу меньше числа ядер используемого компьютера, и число обрабатываемых элементов контейнера в каждом потоке примерно одинаково. В каждом потоке вызывается метод *Calculate(size_t bg, size_t en)* с границами диапазона, предназначенного для обработки в этом потоке. В каждом отдельном потоке расчеты производятся последовательно, но сами потоки выполняются параллельно. Это позволяет существенно ускорить вычисления. Для организации потоков используются инструменты из стандартного заголовочного файла *<future>*. При количестве ядер менее трех, потоки не создаются, а вычисления производятся последовательно с помощью функции *Calculate()*.

При выявлении дефицита у какого-либо моноресурсного склада в процессе расчетов, поток, где найден дефицит, устанавливает флаг *Calculation_Exit* в состояние *true* и завершается. Это дает сигнал другим потокам также завершить работу. Все результаты расчетов записываются во временное хранилище. По завершению работы всех потоков, функция проверяет временное хранилище и ищет дефицит. Функция завершает работу при нахождении первого дефицита и сообщает о размере этого дефицита и наименовании SKU, где найден дефицит.

TLack Calculate_future(size_t icalc)

Аналогичен предыдущему методу. Параметр *icalc* определяет функцию, которая будет выполняться в каждом потоке с границами диапазона, предназначенного для обработки в этом потоке. При *Icalc = 0* будет выполняться *Calculate(size_t bg, size_t en)*, при *Icalc = 1* будет выполняться *CalcPurchase(size_t bg, size_t en)*, при *Icalc = 2* (или любом другом значении, кроме 0 и 1) будет выполняться *CalcPrice(size_t bg, size_t en)*.

Соответственно, для каждого элемента контейнера будут рассчитаны:

- объемы закупок в натуральном выражении и удельные и полные себестоимости отгружаемых партий
- или только натуральные,
- или только стоимостные показатели.

При количестве ядер менее трех, потоки не создаются, а вычисления производятся последовательно с помощью функции *Calculate()*, *CalcPurchase()* или *CalcPrice()* в зависимости от значения параметра *icalc*.

Параметры:

Icalc – признак выбранного для вычисления метода; тип *size_t*.

TLack Calculate_thread()

Этот метод идентичен по назначению и своей структуре методу *Calculate_future()*. Отличие в том, что в нем используются инструменты из стандартного заголовочного файла *<thread>*.

TLack Calculate_thread(size_t)

Этот метод идентичен по назначению и своей структуре методу *Calculate_thread()*. Отличие в том, что в нем используются инструменты из стандартного заголовочного файла *<thread>*.

Выбор метода для проведения расчетов (*Calculate*, *Calculate_future* или *Calculate_thread*) остается за пользователем. Для совершения выбора необходимо принять во внимание широту номенклатуры ресурсов (количество элементов контейнера), длительность (количество периодов) проекта и тип используемого вещественного числа. Создание потоков – ресурсоемкий процесс, и на коротких проектах с малым числом номенклатурных позиций использование последовательных вычислений без создания потоков (метод *Calculate*) может оказаться быстрее. Использование вещественных чисел типа *LongReal* вместо встроенного типа *double* для получения результатов повышенной точности, потребует больше компьютерных ресурсов, и выигрыш от использования параллельных вычислений может стать заметнее.

В таблице ниже (Таблица 10) приведено среднее время вычислений для разных расчетных методов. Для каждого из трех расчетных методов было сделано 150 вычислений: по 50 вычислений с каждым принципом учета запасов. В качестве вещественного числа использовался собственный тип [LongReal](#) с количеством десятичных знаков мантиисы 64. Было установлено 480 периодов проекта. Расчет велся по 100 номенклатурным позициям. Вычисления проводились на ноутбуке ASUS UX303U со следующими характеристиками: Процессор Intel Core i3-6100U CPU @ 2.30 GHz; Оперативная память 4,00 Gb; 64-bit Operation System Windows 10 Home Single Language, x64-based processor. Расчеты проводились в трех параллельных потоках. Исходный код программы для тестирования производительности приведен в [репозитории](#) в папке [froma2/examples/Storage_stress_test](#); файл *main.cpp*.

Таблица 10 Сравнение времени вычислений для разных методов класса `clsStorage`

Названия строк	Названия столбцов		FIFO		LIFO	
	AVG	Стандартное отклонение	Стандартное отклонение	Стандартное отклонение	Стандартное отклонение	Стандартное отклонение
	Среднее время выполнения, сек.	времени выполнения, сек.	Среднее время выполнения, сек.	времени выполнения, сек.	Среднее время выполнения, сек.	времени выполнения, сек.
Calculate	31,0307	0,4646	5,1513	0,0834	1,3561	0,0186
Calculate_future	15,9433	0,5221	2,8892	0,0568	0,7301	0,0152
Calculate_thread	15,8462	0,1773	2,9130	0,0929	0,7279	0,0146

Каждая финансово-экономическая модель строится, как правило, для многократного использования: заказчику модели часто нужны ответы на вопрос «что, если...». Это означает подстановку новых данных и расчет новых результатов. При этом обычно широта номенклатуры ресурсов во всех интересующих заказчика сценариях более-менее одинакова и длительность проекта более-менее укладывается в похожие рамки. Поэтому при создании модели можно попробовать применение всех трех расчетных методов и выбрать наиболее подходящий.

МЕТОДЫ СОХРАНЕНИЯ СОСТОЯНИЯ ОБЪЕКТА В ФАЙЛ И ВОССТАНОВЛЕНИЯ ИЗ ФАЙЛА

`bool StF(ofstream &_outF)`

Метод имплементации записи в файловую переменную текущего экземпляра класса (запись в файл, метод сериализации).

Параметры:

`&_outF` - экземпляр класса [ofstream](#) для записи данных. При инициализации переменной `_outF` необходимо установить флаг бинарного режима открытия потока (не текстового!) [ios::binary](#).

`bool RfF(ifstream &_inF)`

Метод имплементации чтения из файловой переменной текущего экземпляра класса (метод десериализации).

Параметры:

`&_inF` - экземпляр класса [ifstream](#) для чтения данных. При инициализации переменной `_inF` необходимо установить флаг бинарного режима открытия потока (не текстового!) [ios::binary](#).

МЕТОДЫ ВИЗУАЛЬНОГО КОНТРОЛЯ

`void ViewSettings() const`

Функция выводит на экран информацию о настройках для расчета. К настройкам относятся

- общие параметры проекта: длительность проекта, домашняя (основная) валюта, метод учета запасов

- и параметры, относящиеся к конкретному моноресурсному складу: разрешение/запрет на закупки и отгрузки в одном периоде, признак автоматического/ ручного расчета закупок, норматив запаса ресурсов.

void View() const

Функция выводит на экран информацию по всему складу, путем обращения к методу [clsSKU::View\(\)](#) каждого элемента контейнера в цикле.

void ViewChoise(ChoiseData _arr, ReportData flg) const

В зависимости от выбранного признака *_arr*, метод выводит на экран закупки/поступления, балансовые остатки или отгрузки ресурсов; в зависимости от выбранного флага *flg*, вывод этих данных осуществляется в натуральном, удельном стоимостном или полном стоимостном выражении.

Параметры:

_arr – параметр для выбора данных: “purchase” – закупки/ поступления, “balance” – остатки ресурсов на складе, “shipment” – отгрузки ресурсов со склада; тип [ChoiseData](#);

flg – флаг для выбора типа представляемых данных: “volume” - в натуральном, “value” - в полном стоимостном, “price” - в удельном стоимостном измерении; тип [ReportData](#).

КАК ПОЛЬЗОВАТЬСЯ КЛАССОМ *clsStorage*

Класс *clsStorage* – основной класс, с которым придется иметь дело пользователю, моделирующему учетные процессы на складе. Можно рекомендовать следующую последовательность действий:

1. Создаем новый экземпляр класса *clsStorage* в динамической памяти с помощью [конструктора с параметрами](#).
2. Создание элементов контейнера (наполнение контейнера экземплярами класса [clsSKU](#)) можно осуществлять несколькими путями:
 - a. Можно создать *сразу все элементы* контейнера с помощью одного из двух методов [SetStorage](#) с вводом отгрузок и цен в виде массивов вещественных чисел, тип которых определяется псевдонимом [decimal](#) или с вводом отгрузок и цен в виде массивов типа [strItem](#);
 - b. Можно *последовательно* создавать отдельные элементы сразу *внутри* контейнера с помощью метода [SetSKU](#);
 - c. Можно последовательно создавать отдельные элементы контейнера путем копирования или перемещения *ранее созданных* объектов типа *clsSKU* в контейнер с помощью методов копирования или перемещения [SetData](#).
3. При вводе некорректных данных, их изменение в любом из внутренних массивов класса в любом периоде проекта можно осуществить с помощью метода [SetDataItem](#).
4. Вызываем метод [Calculate](#) ([Calculate future](#) или [Calculate thread](#)). Этот метод вызывает у каждого элемента свой метод [clsSKU::Calculate](#), который рассчитывает для данного моноресурсного склада *потребность в ресурсах* (при наличии у него флага [clsSKU::calc](#)), *учетную удельную и полную себестоимости* отгружаемой партии в соответствии с выбранным принципом учета запасов и сообщает о [дефиците](#), если таковой имеется и имени номенклатурной позиции, где он обнаружен.
5. «Достаем» из экземпляра класса массивы с данными, содержащими в т.ч. и искомые расчетные величины: массив с *учетной удельной себестоимостью* ресурсов для всей номенклатуры с помощью метода [GetShipPrice](#) (тип массива определяется псевдонимом [decimal](#)), массив с *отгрузками* в натуральном, удельном и полном стоимостном выражении с помощью метода [GetShip](#), массив с *остатками* в натуральном, удельном и полном стоимостном выражении с помощью метода [GetBal](#), массив с *закупками/ поступлениями* на склад в натуральном, удельном и полном стоимостном выражении с помощью метода [GetPure](#) (типы массивов, указатели на которые

- возвращают три последние функции, [strItem](#)). Все массивы представляют собой одномерные аналоги двумерной матрицы размером [stock.size\(\)*PrCount](#).
6. Если нет необходимости в получении массивов и достаточно получить данные из конкретного массива в каком-либо заданном периоде проекта, можно воспользоваться функцией [GetDataItem](#).
 7. Для отображения данных в стандартный поток [std::cout](#), можно воспользоваться следующими методами:
 - а. методом [ViewSettings](#) – для вывода информации о длительности проекта, домашней валюте, методе учета запасов, о разрешении/запрете на закупки и отгрузки в одном периоде, о признаке автоматического/ ручного расчета закупок, о нормативе запаса ресурсов;
 - б. одним из двух методов визуального контроля: [View](#) или [ViewChoise](#) – для вывода информации о закупках/ поступлениях, остатках на складе и об отгрузках.
 8. После того, как созданный в п.1 экземпляр класса `clsStorage` не нужен, его удаляем.
 9. На любом этапе между п.п. 1 и 8 состояние объекта класса `clsSKU` можно сохранить в файл с последующим восстановлением из этого файла с помощью [методов сериализации и десериализации](#).

В финансово-экономических моделях класс `clsStorage` может присутствовать в виде двух экземпляров разного назначения: один, как *Склад готовой продукции*, другой – как *Склад сырья и материалов*. Между ними может находиться объект другого класса `clsManufactory`, описывающий *Производство*. В такой конструкции, например, для *Склада готовой продукции*, может потребоваться *двухэтапная* работа:

1. На первом этапе в созданный экземпляр класса `clsStorage` вводятся данные о *плановых отгрузках* клиентам в *натуральном выражении* и *норматив запаса* готовых продуктов на складе. Ввод может осуществляться, например, методом [SetStorage\(size t, const strNameMeas*, strItem*, strItem*\)](#). Далее производится расчет *плановых поступлений* в *натуральном выражении* на склад одним из трех вычислительных методов ([Calculate](#), [Calculate future](#) или [Calculate thread](#)). Рассчитанные таким образом данные вводятся в качестве уже исходных данных в другие объекты с помощью их собственных методов ввода (например, в объект «Производство») для вычисления в этих объектах удельной учетной себестоимости (например, производственной себестоимости).
2. На втором этапе данные с *удельной учетной себестоимостью* (например, производственной себестоимостью) *плановых поступлений*, полученные от соседнего объекта, загружаются в склад готовой продукции методами [SetPurPrice](#) или [SetPurchase](#). Далее производится расчет *удельной и полной учетной себестоимости отгружаемой* клиентам продукции одним из трех вычислительных методов ([Calculate](#), [Calculate future](#) или [Calculate thread](#)).

Таким образом, класс `clsStorage` позволяет осуществлять *двухэтапный ввод данных* и *двухэтапные расчеты* для совместимости с другими классами библиотеки FROMA2.

ПРИМЕР ПРОГРАММЫ С КЛАССОМ `clsStorage`

Ниже представлен код программы, демонстрирующей работу класса `clsStorage`.

```

#include <iostream>
#include <warehouse_module.h>           // Подключаем модуль склада

using namespace std;

int main()
{
    setlocale(LC_ALL, "Russian");       // Установка русского языка для вывода
    cout << endl;
    cout << "*****" << endl;
    cout << "*** Тестирование расчетных методов класса clsStorage ***" << endl;
    cout << "*****" << endl << endl;

    /** Общие данные проекта **/

    size_t PrCount = 17;                // Количество периодов проекта
    Currency Cur = RUR;                  // Домашняя валюта проекта
    AccountingMethod Amethod = AVG;     // Принцип учета запасов
    size_t RMCount = 10;                 // Количество номенклатурных позиций
    bool PurchPerm = true;               // Разрешение закупок и отгрузок в одном периоде
    PurchaseCalc flag = calc;           // Разрешение на автоматический расчет закупок

    double Share = 0.5;                 // Запас сырья в каждый период, выраженный
                                        // в доле от объема отгрузок за этот период
    TLack tlack;                        // Дефицит сырья и материалов

    /** Формирование массива имен сырья и материалов с единицами измерения **/

    strNameMeas* RMNames = new strNameMeas[RMCount];
    for(size_t i{}; i<RMCount; i++) {
        (RMNames+i)->name = "Rawmat_" + to_string(i);
        (RMNames+i)->measure = "kg";
    };

    /** Формирование тестового массива отгрузок **/

    strItem* ship = new strItem[PrCount];
    for(size_t i=sZero; i<PrCount; i++) {
        (ship+i)->volume = 40.0;
        (ship+i)->value = (ship+i)->price = 0.0;
    };

    /** Формирование тестового массива закупок (только закупочных цен) **/

    strItem* purc = new strItem[PrCount];
    for(size_t i=sZero; i<PrCount; i++) {
        (purc+i)->volume = (purc+i)->value = 0.0;
        (purc+i)->price = (5.0+i);
    };

    /** Создаем склад для нескольких продуктов **/

    // Создаем контейнер для склада
    clsStorage* Warehouse = new clsStorage(PrCount, Cur, Amethod, RMCount);
    for(size_t i=sZero; i<RMCount; i++) { // Добавляем склад для i-го продукта
        if(!Warehouse->SetSKU((RMNames+i)->name, (RMNames+i)->measure, flag, \
            Share, PurchPerm, ship, purc)) {
            cout << "Ошибка добавления склада для " << (RMNames+i)->name << endl;
            delete[] RMNames;           // Освобождение динамической памяти
            delete[] ship;
            delete[] purc;
            delete Warehouse;
            return EXIT_FAILURE;        // выходим из программы с кодом неудачного завершения
        };
    };
};

```

```

    /** Рассчитываем проект */

    int stop;
    cout << "Приостановка перед работой алгоритма. Для продолжения нажмите ENTER";
    stop = getchar(); // Приостановка до ввода любого символа
    clock_t start = clock(); // Начальное количество тиков процессора, которое сделала
программа
    // Расчет проекта и расчет дефицита.
    // tlack = Warehouse->Calculate(); // Последовательные вычисления
    // tlack = Warehouse->Calculate_future(); // Параллельные вычисления
    tlack = Warehouse->Calculate_thread(); // Параллельные вычисления
    if(tlack.lack>dZero)
        cout << "Дефицит для " << tlack.Name << " равен " << tlack.lack << endl;
    clock_t end = clock(); // Конечное количество тиков процессора, которое сделала
программа
    double seconds = (double)(end - start) / CLOCKS_PER_SEC; // Подсчет секунд
выполнения алгоритма
    printf("Время вычислений: %f секунд\n", seconds); // Вывод времени работы
процессора на экран
    cout << "Приостановка после работы алгоритма. Для продолжения нажмите ENTER";
    stop = getchar();

    /** Методы визуального контроля */
    // Отображение удельной себестоимости отгружаемых ресурсов
    Warehouse->ViewChoise(shipment, price);

    /** Освобождение динамической памяти */

    delete[] RMNames;
    delete[] ship;
    delete[] purc;
    delete Warehouse;

    return EXIT_SUCCESS;
}

```

В методе *Warehouse->ViewChoise(shipment, price)* мы выбрали вывод на экран информации об удельной учетной себестоимости (*price*) отгружаемых (*shipment*) ресурсов.

By price										
Name	Measure	0	1	2	3	4	5	6	7	8
Rawmat_0	RUR/kg	0.00	6.00	6.67	7.56	8.52	9.51	10.50	11.50	12.50
Rawmat_1	RUR/kg	0.00	6.00	6.67	7.56	8.52	9.51	10.50	11.50	12.50
Rawmat_2	RUR/kg	0.00	6.00	6.67	7.56	8.52	9.51	10.50	11.50	12.50
Rawmat_3	RUR/kg	0.00	6.00	6.67	7.56	8.52	9.51	10.50	11.50	12.50
Rawmat_4	RUR/kg	0.00	6.00	6.67	7.56	8.52	9.51	10.50	11.50	12.50
Rawmat_5	RUR/kg	0.00	6.00	6.67	7.56	8.52	9.51	10.50	11.50	12.50
Rawmat_6	RUR/kg	0.00	6.00	6.67	7.56	8.52	9.51	10.50	11.50	12.50
Rawmat_7	RUR/kg	0.00	6.00	6.67	7.56	8.52	9.51	10.50	11.50	12.50
Rawmat_8	RUR/kg	0.00	6.00	6.67	7.56	8.52	9.51	10.50	11.50	12.50
Rawmat_9	RUR/kg	0.00	6.00	6.67	7.56	8.52	9.51	10.50	11.50	12.50

By price										
Name	Measure	9	10	11	12	13	14	15	16	
Rawmat_0	RUR/kg	13.50	14.50	15.50	16.50	17.50	18.50	19.50	20.50	
Rawmat_1	RUR/kg	13.50	14.50	15.50	16.50	17.50	18.50	19.50	20.50	
Rawmat_2	RUR/kg	13.50	14.50	15.50	16.50	17.50	18.50	19.50	20.50	
Rawmat_3	RUR/kg	13.50	14.50	15.50	16.50	17.50	18.50	19.50	20.50	
Rawmat_4	RUR/kg	13.50	14.50	15.50	16.50	17.50	18.50	19.50	20.50	
Rawmat_5	RUR/kg	13.50	14.50	15.50	16.50	17.50	18.50	19.50	20.50	
Rawmat_6	RUR/kg	13.50	14.50	15.50	16.50	17.50	18.50	19.50	20.50	
Rawmat_7	RUR/kg	13.50	14.50	15.50	16.50	17.50	18.50	19.50	20.50	
Rawmat_8	RUR/kg	13.50	14.50	15.50	16.50	17.50	18.50	19.50	20.50	
Rawmat_9	RUR/kg	13.50	14.50	15.50	16.50	17.50	18.50	19.50	20.50	

Исходный код программы приведен в [репозитории](#) в папке [froma2/examples/Storage_example](#); файл *main.cpp*. Содержание файла может незначительно отличаться от приведенного выше.

ПРОИЗВОДСТВО

Основные типы, константы, классы и методы, реализующие алгоритмы для производства, находятся в модуле **MANUFACT**. Модуль *Manufact* представляет собой набор инструментов для моделирования *центра учета затрат* (*Cost Centre*). С его помощью можно аллоцировать затраты на единицу ресурсов/ продукции, услуг и получить *добавленную стоимость*. Инструменты модуля можно применять как для моделирования производства, так и для моделирования любых других подразделений, где создается добавленная стоимость, например на складе, где труд складского персонала увеличивает стоимость хранимых ресурсов¹⁴. Состав модуля приведен в таблице ниже.

Таблица 11 Состав модуля *Manufact*

Состав модуля	Описание	Заметка
void Sum	Вспомогательная функция	Метод суммирует поэлементно два массива, результат сохраняет в первом массиве
void Dif	Вспомогательная функция	Метод вычитает поэлементно два массива, результат сохраняет в первом массиве
decimal *Mult	Вспомогательная функция	Метод возвращает результат умножения элементов двух массивов с одинаковыми индексами друг на друга. Результат возвращается в виде указателя на новый массив.
class clsRecipeItem	Класс технологической карты	Экземпляр класса представляет собой технологическую карту производства одной номенклатурной позиции готовой продукции; содержит методы, позволяющие моделировать учетные процессы в производстве этого единственного продукта
class clsManufactItem	Класс Производство для одной номенклатурной позиции (монопроизводство)	Экземпляр класса представляет собой контейнер для класса технологической карты (объекта типа <i>clsRecipeItem</i>). Содержит исходные, расчетные данные и методы управления объектом типа <i>clsRecipeItem</i>
class clsManufactory	Класс Производство для множества номенклатурных позиций	Экземпляр класса представляет собой контейнер для нескольких объектов типа <i>clsManufactItem</i> и позволяет моделировать учетные процессы в производстве множества номенклатурных единиц

Помимо указанных в таблице членов, в заголовочном файле *manufact_module.h* содержится ряд строковых констант, используемых при выводе информации на экран/файл.

ВСПОМОГАТЕЛЬНЫЕ МЕТОДЫ

inline void Sum(decimal arr1[], const decimal arr2[], size_t N);

Метод суммирует поэлементно два массива (складываются элементы с одинаковыми индексами), результат сохраняет в первом массиве, замещая его первоначальные элементы. Оба массива должны иметь одинаковый размер N. Метод *inline* для встраивания его кода в точку вызова. Метод не проверяет совпадение размерности массивов, не проверяет существование массивов (не nullptr), это ответственность пользователя.

Параметры:

arr1 – указатель на первый массив; тип вещественного числа определяется псевдонимом *decimal*;

arr2 – константный указатель на второй массив; тип вещественного числа определяется псевдонимом *decimal*;

N - размерность обоих массивов; стандартный тип *size_t*.

¹⁴ Моделирование склада совместным использованием классов *clsStorage* и *clsManufactory* описано в разделе «СОЗДАНИЕ МОДЕЛИ С НЕСКОЛЬКИМИ ЦЕНТРАМИ ЗАТРАТ».

inline void Dif(decimal arr1[], const decimal arr2[], size_t N)

Метод вычитает поэлементно два массива (разность между элементом первого массива и элементом второго массива с одинаковыми индексами), результат сохраняет в первом массиве, замещая его первоначальные элементы. Оба массива должны иметь одинаковый размер N. Метод *inline* для встраивания его кода в точку вызова. Метод не проверяет совпадение размерности массивов, не проверяет существование массивов (не nullptr), это ответственность пользователя.

Параметры:

arr1 – указатель на первый массив; тип вещественного числа определяется псевдонимом *decimal*;

arr2 – константный указатель на второй массив; тип вещественного числа определяется псевдонимом *decimal*;

N - размерность обоих массивов; стандартный тип *size_t*.

inline decimal *Mult(const decimal arr1[], const decimal arr2[], const size_t N)

Метод возвращает результат умножения элементов двух массивов с одинаковыми индексами друг на друга. Результат возвращается в виде указателя на новый массив. Тип вещественного числа возвращаемого массива определяется псевдонимом *decimal*. Оба массива должны иметь одинаковый размер N. Метод *inline* для встраивания его кода в точку вызова. Метод не проверяет совпадение размерности массивов, не проверяет существование массивов (не nullptr), это ответственность пользователя. Если происходит ошибка выделения памяти новому массиву, метод возвращает *nullptr*.

Параметры:

arr1 – константный указатель на первый массив; тип вещественного числа определяется псевдонимом *decimal*;

arr2 – константный указатель на второй массив; тип вещественного числа определяется псевдонимом *decimal*;

N - размерность обоих массивов; стандартный тип *size_t*.

КЛАСС clsRecipeItem

Экземпляр класса *clsRecipeItem* представляет собой *рецептуру/ технологическую карту* производства одной номенклатурной позиции готовой продукции или услуги; содержит методы, позволяющие *моделировать учетные процессы в однократном* производстве этого *единственного* продукта.

Основные *задачи*, которые можно решить применением класса *clsRecipeItem*:

1. Сохранить и, при необходимости, вернуть рецептуру / технологическую карту продукта;
2. Рассчитать потребность в разных ресурсах и график обеспечения ими для *однократного* производства одного какого-то конкретного продукта;
3. Рассчитать объем, полную и удельную ресурсную себестоимость незавершенного производства и готового продукта одного конкретного наименования.

ПОЛЯ КЛАССА clsRecipeItem

Таблица 12 Поля класса *clsRecipeItem*. Секция *private*

Поле класса	Описание	Заметка
string name	Тип <i>string</i> . Название продукта	
string measure	Тип <i>string</i> . Название единицы измерения натурального объема продукта	

size_t duration	Тип <i>size_t</i> . Длительность производственного цикла	
size_t rcount	Тип <i>size_t</i> . Количество позиций в номенклатуре ресурсов, требуемых для производства продукта	
strNameMeas *rnames	Тип <i>strNameMeas</i> . Указатель на одномерный динамический массив с названиями ресурсов и единицами их измерения. Размерность <i>rcount</i>	
decimal *recipeitem	Тип <i>decimal</i> . Указатель на одномерный динамический массив с рецептурой/технологической картой продукта.	Массив представляет собой одномерный аналог двумерной матрицы размером <i>rcount*duration</i> . Обращение к <i>i,j</i> -элементу матрицы производится так: <i>*(recipeitem + duration * i + j)</i> , где <i>i</i> - номер строки, <i>j</i> - номер столбца. Условная (<i>i,j</i>)-ячейка матрицы содержит расход <i>i</i> -го ресурса на производство единицы продукта в <i>j</i> -м периоде от начала его производственного цикла

Методы класса *clsRecipeItem* можно условно разделить на следующие группы:

- Конструкторы, деструктор, перегруженные операторы;
- Алгоритмы учета (методы в секции Private и методы в секции Public);
- Get-методы;
- Методы сохранения состояния объекта в файл и восстановления из файла;
- Методы визуального контроля

Ниже подробно описываются все методы.

КОНСТРУКТОРЫ, ДЕКТРУКТОР, ПЕРЕГРУЖЕННЫЕ ОПЕРАТОРЫ

clsRecipeItem()

Конструктор по умолчанию (без параметров). Создает и инициализирует переменные *name* = "", *measure* = "", *duration* = 0, *rcount* = 0. Динамические массивы *rnames* и *recipeitem* не создаются, указатели на них устанавливаются в *nullptr*.

clsRecipeItem(const string &_name, const string &_measure, const size_t _duration, const size_t _rcount, const strNameMeas _rnames[], const decimal _recipeitem[]);

Конструктор с вводом названия продукта, единицы его натурального измерения, длительности производственного цикла, массивов с названиями ресурсов, участвующих в технологическом цикле, рецептуры/ технологической карты продукта.

Параметры:

&_name – ссылка на название продукта; устанавливает значение поля класса *name*; тип *string*;

&_measure – ссылка на название единицы измерения натурального объема продукта; устанавливает значение поля класса *measure*; тип *string*;

_duration – длительность производственного цикла; устанавливает значение поля класса *duration*; тип *size_t*.

_rcount – количество позиций ресурсов, требуемых для производства продукта; устанавливает значение поля класса *rcount*; тип *size_t*.

_rnames[] - указатель на массив с наименованиями ресурсов и единицами их измерения размером *_rcount*; устанавливает значение поля класса *rnames*; тип элемента массива *strNameMeas*;

`_recipeitem[]` – указатель на массив с рецептурами (одномерный аналог матрицы размерностью `_rcount*
_duration`); устанавливает значение поля класса `recipeitem`; вещественный тип элемента массива определяется псевдонимом `decimal`.

clsRecipeItem(const clsRecipeItem &obj)

Конструктор копирования.

Параметры:

`&obj` – ссылка на копируемый константный экземпляр класса `clsRecipeItem`.

clsRecipeItem(clsRecipeItem &&obj)

Конструктор перемещения.

Параметры:

`&&obj` - перемещаемый экземпляр класса `clsRecipeItem`.

clsRecipeItem &operator= (const clsRecipeItem &obj)

Перегрузка оператора присваивания копированием.

Параметры:

`&obj` – ссылка на копируемый константный экземпляр класса `clsRecipeItem`.

clsRecipeItem &operator= (clsRecipeItem &&obj)

Перегрузка оператора присваивания перемещением

Параметры:

`&&obj` - перемещаемый экземпляр класса `clsRecipeItem`.

~clsRecipeItem()

Деструктор.

void swap(clsRecipeItem& obj) noexcept

Функция обмена значениями между экземплярами класса.

Параметры:

`&obj` – ссылка на обмениваемый экземпляр класса `clsRecipeItem`.

АЛГОРИТМЫ УЧЕТА. СЕКЦИЯ PRIVATE

decimal *CalcRawMatVolumItem(const size_t PrCount, const size_t period, const decimal volume) const

Метод рассчитывает объем потребления ресурсов в каждом периоде *технологического цикла* в зависимости от требуемого объема готового продукта на выходе этого цикла. Объем рассчитывается в натуральном выражении. Метод рассчитывает потребность в ресурсах только для одного *единичного производственного цикла* (ЕПЦ). ЕПЦ — это однократный процесс производства одного наименования продукта в каком-либо отдельно взятом периоде проекта.

Метод возвращает указатель на вновь создаваемый одномерный динамический массив, аналог двумерной матрицы, размером `rcount*PrCount` с рассчитанными объемами потребных в производстве ресурсов: число строк матрицы совпадает с числом позиций ресурсов, число столбцов – с длительностью проекта. (нулевой

период, соответствующий стартовому балансу, не задействуется, остается равным нулю). Тип возвращаемых значений массива определяется псевдонимом [decimal](#).

Параметры:

PrCount - число периодов проекта; тип *size_t*;

period - номер периода проекта, в котором требуется готовый произведенный продукт в заданном количестве; тип *size_t*;

volume - требуемый объем этого продукта в заданном периоде; тип вещественного числа определяется псевдонимом [decimal](#).

decimal *CalcWorkingVolumItem(const size_t PrCount, const size_t period, const decimal volume) const

Метод рассчитывает объемы незавершенного производства в каждом периоде *технологического цикла* в зависимости от требуемого объема готового продукта на выходе этого цикла. Объемы рассчитываются в натуральном выражении. Также, как и предыдущий метод, данная функция рассчитывает объемы незавершенного производства только для одного *единичного производственного цикла*.

Метод возвращает указатель на вновь создаваемый одномерный динамический массив рассчитанных объемов незавершенного производства в натуральном выражении размером PrCount. Тип возвращаемых значений массива определяется псевдонимом [decimal](#).

Параметры:

PrCount - число периодов проекта; тип *size_t*;

period - номер периода проекта, в котором требуется готовый произведенный продукт в заданном количестве; тип *size_t*;

volume - требуемый объем этого продукта в заданном периоде; тип вещественного числа определяется псевдонимом [decimal](#).

decimal *CalcWorkingVolume(const size_t PrCount, const strItem volume[]) const

Метод рассчитывает объемы незавершенного производства в *каждом* периоде проекта в зависимости от требуемого объема готового продукта в *каждом* периоде проекта. Объемы рассчитываются в натуральном выражении.

Метод суммирует элементы массивов с одинаковыми индексами, полученных вызовом функции [CalcWorkingVolumItem](#) для каждого периода проекта: начиная с периода, равного значению в поле [duration](#) и до периода, равного предыдущему значению из параметра PrCount.

Метод возвращает указатель на вновь создаваемый одномерный динамический массив рассчитанных объемов незавершенного производства в натуральном выражении размером *PrCount*. Тип возвращаемых значений массива определяется псевдонимом [decimal](#).

Параметры:

PrCount - число периодов проекта; тип *size_t*;

volume[] – указатель на массив размерности *PrCount* с объемами производства продукта по периодам (план выпуска продукта); Тип значений массива определяется псевдонимом [decimal](#).

decimal *CalcWorkingValueItem(const size_t _PrCount, const size_t period, const decimal rmpPrice[], const decimal volume) const

Метод рассчитывает стоимость потребляемых ресурсов в каждом периоде *технологического цикла* в зависимости от требуемого объема готового продукта на выходе этого цикла и цен на ресурсы. Стоимость

рассчитывается в денежном эквиваленте. Метод рассчитывает стоимость потребляемых ресурсов только для одного *единичного производственного цикла*.

Метод перемножает элементы массива с объемами потребляемых ресурсов в натуральном выражении, полученного с помощью вызова функции [CalcRawMatVolumeItem](#), на элементы массива с ценами этих ресурсов, передаваемого в функцию в качестве параметра.

Метод возвращает указатель на вновь создаваемый одномерный динамический массив со стоимостью потребляемых ресурсов размером *PrCount*. Тип возвращаемых значений массива определяется псевдонимом [decimal](#).

Параметры:

_PrCount - число периодов проекта; тип *size_t*;

period - номер периода проекта, в котором требуется готовый произведенный продукт в заданном количестве; тип *size_t*;

rmprice[] – указатель на одномерный массив, аналог двумерной матрицы, размером *rcount* _PrCount*, с ценами ресурсов; Тип значений массива определяется псевдонимом [decimal](#);

volume - требуемый объем этого продукта в заданном периоде; тип вещественного числа определяется псевдонимом [decimal](#).

decimal *CalcWorkingValue(const size_t _PrCount, const size_t period, const decimal rmprice[], const decimal volume) const

Метод рассчитывает *аккумулированную* стоимость потребляемых ресурсов в каждом периоде *технологического цикла* в зависимости от требуемого объема готового продукта на выходе этого цикла и цен на ресурсы. Аккумулированная стоимость рассчитывается в денежном эквиваленте. Метод рассчитывает аккумулярованную стоимость потребляемых ресурсов только для одного *единичного производственного цикла*.

Метод создает и возвращает указатель на массив, элементы которого представляют собой сумму текущего и всех предыдущих элементов массива со стоимостью потребляемых ресурсов, полученных методом [CalcWorkingValueItem](#).

Метод возвращает указатель на вновь создаваемый одномерный динамический массив с аккумулярованной стоимостью потребляемых ресурсов размером *PrCount*. Тип возвращаемых значений массива определяется псевдонимом [decimal](#).

Полученный массив де-факто представляет собой сумму стоимостей незавершенного производства и готовой продукции. Последующие методы ([CalcWorkingBalanceItem](#) и [CalcProductBalanceItem](#)) призваны разделить их между собой и вернуть нужный результат: стоимость незавершенного производства и стоимость готового продукта по отдельности.

Параметры:

_PrCount - число периодов проекта; тип *size_t*;

period - номер периода проекта, в котором требуется готовый произведенный продукт в заданном количестве; тип *size_t*;

rmprice[] – указатель на одномерный массив, аналог двумерной матрицы, размером *rcount* _PrCount*, с ценами ресурсов; Тип значений массива определяется псевдонимом [decimal](#);

volume - требуемый объем этого продукта в заданном периоде; тип вещественного числа определяется псевдонимом [decimal](#).

decimal *CalcWorkingBalanceItem(const size_t _PrCount, const size_t period, const decimal rmprice[], const decimal volume) const

Метод рассчитывает себестоимость незавершенного производства в каждом периоде *технологического цикла* в зависимости от требуемого объема готового продукта на выходе этого цикла и цен на ресурсы. Себестоимость рассчитывается в денежном эквиваленте. Метод рассчитывает себестоимость незавершенного производства только для одного *единичного производственного цикла*.

Метод получает массив с аккумулярованной стоимостью потребляемых ресурсов (вызывая функцию [CalcWorkingValue](#)) и массив с объемами незавершенного производства в натуральном выражении (вызывая функцию [CalcWorkingVolumeItem](#)). Далее в методе обнуляются те элементы массива с аккумулярованной себестоимостью ресурсов, которые равны нулю в массиве с объемами незавершенного производства и имеют тот же индекс, что и элементы первого массива. Полученный «прореженный» массив содержит себестоимость незавершенного производства и возвращается методом. Метод возвращает указатель на вновь создаваемый одномерный динамический массив с элементами, тип которых определяется псевдонимом [decimal](#).

Параметры:

_PrCount - число периодов проекта; тип *size_t*;

period - номер периода проекта, в котором требуется готовый произведенный продукт в заданном количестве; тип *size_t*;

rmprice[] – указатель на одномерный массив, аналог двумерной матрицы, размером *rcount*_PrCount*, с ценами ресурсов; Тип значений массива определяется псевдонимом [decimal](#);

volume - требуемый объем этого продукта в заданном периоде; тип вещественного числа определяется псевдонимом [decimal](#).

decimal *CalcProductBalanceItem(const size_t _PrCount, const size_t period, const decimal rmprice[], const decimal volume) const

Метод рассчитывает себестоимость готового продукта на конец *технологического цикла* в зависимости от требуемого объема этого продукта на выходе этого цикла и цен на ресурсы. Себестоимость рассчитывается в денежном эквиваленте. Метод рассчитывает себестоимость готового продукта только для одного *единичного производственного цикла*.

Метод получает массив с аккумулярованной стоимостью потребляемых ресурсов (вызывая функцию [CalcWorkingValue](#)) и обнуляет в нем те элементы массива, индекс которых не совпадает с заданным периодом. Полученный «прореженный» массив содержит себестоимость готового продукта и возвращается методом. Метод возвращает указатель на вновь создаваемый одномерный динамический массив с элементами, тип которых определяется псевдонимом [decimal](#).

Параметры:

_PrCount - число периодов проекта; тип *size_t*;

period - номер периода проекта, в котором требуется готовый произведенный продукт в заданном количестве; тип *size_t*;

rmprice[] – указатель на одномерный массив, аналог двумерной матрицы, размером *rcount*_PrCount*, с ценами ресурсов; Тип значений массива определяется псевдонимом [decimal](#);

volume - требуемый объем этого продукта в заданном периоде; тип вещественного числа определяется псевдонимом [decimal](#).

void clsEraser()

Метод "обнуляет" все поля экземпляра класса: *name* = "", *measure* = "", *duration* = 0, *rcount* = 0. Указатели на динамические массивы *rnames* и *recipeitem* устанавливаются в *nullptr*. Метод используется в конструкторе перемещения. Внимание!!! Этот метод *не удаляет* массивы, на которые указывают указатели *rnames* и *recipeitem*!

```
void View_f_Item(const size_t PrCount, const strItem ProPlan[], const decimal rmprice[], const string& _hmcu, decimal* (clsRecipItem::*f)(const size_t, const size_t, const decimal*, const decimal) const) const
```

Служебная функция для реализации визуального контроля. Позволяет реализовать контроль работоспособности функций: [CalcWorkingValueItem](#), [CalcWorkingValue](#), [CalcWorkingBalanceItem](#) и [CalcProductBalanceItem](#). Используется функциями: [ViewWorkingValueItem](#), [ViewWorkingValue](#), [ViewWorkingBalanceItem](#) и [ViewProductBalanceItem](#) при отладке.

Параметры:

PrCount - число периодов проекта; тип *size_t*;

ProPlan[] - указатель на массив с планом выхода готовой продукции размерностью *PrCount*; тип элемента массива [strItem](#);

rmprice[] – указатель на одномерный массив, аналог двумерной матрицы, размером *rcount* PrCount*, с ценами ресурсов; тип значений массива определяется псевдонимом [decimal](#);

&_hmcu – ссылка на наименование денежной единицы (валюты); тип *string*;

decimal* (clsRecipItem::*f)(const size_t, const size_t, const decimal*, const decimal) const – указатель на одну из функций: [CalcWorkingValueItem](#), [CalcWorkingValue](#), [CalcWorkingBalanceItem](#) или [CalcProductBalanceItem](#).

```
void View_f_Balance(const size_t _PrCount, const strItem ProPlan[], const decimal rmprice[], const string& _hmcu, strItem* (clsRecipItem::*f)(const size_t, const decimal*, const strItem*) const) const
```

Служебная функция для реализации визуального контроля. Позволяет реализовать контроль работоспособности функций: [CalcWorkingBalance](#) и [CalcProductBalance](#). Используется функциями: [ViewWorkingBalance](#) и [ViewProductBalance](#) при отладке.

Параметры:

_PrCount - число периодов проекта; тип *size_t*;

ProPlan[] - указатель на массив с планом выхода готовой продукции размерностью *PrCount*; тип элемента массива [strItem](#);

rmprice[] – указатель на одномерный массив, аналог двумерной матрицы, размером *rcount* PrCount*, с ценами ресурсов; тип значений массива определяется псевдонимом [decimal](#);

&_hmcu – ссылка на наименование денежной единицы (валюты); тип *string*;

strItem* (clsRecipItem::*f)(const size_t, const decimal*, const strItem*) const – указатель на одну из функций [CalcWorkingBalance](#) или [CalcProductBalance](#).

АЛГОРИТМЫ УЧЕТА. СЕКЦИЯ PUBLIC

```
strItem *CalcWorkingBalance(const size_t _PrCount, const decimal rmprice[], const strItem ProPlan[]) const
```

Метод рассчитывает объем, удельную и полную себестоимость незавершенного производства для конкретного продукта, выпускаемого на протяжении всего проекта. Метод возвращает указатель на вновь создаваемый динамический массив типа [strItem](#) размером, равным числу периодов проекта *_PrCount*.

Метод использует функции [CalcWorkingVolume](#), [CalcWorkingBalanceItem](#).

Параметры:

_PrCount - число периодов проекта; тип *size_t*;

rmprice[] – указатель на одномерный массив, аналог двумерной матрицы, размером *rcount* PrCount*, с ценами ресурсов; тип значений массива определяется псевдонимом [decimal](#);

ProPlan[] - указатель на массив с планом выхода готовой продукции размерностью *PrCount*; тип элемента массива [strItem](#).

strItem *CalcProductBalance(const size_t _PrCount, const decimal rmpri[], const strItem ProPlan[]) const

Метод рассчитывает объем, удельную и полную себестоимость готового продукта, выпускаемого на протяжении всего проекта. Метод возвращает указатель на вновь создаваемый динамический массив типа [strItem](#) размером, равным числу периодов проекта *_PrCount*.

Метод использует функцию [CalcProductBalanceItem](#).

Параметры:

_PrCount - число периодов проекта; тип *size_t*;

rmpri[] – указатель на одномерный массив, аналог двумерной матрицы, размером *rcount* _PrCount*, с ценами ресурсов; тип значений массива определяется псевдонимом [decimal](#);

ProPlan[] - указатель на массив с планом выхода готовой продукции размерностью *PrCount*; тип элемента массива [strItem](#).

decimal* CalcRawMatVolume(const size_t PrCount, const strItem volume[]) const

Метод рассчитывает и возвращает объем потребления ресурсов в натуральном выражении для всего плана выпуска продукта в динамике, - по периодам. Метод возвращает указатель на вновь создаваемый одномерный динамический массив, аналог двумерной матрицы размером *rcount * PrCount*, строки которой представляют собой название ресурсов, а столбцы - периоды проекта. Тип элементов массива определяется псевдонимом [decimal](#).

Параметры:

PrCount - число периодов проекта; тип *size_t*;

volume[] – указатель на массив размерности *PrCount* с объемами производства продукта по периодам (план выпуска продукта); Тип значений массива определяется псевдонимом [decimal](#).

GET- МЕТОДЫ

strNameMeas *GetRawNamesItem() const

Метод возвращает указатель на вновь создаваемый одномерный динамический массив с наименованиями ресурсов и названиями их единиц измерения. Массив создается как *копия* внутреннего массива [rnames](#).

Размер создаваемого массива [rcount](#). Тип элементов массива [strNameMeas](#).

const strNameMeas* GetRefRawNamesItem() const

Метод возвращает *константный указатель* на внутренний массив с наименованиями ресурсов и названиями их единиц измерения [rnames](#). Тип элементов массива [strNameMeas](#).

decimal* GetRecipeitem() const

Метод возвращает указатель на вновь создаваемый одномерный динамический массив с рецептурой/ технологической картой продукта (аналог двумерной матрицы размером *rcount*duration*). Массив создается как *копия* внутреннего массива [recipeitem](#). Тип элементов массива определяется псевдонимом [decimal](#).

const decimal* GetRefRecipeitem() const

Метод возвращает *константный указатель* на внутренний массив с рецептурой/ технологической картой продукта [recipeitem](#). Тип элементов массива определяется псевдонимом [decimal](#).

const size_t GetDuration() const

Метод возвращает длительность производственного цикла (значение поля [duration](#)). Тип возвращаемого значения [size_t](#).

const size_t GetRCount() const

Метод возвращает количество позиций ресурсов, используемых в технологическом цикле (значение поля [rcount](#)). Тип возвращаемого значения [size_t](#).

const string& GetName() const

Метод возвращает ссылку на наименование продукта (ссылка на поле [name](#)). Тип возвращаемого значения ссылка на [string](#).

const string& GetMeasure() const

Метод возвращает ссылку на наименование единицы измерения натурального объема продукта (ссылка на поле [measure](#)). Тип возвращаемого значения ссылка на [string](#).

МЕТОДЫ СОХРАНЕНИЯ СОСТОЯНИЯ ОБЪЕКТА В ФАЙЛ И ВОССТАНОВЛЕНИЯ ИЗ ФАЙЛА**bool StF(ofstream &_outF)**

Метод имплементации записи в файловый поток текущего состояния экземпляра класса (запись в файловый поток, метод сериализации). Название метода является аббревиатурой, расшифровывается “StF” как “Save to File”. При удачной сериализации возвращает [true](#), при ошибке записи возвращает [false](#).

Параметры:

&_outF - экземпляр класса [ofstream](#) для записи данных. При инициализации переменной _outF необходимо установить флаг бинарного режима открытия потока (не текстового!) [ios::binary](#).

bool RfF(ifstream &_inF)

Метод имплементации чтения из файлового потока текущего состояния экземпляра класса (чтение из файлового потока, метод десериализации). Название метода является аббревиатурой, расшифровывается “RfF” как “Read from File”. При удачной десериализации возвращает [true](#), при ошибке чтения возвращает [false](#).

Параметры:

&_inF - экземпляр класса [ifstream](#) для чтения данных. При инициализации переменной _inF необходимо установить флаг бинарного режима открытия потока (не текстового!) [ios::binary](#).

bool SaveToFile(const string _filename)

Метод записи текущего состояния экземпляра класса в файл. При удачной записи в файл, возвращает [true](#), при ошибке записи возвращает [false](#).

Параметры:

_filename – имя файла; тип [string](#).

bool ReadFromFile(const string _filename)

Метод восстановления состояния экземпляра класса из файла. При удачном чтении из файла, возвращает [true](#), при ошибке чтения возвращает [false](#).

_filename – имя файла; тип [string](#).

МЕТОДЫ ВИЗУАЛЬНОГО КОНТРОЛЯ

void ViewMainParams() const

Метод выводит в стандартный поток [std::cout](#) основные параметры класса: *название* продукта (контроль работоспособности метода [GetName](#)), *единицу измерения* продукта (контроль работоспособности метода [GetMeasure](#)), *длительность технологического цикла* (контроль работоспособности метода [GetDuration](#)), *количество позиций ресурсов* в рецептуре/ технологической карте (контроль работоспособности метода [GetRCount](#)).

void ViewRawNames() const

Метод выводит в стандартный поток [std::cout](#) основные параметры класса: *наименование ресурсов* (контроль работоспособности метода [GetRawNamesItem](#)).

void ViewRefRawNames() const

Метод выводит в стандартный поток [std::cout](#) основные параметры класса: *наименование ресурсов* (контроль работоспособности метода [GetRefRawNamesItem](#)).

void ViewRecipeItem() const

Метод выводит в стандартный поток [std::cout](#) основные параметры класса: рецептуру/ технологическую карту продукта (контроль работоспособности метода [GetRecipeItem](#)).

void ViewRefRecipeItem() const

Метод выводит в стандартный поток [std::cout](#) основные параметры класса: рецептуру/ технологическую карту продукта (контроль работоспособности метода [GetRefRecipeItem](#)).

void ViewRawMatVolume(const size_t PrCount, const strItem ProPlan[]) const

Метод выводит в стандартный поток [std::cout](#) объем потребления ресурсов в натуральном выражении для всего плана выпуска продукта в динамике, - по периодам (контроль работоспособности метода [CalcRawMatVolume](#)).

Параметры:

PrCount - число периодов проекта; тип *size_t*;

ProPlan[] – указатель на массив размерности *PrCount* с объемами производства продукта по периодам (план выпуска продукта); Тип значений массива определяется псевдонимом [decimal](#).

void ViewWorkingValueItem(const size_t PrCount, const strItem ProPlan[], const decimal rmprice[], const string& hmcure) const

Метод выводит в стандартный поток [std::cout](#) стоимость потребляемых ресурсов в каждом периоде *технологического цикла* (контроль работоспособности метода [CalcWorkingValueItem](#)). Информация выводится по *всем циклам проекта* в виде таблицы, где строки - единичные технологические циклы, столбцы – расход в денежном эквиваленте по периоду.

Параметры:

PrCount - число периодов проекта; тип *size_t*;

ProPlan[] – указатель на массив размерности *PrCount* с объемами производства продукта по периодам (план выпуска продукта); Тип значений массива определяется псевдонимом [decimal](#).

rmprice[] – указатель на одномерный массив, аналог двумерной матрицы, размером *rcount*PrCount*, с ценами ресурсов; Тип значений массива определяется псевдонимом [decimal](#);

&hmcu – ссылка на наименование денежной единицы (валюты); тип *string*.

void ViewWorkingValue(const size_t PrCount, const strItem ProPlan[], const decimal rmprice[], const string& hmcu) const

Метод выводит в стандартный поток [std::cout](#) аккумулярованную стоимость потребляемых ресурсов в каждом периоде *технологического цикла* (контроль работоспособности метода [CalcWorkingValue](#)). Информация выводится по *всем циклам проекта* в виде таблицы, где строки - единичные технологические циклы, столбцы – аккумулярованный расход в денежном эквиваленте по периоду.

Параметры:

PrCount - число периодов проекта; тип *size_t*;

ProPlan[] – указатель на массив размерности *PrCount* с объемами производства продукта по периодам (план выпуска продукта); Тип значений массива определяется псевдонимом [decimal](#).

rmprice[] – указатель на одномерный массив, аналог двумерной матрицы, размером *rcount*PrCount*, с ценами ресурсов; Тип значений массива определяется псевдонимом [decimal](#);

&hmcu – ссылка на наименование денежной единицы (валюты); тип *string*.

void ViewWorkingBalanceItem(const size_t PrCount, const strItem ProPlan[], const decimal rmprice[], const string& hmcu) const

Метод выводит в стандартный поток [std::cout](#) себестоимость незавершенного производства в каждом периоде *технологического цикла* (контроль работоспособности метода [CalcWorkingBalanceItem](#)). Информация выводится по *всем циклам проекта* в виде таблицы, где строки - единичные технологические циклы, столбцы - незавершенное производство по периоду.

Параметры:

PrCount - число периодов проекта; тип *size_t*;

ProPlan[] – указатель на массив размерности *PrCount* с объемами производства продукта по периодам (план выпуска продукта); Тип значений массива определяется псевдонимом [decimal](#).

rmprice[] – указатель на одномерный массив, аналог двумерной матрицы, размером *rcount*PrCount*, с ценами ресурсов; Тип значений массива определяется псевдонимом [decimal](#);

&hmcu – ссылка на наименование денежной единицы (валюты); тип *string*.

void ViewWorkingBalance(const size_t _PrCount, const strItem ProPlan[], const decimal rmprice[], const string& hmcu) const

Метод выводит в стандартный поток [std::cout](#) объем, удельную и полную себестоимость незавершенного производства для конкретного продукта, выпускаемого на протяжении всего проекта (контроль работоспособности метода [CalcWorkingBalance](#)). Информация выводится в виде трех таблиц: объем, удельная себестоимость, полная себестоимость. Каждая таблица имеет одну строку с названием продукта и столбцы по количеству периодов проекта.

Параметры:

_PrCount - число периодов проекта; тип *size_t*;

ProPlan[] – указатель на массив размерности *PrCount* с объемами производства продукта по периодам (план выпуска продукта); Тип значений массива определяется псевдонимом [decimal](#).

rmprice[] – указатель на одномерный массив, аналог двумерной матрицы, размером *rcount*PrCount*, с ценами ресурсов; Тип значений массива определяется псевдонимом [decimal](#);

&hmcu – ссылка на наименование денежной единицы (валюты); тип *string*.

void ViewProductBalanceItem(const size_t PrCount, const strItem ProPlan[], const decimal rmprice[], const string& hmcure) const

Метод выводит в стандартный поток `std::cout` себестоимость готового продукта на конец *технологического цикла* (контроль работоспособности метода `CalcProductBalanceItem`). Информация выводится по *всем циклам проекта* в виде таблицы, где строки - единичные технологические циклы, столбцы - себестоимость готового продукта по периоду.

Параметры:

PrCount - число периодов проекта; тип *size_t*;

ProPlan[] – указатель на массив размерности *PrCount* с объемами производства продукта по периодам (план выпуска продукта); Тип значений массива определяется псевдонимом *decimal*.

rmprice[] – указатель на одномерный массив, аналог двумерной матрицы, размером *rcount* PrCount*, с ценами ресурсов; Тип значений массива определяется псевдонимом *decimal*;

&hmcure – ссылка на наименование денежной единицы (валюты); тип *string*.

void ViewProductBalance(const size_t _PrCount, const strItem ProPlan[], const decimal rmprice[], const string& hmcure) const

Метод выводит в стандартный поток `std::cout` объем, удельную и полную себестоимость готового продукта, выпускаемого на протяжении всего проекта (контроль работоспособности метода `CalcProductBalance`). Информация выводится в виде трех таблиц: объем, удельная себестоимость, полная себестоимость. Каждая таблица имеет одну строку с названием продукта и столбцы по количеству периодов проекта.

Параметры:

_PrCount - число периодов проекта; тип *size_t*;

ProPlan[] – указатель на массив размерности *PrCount* с объемами производства продукта по периодам (план выпуска продукта); Тип значений массива определяется псевдонимом *decimal*.

rmprice[] – указатель на одномерный массив, аналог двумерной матрицы, размером *rcount* PrCount*, с ценами ресурсов; Тип значений массива определяется псевдонимом *decimal*;

&hmcure – ссылка на наименование денежной единицы (валюты); тип *string*.

КЛАСС clsManufactItem

Если класс `clsRecipeItem` предоставляет основные расчетные *методы* позволяющие моделировать учетные процессы в производстве единственного продукта, то класс `clsManufactItem` инкапсулирует эти *методы*, *исходные данные* и *результаты расчетов* и представляет собой *инструмент* моделирования частичных затрат на основе переменных затрат полноценного производства для единственного наименования продукта (производства монопродукта, однотоварного производства).

К задачам, которые можно решить применением класса `clsRecipeItem`, добавляются возможности по расчету и сохранению данных:

1. Количества периодов проекта;
2. Плана производства продукта в натуральном, удельном и полном стоимостном выражении;
3. Плана поставок ресурсов в производство в натуральном выражении;
4. Цен ресурсов и график их изменения;
5. Плана незавершенного производства в натуральном, удельном и полном стоимостном выражении.

На рисунке ниже (Рисунок 4) представлена схема информационных потоков однотоварного производства.

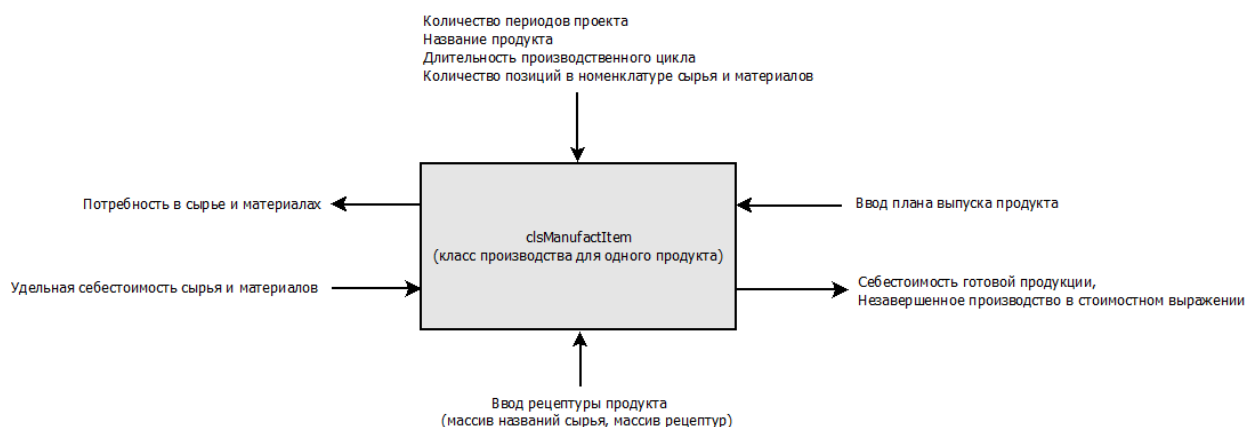


Рисунок 4 Принципиальная схема информационных потоков класса clsManufactItem

ПОЛЯ КЛАССА clsManufactItem

Таблица 13 Поля класса clsManufactItem. Секция private

Поле класса	Описание	Заметка
size_t PrCount	Тип <i>size_t</i> . Количество периодов проекта	
const string* name	Указатель на название продукта. Тип данных <i>string</i> .	В конструкторе без параметров равен <i>nullptr</i> . В конструкторах с параметрами указывает на одноименное поле объекта <i>Recipe</i> (рецептура/технологическая карта продукта)
const string* measure	Указатель на название единицы измерения натурального объема продукта. Тип данных <i>string</i> .	В конструкторе без параметров равен <i>nullptr</i> . В конструкторах с параметрами указывает на одноименное поле объекта <i>Recipe</i>
size_t duration	Тип <i>size_t</i> . Длительность производственного цикла	В конструкторе без параметров равен нулю. В конструкторах с параметрами значение устанавливается при создании объекта класса как копия одноименного поля объекта <i>Recipe</i>
size_t rcount	Тип <i>size_t</i> . Количество позиций в номенклатуре ресурсов, требуемых для производства продукта	В конструкторе без параметров равен нулю. В конструкторах с параметрами значение устанавливается при создании объекта класса как копия одноименного поля объекта <i>Recipe</i>
clsRecipeItem *Recipe	Указатель на рецептуру/ технологическую карту продукта. Тип объекта clsRecipeItem	В конструкторе без параметров равен <i>nullptr</i> . В конструкторах с параметрами вызывает один из конструкторов класса <i>clsRecipeItem</i>
strItem *ProductPlan	Указатель на массив с планом производства продукта. Массив одномерный, размером <i>PrCount</i> , Тип элемента массива strItem	При создании объекта класса устанавливается равным <i>nullptr</i> . В дальнейшем поля <i>volume</i> вводятся, как исходные данные, поля <i>price</i> и <i>value</i> рассчитываются. В конструкторах копирования/ перемещения принимает состояние одноименного копируемого/ перемещаемого объекта
decimal *RawMatPurchPlan	Указатель на массив с планом поставок ресурсов в натуральном выражении. Массив одномерный, аналог двумерной матрицы: строки – ресурсы, столбцы – объемы поставок в разные периоды проекта; размер массива <i>rcount*PrCount</i> . Тип вещественного числа элемента массива определяется псевдонимом decimal	При создании объекта класса устанавливается равным <i>nullptr</i> . В дальнейшем значения элементов массива рассчитываются. В конструкторах копирования/ перемещения принимает состояние одноименного копируемого/ перемещаемого объекта
decimal *RawMatPrice	Указатель на массив с ценами ресурсов. Массив одномерный, аналог двумерной	При создании объекта класса устанавливается равным <i>nullptr</i> . В дальнейшем значения

	матрицы: строки – ресурсы, столбцы – цены в разные периоды проекта; размер массива <i>rcount*PrCount</i> . Тип вещественного числа элемента массива определяется псевдонимом <i>decimal</i>	элементов массива вводятся, как исходные данные. В конструкторах копирования/перемещения принимает состояние одноименного копируемого/перемещаемого объекта
strItem *Balance	Указатель на массив с планом незавершенного производства. Массив одномерный, размером <i>PrCount</i> , Тип элемента массива <i>strItem</i>	При создании объекта класса устанавливается равным <i>nullptr</i> . В дальнейшем значения элементов массива рассчитываются. В конструкторах копирования/перемещения принимает состояние одноименного копируемого/перемещаемого объекта

Методы класса *clsManufactItem* можно условно разделить на следующие группы:

- Конструкторы, деструктор, перегруженные операторы;
- Вычислительные методы;
- Get-методы;
- Set – методы
- Методы сохранения состояния объекта в файл и восстановления из файла;
- Методы визуального контроля

Ниже подробно описываются все методы.

КОНСТРУКТОРЫ, ДЕКТРУКТОР, ПЕРЕГРУЖЕННЫЕ ОПЕРАТОРЫ

clsManufactItem()

Конструктор по умолчанию (без параметров). Создает и инициализирует переменные *PrCount* = *duration* = *rcount* = 0; указатели на *name* и *measure* устанавливаются в *nullptr*; объект *Recipe* и динамические массивы *ProductPlan*, *RawMatPurchPlan*, *RawMatPrice* и *Balance* не создаются, указатели на них устанавливаются в *nullptr*.

clsManufactItem(const size_t _PrCount, const clsRecipeItem &obj)

Конструктор с вводом длительности проекта и рецептуры/ технологической карты.

Параметры:

_PrCount – число периодов проекта; устанавливает значение поля *PrCount*; тип *size_t*;

&obj – рецептура/ технологическая карта продукта; ссылка на экземпляр класса *clsRecipeItem*. Объект *obj* копируется в поле *Recipe*.

Поле *PrCount* инициализируется значением из параметра *_PrCount*; поля *Recipe*, *name*, *measure*, *duration*, *rcount* инициализируются значениями из *obj*; динамические массивы *ProductPlan*, *RawMatPurchPlan*, *RawMatPrice* и *Balance* не создаются, указатели на них устанавливаются в *nullptr*.

clsManufactItem(const size_t _PrCount, clsRecipeItem &&obj)

Конструктор с вводом длительности проекта и рецептуры/ технологической карты.

Параметры:

_PrCount – число периодов проекта; устанавливает значение поля *PrCount*; тип *size_t*;

&obj – рецептура/ технологическая карта продукта; rvalue-ссылка на экземпляр класса *clsRecipeItem*. Объект *obj* перемещается в поле *Recipe*.

Поле *PrCount* инициализируется значением из параметра *_PrCount*; поля *Recipe*, *name*, *measure*, *duration*, *rcount* инициализируются значениями из *obj*; динамические массивы *ProductPlan*, *RawMatPurchPlan*, *RawMatPrice* и *Balance* не создаются, указатели на них устанавливаются в *nullptr*.

clsManufactItem(const size_t _PrCount, const string &_name, const string &_measure, const size_t _duration, const size_t _rcount, const strNameMeas _rnames[], const decimal _recipeitem[])

Конструктор с параметрами, создает внутри себя экземпляр класса рецептов.

Параметры:

_PrCount – число периодов проекта; устанавливает значение поля [PrCount](#); тип *size_t*;

&_name – ссылка на название продукта; используется как параметр в конструкторе класса [clsRecipeItem](#) для создания объекта в поле [Recipe](#) и устанавливает указатель [name](#) на одноименное поле объекта *Recipe*; тип *string*;

&_measure – ссылка на наименование единицы измерения натурального объема продукта; используется как параметр в конструкторе класса [clsRecipeItem](#) для создания объекта в поле [Recipe](#) и устанавливает указатель [measure](#) на одноименное поле объекта *Recipe*; тип *string*;

_duration – длительность производственного цикла; используется как параметр в конструкторе класса [clsRecipeItem](#) для создания объекта в поле [Recipe](#) и устанавливает значение в поле [duration](#); тип *size_t*;

_rcount – количество позиций ресурсов, требуемых для производства продукта; используется как параметр в конструкторе класса [clsRecipeItem](#) для создания объекта в поле [Recipe](#) и устанавливает значение поля [rcount](#); тип *size_t*;

_rnames[] – указатель на массив с наименованиями ресурсов и единицами их измерения размером *_rcount*; используется как параметр в конструкторе класса [clsRecipeItem](#) для создания объекта в поле [Recipe](#); тип элемента массива [strNameMeas](#);

_recipeitem[] – указатель на массив с рецептурами (одномерный аналог матрицы размерностью *_rcount***_duration*); используется как параметр в конструкторе класса [clsRecipeItem](#) для создания объекта в поле [Recipe](#); вещественный тип элемента массива определяется псевдонимом [decimal](#).

clsManufactItem(const clsManufactItem &obj)

Конструктор копирования.

Параметры:

&obj – ссылка на копируемый константный экземпляр класса *clsManufactItem*.

clsManufactItem(clsManufactItem &&obj)

Конструктор перемещения.

Параметры:

&&obj - перемещаемый экземпляр класса *clsManufactItem*.

clsManufactItem &operator= (const clsManufactItem &obj)

Перегрузка оператора присваивания копированием

Параметры:

&obj – ссылка на копируемый константный экземпляр класса *clsManufactItem*.

clsManufactItem &operator= (clsManufactItem &&obj)

Перегрузка оператора присваивания перемещением

Параметры:

&&obj - перемещаемый экземпляр класса *clsManufactItem*.

bool operator == (const string &Rightname) const

Переопределение оператора сравнения для поиска экземпляра объекта по наименованию продукта

Параметры:

&Rightname – ссылка на строку символов, с которой сравнивается значение, на которое указывает поле [name](#) экземпляра класса.

~clsManufactItem()

Деструктор.

void swap(clsManufactItem& obj) noexcept

Функция обмена значениями между объектами типа *clsManufactItem*.

Параметры:

&obj – ссылка на обмениваемый экземпляр класса *clsManufactItem*.

АЛГОРИТМЫ УЧЕТА. СЕКЦИЯ PRIVATE

void clsEraser()

Метод "обнуляет" все поля экземпляра класса: [PrCount](#) = [duration](#) = [rcount](#) = 0, устанавливает указатели [name](#) и [measure](#) = *nullptr*. Устанавливает указатели [Recipe](#), [ProductPlan](#), [RawMatPurchPlan](#), [RawMatPrice](#) и [Balance](#), в *nullptr*. Метод используется в конструкторе перемещения. Внимание!!! Метод не удаляет существующие массивы, на которые указывают указатели!

АЛГОРИТМЫ УЧЕТА. СЕКЦИЯ PUBLIC

void CalcRawMatPurchPlan()

Метод вызывает функцию [clsRecipeItem::CalcRawMatVolume](#) объекта из поля [Recipe](#) и рассчитывает объем потребления ресурсов в натуральном выражении для всего плана выпуска продукта и создает и заполняет массив [RawMatPurchPlan](#).

bool CalculatItem()

Метод вызывает функции [clsRecipeItem::CalcWorkingBalance](#) и [clsRecipeItem::CalcProductBalance](#) объекта из поля [Recipe](#) и рассчитывает объем, удельную и полную себестоимость незавершенного производства и готовой продукции для конкретного продукта, выпускаемого на протяжении всего проекта. Если вычисления проведены без ошибок, метод возвращает *true*, в противном случае – *false*.

GET-МЕТОДЫ

const size_t& GetPrCount() const

Возвращает const-ссылку на количество периодов проекта. Возвращается ссылка на тип *size_t*.

const size_t& GetRCount() const

Возвращает const-ссылку на количество позиций сырья, участвующего в производстве. Возвращается ссылка на тип *size_t*.

const size_t& GetDuration() const

Возвращает const-ссылку на длительность производственного цикла. Возвращается ссылка на тип *size_t*.

const string* GetName() const

Возвращает const-указатель на наименование продукта. Возвращает указатель на тип *string*.

const string* GetMeasure() const

Возвращает const-указатель на наименование единицы измерения натурального объема продукта. Возвращает указатель на тип *string*.

const decimal* GetRefRecipe() const

Метод вызывает функцию [clsRecipeItem::GetRefRecipeItem](#) объекта из поля [Recipe](#) и возвращает const-указатель на внутренний массив с рецептурами объекта из этого поля. Вещественный тип элементов массива определяется псевдонимом [decimal](#).

const strNameMeas* GetRefRawNames() const

Метод вызывает функцию [clsRecipeItem::GetRefRawNamesItem](#) объекта из поля [Recipe](#) и возвращает const-указатель на внутренний массив с наименованиями ресурсов и единицами их натурального измерения объекта из этого поля. Тип элементов массива [strNameMeas](#).

const decimal* GetRawMatPurchPlan() const

Метод возвращает константный указатель на массив [RawMatPurchPlan](#) с объемом потребления ресурсов в натуральном выражении для всего плана выпуска продукта. Вещественный тип элементов массива определяется псевдонимом [decimal](#).

const decimal* GetRawMatPrice() const

Метод возвращает константный указатель на массив [RawMatPrice](#) с ценами на ресурсы. Вещественный тип элементов массива определяется псевдонимом [decimal](#).

const strItem* GetBalance() const

Метод возвращает константный указатель на массив [Balance](#) с объемом, удельной и полной себестоимостью незавершенного производства для конкретного продукта, выпускаемого на протяжении всего проекта. Если массив не иницирован расчетными данными, то возвращает *nullptr*. Тип элемента массива [strItem](#).

const strItem* GetProductPlan() const

Метод возвращает константный указатель на массив [ProductPlan](#) с объемом, удельной и полной себестоимостью готовой продукции для конкретного продукта, выпускаемого на протяжении всего проекта. Если массив не иницирован исходными данными, то возвращает *nullptr*. Если в массиве отсутствуют расчетные данные, то возвращает только натуральные объемы (значения полей [strItem](#).volume). Тип элемента массива [strItem](#).

SET – МЕТОДЫ

bool SetProductPlan(const strItem _ProductPlan[])

Метод ввода плана выпуска продукта (объем выпуска в натуральном выражении и график выпуска).

Параметры:

[_ProductPlan\[\]](#) – указатель на массив с планом выпуска продукта в натуральном выражении. Тип элемента массива [strItem](#). В каждом элементе массива полезной информацией заполнены поля *volume*; поля *price* и *value* содержат нули. В случае удачного завершения работы метода, возвращается *true*, иначе – возвращается *false*.

bool SetRawMatPrice(const decimal _Price[])

Метод ввода цен на ресурсы. Предполагается, что после получения складом ресурсов информации о потребности в ресурсах (с помощью метода [GetRawMatPurchPlan](#)), склад возвращает информацию об учетных ценах на эти ресурсы. Эта информация с помощью данного метода *копируется* в массив [RawMatPrice](#) размером [rcount*PrCount](#). В случае удачного завершения работы метода, возвращается *true*, иначе – возвращается *false*.

Параметры:

_Price – указатель на *копируемый* массив с ценами ресурсов; массив является одномерным аналогом двумерной матрицы, размером *rcount*PrCount*. Вещественный тип элемента массива определяется псевдонимом [decimal](#).

bool SetRawMatPrice(const strItem _Price[])

Аналогичен предыдущему методу, но элементы входного массива имеют тип [strItem](#) (используются поля *price*).

bool MoveRawMatPrice(decimal* &_Price)

Метод ввода цен на ресурсы. Предполагается, что после получения складом ресурсов информации о потребности в ресурсах (с помощью метода [GetRawMatPurchPlan](#)), склад возвращает информацию об учетных ценах на эти ресурсы. Эта информация с помощью данного метода *перемещается* в массив [RawMatPrice](#) размером [rcount*PrCount](#). В случае удачного завершения работы метода, возвращается *true*, иначе – возвращается *false*.

Параметры:

_Price – ссылка на указатель на *перемещаемый* массив с ценами ресурсов; массив является одномерным аналогом двумерной матрицы, размером *rcount*PrCount*. После окончания работы метода *_Price* принимает значение *nullptr*. Вещественный тип элемента массива определяется псевдонимом [decimal](#).

МЕТОДЫ СОХРАНЕНИЯ СОСТОЯНИЯ ОБЪЕКТА В ФАЙЛ И ВОССТАНОВЛЕНИЯ ИЗ ФАЙЛА**bool StF(ofstream &_outF)**

Метод имплементации записи в файловый поток текущего состояния экземпляра класса (запись в файловый поток, метод сериализации). Название метода является аббревиатурой, расшифровывается “StF” как “Save to File”. При удачной сериализации возвращает *true*, при ошибке записи возвращает *false*.

Параметры:

&_outF - экземпляр класса [ofstream](#) для записи данных. При инициализации переменной *_outF* необходимо установить флаг бинарного режима открытия потока (не текстового!) [ios::binary](#).

bool RfF(ifstream &_inF)

Метод имплементации чтения из файлового потока текущего состояния экземпляра класса (чтение из файлового потока, метод десериализации). Название метода является аббревиатурой, расшифровывается “RfF” как “Read from File”. При удачной десериализации возвращает *true*, при ошибке чтения возвращает *false*.

Параметры:

&_inF - экземпляр класса [ifstream](#) для чтения данных. При инициализации переменной *_inF* необходимо установить флаг бинарного режима открытия потока (не текстового!) [ios::binary](#).

bool SaveToFile(const string _filename)

Метод записи текущего состояния экземпляра класса в файл. При удачной записи в файл, возвращает *true*, при ошибке записи возвращает *false*.

Параметры:

`_filename` – имя файла; тип *string*.

bool ReadFromFile(const string _filename)

Метод восстановления состояния экземпляра класса из файла. При удачном чтении из файла, возвращает *true*, при ошибке чтения возвращает *false*.

`_filename` – имя файла; тип *string*.

МЕТОДЫ ВИЗУАЛЬНОГО КОНТРОЛЯ**void ViewMainParams() const**

Метод выводит в стандартный поток `std::cout` основные параметры класса: *длительность* проекта (контроль работоспособности метода `GetPrCount`), *название* продукта (контроль работоспособности метода `GetName`), *единицу измерения* натурального объема продукта (контроль работоспособности метода `GetMeasure`), *длительность технологического цикла* (контроль работоспособности метода `GetDuration`), *количество позиций ресурсов* в рецептуре/ технологической карте (контроль работоспособности метода `GetRCount`).

void ViewRefRecipe() const

Метод выводит в стандартный поток `std::cout` рецептуру/ технологическую карту продукта (контроль работоспособности методов `GetRefRecipe` и `GetRefRawNames`). Метод использует функцию вывода информации в формате таблицы `ArrPrint`.

void ViewRawMatPurchPlan() const

Метод выводит в стандартный поток `std::cout` *потребность в ресурсах* для продукта (контроль работоспособности методов `GetRCount`, `GetRefRawNames`, `GetRawMatPurchPlan`, `GetPrCount`). Перед вызовом данного метода необходимо провести вычисления методом `CalcRawMatPurchPlan`, иначе метод `GetRawMatPurchPlan` вернет *nullptr*. Метод использует функцию вывода информации в формате таблицы `ArrPrint`.

void ViewRawMatPrice(const string& hmcu) const

Метод выводит в стандартный поток `std::cout` *цены на ресурсы* для продукта (контроль работоспособности методов `GetRCount`, `GetRefRawNames`, `GetRawMatPrice`, `GetPrCount`). Перед вызовом данного метода необходимо ввести цены на ресурсы методом `SetRawMatPrice`, иначе метод `GetRawMatPrice` вернет *nullptr*. Метод использует функцию вывода информации в формате таблицы `ArrPrint`.

Параметры:

`&hmcu` – ссылка на название денежной единицы. Тип *string*.

void ViewCalculate(const string& hmcu) const

Метод выводит в стандартный поток `std::cout` *незавершенное производство* в *натуральном, удельном и полном стоимостном* выражении (контроль работоспособности метода `GetBalance`), а также *готовую продукцию* в *натуральном, удельном и полном стоимостном* выражении (контроль работоспособности метода `GetProductPlan`). Перед вызовом данного метода необходимо:

- Создать объект класса и ввести длительность проекта, рецептуру/ технологическую карту, план выпуска продукта (например, используя соответствующий `конструктор` и метод `SetProductPlan`);
- Рассчитать потребность в ресурсах для производства продукта (используя метод `CalcRawMatPurchPlan`);
- Ввести цены на ресурсы (используя метод `SetRawMatPrice`);
- Провести расчет незавершенного производства и готовой продукции (используя метод `CalculateItem`).

Если пропустить какой-либо из перечисленных выше пунктов, массивы [Balance](#) и [ProductPlan](#) будут не заполнены расчетными данными, и методы [GetBalance](#) и [GetProductPlan](#) вернут некорректные данные. Таким образом, данный метод является финализирующим в цепочке визуального контроля работоспособности методов класса.

Метод использует функцию вывода информации в формате таблицы [ArrPrint](#).

Параметры:

&hmcur – ссылка на название денежной единицы. Тип *string*.

ПРИМЕР ПРОГРАММЫ С КЛАССОМ `clsManufactItem`

Ниже представлен листинг программы, демонстрирующей работу класса `clsManufactItem`.


```

#include <iostream>
#include <Manufact_module.h>

using namespace std;

int main() {

    setlocale(LC_ALL, "Russian");          // Установка русского языка для вывода

    /** Основные параметры проекта **/
    const size_t PrCount = 9;              // Количество периодов проекта
    const string Cur = "RUR";              // Домашняя валюта проекта

    /** Продукт **/
    const string Name_00 = "Фрикасе из курицы с зеленым горошком";
    const string Meas_00 = "шт.";

    /** План отгрузок готовой продукции **/

    const strItem PPlan[PrCount] { {0, 0, 0}, {5003, 0, 0}, {8482, 0, 0},
                                     {11825, 0, 0}, {15137, 0, 0}, {18428, 0, 0},
                                     {21706, 0, 0}, {24975, 0, 0}, {28239, 0, 0}};

    /** Исходные данные для рецептуры продукта **/
    const size_t rcount_00 = 9;            // Число наименований ресурсов
    const size_t duratio_00 = 1;           // Длительность производственного цикла

    const strNameMeas rnames_00[rcount_00] // Номенклатура ресурсов
    { {"Горошек зеленый с/м", "кг."}, {"Итальянские травы", "кг."},
      {"Курица бедро", "кг."}, {"Молоко", "кг."}, \
      {"Морковь", "кг."}, {"Мука пшеничная", "кг."}, {"Соль", "кг."},
      {"Упаковка", "шт."}, {"Чеснок", "кг."}};

    const decimal recipe_00[rcount_00*duratio_00] {0.1200,
                                                       0.0012,
                                                       0.2700,
                                                       0.1200,
                                                       0.1320,
                                                       0.0120,
                                                       0.0042,
                                                       1.0000,
                                                       0.0024 };

    /** Создаем рецептуру продукта **/
    clsRecipeItem* rcp_00 =
        new clsRecipeItem(Name_00, Meas_00, duratio_00, rcount_00,
                           rnames_00, recipe_00);

    /** Создаем единичное производство **/
    clsManufactItem* man_00 = new clsManufactItem(PrCount, move(*rcp_00));
    man_00->SetProductPlan(PPlan); // Вводим план отгрузок готовой продукции
    man_00->CalcRawMatPurchPlan(); // Рассчитываем план потребления ресурсов

    /** Отдаем на склад ресурсов план потребления сырья и материалов (например,
    так: const decimal* RMPlan = man_00->GetRawMatPurchPlan()), и получаем от
    склада данные о себестоимости необходимых нам ресурсов. Например, склад
    вернул нам следующий массив: **/

```

```

const decimal RMRpice[rcount_00*PrCount] {
100.00, 101.00, 102.01, 103.03, 104.06, 105.10, 106.15, 107.21, 108.28,
4062.50, 4103.13, 4144.16, 4185.60, 4227.45, 4269.73, 4312.43, 4355.55, 4399.11,
225.00, 227.25, 229.52, 231.82, 234.14, 236.48, 238.84, 241.23, 243.64,
39.00, 39.39, 39.78, 40.18, 40.58, 40.99, 41.40, 41.81, 42.23,
15.00, 15.15, 15.30, 15.45, 15.61, 15.77, 15.92, 16.08, 16.24,
41.00, 41.41, 41.82, 42.24, 42.66, 43.09, 43.52, 43.96, 44.40,
19.00, 19.19, 19.38, 19.58, 19.77, 19.97, 20.17, 20.37, 20.57,
20.00, 20.20, 20.40, 20.61, 20.81, 21.02, 21.23, 21.44, 21.66,
105.00, 106.05, 107.11, 108.18, 109.26, 110.36, 111.46, 112.57, 113.70};

if(man_00->SetRawMatPrice(RMRpice)); // Вводим цены ресурсов в производство

/** Рассчитываем учетную себестоимость готовой продукции */
if(!man_00->CalculateItem()) cout << "Ошибка расчета" << endl;
/** Выводим на экран информацию о готовом продукте и незавершенном производстве */
man_00->ViewCalculate(Cur);

delete rcp_00;
delete man_00;
return 0;
}

```

Программа выводит на экран информацию о незавершенном производстве и готовом продукте.

Контроль работы методов GetBalance и GetProductPlan

Незавершенное производство в натуральном, удельном и полном стоимостном выражении

By volume		0	1	2	3	4	5	6	7	8
Name	Measure									
Фрикасе из кури шт.		0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

By price		0	1	2	3	4	5	6	7	8
Name	Measure									
Фрикасе из кури RUR/шт.		0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

By value		0	1	2	3	4	5	6	7	8
Name	Measure									
Фрикасе из кури RUR		0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00	0.00

Готовое производство в натуральном, удельном и полном стоимостном выражении

By volume		0	1	2	3	4	5	6	7	8
Name	Measure									
Фрикасе из кури шт.		0.00	5003.00	8482.00	11825.00	15137.00	18428.00	21706.00	24975.00	28239.00

By price		0	1	2	3	4	5	6	7	8
Name	Measure									
Фрикасе из кури RUR/шт.		0.00	106.16	107.22	108.30	109.38	110.47	111.57	112.69	113.82

By value		0	1	2	3	4	5	6	7	8
Name	Measure									
Фрикасе из кури RUR		0.00	531117.95	909423.86	1280617.56	1655612.98	2035769.43	2421816.87	2814356.88	3214125.87

Process returned 0 (0x0) execution time : 0.144 s
Press any key to continue.

Поскольку длительность технологического цикла равна одному периоду, незавершенное производство отсутствует.

Исходный код программы приведен в [репозитории](#) в папке [froma2/examples/ManufactItem example](#); файл *main.cpp*. Содержание файла может незначительно отличаться от приведенного выше.

КЛАСС clsManufactory

Класс *clsManufactory* предназначен для моделирования учетных процессов на производстве. Экземпляр класса *clsManufactory* описывает полностью работоспособное производство, выпускающее множество номенклатурных позиций готовой продукции и представляет собой *инструмент* моделирования частичных затрат на основе переменных затрат полноценного производства для неограниченного числа продуктов. Класс представляет собой контейнер для нескольких экземпляров класса *clsManufactItem* (однотоварных производств).

Также как для класса *clsManufactItem*, применением класса *clsManufactory* решаются задачи:

1. Рассчитать потребность в разных ресурсах и график обеспечения ими для производства множества номенклатурных позиций готовой продукции;
2. Принять учетные цены всех необходимых производству ресурсов и график их изменения;
3. Рассчитать объем, полную и удельную ресурсную себестоимость незавершенного производства и готовой продукции множества номенклатурных позиций.

ПОЛЯ КЛАССА *clsManufactory*

Таблица 14 Поля класса *clsManufactory*. Секция *Private*

Поле класса	Описание	Заметка
size_t PrCount	Тип <i>size_t</i> . Количество периодов проекта	Значение устанавливается при создании объекта класса. Может корректироваться. В случае отличия одноименных полей элементов контейнера (объектов <i>clsManufactItem</i>), их значения устанавливаются равными данной величине.
size_t RMCount	Тип <i>size_t</i> . Полное количество позиций в номенклатуре ресурсов	Значение устанавливается при создании объекта класса. Может корректироваться.
Currency hmcu	Тип <i>Currency</i> . Домашняя (основная) валюта.	Значение устанавливается при создании объекта класса. По умолчанию принимает значение RUR (индекс 0) и соответствует валюте «Российский рубль».
strNameMeas *RMNames	Тип <i>strNameMeas</i> . Указатель на полный массив с названиями ресурсов и наименованиями их единиц измерения. Размерность <i>RMCount</i>	Этот массив в отличие от массивов <i>clsRecipeItem::*rnames</i> содержит полный список всех позиций сырья и материалов. Соответственно, <i>RMCount</i> >= <i>clsRecipeItem::rcount</i> .
vector <clsManufactItem> Manuf	Тип <i>vector<clsManufactItem></i> . Контейнер для множества экземпляров класса <i>clsManufactItem</i> (однотоварных производств)	Статическая переменная
clsProgress_shell<type_progress>* pshell{nullptr}	Тип указатель на класс <i>clsProgress_shell</i> . Адрес внешнего объекта-оболочки для индикатора прогресса. Тип самого индикатора определяется псевдонимом <i>type_progress</i> .	При создании объекта класса устанавливается значение <i>nullptr</i> : по умолчанию индикация прогресса не осуществляется

Методы класса *clsManufactory* можно условно разделить на следующие группы:

- Конструкторы, деструктор, перегруженные операторы;
- Set-методы;
- Сервисные методы;
- Расчетные методы;
- Get-методы (методы в секции *Private* и методы в секции *Public*);
- Методы сохранения состояния объекта в файл и восстановления из файла;
- Методы визуального контроля

Ниже подробно описываются все методы

КОНСТРУКТОРЫ, ДЕКТРУКТОР, ПЕРЕГРУЖЕННЫЕ ОПЕРАТОРЫ

clsManufactory()

Конструктор по умолчанию (без параметров). Создает и инициализирует переменные: `PrCount = RMCOUNT = 0;`
`hmcure = RUR;` создает на стеке пустой контейнер `Manuf`; Устанавливает указатель `RMNames = nullptr`.

clsManufactory(const size_t _PrCount, const size_t _RMCOUNT, const strNameMeas _RMNames[], const size_t msize)

Конструктор с вводом длительности проекта, массива с наименованиями и единицами измерения ресурсов и резервированием памяти для заданного числа однотоварных производств.

Параметры:

`_PrCount` – количество периодов проекта; устанавливает значение поля `PrCount`; тип `size_t`;

`_RMCOUNT` – количество номенклатурных позиций ресурсов, используемых в производстве; устанавливает значение поля `RMCOUNT`; тип `size_t`;

`_RMNames` – указатель на массив с наименованиями и единицами измерения ресурсов, используемых в производстве; устанавливает значение поля `RMNames`; тип `strNameMeas`;

`msize` – предварительное количество однотоварных производств (объектов типа `clsManufactItem`) в контейнере.

clsManufactory(const size_t _PrCount, const size_t _RMCOUNT, const strNameMeas _RMNames[], const size_t msize, const Currency _cur, const clsRecipItem _Recipies[])

Конструктор с параметрами, позволяющими создать индивидуальные производства с заданными валютой и рецептурами.

Параметры:

`_PrCount` – количество периодов проекта; устанавливает значение поля `PrCount`; тип `size_t`;

`_RMCOUNT` – количество номенклатурных позиций ресурсов, используемых в производстве; устанавливает значение поля `RMCOUNT`; тип `size_t`;

`_RMNames` – указатель на массив с наименованиями и единицами измерения ресурсов, используемых в производстве; устанавливает значение поля `RMNames`; тип `strNameMeas`;

`msize` – предварительное количество однотоварных производств (объектов типа `clsManufactItem`) в контейнере;

`_cur` - валюта проекта; тип `Currency`;

`_Recipies` - указатель на массив с рецептурами продуктов; тип элемента массива `clsRecipItem`.

Наименования ресурсов и рецептуры копируются.

clsManufactory(const size_t _PrCount, const size_t _RMCOUNT, strNameMeas* &_RMNames, const size_t msize, const Currency _cur, clsRecipItem* &_Recipies)

Аналогичен предыдущему конструктору, но наименования ресурсов и рецептуры перемещаются.

clsManufactory(const clsManufactory &obj)

Конструктор копирования.

Параметры:

`&obj` – ссылка на копируемый константный экземпляр класса `clsManufactory`.

clsManufactory(clsManufactory &&obj)

Конструктор перемещения.

Параметры:

&&obj – ссылка на перемещаемый константный экземпляр класса *clsManufactory*.

void swap(clsManufactory& obj) noexcept

Функция обмена значениями между экземплярами класса.

Параметры:

&obj – ссылка на обмениваемый экземпляр класса *clsManufactory*.

clsManufactory& operator=(const clsManufactory &obj)

Перегрузка оператора присваивания копированием.

Параметры:

&obj – ссылка на копируемый константный экземпляр класса *clsManufactory*.

clsManufactory& operator=(clsManufactory&&)

Перегрузка оператора присваивания перемещением

Параметры:

&&obj - перемещаемый экземпляр класса *clsManufactory*.

~clsManufactory()

Деструктор.

SET-МЕТОДЫ.

Все Set-методы класса находятся в секции Public.

void Set_progress_shell(clsProgress_shell<type_progress>* val)

Метод присваивает указателю *pshell* адрес внешнего объекта-оболочки индикатора прогресса. Тип оболочки *clsProgress_shell*, тип индикатора прогресса определяется псевдонимом *type_progress*.

Параметры:

*val – указатель на внешний объект-оболочку для класса индикатора прогресса.

По окончании использования индикатора, необходимо вернуть указатель *pshell* в *nullptr* командой *Set_progress_shell(nullptr)*.

void Set_progress_message(string&& _message)

Метод устанавливает новое значение сообщения во время вывода индикатора. Вызывает метод *clsProgress_shell::SetMessage*.

Параметры:

_message – ссылка на сообщение, являющееся началом индикатора прогресса; тип указатель на *string*.

void Set_progress_maxcount(const int _mx)

Метод устанавливает новое значение максимального числа итераций для индикатора прогресса.

Параметры:

`_mx` – новое значение максимального числа итераций; тип *int*.

void SetCurrency(const Currency &_cur)

Метод устанавливает основную валюту проекта.

Параметры:

`&_cur` – ссылка на основную валюту проекта. Тип [Currency](#).

bool SetManufItem(const string &_name, const string &_measure, const size_t _duration, const size_t _rcount, const strNameMeas _rnames[], const decimal _recipe[], const strItem _pplan[])

Метод создания производства для конкретного продукта. Метод проверяет совпадение имен нового и имеющихся в контейнере продуктов; если наименование продукта уникально (нет совпадений), то создает новый экземпляр класса [clsManufactItem](#) непосредственно в контейнере и возвращает *true*; иначе производство не создается, и метод возвращает *false*. Метод также возвращает *false*, если длительность технологического цикла или количество позиций ресурсов равно нулю или же отсутствуют массивы с наименованиями ресурсов, рецептура/технологическая карта или план отгрузок готовой продукции.

Параметры:

`&_name` – наименование продукта; тип *string*;

`&_measure` – единица измерения натурального объема продукта; тип *string*;

`_duration` – длительность технологического цикла; тип *size_t*;

`_rcount` – количество позиций ресурсов, требуемых для производства продукта; тип *size_t*;

`_rnames[]` – указатель на массив с именами и единицами измерения готового продукта, размерностью

rcount; тип элемента массива [strNameMeas](#);

`_recipe[]` – указатель на одномерный массив с рецептурой, являющийся аналогом двумерной матрицы, размером *rcount*duration*;

`_pplan[]` – указатель на массив с планом отгрузки готового продукта, размером [PrCount](#). Тип элемента массива [strItem](#).

Метод проверяет корректность параметров и вызывает непосредственно в контейнере [Manuf конструктор с параметрами](#) класса *clsManufactItem*. Следом за ним вызывается метод [clsManufactItem::SetProductPlan](#) для ввода плана отгрузки готового продукта.

Метод позволяет создавать производство при отсутствии плана отгрузки `_pplan[] = nullptr`.

bool SetManufItem(const clsRecipeItem &obj, const strItem _pplan[])

Метод имеет назначение, что и предыдущий. В методе также проверяется совпадение имени продукта из передаваемого в качестве параметра объекта типа [clsRecipeItem](#) и имен имеющихся в контейнере продуктов; если наименование продукта уникально (нет совпадений), то метод создает новый экземпляр класса [clsManufactItem](#) непосредственно в контейнере и возвращает *true*; иначе производство не создается, и метод возвращает *false*. Метод вызывает [соответствующий конструктор](#) класса *clsManufactItem* непосредственно в контейнере. Следом за ним вызывается метод [clsManufactItem::SetProductPlan](#) для ввода плана отгрузки готового продукта.

Параметры:

`&obj` – копируемый объект рецептура/технологическая карта типа *clsRecipeItem*;

`_pplan[]` – указатель на массив с планом отгрузки готового продукта, размером *PrCount*. Тип элемента массива *strItem*.

Метод позволяет создавать производство при отсутствии плана отгрузки `_pplan[] = nullptr`.

bool SetManufItem(clsRecipeItem &&obj, const strItem _pplan[])

Метод, аналогичный предыдущему, но объект рецептура/технологическая карта перемещается в конструкторе класса *clsManufactItem*.

`&&obj` – перемещаемый объект рецептура/технологическая карта типа *clsRecipeItem*;

`_pplan[]` – указатель на массив с планом отгрузки готового продукта, размером *PrCount*. Тип элемента массива *strItem*.

Метод позволяет создавать производство при отсутствии плана отгрузки `_pplan[] = nullptr`.

bool SetManufItem(const clsManufactItem &obj)

Метод имеет назначение, что и предыдущие (создание производства для конкретного продукта). В методе также проверяется совпадение имени продукта из передаваемого в качестве параметра объекта типа *clsManufactItem* и имен имеющихся в контейнере продуктов. Если наименование продукта уникально (нет совпадений), то метод создает новый экземпляр класса *clsManufactItem* непосредственно в контейнере и возвращает *true*; иначе производство не создается, и метод возвращает *false*.

Параметры:

`&obj` – копируемый экземпляр класса *clsManufactItem*.

bool SetManufItem(clsManufactItem &&obj)

Аналогичен предыдущему методу, но объект-параметр типа *clsManufactItem* перемещается в контейнер.

Параметры:

`&&obj` – перемещаемый экземпляр класса *clsManufactItem*.

bool SetProdPlan(const strItem _ProdPlan[])

Метод вводит план выпуска всех продуктов в производство. Данные вводятся копированием.

Параметры:

`_ProdPlan` - указатель на полный массив с планом отгрузок *всех* продуктов, являющийся аналогом двумерной матрицы размером *Manuf.size()*PrCount*. Тип элемента массива *strItem*. При благополучном завершении операции возвращает *true*.

bool SetProdPlan(strItem* &_ProdPlan)

Метод вводит план выпуска всех продуктов в производство. Данные вводятся перемещением.

Параметры:

`_ProdPlan` - ссылка на указатель на полный массив с планом выпуска всех продуктов, являющийся аналогом двумерной матрицы размером *Manuf.size()*PrCount*. Тип элемента массива *strItem*. При благополучном завершении операции возвращает *true*.

bool SetRawMatPrice(const decimal _Price[])

Несмотря на формальное отнесение данного метода к Set-методам, он по сути является *вычислительным*. Метод вводит цены на ресурсы в экземпляр класса «производство». Данные вводятся копированием.

Параметры:

`_Price` – указатель на массив с ценами ресурсов; массив одномерный, аналог двумерной матрицы размером `RMCount * PrCount`. Вещественный тип элемента массива определяется псевдонимом `decimal`. При благополучном завершении операции возвращает `true`.

bool SetRawMatPrice(const strItem _Price[])

Аналогичен предыдущему методу, но элементы массива представляют собой структуру `strItem`, из которой используются только поля `price`.

bool SetRawMatPrice(strItem* &_Price)

Аналогичен предыдущему методу, но после копирования исходный массив уничтожается, а указатель `_Price` становится равным `nullptr`. (Данные во внутренние массивы не перемещаются, т.к. массив в параметрах метода и внутренние массивы имеют разный тип: `strItem` и `decimal` соответственно.)

Параметры:

`_Price` – ссылка на указатель на массив с ценами ресурсов; массив одномерный, аналог двумерной матрицы размером `RMCount * PrCount`. Вещественный тип элемента массива определяется псевдонимом `decimal`. При благополучном завершении операции возвращает `true`.

СЕРВИСНЫЕ МЕТОДЫ

bool IncreaseCapacity(size_t msize)

Новые производства добавляются в контейнер без ограничений по количеству. Однако, чем больше элементов уже содержит контейнер, тем больше ресурсов и времени затрачивается на добавление новых. Это связано с особенностью контейнеров типа `vector`. Для повышения эффективности работы класса `clsManufactory` в [конструкторе с параметрами](#) предусмотрена возможность резервирования числа однотоварных производств. Это наилучшее с точки зрения эффективности решение. Другой возможностью является возможность увеличить «количество вакантных мест» для новых производств, если экземпляр класса `clsManufactory` создавался без резервирования.

Данный метод увеличивает память вектора для эффективного добавления новых элементов. Однако сам метод вызывает копирование и удаление всех элементов вектора, что весьма затратно. Лучше использовать этот метод в самом начале создания мульти-товарного производства и потом не менять размер вектора.

Параметры:

`msize` – новый размер резервируемой памяти контейнера (новое число элементов типа `clsManufactItem` в контейнере: занятых и вакантных в сумме).

ВЫЧИСЛИТЕЛЬНЫЕ МЕТОДЫ

void CalcRawMatPurchPlan(size_t bg, size_t en)

Метод рассчитывает объем потребления ресурсов в натуральном выражении в соответствии с планом выпуска для каждого продукта (каждого однотоварного производства из контейнера) в диапазоне: от продукта с индексом `bg` до продукта с индексом `(en-1)`.

Параметры:

`Bg` – нижняя граница диапазона продуктов (равен начальному индексу элемента контейнера из диапазона); тип `size_t`;

En – верхняя граница диапазона продуктов (равен следующему за конечным индексом диапазона индексу элемента контейнера); *size_t*;

Метод поочередно вызывает методы [clsManufactItem::CalcRawMatPurchPlan](#) у каждого элемента контейнера в диапазоне индексов от *bg* до (*en-1*) включительно и формирует массивы [clsManufactItem::RawMatPurchPlan](#).

void CalcRawMatPurchPlan()

Метод, аналогичный предыдущему, обрабатывает *все* элементы контейнера. Аналогичен методу [CalcRawMatPurchPlan\(size_t bg, size_t en\)](#) с параметрами *bg* = 0, *en* = [Manuf.size\(\)-1](#)).

void CalcRawMatPurchPlan_future()

Метод является альтернативой методу [CalcRawMatPurchPlan](#). В методе [CalcRawMatPurchPlan_future](#) расчеты производятся в параллельных потоках, число которых на единицу меньше числа ядер используемого компьютера, и число обрабатываемых элементов контейнера в каждом потоке примерно одинаково. В каждом потоке вызывается метод [CalcRawMatPurchPlan\(size_t bg, size_t en\)](#) с границами диапазона, предназначенного для обработки в этом потоке. В каждом отдельном потоке расчеты производятся последовательно, но сами потоки выполняются параллельно. Это позволяет существенно ускорить вычисления. Для организации потоков используются инструменты из стандартного заголовочного файла [<future>](#).

void CalcRawMatPurchPlan_thread()

Этот метод идентичен по назначению и своей структуре предыдущему методу. Отличие в том, что в нем используются инструменты из стандартного заголовочного файла [<thread>](#).

Выбор метода из приведенных выше ([CalcRawMatPurchPlan](#), [CalcRawMatPurchPlan_future](#) или [CalcRawMatPurchPlan_thread](#)) остается за пользователем. Рекомендации те же, что и при [выборе методов расчета](#) в классе [clsStorage](#).

bool Calculate(size_t bg, size_t en)

Метод используется после ввода в экземпляр класса [clsManufactory](#) данных о ценах на ресурсы, которые используются в производстве продуктов (метод [SetRawMatPrice](#)) и рассчитывает объем, удельную и полную себестоимость незавершенного производства и готовой продукции для продуктов с индексами в диапазоне от *bg* до *en-1*, выпускаемых на протяжении всего проекта. Метод поочередно вызывает методы [clsManufactItem::CalculateItem](#) для каждого единичного производства; формирует массивы продуктов и баланса незавершенного производства для каждого продукта из заданного диапазона. Метод возвращает *true*, если в контейнере есть элементы и вычисления во всех элементах диапазона вернули *true*; иначе метод возвращает *false*.

Параметры:

Bg – нижняя граница диапазона продуктов (равен начальному индексу элемента контейнера из диапазона); тип *size_t*;

En – верхняя граница диапазона продуктов (равен следующему за конечным индексом диапазона индексу элемента контейнера); *size_t*;

bool Calculate()

Метод, аналогичный предыдущему, обрабатывает *все* элементы контейнера. Аналогичен методу [Calculate\(size_t bg, size_t en\)](#) с параметрами *bg* = 0, *en* = [Manuf.size\(\)-1](#)). Метод возвращает *true*, если в контейнере есть элементы и вычисления во всех элементах контейнера вернули *true*; иначе метод возвращает *false*.

bool Calculate_future()

Метод является альтернативой методу [Calculate](#). В методе [Calculate_future](#) расчеты производятся в параллельных потоках, число которых на единицу меньше числа ядер используемого компьютера, и число обрабатываемых элементов контейнера в каждом потоке примерно одинаково. В каждом потоке вызывается метод [Calculate\(size t bg, size t en\)](#) с границами диапазона, предназначенного для обработки в этом потоке. В каждом отдельном потоке расчеты производятся последовательно, но сами потоки выполняются параллельно. Это позволяет существенно ускорить вычисления. Для организации потоков используются инструменты из стандартного заголовочного файла [<future>](#). Метод возвращает *true*, если в контейнере есть элементы и вычисления во всех элементах контейнера вернули *true*; иначе метод возвращает *false*.

bool Calculate_thread()

Этот метод идентичен по назначению и своей структуре предыдущему методу. Отличие в том, что в нем используются инструменты из стандартного заголовочного файла [<thread>](#). Метод также возвращает *true*, если в контейнере есть элементы и вычисления во всех элементах контейнера вернули *true*; иначе метод возвращает *false*.

Выбор метода из приведенных выше ([Calculate](#), [Calculate_future](#) или [Calculate_thread](#)) остается за пользователем. Рекомендации те же, что и при [выборе методов расчета](#) в классе [clsStorage](#).

В таблице ниже (Таблица 15) приведено среднее время вычислений для разных расчетных методов. Комбинации методов следующие:

- [Calculate](#) + [CalcRawMatPurchPlan](#);
- [Calculate_future](#) + [CalcRawMatPurchPlan_future](#);
- [Calculate_thread](#) + [CalcRawMatPurchPlan_thread](#).

Для каждой из приведенных выше комбинаций было сделано не менее 50 вычислений. В качестве вещественного числа использовался собственный тип [LongReal](#) с количеством десятичных знаков мантиисы 64. Было установлено 480 периодов проекта. Расчет велся по 100 номенклатурным позициям продукции и 100 номенклатурным позициям ресурсов. Вычисления проводились на ноутбуке ASUS UX303U со следующими характеристиками: Процессор Intel Core i3-6100U CPU @ 2.30 GHz; Оперативная память 4,00 Gb; 64-bit Operation System Windows 10 Home Single Language, x64-based processor. Расчеты проводились в трех параллельных потоках. Исходный код программы для тестирования производительности приведен в [репозитории](#) в папке [froma2/examples/Manufactory_stress_test](#); файл *main.cpp*.

Таблица 15 Сравнение времени вычислений для разных методов класса [clsManufactory](#)

Названия строк	Среднее время выполнения, сек.	Станд. отклонение времени выполнения, сек.
Calculate + CalcRawMatPurchPlan	52,8318	0,6103
Calculate_future + CalcRawMatPurchPlan_future	27,3464	0,7651
Calculate_thread + CalcRawMatPurchPlan_thread	27,4714	0,4999

GET-МЕТОДЫ. СЕКЦИЯ PRIVATE

strItem* gettotal(const strItem* (clsManufactItem::*f)() const) const

Метод создает динамический массив и возвращает на него указатель. Созданный одномерный массив, является аналогом двумерной матрицы с *балансами незавершенного производства* (при подстановке вместо *f* метода [clsManufactItem::GetBalance](#)) или *планами выхода всех продуктов* (при подстановке вместо *f* метода [clsManufactItem::GetProductPlan](#)) в натуральном, удельном и полном стоимостном выражении. Размер матрицы [Manuf.size\(\)*PrCount](#). Каждый элемент матрицы имеет тип [strItem](#). Используется в методах класса [clsManufactory](#): [GetTotalBalance](#) и [GetTotalProduct](#).

Параметры:

const strItem (clsManufactItem::*f)() const* – указатель на один из методов класса [clsManufactItem](#): [GetBalance](#) – функция возврата константного указателя на массив [Balance](#) с объемом, удельной и полной себестоимостью

незавершенного производства для конкретного продукта, *GetProductPlan* – функция возврата константного указателя на массив *ProductPlan* с объемом, удельной и полной себестоимостью готовой продукции для конкретного продукта.

GET-МЕТОДЫ. СЕКЦИЯ PUBLIC

const size_t GetRMCount() const

Метод возвращает общее число позиций ресурсов, участвующих в производстве. Тип *size_t*.

const size_t GetPrCount() const

Метод возвращает число периодов проекта. Тип *size_t*.

const size_t GetProdCount() const

Метод возвращает число производимых продуктов. Тип *size_t*.

const string GetCurrency() const

Метод возвращает основную валюту проекта в виде текстовой строки. Тип *string*.

const size_t GetHomeCurrency() const

Метод возвращает основную валюту проекта в виде значения типа *size_t*, совпадающим со значением индекса из типа данных перечисления *Currency*.

strNameMeas *GetProductDescription() const

Метод возвращает указатель на вновь создаваемый массив с именами и единицами измерения всех продуктов. Размер массива равен *Manuf.size()*. Тип элемента массива *strNameMeas*.

const strNameMeas *GetRMNames() const

Метод возвращает константный указатель на внутренний массив *RMNames* с именами и единицами измерения всех ресурсов. Размер массива равен *RMCount*. Тип элемента массива *strNameMeas*. Массив доступен только на чтение.

strNameMeas *GetRawMatDescription() const

Метод возвращает указатель на вновь создаваемый массив с именами и единицами измерения всех ресурсов. Размер массива равен *RMCount*. Тип элемента массива *strNameMeas*.

decimal *GetRawMatPurchPlan() const

Несмотря на формальное отнесение данного метода к Get-методам, он по сути является *вычислительным*. Метод вызывается *после* проведения расчетов потребности в ресурсах методами *CalcRawMatPurchPlan*, *CalcRawMatPurchPlan_future* или *CalcRawMatPurchPlan_thread* и возвращает указатель на вновь создаваемый одномерный массив, являющийся аналогом двумерной матрицы размером *RMCount*PrCount*, в котором содержится потребность для каждого наименования ресурса в каждом периоде проекта. Потребность в ресурсах суммарная, по всем однотоварным производствам. Тип вещественного числа элемента массива определяется псевдонимом *decimal*.

strItem *GetRMPurchPlan() const

Данный метод возвращает тот же результат, что и предыдущий метод, но тип элементов массива *strItem*. В каждом элементе массива заполнены только поля *volume*, поля *price* и *value* нулевые.

strItem *GetTotalBalance() const

Метод возвращает указатель на вновь создаваемый одномерный массив, являющийся аналогом двумерной матрицы с балансами незавершенного производства для всех продуктов. Размер матрицы `Manuf.size()*PrCount`. Каждый элемент матрицы имеет тип `strItem`, т.е. имеет в своем составе значения *volume*, *price* и *value* для незавершенного производства.

strItem *GetTotalProduct() const

Метод возвращает указатель на вновь создаваемый одномерный массив, являющийся аналогом двумерной матрицы с планами выхода всех продуктов в натуральном, удельном и полном стоимостном выражении. Размер матрицы `Manuf.size()*PrCount`. Каждый элемент матрицы имеет тип `strItem`, т.е. имеет в своем составе значения *volume*, *price* и *value* для готового продукта.

const decimal* GetRecipeItem(const string& Name) const

Метод возвращает константный указатель на внутренний массив с рецептурами `clsManufactItem::clsRecipeItem::recipeitem` для производства продукта с именем *Name*. Тип вещественного числа элемента массива определяется псевдонимом `decimal`. Массив доступен только на чтение.

Параметры:

&Name – ссылка на строку с названием продукта. Тип *string*.

const decimal* GetRecipeItem(const size_t _ind) const

Перегруженный предыдущий метод; возвращает константный указатель на внутренний массив с рецептурами `clsManufactItem::clsRecipeItem::recipeitem` для производства продукта с индексом *_ind*. Тип вещественного числа элемента массива определяется псевдонимом `decimal`. Массив доступен только на чтение.

Параметры:

_ind – индекс элемента в контейнере `Manuf`. Тип *size_t*.

const strNameMeas* GetRawNamesItem(const string& Name) const

Метод возвращает константный указатель на внутренний массив с наименованиями ресурсов и единицами их измерения `clsManufactItem::clsRecipeItem::rnames` для производства продукта с именем *Name*. Тип элементов массива `strNameMeas`. Массив доступен только на чтение.

Параметры:

&Name – ссылка на строку с названием продукта. Тип *string*.

const strNameMeas* GetRawNamesItem(const size_t _ind) const

Перегруженный предыдущий метод; возвращает константный указатель на внутренний массив с наименованиями ресурсов и единицами их измерения `clsManufactItem::clsRecipeItem::rnames` для производства продукта с индексом *_ind*. Тип элементов массива `strNameMeas`. Массив доступен только на чтение.

Параметры:

_ind – индекс элемента в контейнере `Manuf`. Тип *size_t*.

const size_t GetDuration(const string& Name) const

Метод возвращает длительность производственного цикла в рецептуре продукта с именем *Name*. Тип возвращаемого значения *size_t*.

Параметры:

&Name – ссылка на строку с названием продукта. Тип *string*.

const size_t GetDuration(const size_t _ind) const

Перегруженный предыдущий метод; возвращает длительность производственного цикла в рецептуре продукта с индексом *_ind*. Тип возвращаемого значения *size_t*.

Параметры:

_ind – индекс элемента в контейнере [Manuf](#). Тип *size_t*.

const size_t GetRCount(const string& Name) const

Метод возвращает число позиций ресурсов, участвующих в рецептуре продукта с именем *Name* (размер массива с наименованиями сырья, возвращаемого функцией [GetRawNamesItem](#)). Тип возвращаемого значения *size_t*.

Параметры:

&Name – ссылка на строку с названием продукта. Тип *string*.

const size_t GetRCount(const size_t _ind) const

Перегруженный предыдущий метод; возвращает число позиций ресурсов, участвующих в рецептуре продукта с индексом *_ind* (размер массива с наименованиями сырья, возвращаемого функцией [GetRawNamesItem](#)). Тип возвращаемого значения *size_t*.

Параметры:

_ind – индекс элемента в контейнере [Manuf](#). Тип *size_t*.

const strNameMeas GetNameItem(const size_t _ind) const

Метод возвращает наименование продукта и единицу его измерения для продукта с индексом *_ind*. Тип возвращаемого значения [strNameMeas](#).

Параметры:

_ind – индекс элемента в контейнере [Manuf](#). Тип *size_t*.

МЕТОДЫ СОХРАНЕНИЯ СОСТОЯНИЯ ОБЪЕКТА В ФАЙЛ И ВОССТАНОВЛЕНИЯ ИЗ ФАЙЛА**bool StF(ofstream &_outF)**

Метод имплементации записи в файловый поток текущего состояния экземпляра класса (запись в файловый поток, метод сериализации). Название метода является аббревиатурой, расшифровывается “StF” как “Save to File”. При удачной сериализации возвращает *true*, при ошибке записи возвращает *false*.

Параметры:

&_outF - экземпляр класса [ofstream](#) для записи данных. При инициализации переменной *_outF* необходимо установить флаг бинарного режима открытия потока (не текстового!) [ios::binary](#).

bool RfF(ifstream &_inF)

Метод имплементации чтения из файлового потока текущего состояния экземпляра класса (чтение из файлового потока, метод десериализации). Название метода является аббревиатурой, расшифровывается “RfF” как “Read from File”. При удачной десериализации возвращает *true*, при ошибке чтения возвращает *false*.

Параметры:

&_inF - экземпляр класса [ifstream](#) для чтения данных. При инициализации переменной *_inF* необходимо установить флаг бинарного режима открытия потока (не текстового!) [ios::binary](#).

bool SaveToFile(const string _filename)

Метод записи текущего состояния экземпляра класса в файл. При удачной записи в файл, возвращает *true*, при ошибке записи возвращает *false*.

Параметры:

_filename – имя файла; тип *string*.

bool ReadFromFile(const string _filename)

Метод восстановления состояния экземпляра класса из файла. При удачном чтении из файла, возвращает *true*, при ошибке чтения возвращает *false*.

Параметры:

_filename – имя файла; тип *string*.

МЕТОДЫ ВИЗУАЛЬНОГО КОНТРОЛЯ
void ViewProjectParametrs() const

Метод выводит в стандартный поток [std::cout](#) основные параметры класса: *длительность* проекта (контроль работоспособности метода [GetPrCount](#)), *количество* номенклатурных позиций *готовой продукции* (контроль работоспособности метода [GetProdCount](#)) и *количество* номенклатурных позиций *ресурсов* (контроль работоспособности метода [GetRMCount](#)).

void ViewRawMatPurchPlan() const

Метод выводит в стандартный поток [std::cout](#) потребность в ресурсах для всех продуктов в динамике (контроль работоспособности метода [GetRawMatPurchPlan](#)).

void ViewRawMatPrice() const

Метод выводит в стандартный поток [std::cout](#) цены на ресурсы (контроль работоспособности метода [SetRawMatPrice](#)).

void ViewCalculate() const

Метод выводит в стандартный поток [std::cout](#) *незавершенное производство* в *натуральном, удельном и полном стоимостном* выражении, а также *готовую продукцию* в *натуральном, удельном и полном стоимостном* выражении для всех продуктов. Для каждого элемента контейнера метод вызывает его собственную функцию [clsManufactItem::ViewCalculate](#). Для каждого продукта метод выводит шесть однострочных таблиц:

- незавершенное производство в натуральном выражении;
- незавершенное производство в удельном стоимостном выражении;
- незавершенное производство в полном стоимостном выражении;
- готовая продукция в натуральном выражении;
- готовая продукция в удельном стоимостном выражении;
- готовая продукция в полном стоимостном выражении.

Количество выводимых таблиц равно числу продуктов умноженному на шесть.

void ViewBalance() const

Метод выводит в стандартный поток [std::cout](#) *незавершенное производство* в *натуральном, удельном и полном стоимостном* выражении для всех продуктов (контроль работоспособности методов [GetProductDescription](#) и [GetTotalBalance](#)). Метод выводит три таблицы:

- незавершенное производство в натуральном выражении для всех продуктов;
- незавершенное производство в удельном стоимостном выражении для всех продуктов;
- незавершенное производство в полном стоимостном выражении для всех продуктов.

void ViewProductPlan() const

Метод выводит в стандартный поток `std::cout` *готовую продукцию в натуральном, удельном и полном стоимостном* выражении для всех продуктов (контроль работоспособности методов [GetProductDescription](#) и [GetTotalProduct](#)). Метод выводит три таблицы:

- готовая продукция в натуральном выражении для всех продуктов;
- готовая продукция в удельном стоимостном выражении для всех продуктов;
- готовая продукция в полном стоимостном выражении для всех продуктов.

void ViewRecipes() const

Метод поочередно выводит в стандартный поток `std::cout` рецептуры всех продуктов. Для каждого элемента контейнера метод вызывает его собственную функцию [clsManufactItem::ViewRefRecipe](#).

ПРИМЕР ПРОГРАММЫ С КЛАССОМ `clsManufactory`

Ниже представлен листинг программы, демонстрирующей работу класса `clsManufactory`. Исходный код программы приведен в [репозитории](#) в папке [froma2/examples/Manufactory_example](#); файл `main.cpp`. Содержание файла может незначительно отличаться от приведенного ниже.

```

#include <iostream>
#include <manufact_module.h>    // Подключаем модуль "производство"

using namespace std;

int main() {

    setlocale(LC_ALL, "Russian");    // Установка русского языка для вывода

    /** Основные параметры проекта **/
    size_t PrCount = 8;    // Длительность проекта
    size_t RMCCount = 10;    // Полное число наименований ресурсов
    size_t MSize = 5;    // Число позиций в ассортименте готовой продукции
    size_t rcount = 5;    // Число наименований ресурсов в рецептуре продукта
    size_t duratio = 7;    // Длительность производственного цикла

    /** Формирование полного массива имен ресурсов с единицами измерения **/
    strNameMeas* RMNames = new strNameMeas[RMCCount];
    for(size_t i = sZero; i < RMCCount; i++) {
        (RMNames+i)->name = "Rawmat_" + to_string(i);
        (RMNames+i)->measure = "kg";
    };

    /** Формирование массива цен (удельной себестоимости) для ресурсов **/
    decimal* RMPrice = new decimal[RMCCount*PrCount];
    for(size_t i = sZero; i < RMCCount; i += 5) {
        for(size_t j = sZero; j < PrCount; j++) {
            *(RMPrice + PrCount*i + j) = 10.0;
            *(RMPrice + PrCount*(i+1) + j) = 5.0;
            *(RMPrice + PrCount*(i+2) + j) = 20.0;
            *(RMPrice + PrCount*(i+3) + j) = 2.5;
            *(RMPrice + PrCount*(i+4) + j) = 50.0;
        };
    };

    /** Рецепттура для единичного продукта **/
    strNameMeas* rnames = new strNameMeas[rcount] { {"Rawmat_0", "kg"}, {"Rawmat_1", "kg"},
    {"Rawmat_2", "kg"}, {"Rawmat_3", "kg"}, {"Rawmat_4", "kg"} };    // Номенклатура ресурсов
    decimal* recipe = new decimal[rcount*duratio]{
        10.0, 10.0, 10.0, 10.0, 10.0, 10.0, 10.0, 10.0,    // Рецепттура
        50.0, 0.0, 0.0, 50.0, 0.0, 0.0, 0.0, 0.0,
        30.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0,
        0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 20.0,
        0.0, 0.0, 0.0, 10.0, 0.0, 0.0, 0.0 };

    /** Формирование плана выпуска продуктов **/
    strItem* ProPlan = new strItem[PrCount];
    for(size_t i = sZero; i < PrCount; i++) {
        (ProPlan+i)->volume = 10.0;
        (ProPlan+i)->value = (ProPlan+i)->price = 0.0;
    };

    /** Формирование списка продуктов **/
    strNameMeas* ProdNames = new strNameMeas[MSize];
    for(size_t i = sZero; i < MSize; i++) {
        (ProdNames+i)->name = "Product_" + to_string(i);
        (ProdNames+i)->measure = "kg";
    };

    /** Создание производства **/
    clsManufactory* MMM = new clsManufactory(PrCount, RMCCount, RMNames, MSize);

```



```

/** Ввод продуктов, рецептов и плана выпуска продуктов */
for(size_t i=sZero; i<MSize; i++)
    if(!MMM->SetManufItem((ProdNames+i)->name,\
        (ProdNames+i)->measure, duratio, rcount,\
        rnames, recipe, ProPlan))
        cout << "Ошибка добавления производства для "\
        << (ProdNames+i)->name << " продукта" << endl;

/** Расчет потребности в сырье и материалах */
MMM->CalcRawMatPurchPlan();

/** Ввод цен сырья и материалов */
MMM->SetRawMatPrice(RMPrice);

/** Расчет себестоимости продуктов и незавершенного производства */
MMM->Calculate();

/** Выборочный визуальный контроль работоспособности */
MMM->ViewProjectParameters();
MMM->ViewBalance();
MMM->ViewProductPlan();

```

Вывод на экран выбранной информации представлен ниже.

```

*** Основные параметры проекта ***
Количество периодов проекта      8
Общее число позиций продуктов     5
Общее число позиций сырья и материалов 10
Валюта проекта                    RUR

```

*** Незавершенное производство ***

By volume		0	1	2	3	4	5	6	7
Name	Measure								
Product_0	kg	0.00	10.00	10.00	10.00	10.00	10.00	10.00	0.00
Product_1	kg	0.00	10.00	10.00	10.00	10.00	10.00	10.00	0.00
Product_2	kg	0.00	10.00	10.00	10.00	10.00	10.00	10.00	0.00
Product_3	kg	0.00	10.00	10.00	10.00	10.00	10.00	10.00	0.00
Product_4	kg	0.00	10.00	10.00	10.00	10.00	10.00	10.00	0.00

By price		0	1	2	3	4	5	6	7
Name	Measure								
Product_0	RUR/kg	0.00	950.00	1050.00	1150.00	2000.00	2100.00	2200.00	0.00
Product_1	RUR/kg	0.00	950.00	1050.00	1150.00	2000.00	2100.00	2200.00	0.00
Product_2	RUR/kg	0.00	950.00	1050.00	1150.00	2000.00	2100.00	2200.00	0.00
Product_3	RUR/kg	0.00	950.00	1050.00	1150.00	2000.00	2100.00	2200.00	0.00
Product_4	RUR/kg	0.00	950.00	1050.00	1150.00	2000.00	2100.00	2200.00	0.00

By value		0	1	2	3	4	5	6	7
Name	Measure								
Product_0	RUR	0.00	9500.00	10500.00	11500.00	20000.00	21000.00	22000.00	0.00
Product_1	RUR	0.00	9500.00	10500.00	11500.00	20000.00	21000.00	22000.00	0.00
Product_2	RUR	0.00	9500.00	10500.00	11500.00	20000.00	21000.00	22000.00	0.00
Product_3	RUR	0.00	9500.00	10500.00	11500.00	20000.00	21000.00	22000.00	0.00
Product_4	RUR	0.00	9500.00	10500.00	11500.00	20000.00	21000.00	22000.00	0.00

*** Готовая продукция ***

By volume		0	1	2	3	4	5	6	7
Name	Measure								
Product_0	kg	0.00	0.00	0.00	0.00	0.00	0.00	0.00	10.00
Product_1	kg	0.00	0.00	0.00	0.00	0.00	0.00	0.00	10.00
Product_2	kg	0.00	0.00	0.00	0.00	0.00	0.00	0.00	10.00
Product_3	kg	0.00	0.00	0.00	0.00	0.00	0.00	0.00	10.00
Product_4	kg	0.00	0.00	0.00	0.00	0.00	0.00	0.00	10.00

By price		0	1	2	3	4	5	6	7
Name	Measure								
Product_0	RUR/kg	0.00	0.00	0.00	0.00	0.00	0.00	0.00	2350.00
Product_1	RUR/kg	0.00	0.00	0.00	0.00	0.00	0.00	0.00	2350.00
Product_2	RUR/kg	0.00	0.00	0.00	0.00	0.00	0.00	0.00	2350.00
Product_3	RUR/kg	0.00	0.00	0.00	0.00	0.00	0.00	0.00	2350.00
Product_4	RUR/kg	0.00	0.00	0.00	0.00	0.00	0.00	0.00	2350.00

By value		0	1	2	3	4	5	6	7
Name	Measure								
Product_0	RUR	0.00	0.00	0.00	0.00	0.00	0.00	0.00	23500.00
Product_1	RUR	0.00	0.00	0.00	0.00	0.00	0.00	0.00	23500.00
Product_2	RUR	0.00	0.00	0.00	0.00	0.00	0.00	0.00	23500.00
Product_3	RUR	0.00	0.00	0.00	0.00	0.00	0.00	0.00	23500.00
Product_4	RUR	0.00	0.00	0.00	0.00	0.00	0.00	0.00	23500.00

КАК ПОЛЬЗОВАТЬСЯ КЛАССАМИ МОДУЛЯ MANUFACT

Классы *clsRecipelItem*, *clsManufactItem* и *clsManufactory* предназначены для описания учетных процессов в производстве¹⁵ при моделировании *центра учета затрат* (*Cost Centre*). Объединяющим классом является класс *clsManufactory*. Основная задача классов - расчет ресурсной себестоимости готовой продукции/услуг и незавершенного производства.

Общая логика работы классов представлена на рисунке ниже (Рисунок 5):

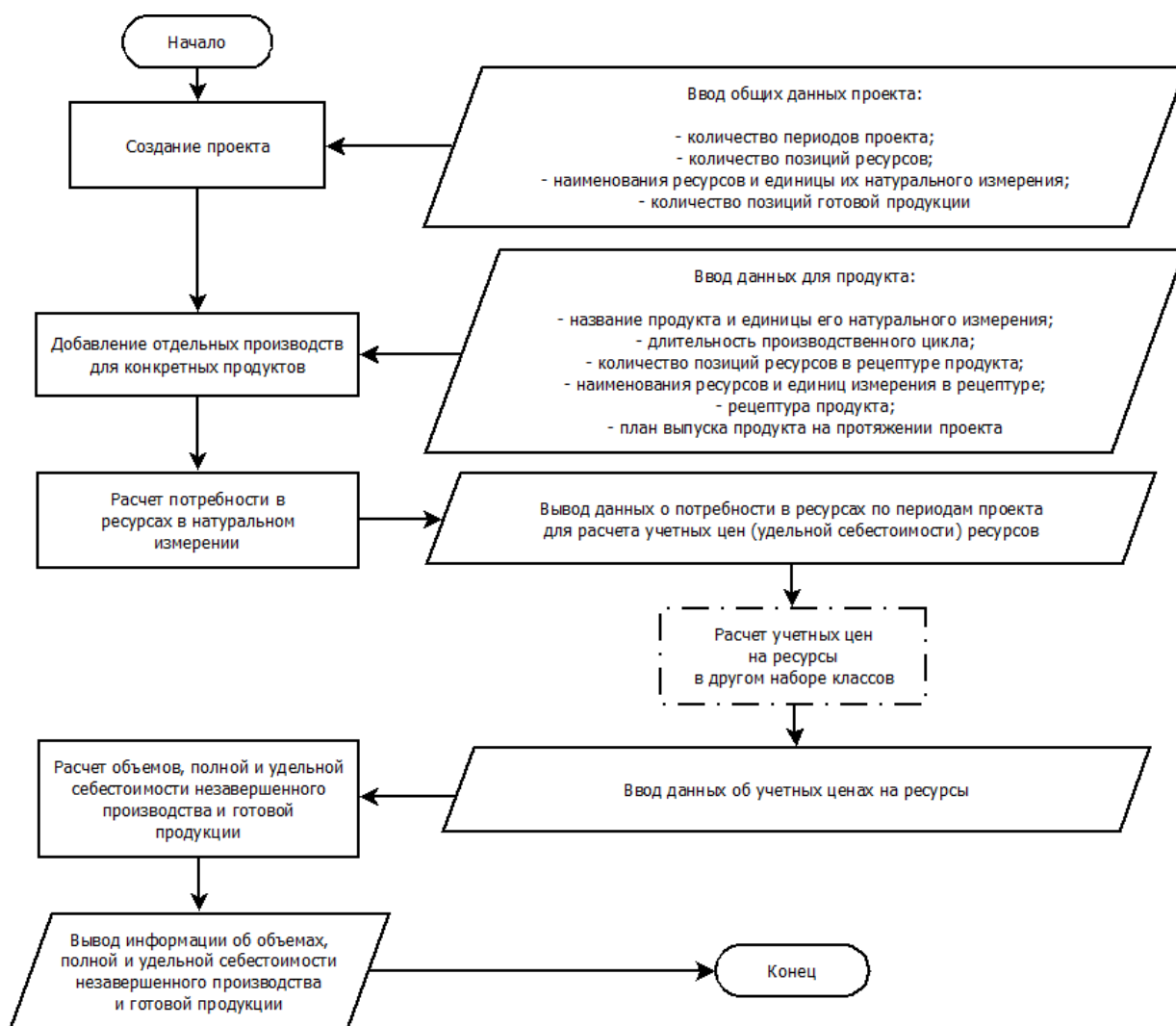


Рисунок 5 Принципиальная схема взаимодействия классов модуля manufact

Роли классов в этом процессе следующие:

- класс **clsRecipelItem** - ядро всего алгоритма, содержит описание рецептуры/ технологической карты одной номенклатурной единицы (*SKU*) продукции и методы расчета потребности в ресурсах для производства этого продукта, методы расчета объемов, полной и удельной себестоимости незавершенного производства и готового продукта;

¹⁵ Термин «производство» будем понимать широко, как логически обособленную совокупность процессов создания добавленной стоимости.

- класс **clsManufactItem** - контейнер для экземпляра класса *clsRecipeItem* и хранилище данных о количестве периодов проекта, плане выпуска одного вида продукта, потребности в ресурсах для производства этого продукта, учетных цен на ресурсы для его производства, объемов, полной и удельной себестоимости незавершенного производства и готового продукта; этот класс представляет собой полноценное производство для одного вида продукта; Set-методы вводят данных во внутренние поля, Calc-методы обращаются к Calc-методам внутреннего объекта типа *clsRecipeItem* и заполняют поля расчетными данными, Get-методы возвращают данные из внутренних полей;
- класс **clsManufactory** - контейнер для нескольких экземпляров класса *clsManufactItem*, - аналог полноценного производства ассортимента продукции и хранилище данных о количестве периодов проекта, полном количестве позиций ресурсов, используемых в производстве, наименованиях и единицах измерения полного списка ресурсов; Set-методы класса позволяют добавить в контейнер производство какого-либо конкретного продукта, учетные цены на все ресурсы; Calc_методы обращаются к Calc-методам отдельных производств (объектам типа *clsManufactItem*), Get-методы возвращают объединенную информацию.

Все классы имеют в своем составе методы сериализации и десериализации и методы визуального контроля.

Схема вложенности классов и отношений между ними приведена на рисунке ниже (Рисунок 6).

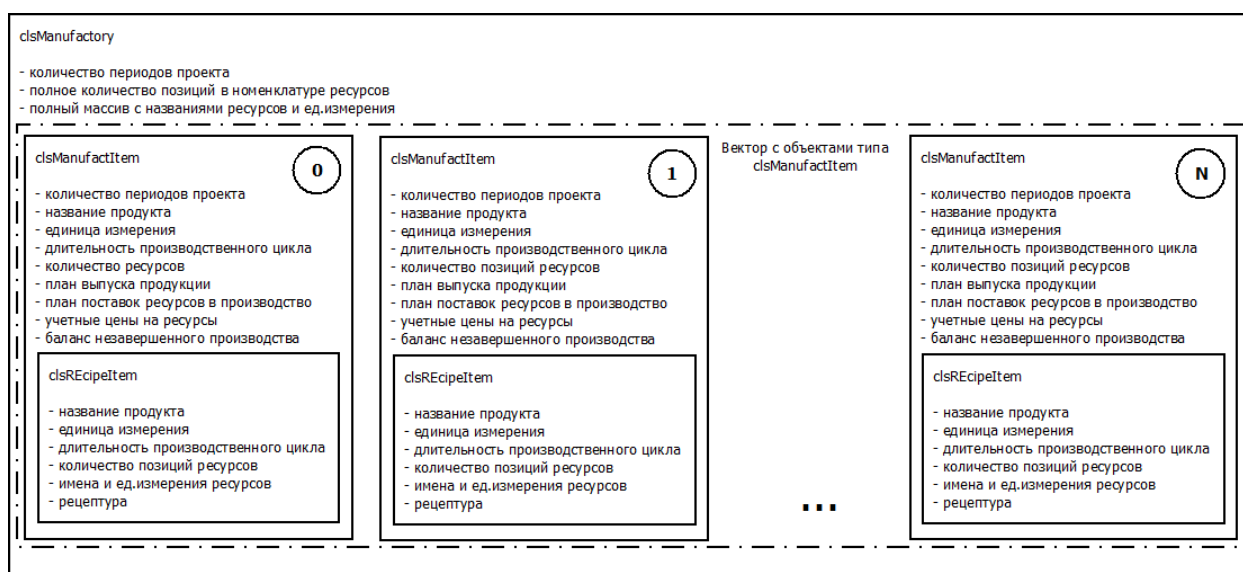


Рисунок 6 Схема отношений между экземплярами классов модуля manufact

СОЗДАНИЕ ПРОЕКТА

Создание проекта начинается с создания экземпляра класса *clsManufactory*. Для создания используется, например, [конструктор](#) с вводом длительности проекта, массива с наименованиями и единицами измерения ресурсов и резервированием памяти для заданного числа одотоварных производств.

На этом этапе вводятся исходные данные:

- количество периодов проекта;
- полное количество позиций в номенклатуре сырья и материалов;

- наименования и единицы измерения номенклатурных позиций сырья и материалов в виде массива;
- предварительное количество позиций в номенклатуре готовой продукции.

Созданный проект пока пуст и не имеет в своем составе ни одного производства. Это лишь [контейнер](#), в который могут быть добавлены отдельные производства, - экземпляры класса [clsManufactItem](#), - для конкретных наименований продукции.

ДОБАВЛЕНИЕ ПРОИЗВОДСТВА ДЛЯ КОНКРЕТНОГО ПРОДУКТА

При добавлении производства для конкретного продукта требуется информация:

- наименование продукта;
- единица измерения его натурального количества;
- длительность производственного цикла;
- число номенклатурных позиций сырья и материалов, участвующих в производстве данного продукта;
- наименования сырья и материалов и единицы измерения этих позиций сырья и материалов; при этом эти наименования и единицы измерения должны быть подмножеством введенного на этапе создания проекта множества имен и единиц измерения для сырья и материалов;
- рецептура продукта;
- план выпуска продукта на протяжении проекта.

В результате добавления нового производства в контейнере появляется новый элемент типа [clsManufactItem](#). Добавление производства для конкретного продукта осуществляется с помощью метода [clsManufactory::SetManufItem](#). Существуют несколько вариантов этого метода:

- с *копированием* ранее созданного экземпляра класса [clsManufactItem](#) в новый элемент контейнера;
- с *перемещением* ранее созданного экземпляра класса [clsManufactItem](#) в новый элемент контейнера;
- с *копированием* ранее созданного экземпляра класса [clsRecipeItem](#) и копированием массива с планом выпуска продукта в новый элемент контейнера;
- с *перемещением* ранее созданного экземпляра класса [clsRecipeItem](#) и копированием массива с планом выпуска продукта в новый элемент контейнера;
- с *вводом параметров*, указанных в предыдущем абзаце и непосредственным созданием экземпляра класса [clsManufactItem](#) в новом элементе вектора.

РАСЧЕТ ПОТРЕБНОСТИ В СЫРЬЕ И МАТЕРИАЛАХ ДЛЯ ПРОИЗВОДСТВА ПРОДУКЦИИ

Следующим этапом после ввода всех планируемых к производству продуктов, является расчет потребности в ресурсах в натуральном измерении для всех продуктов. Расчет осуществляется с помощью одного из трех методов на выбор: [clsManufactory::CalcRawMatPurchPlan](#), [clsManufactory::CalcRawMatPurchPlan_future](#) или [clsManufactory::CalcRawMatPurchPlan_thread](#). Каждый из этих методов вызывает поочередно одноименные методы для каждого элемента контейнера типа [clsManufactItem](#), которые в свою очередь, вызывают методы [clsRecipeItem::CalcRawMatVolume](#) экземпляра класса рецептур каждого продукта (см. Рисунок 7).

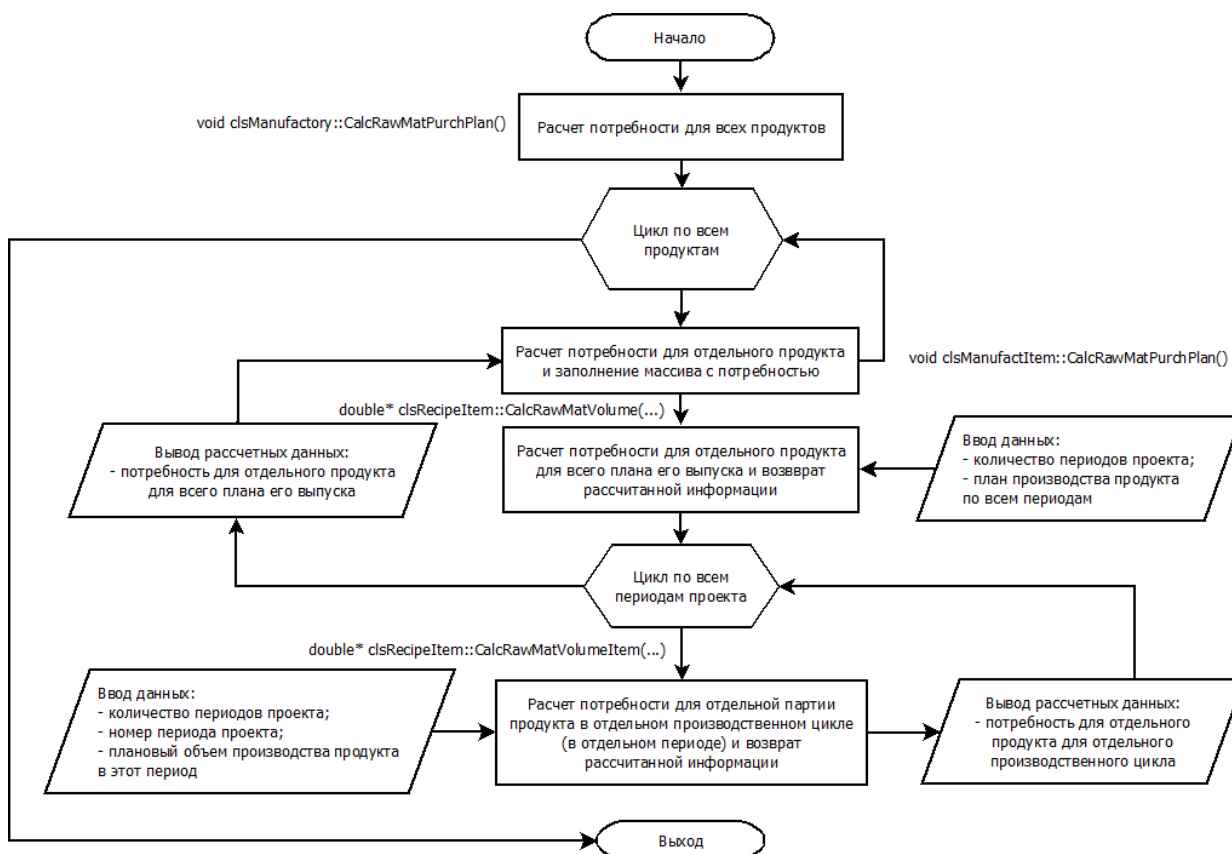


Рисунок 7 Расчет потребности в ресурсах в натуральном выражении для всех продуктов

Метод не имеет параметров, он использует информацию, введенную ранее на этапе создания производств для конкретных продуктов. Метод не возвращает рассчитанную информацию, представляя собой исключительно расчетный инструмент. Результаты расчетов для каждого наименования продукта записываются в соответствующие элементы контейнера в массивы [clsManufactItem::RawMatPurchPlan](#).

ПОЛУЧЕНИЕ ИНФОРМАЦИИ О ПОТРЕБНОСТИ В РЕСУРСАХ ДЛЯ ПРОИЗВОДСТВА ПРОДУКЦИИ

Для получения информации о потребности в ресурсах, рассчитанной на предыдущем этапе, используется один из двух методов: [decimal* clsManufactory::GetRawMatPurchPlan](#) или [strItem* clsManufactory::GetRMPurchPlan](#). Первый метод возвращает информацию о потребностях в ресурсах в виде указателя на массив вещественных чисел типа [decimal](#), другой – указатель на массив типа [strItem](#), в котором заполнены только поля *volume*, остальные поля заполнены нулями. Оба массива одномерные, являются аналогами двумерной матрицы с числом строк, равным общему числу позиций сырья и материалов [RMCount](#) и числом столбцов, равному числу периодов проекта [PrCount](#). Обращение к *(i,j)*-элементу такой «матрицы» производится так:

```
*(arrayref + PrCount * i + j),
```

где i - номер строки, j - номер столбца, *arrayref* – имя массива, выполняющее роль указателя на первый элемент массива. Условная *(i,j)-ячейка* матрицы содержит расход i -го наименования сырья на производство всех продуктов в j -м периоде проекта.

Массив создается вновь. Ответственность за освобождение памяти после использования этого массива лежит на программисте, использующем данный набор классов. После использования полученного массива (например, с именем *arrayref*), требуется его ручное удаление командой *delete[]*.

РАСЧЕТ УЧЕТНЫХ ЦЕН НА СЫРЬЕ И МАТЕРИАЛЫ

Для расчета учетных цен на ресурсы используются сторонние программные инструменты. Например, рекомендуются к использованию инструменты «[Склад ресурсов](#)» с классами [clsSKU](#) и [clsStorage](#).

Предполагается, что после получения *складом* информации о потребности в ресурсах, склад возвращает информацию об учетных ценах на сырье и материалы (например, с помощью методов [clsStorage::GetShipPrice](#) или [clsStorage::GetShip](#)).

ВВОД УЧЕТНЫХ ЦЕН НА СЫРЬЕ И МАТЕРИАЛЫ

Ввод учетных цен на ресурсы осуществляется методом [clsManufactory::SetRawMatPrice](#).

Каждый элемент контейнера содержит свой собственный массив цен [clsManufactItem::RawMatPrice](#), отличающийся от общего массива, передаваемого указанным выше методом. В общем массиве цен присутствует *полный список* ресурсов, а в массивах *clsManufactItem::RawMatPrice* находятся только те позиции ресурсов, которые участвуют в производстве данного конкретного продукта. Поэтому метод *clsManufactory::SetRawMatPrice* «разделяет» общий массив на ряд частных массивов. Принципиальная схема алгоритма приведена ниже (Рисунок 8).

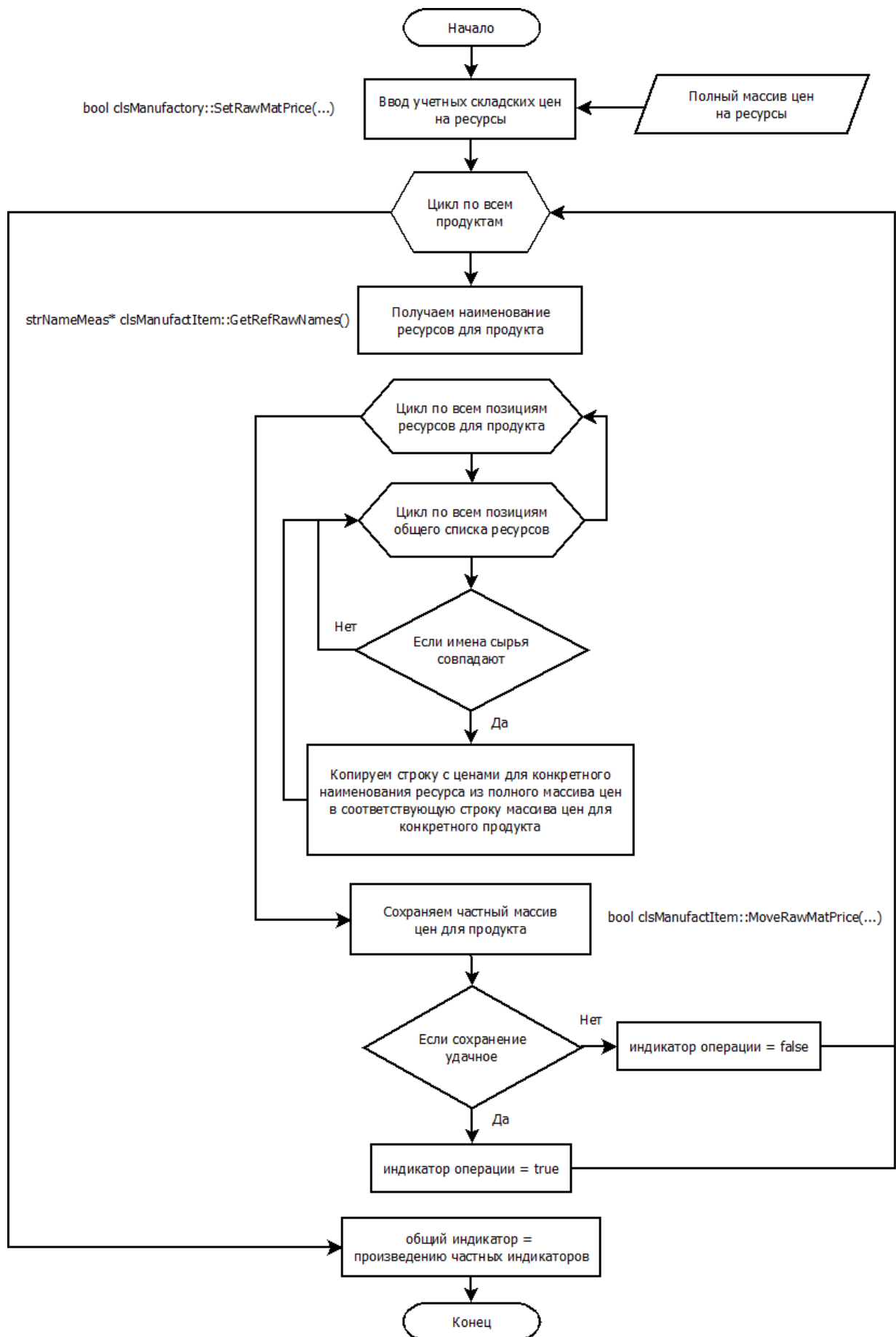


Рисунок 8 Ввод учетных цен на ресурсы

РАСЧЕТ СЕБЕСТОИМОСТИ НЕЗАВЕРШЕННОГО ПРОИЗВОДСТВА И ГОТОВОЙ ПРОДУКЦИИ

Следующим этапом после ввода учетных цен на ресурсы является, собственно, расчет себестоимости незавершенного производства и готовой продукции. Расчет осуществляется с помощью одного из трех методов на выбор: `clsManufactory::Calculate`, `clsManufactory::Calculate_future` или `clsManufactory::Calculate_thread`. Каждый из этих методов вызывает поочередно для каждого элемента контейнера функцию `clsManufactItem::CalculateItem` которая, в свою очередь, вызывает методы `clsRecipeItem::CalcWorkingBalance` и `clsRecipeItem::CalcProductBalance` экземпляра класса рецептов каждого продукта (см. Рисунок 9).

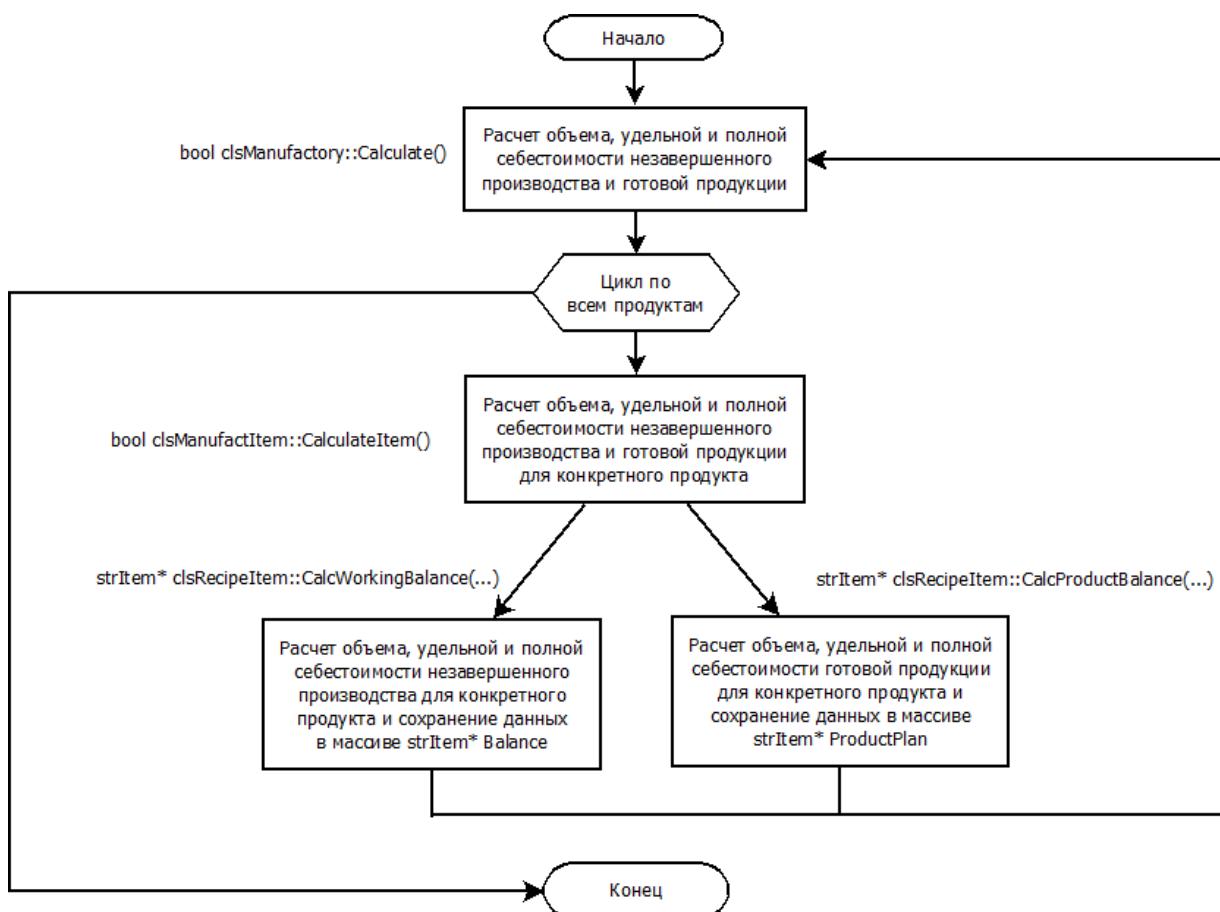


Рисунок 9 Расчет объема, удельной и полной себестоимости незавершенного производства и готовой продукции

Метод не имеет параметров, он использует информацию, полученную ранее. Метод не возвращает рассчитанную информацию, представляя собой исключительно расчетный инструмент. Результаты расчетов для каждого наименования продукта записываются в соответствующие элементы контейнера в массивы `clsManufactItem::Balance` и `clsManufactItem::ProductPlan`.

ПОЛУЧЕНИЕ ИНФОРМАЦИИ О СЕБЕСТОИМОСТИ НЕЗАВЕРШЕННОГО ПРОИЗВОДСТВА И ГОТОВОЙ ПРОДУКЦИИ

Для получения информации о себестоимости незавершенного производства и готовой продукции, рассчитанной на предыдущем этапе, используются методы:

- `strItem* clsManufactory::GetTotalBalance` – для получения указателя на массив с балансами незавершенного производства для всех продуктов;
- `strItem* clsManufactory::GetTotalProduct` – для получения указателя на массив с планами выхода всех продуктов.

Оба массива одномерные, являются аналогами двумерной матрицы с числом строк, равным общему числу продуктов `Manuf.size()` и числом столбцов, равному числу периодов проекта `PrCount`. Тип элемента массива `strItem`. Поля `volume` неизменны с момента добавления производства для конкретного продукта, поля `price` и `value` – результаты расчетов, произведенных на предыдущем этапе.

Массивы создаются вновь. Ответственность за освобождение памяти после использования этих массивов лежит на программисте, использующем данный набор классов.

МОДЕЛИРОВАНИЕ С ПОМОЩЬЮ КЛАССОВ `clsStorage` И `clsManufactory`

При моделировании предприятия необходимо принимать во внимание, сколько *центров затрат* (*Cost Centre*) планируется выделить в модели. Центр затрат может быть описан только классом `clsManufactory`. С его помощью можно аллокировать затраты на единицу ресурсов/ продукции, услуг и получить *добавленную стоимость*. В отличие от класса `clsStorage`, в котором такой функционал отсутствует.

СОЗДАНИЕ МОДЕЛИ С ОДНИМ ЦЕНТРОМ ЗАТРАТ

Пример 1, описанный в разделе «Возможности FROMA2», представляет предприятие с *тремя подразделениями* (Склад сырья и материалов, Производство и Склад готовой продукции), но одним *центром затрат*. Принципиальная схема модели приведена на рисунке ниже. Модель реализована в виде Программы для расчета полных и удельных переменных затрат Model_1, поставляемой в составе примеров библиотеки.

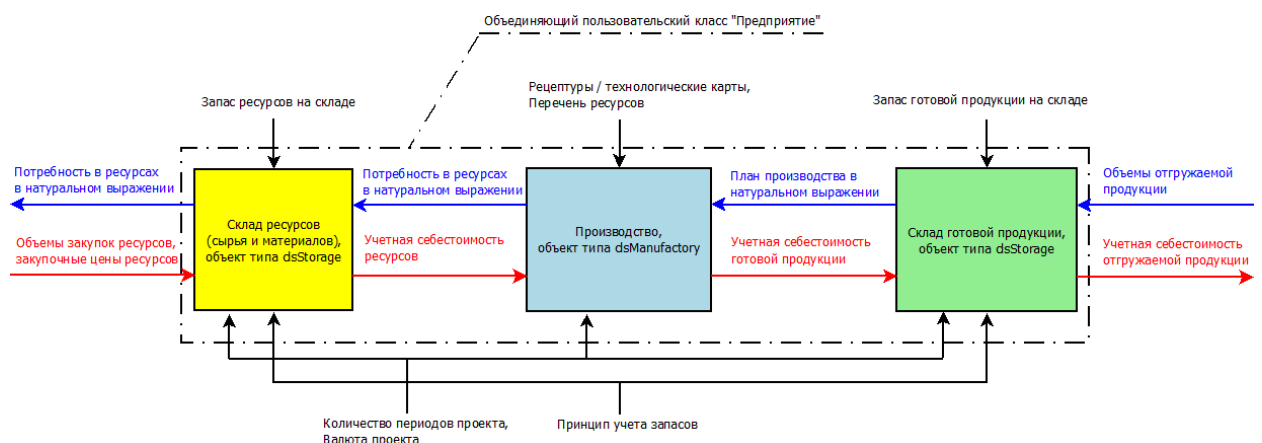


Рисунок 10 Принципиальная схема модели с одним центром затрат

Информационные потоки о готовой продукции и ресурсах в *натуральном выражении* представлены на схеме *синими стрелками*. Информационные потоки о *стоимостных показателях* готовой продукции и ресурсов представлены на схеме *красными стрелками*.

Рассмотрим подробнее *алгоритм расчета* учетной себестоимости отгружаемой со склада готовой продукции.

1. Для описания предприятия целесообразно создать *объединяющий* класс «Предприятие», в котором полями будут выступать Склад готовой продукции (экземпляр класса `clsStorage`), Производство (экземпляр класса `clsManufactory`) и Склад готовой продукции (экземпляр класса `clsStorage`), а также поля, обеспечивающие хранение и передачу данных между ними. Например, как во фрагменте кода ниже.

```

class clsEnterprise {
...
    size_t PrCount;           // Количество периодов проекта
    Currency Cur;             // Домашняя валюта проекта
    AccountingMethod Amethod; // Принцип учета запасов
    size_t ProdCount;         // Полное число выпускаемых продуктов
    strItem* Ship;            // Указатель на массив отгрузок со склада
    strNameMeas* ProdNames;   // Указатель на массив названий и ед. измерения продуктов
    size_t RMCount;           // Полное число наименований ресурсов
    strNameMeas* RMNames;     // Указатель на массив названий и ед. измерения ресурсов
    decimal* Purch;          // Указатель на массив цен на ресурсы
    clsStorage* Warehouse;    // Указатель на склад готовой продукции
    clsManufactory* Manufactory; // Указатель на производство
    clsStorage* RawMatStock;   // Указатель на склад сырья и материалов
...
}

```

2. Далее необходимо создать Склад готовой продукции, ввести в него длительность проекта, данные об отгрузках готовой продукции в натуральном выражении, норматив остатков продукции на складе, принцип учета запасов и рассчитать потребность в продукции, получаемой из производства (Производственный план). Подробнее об использовании класса clsStorage см. раздел «Как пользоваться классом clsStorage».
3. На следующем шаге мы создаем Производство, вводим в него длительность проекта, рецептуры/технологические карты и полученный на предыдущем этапе производственный план. Рассчитываем потребность в ресурсах для обеспечения выполнения производственного плана. Подробнее об использовании класса clsManufactory см. раздел «Как пользоваться классами модуля manufact».
4. Далее необходимо создать Склад ресурсов, ввести в него длительность проекта, данные об отгрузках ресурсов в производство в натуральном выражении, норматив остатков продукции на складе, принцип учета запасов и рассчитать потребность в ресурсах, закупаемых предприятием.
5. После того, как определен план закупок ресурсов и проведены переговоры с поставщиками, сотрудники предприятия имеют на руках окончательную версию графика и объемов закупок ресурсов и цен на них. Этот график и цены ресурсов вводятся в Склад ресурсов и рассчитывается учетная себестоимость этих ресурсов на выходе со склада.
6. Эта учетная себестоимость ресурсов вводится в Производство и рассчитывается учетная себестоимость готовой продукции.
7. Учетная себестоимость готовой продукции вводится в Склад готовой продукции и рассчитывается учетная себестоимость отгружаемой с этого склада продукции.

Нетрудно заметить, что алгоритм расчета учетной себестоимости отгружаемой со Склада готовой продукции является *двухпроходным*: сначала *от клиентов к поставщикам* идет расчет *объемных* показателей в натуральном выражении, потом *от поставщиков к клиентам* идет расчет *денежных* показателей. Эту специфику следует учитывать при создании агрегированных расчетных методов объединяющего класса.

СОЗДАНИЕ МОДЕЛИ С НЕСКОЛЬКИМИ ЦЕНТРАМИ ЗАТРАТ

Рассмотрим пример модели, в которой *три подразделения* (Склад сырья и материалов, Производство и Склад готовой продукции) и, соответственно, *три одноименных центра затрат*. Принципиальная схема модели приведена на рисунке ниже.

Как и на предыдущем рисунке, информационные потоки о готовой продукции и ресурсах в *натуральном выражении* представлены на схеме *синими стрелками*. Информационные потоки о *стоимостных показателях* готовой продукции и ресурсов представлены на схеме *красными стрелками*.

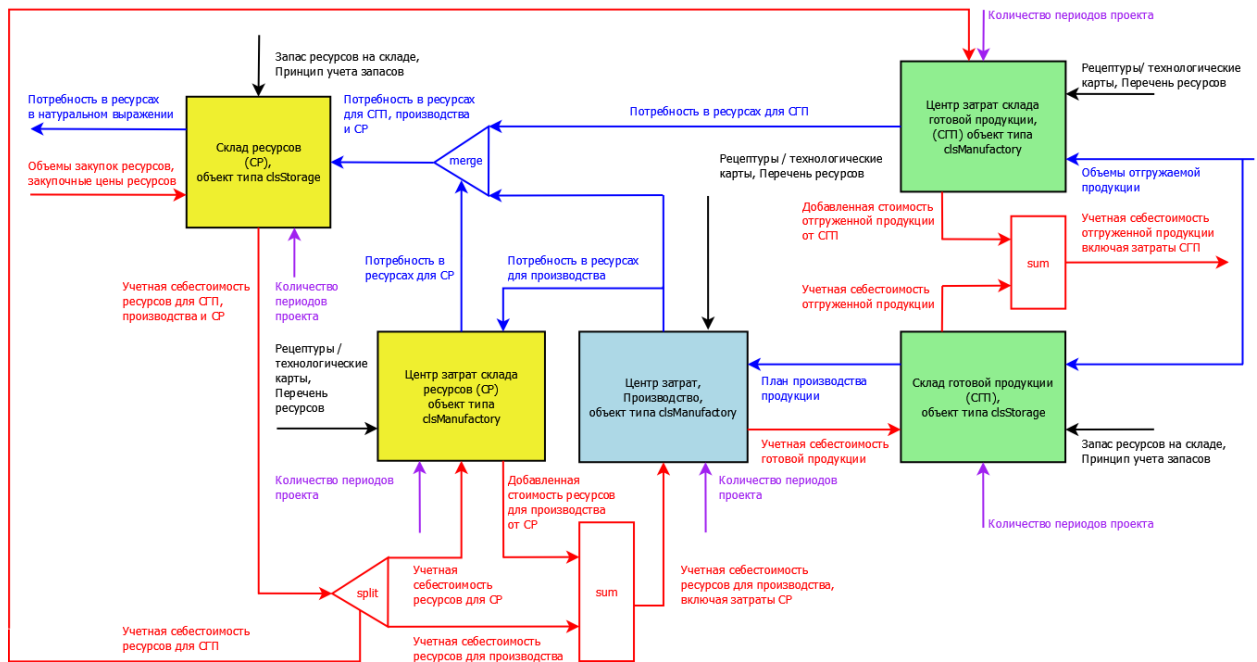


Рисунок 11 Принципиальная схема модели с тремя центрами затрат

На рисунке *Склад готовой продукции (СП)* и *Склад ресурсов (CP)*, каждый из них представлен *двумя объектами*: СП – прямоугольниками **зеленого цвета**, CP – прямоугольниками **желтого цвета**. Каждый склад имеет в своем составе *Центр учета затрат*, представленный экземпляром класса *clsManufactory* и *Центр учета запасов*, представленный экземпляром класса *clsStorage*.

Рассмотрим подробнее *алгоритм расчета* учетной себестоимости отгружаемой со склада готовой продукции.

1. Как и в предыдущем примере, для описания предприятия целесообразно предусмотреть *объединяющий* класс «Предприятие». Но в данном случае, также, может быть полезным дополнительно предусмотреть объединяющие классы для складов: отдельно объединяющий класс для СП, в котором полями будут выступать Центр учета затрат (экземпляр класса *clsManufactory*) и Центр учета запасов (экземпляр класса *clsStorage*), и объединяющий класс для CP с аналогичными полями. Объединяющие классы складов, в свою очередь, станут полями объединяющего класса для всего предприятия. (Для облегчения читаемости рисунка, мы не стали наносить на него линии объединяющих классов.)
2. После создания экземпляра класса «Предприятие», мы создаем *Склад готовой продукции* и в нем создаем *Центр учета запасов* СП и *Центр учета затрат* СП. В оба этих центра вводим данные: длительность проекта, данные об отгрузках готовой продукции в натуральном выражении. В Центр учета запасов ещё вводятся принцип учета запасов и норматив остатков продукции на складе. В Центр учета затрат вводятся: перечень ресурсов, используемых на СП и технологические карты, описывающие применение этих ресурсов на СП. *Технологические карты* для центра затрат СП содержат информацию о затратах СП на единицу отгружаемой продукции, например, расход групповой упаковочной тары (если упаковка происходит на складе) и трудозатраты на упаковочный процесс. В Центре учета запасов рассчитывается потребность в продукции, получаемой из производства (Производственный план). В Центре учета затрат рассчитывается потребность в ресурсах для СП.
3. На следующем шаге мы создаем *Производство*, вводим в него длительность проекта, перечень ресурсов для производства продукции (например, список сырья и материалов, используемых на производстве), рецептуры/ технологические карты и полученный на предыдущем шаге производственный план. Рассчитываем потребность в ресурсах для обеспечения выполнения производственного плана.

4. Далее необходимо создать *Склад ресурсов* и в нем создать *Центр учета запасов* СР и *Центр учета затрат* СР. В оба этих центра вводится длительность проекта. В Центр учета затрат ещё вводятся: перечень ресурсов, используемых на СР и технологические карты, а также потребность в ресурсах на производстве. *Технологические карты* для центра затрат СР содержат информацию о затратах СР на единицу отгружаемых ресурсов, например, трудозатраты на сортировку сырья и материалов. Производится расчет потребности в ресурсах для СР.
5. В объединяющем классе для СР необходимо предусмотреть *функцию, производящую слияние* массивов с информацией о потребности в ресурсах для СР, потребности в ресурсах для производства и потребности в ресурсах для СГП в единый массив с общей потребностью в ресурсах (на схеме эта функция обозначена *синим треугольником с надписью "merge"*). Этот общий массив вводится в Центр учета запасов СР. В Центр учета запасов ещё вводятся принцип учета запасов и норматив остатков продукции на Складе ресурсов. Производится расчет потребности в закупках всех ресурсов в натуральном выражении (План закупок ресурсов).
6. После того, как определен план закупок ресурсов и проведены переговоры с поставщиками, сотрудники предприятия имеют на руках окончательную версию графика и объемов закупок ресурсов и цен на них. Этот график и цены ресурсов вводятся в *Центр учета запасов* Склада ресурсов и рассчитывается учетная себестоимость этих ресурсов на выходе со склада.
7. В объединяющем классе для СР необходимо предусмотреть ещё одну *функцию, производящую разделение* общего массива с учетными стоимостями ресурсов на массив с учетной себестоимостью ресурсов для СР, массив с учетной себестоимостью ресурсов для СГП и массив с учетной себестоимостью ресурсов для Производства (на схеме эта функция обозначена *красным треугольником с надписью "split"*).
8. Полученный в результате разделения массив с учетной себестоимостью ресурсов для СГП передается для ввода в *Центр учета затрат* СГП; массив с учетной себестоимостью ресурсов для СР вводится в *Центр учета затрат* СР, в котором производится расчет добавленной на СР стоимости ресурсов для производства. Полученный в результате расчета массив с добавленной стоимостью суммируется поэлементно с массивом с учетной себестоимостью ресурсов для Производства (например, с помощью функции *Sum* из модуля Manufact; на схеме обозначена *прямоугольником красного цвета с надписью "sum"*). Результат суммирования передается для ввода в *Производство*.
9. Учетная себестоимость ресурсов для производства, включающая в себя добавленную на СР стоимость вводится в *Производство* и рассчитывается учетная себестоимость готовой продукции.
10. Учетная себестоимость готовой продукции вводится в *Центр учета запасов* СГП; учетная себестоимость ресурсов для СГП вводится в *Центр учета затрат* СГП. В Центре учета запасов рассчитывается учетная себестоимость отгружаемой продукции, а в Центре учета затрат рассчитывается добавленная на СГП стоимость отгружаемой продукции. Полученные в результате этих расчетов массив с учетной себестоимостью отгружаемой продукции и массив с добавленной на СГП стоимостью отгружаемой продукции суммируются поэлементно (например, с помощью той же функции *Sum*). Результат суммирования и представляет собой *искомую учетную себестоимость* отгружаемой со склада готовой продукции с учетом вносимых складами затрат.

ИМПОРТ И ЭКСПОРТ ДАННЫХ

Обмен данными между объектами библиотеки FROMA2 и сторонними программами осуществляется с помощью *CSV-файлов*. Основные типы, константы, классы и методы, реализующие эти алгоритмы, находятся в модуле *Imprex*. Состав модуля приведен в таблице ниже.

Таблица 16 Состав модуля Imprex

Состав модуля	Описание	Заметка
---------------	----------	---------

const size_t ex_precision	Константа, определяющая длину мантиссы вещественных чисел при преобразовании этих чисел в строковое представление; тип <i>size_t</i>	Рассчитывается на этапе компиляции (в выражении используется спецификатор constexpr). Если в расчётах используются вещественные числа типа LongReal , то длина мантиссы равна manDigits , если встроенные типы, - то max_digits10 . Используется в методах clsImpex::csvExport и clsImpex::Import .
class clsImpex	Класс для обмена данными между объектами библиотеки FROMA2 и сторонними программами	Экземпляр класса позволяет импортировать данные из CSV-файлов , а также массивов типа strNameMeas и strItem во внутреннее хранилище. Из внутреннего хранилища экземпляр класса позволяет экспортировать эти данные в CSV-файл, в массивы типа decimal , strNameMeas , strItem и string . В классе предусмотрена операция транспонирования
constexpr bool is_decimal_bool_or_PurchaseCalc_v	Функция, выполняемая на этапе компиляции, ограничивающая перечень возможных типов для метода ImportSingleArray	Допустимые значения типов: bool, size_t и decimal .
Внеклассовые функции	ImportSingleArray , Import_Recipes	Описание функций ниже

КЛАСС clsImpex

Основные задачи, которые можно решать с помощью класса *clsImpex*:

1. Импорт данных из [CSV-файла](#) во внутреннее хранилище;
2. Импорт данных из массива типа [strNameMeas](#) и [strItem](#) во внутреннее хранилище;
3. Транспонирование данных, представленных во внутреннем хранилище матрицей;
4. Экспорт данных из внутреннего хранилища в [CSV-файл](#);
5. Выборка данных, заданных начальными и конечными индексами строк и столбцов из внутреннего хранилища (представляющего собой матрицу) и получение этих данных в виде динамического массива вещественного типа [decimal](#); используется для получения численных данных;
6. Выборка данных, заданных начальными и конечными индексами строк и столбцов из внутреннего хранилища и получение этих данных в виде динамического массива типа [strItem](#); используется для получения численных данных;
7. Выборка данных, заданных начальными и конечными индексами строк и столбцов из внутреннего хранилища и получение этих данных в виде динамического массива типа [strNameMeas](#); используется для получения наименований продуктов/ресурсов и единиц их натурального измерения;
8. Выборка данных, заданных начальными и конечными индексами строк и столбцов из внутреннего хранилища и получение этих данных в виде динамического массива типа [string](#); используется для получения только наименований продуктов/ресурсов;
9. Получение *числа строк и столбцов матрицы*, являющейся внутренним хранилищем.

ПОЛЯ КЛАССА clsImpex

Таблица 17 Поля класса `clsImpex`. Секция `Private`

Поле класса	Описание	Заметка
<code>vector<std::vector<std::string>> m_data</code>	Внутреннее хранилище. Тип <code>vector<std::vector<std::string>></code>	Хранит импортированные данные в виде прямоугольной матрицы
<code>size_t m_rowcount</code>	Тип <code>size_t</code> . Количество строк матрицы внутреннего хранилища	
<code>size_t m_colcount</code>	Тип <code>size_t</code> . Количество столбцов матрицы внутреннего хранилища	
<code>char separator</code>	Тип <code>char</code> . Разделитель между значениями в <i>CSV-файле</i> .	

Методы класса `clsImpex` можно условно разделить на следующие группы:

- Конструкторы, перегруженные операторы, сервисные функции;
- Методы импорта;
- Методы преобразования;
- Методы экспорта;
- Get-методы;
- Методы визуального контроля.

Ниже подробно описываются все методы.

КОНСТРУКТОРЫ, ПЕРЕГРУЖЕННЫЕ ОПЕРАТОРЫ, СЕРВИСНЫЕ ФУНКЦИИ

`clsImpex()`

Конструктор по умолчанию (без параметров). Создает пустое внутреннее хранилище `m_data`. Создает и инициализирует переменные: `m_rowcount = m_colcount = 0`, `separator = ';' ;`.

`clsImpex(ifstream& ifs, const char& ch)`

Конструктор с параметрами. Импортирует данные из файлового потока во внутреннее хранилище.

Параметры:

`&ifs` – файловый поток, открытый для чтения; тип `ifstream`;

`&ch` – разделитель, используемый в файловом потоке `&ifs`; тип `char`.

`clsImpex(const size_t ncount, const nmBTypes::strNameMeas names[], const nmBTypes::strItem data[], const size_t dcount, nmBTypes::ReportData flg)`

Конструктор с параметрами. Импортирует данные из двух массивов: массива с наименованиями ресурсов/продуктов и единицами их натурального измерения и массива с данными. Импорт производится во внутреннее хранилище.

Параметры:

`ncount` – число строк, равное числу элементов массива `names`; тип `size_t`;

`names` – указатель на массив с наименованиями строк и единицами измерения; тип `strNameMeas`;

`data` – указатель на массив с данными; представляет собой одномерный массив, аналог двумерной матрицы размером `ncount*dcount`; тип элементов массива `strItem`;

`dcount` – число столбцов двумерной матрицы, хранимой в массиве `data`; тип `size_t`;

flag – флаг, определяющий тип используемых данных: volume, price или value из структуры типа *strItem*; тип параметра [ReportData](#).

clsImpex(const clsImpex& other)

Конструктор копирования.

Параметры:

& other – ссылка на копируемый константный экземпляр класса *clsImpex*.

clsImpex(clsImpex&& other)

Конструктор перемещения.

Параметры:

&& other – перемещаемый экземпляр класса *clsImpex*.

void swap(clsImpex& other) noexcept

Функция обмена значениями между экземплярами класса.

Параметры:

& other – ссылка на обмениваемый экземпляр класса *clsImpex*.

clsImpex& operator=(const clsImpex& other)

Перегрузка оператора присваивания копированием.

Параметры:

& other – ссылка на копируемый константный экземпляр класса *clsImpex*.

clsImpex& operator=(clsImpex &&obj)

Перегрузка оператора присваивания перемещением

Параметры:

&& other – перемещаемый экземпляр класса *clsImpex*.

void reset()

Сбрасывает состояние объекта до значения по умолчанию. Метод [vector::clear](#) не гарантирует успешного перераспределения памяти, в настоящем методе используется альтернатива ему. Кроме того, данный метод устанавливает счетчики строк и столбцов внутреннего хранилища в ноль, а сепаратор в значение по умолчанию (';').

bool is_Empty() const

Возвращает *true*, если внутреннее хранилище (контейнер m_data) пустое.

МЕТОДЫ ИМПОРТА

bool Import(ifstream& ifs, const char& ch)

Метод импортирует данные из файлового потока во внутреннее хранилище. В случае успешной операции возвращает *true*.

Параметры:

&ifs – файловый поток, открытый для чтения; тип [ifstream](#);

&ch – разделитель, используемый в файловом потоке &ifs; тип *char*;

bool Import(const size_t ncount, const nmBPTypes::strNameMeas names[], const nmBPTypes::strItem data[], const size_t dcount, nmBPTypes::ReportData flg)

Метод импортирует данные из двух массивов: массива с наименованиями ресурсов/ продуктов и единицами их натурального измерения и массива с данными. Импорт производится во внутреннее хранилище. В случае успешной операции возвращает *true*. Данные импортируются с длиной мантиссы, определяемой константой [ex_precision](#).

Параметры:

ncount – число строк, равное числу элементов массива names; тип *size_t*;

names – указатель на массив с наименованиями строк и единицами измерения; тип [strNameMeas](#);

data – указатель на массив с данными; представляет собой одномерный массив, аналог двумерной матрицы размером ncount*dcount; тип элементов массива [strItem](#);

dcount – число столбцов двумерной матрицы, хранимой в массиве data; тип *size_t*;

flg – флаг, определяющий тип используемых данных: *volume*, *price* или *value* из структуры типа *strItem*; тип параметра [ReportData](#).

МЕТОДЫ ПРЕОБРАЗОВАНИЯ

void Transpon()

Метод транспонирует матрицу [m_data](#).

МЕТОДЫ ЭКСПОРТА

void csvExport(ofstream& ofs) const

Метод экспорта данных из внутреннего хранилища в *CSV-файл* с разделителем, заданным переменной [separator](#). Данные экспортируются с длиной мантиссы, определяемой константой [ex_precision](#).

Параметры:

&ofs – файловый поток, открытый для чтения; тип [ifstream](#);

GET-МЕТОДЫ

decimal* GetDecimal(const size_t brow, const size_t erow, const size_t bcol, const size_t ecol) const

Метод возвращает выбранный из внутреннего хранилища *фрагмент* данных, конвертированных в тип [decimal](#).

Параметры:

brow – индекс начальной строки выбранного фрагмента данных из внутреннего хранилища; тип *size_t*;

erow – индекс конечной строки выбранного фрагмента данных из внутреннего хранилища; тип *size_t*;

bcol – индекс начального столбца выбранного фрагмента данных из внутреннего хранилища; тип *size_t*;

ecol – индекс конечного столбца выбранного фрагмента данных из внутреннего хранилища; тип *size_t*.

Метод возвращает указатель на одномерный динамический массив, являющийся аналогом двумерной матрицы размером $(erow-brow+1)*(ecol-bcol+1)$. Вещественный тип элементов массива определяется псевдонимом [decimal](#).

Для совместимости с параметрами методов классов *clsStorage* и *clsManufactory* матрица внутреннего хранилища (в векторе [m_data](#)) должна иметь горизонтальную ориентацию (разные периоды - в разных столбцах). В противном случае перед применением настоящего метода матрицу необходимо транспонировать с помощью метода [Transpon](#).

```
template<typename U>
```

```
constexpr bool is_decimal_bool_or_size_t_v =
```

```
std::is_same<U, decimal>::value || std::is_same<U, bool>::value || std::is_same<U, size_t>::value;
```

Ограничение типа для функции GetData.

```
template<typename T, class=std::enable_if_t<is_decimal_bool_or_size_t_v<T>>>
```

```
T* GetData(const size_t brow, const size_t erow, const size_t bcol, const size_t ecol) const;
```

Шаблонный метод возвращает выбранный из внутреннего хранилища *фрагмент* данных, конвертированных в тип [decimal](#), [bool](#) или [size_t](#).

Параметры:

brow – индекс начальной строки выбранного фрагмента данных из внутреннего хранилища; тип [size_t](#);

erow – индекс конечной строки выбранного фрагмента данных из внутреннего хранилища; тип [size_t](#);

bcol – индекс начального столбца выбранного фрагмента данных из внутреннего хранилища; тип [size_t](#);

ecol – индекс конечного столбца выбранного фрагмента данных из внутреннего хранилища; тип [size_t](#).

Метод возвращает указатель на одномерный динамический массив, являющийся аналогом двумерной матрицы размером $(erow-brow+1)*(ecol-bcol+1)$.

Предназначен для получения данных о [нормативе запасов на складе](#), о [флагах разрешения на отгрузку и закупку в одном периоде](#) и [флагах авторасчета](#).

Для совместимости с параметрами методов классов *clsStorage* и *clsManufactory* матрица внутреннего хранилища (в векторе [m_data](#)) должна иметь горизонтальную ориентацию (разные периоды - в разных столбцах). В противном случае перед применением настоящего метода матрицу необходимо транспонировать с помощью метода [Transpon](#).

```
nmBPTypes::strNameMeas* GetNames(const size_t brow, const size_t erow, const size_t idName, const size_t idMeas) const
```

Метод возвращает выбранный из внутреннего хранилища *фрагмент* данных, конвертированных в тип [strNameMeas](#). Используется для получения массива с наименованиями ресурсов/ продуктов и единиц их натурального измерения.

Параметры:

brow – индекс начальной строки выбранного фрагмента данных из внутреннего хранилища; тип [size_t](#);

erow – индекс конечной строки выбранного фрагмента данных из внутреннего хранилища; тип [size_t](#);

idName – номер столбца с названиями ресурсов/ продуктов; тип [size_t](#);

idMeas – номер столбца с названиями единиц измерения натурального выражения ресурсов/ продуктов; тип [size_t](#).

Метод возвращает одномерный динамический массив размером $(erow-brow+1)$. Тип элементов массива [strNameMeas](#).

Для совместимости с параметрами методов классов *clsStorage* и *clsManufactory* матрица внутреннего хранилища (в векторе [m_data](#)) должна иметь горизонтальную ориентацию (разные периоды - в разных столбцах). В противном случае перед применением настоящего метода матрицу необходимо транспонировать с помощью метода [Transpon](#).

string* GetNames(const size_t brow, const size_t erow, const size_t idName) const

Метод возвращает выбранный из внутреннего хранилища *фрагмент* данных, тип которых совпадает с типом данных хранилища, - тип *string*. Используется для получения массива с наименованиями ресурсов/ продуктов или наименованиями единиц их натурального измерения.

Параметры:

brow – индекс начальной строки выбранного фрагмента данных из внутреннего хранилища; тип *size_t*;

erow – индекс конечной строки выбранного фрагмента данных из внутреннего хранилища; тип *size_t*;

idName – номер столбца с названиями ресурсов/ продуктов; тип *size_t*;

Метод возвращает одномерный динамический массив размером $(erow-brow+1)$. Тип элементов массива *string*.

Для совместимости с параметрами методов классов *clsStorage* и *clsManufactory* матрица внутреннего хранилища (в векторе [m_data](#)) должна иметь горизонтальную ориентацию (разные периоды - в разных столбцах). В противном случае перед применением настоящего метода матрицу необходимо транспонировать с помощью метода [Transpon](#).

strItem* GetStrItem(const size_t brow, const size_t erow, const size_t bcol, const size_t ecol, nmBPTypes::ReportData flg) const

Метод возвращает выбранный из внутреннего хранилища *фрагмент* данных, конвертированных в структурный тип [strItem](#).

Параметры:

brow – индекс начальной строки выбранного фрагмента данных из внутреннего хранилища; тип *size_t*;

erow – индекс конечной строки выбранного фрагмента данных из внутреннего хранилища; тип *size_t*;

bcol – индекс начального столбца выбранного фрагмента данных из внутреннего хранилища; тип *size_t*;

ecol – индекс конечного столбца выбранного фрагмента данных из внутреннего хранилища; тип *size_t*;

flg – флаг, определяющий тип выбираемых данных: *volume*, *price* или *value* из структуры типа *strItem*; тип параметра [ReportData](#).

Метод возвращает одномерный динамический массив, являющийся аналогом двумерной матрицы размером $(erow-brow+1)*(ecol-bcol+1)$. Тип элементов массива [strItem](#). Используемые поля заполняются данными из матрицы, неиспользуемые – нулями.

Для совместимости с параметрами методов классов *clsStorage* и *clsManufactory* матрица внутреннего хранилища (в векторе [m_data](#)) должна иметь горизонтальную ориентацию (разные периоды - в разных столбцах). В противном случае перед применением настоящего метода матрицу необходимо транспонировать с помощью метода [Transpon](#).

size_t GetRowCount() const

Метод возвращает число строк матрицы внутреннего хранилища; тип *size_t*.

size_t GetColCount() const

Метод возвращает число столбцов матрицы внутреннего хранилища; тип `size_t`.

МЕТОДЫ ВИЗУАЛЬНОГО КОНТРОЛЯ

`void View(ostream& os) const`

Метод визуального контроля импортированной из файла информации. Выводит содержимое внутреннего хранилища в поток типа `ostream`. Используется в отладочных целях. При выводе каждый элемент матрицы, представляющий собой тип `string`, обрезается до 15 символов.

Параметры:

`&os` – выходной поток для вывода информации; тип `ostream`.

ПРИМЕР ПРОГРАММЫ С КЛАССОМ `clsImpex`

Ниже представлен листинг программы, демонстрирующей работу класса `clsImpex`.

```
#include <iostream>
#include <fstream>
#include <impex_module.h>
#include <common_values.hpp>

using namespace std;
using namespace nmBPTypes;

int main(int argc, char* argv[]) {
    setlocale(LC_ALL, "Russian"); // Установка русского языка

    string dir = "../examples/Impex_example/"; // Расположение файлов данных

    /** Импортируем данные из CSV-файла во внутреннее хранилище */
    ifstream input(dir + "ship.csv"); // Связываем файл с потоком
    const char ch = ','; // Выбираем разделитель
    clsImpex* Data = new clsImpex(input, ch); // Импортируем данные из файла
    input.close(); // Закрываем файл

    /** Визуальный контроль результата импорта */
    ofstream View_ship(dir + "View_ship.txt"); // Связываем файл с потоком
    Data->View(View_ship); // Отображаем хранилище
    View_ship.close(); // Закрываем файл

    /** Проверка работы Get - методов */
    size_t PrCount = Data->GetColCount()-2; // Число периодов проекта
    size_t ProdCount = Data->GetRowCount()-1; // Число продуктов
    cout << "Число периодов проекта " << PrCount << endl;
    cout << "Число продуктов " << ProdCount << endl;

    strItem* Ship = Data->GetstrItem(1, 6, 2, 61, volume); // Получаем массив
    if(!Ship) cout << "Ship = nullptr" << endl;
    strNameMeas* ProdNames = Data->GetNames(1, 6, 0, 1); // Получаем массив
    if(!ProdNames) cout << "ProdNames = nullptr" << endl;
```

```

    /** Печатаем в файл полученные массивы в виде таблицы */
    ofstream Output(dir + "Array.txt"); // Связываем файл с потоком
    Output << setw(15) << "Name" << setw(15) << "Measure"; // Заголовки
    for(size_t j{}; j<PrCount; j++)
        Output << setw(15) << j;
    Output << endl;
    for(size_t i{}; i<ProdCount; i++) { // Данные
        Output << setw(15) << ((ProdNames+i)->name).substr(0,15) << setw(15) <<
        (ProdNames+i)->measure;
        for(size_t j{}; j<PrCount; j++)
            Output << fixed << setprecision(7) << setw(15) <<
            (Ship+PrCount*i+j)->volume;
        Output << endl;
    };
    Output.close(); // Закрываем файл

    /** Транспонирование матрицы внутреннего хранилища */
    Data->Transpon();

    /** Визуальный контроль результата транспонирования */
    ofstream View_ship_T(dir + "View_ship_T.txt"); // Связываем файл с потоком
    Data->View(View_ship_T); // Выводим результат в файл
    View_ship_T.close(); // Закрываем файл

    /** Экспорт транспонированной матрицы внутреннего хранилища в CSV-файл */
    ofstream ship_T(dir + "ship_T.csv"); // Связываем файл с потоком
    Data->csvExport(ship_T); // Экспорт транспонированной матрицы
    ship_T.close(); // Закрываем выходной файл

    delete Data; // Удаляем класс импорта-экспорта
    delete[] Ship; // Удаляем массив
    delete[] ProdNames; // Удаляем массив
    return 0;
}

```

Исходный код программы приведен в [репозитории](#) в папке [froma2/examples/Impex_example](#); файл main.cpp.

ВНЕКЛАССОВЫЕ ФУНКЦИИ

Эти функции предназначены для импорта сразу нескольких типов данных одновременно и преобразования импортированных данных к формату, пригодному для непосредственного ввода в методы классов

[clsStorage](#), [clsManufactory](#), [clsAgregate](#). Эти функции используют класс [clsImpex](#).

bool ImportSingleArray(const string filename, const char _ch, size_t hcols, size_t hrows, ReportData flg, strItem* &_data, strNameMeas* &_names, size_t& ColCount, size_t& RowCount)

Метод импортирует названия номенклатурных позиций и единиц их натурального измерения, а также числовые данные. Импортируемый файл должен быть в формате [CSV](#). Возвращает *true* при удачном импорте. Иначе возвращает *false*. Часть параметров (*_data*, *_names*, *ColCount*, *RowCount*) является выходными данными.

Параметры:

filename – имя файла для импорта; тип *string*;

_ch – символ разделителя между полями в файле; тип *char*;

hcols – число столбцов с заголовками (см. Рисунок 16); тип *size_t*;

hrows – число строк с заголовками (см. Рисунок 16); тип *size_t*;

flg – флаг, определяющий тип импортируемых данных: "volume" - объемы в натуральном выражении, "price" - цены, "value" – стоимость; тип [ReportData](#);

_data – ссылка на указатель на формируемый массив с числовыми данными; тип элементов массива [strItem](#); размер массива *RowCount* ColCount*; выходные данные;

`_names` – ссылка на указатель на формируемый массив с наименованиями и единицами измерения номенклатурных позиций; тип элементов массива [strNameMeas](#); выходные данные;

`ColCount` – ссылка на формируемую переменную с числом периодов проекта; тип `size_t`; выходные данные;

`RowCount` – ссылка на формируемую переменную с числом номенклатурных позиций; тип `size_t`; выходные данные.

```
template<typename T, class=std::enable_if_t<is_decimal_bool_or_PurchaseCalc_v<T>>>
bool ImportSingleArray(const string _filename, const char _ch, size_t hcols, size_t hrows, \
    T* &_amp;_data, strNameMeas* &_amp;_names, size_t& ColCount, size_t& RowCount)
```

Аналогичен предыдущему методу, но элементы массива данных `_data` могут иметь следующие возможные типы: [decimal](#), [bool](#) или `size_t`. Используется для формирования массивов с нормативами запасов на складе, флагами разрешения/ запрета поступлений и отгрузок в одном и том же периоде и флагов автоматического/ ручного расчёта объемов поступлений на склад.

```
bool Import_Recipes(const string _prefixname, const char _ch, size_t hcols, size_t hrows, vector<clsRecipeItem>
    &_amp;_Recipe, size_t _Count, const string& msk, const string& _dir)
```

Метод читает информацию из файлов с именами, содержащими префикс имени рецептуры/ технологической карты `_prefixname`. Обрабатываются все файлы, удовлетворяющие маске `msk` и лежащие в одной папке (подпапки не обрабатываются). Метод заполняет контейнер рецептур/ технологических карт `_Recipe`.

Параметры:

`_prefixname` – префикс имен файлов технологических карт; тип `string`;

`_ch` – разделитель, используемый в файлах типа CSV; тип `char`;

`hcols` – количество столбцов с заголовками; тип `size_t`;

`hrows` – количество строк с заголовками в файлах; тип `size_t`;

`_Recipe` – выходной контейнер с рецептурами; тип [vector<clsRecipeItem>](#); выходные данные;

`_Count` – предварительный размер контейнера `_Recipe`; тип `size_t`; используется для начального резервирования памяти для контейнера;

`msk` – маска [regex регулярного выражения](#) для поиска подходящих файлов;

`_dir` – имя папки, где будет происходить поиск файлов технологических карт; тип `string`.

АРИФМЕТИКА ДЛИННЫХ ВЕЩЕСТВЕННЫХ ЧИСЕЛ

Как было сказано в разделе «[ТИП ВЕЩЕСТВЕННЫХ ЧИСЕЛ](#)», вещественные числа в библиотеке FROMA2 по желанию программиста могут быть представлены либо стандартным для C++ типом `double`, либо собственным типом библиотеки `LongReal`, реализация которого представлена в модуле [LONGREAL](#). Модуль `LONGREAL` представлен заголовочным файлом `LongReal_module.h` и файлом реализации `LongReal_lib.cpp`. В модуле содержатся классы и методы, реализующие вещественные числа и основные арифметические и логические операции над ними.

Каждый экземпляр класса `LongReal` являет собой вещественное число. Внутреннее представление вещественного числа типа `LongReal` включает в себя знак числа, мантиссу и экспоненту и выражается формулой:

$$\text{sign} * 0.d_1d_2\dots d_n * 10^{\text{exponent}},$$

где

sign – знак числа;
d₁, d₂, d_n – цифры мантиссы числа;
exponent – экспонента числа;
 знак [^] – знак возведения в степень.

Таблица 18 Состав модуля LongReal

Состав модуля	Описание	Заметка
using SType = bool	Псевдоним типа	Тип <i>знака</i> вещественного числа типа LongReal: <i>true</i> для отрицательного и <i>false</i> для положительного
using DType = short int	Псевдоним типа	Тип цифры <i>мантиссы</i> числа (каждая цифра имеет этот тип). Обязательно тип со знаком (знак используется временно при промежуточных вычислениях)
using EType = int	Псевдоним типа	Тип числа <i>экспоненты</i>
const EType MinEType	Минимальное значение экспоненты. Тип <i>EType</i>	При выполнении операций с двумя операндами для исключения ситуации с выходом экспоненты результирующего числа за пределы диапазона заданного типа, необходимо ограничивать минимальное значение экспоненты числом <i>numeric_limits<EType>::min()/2+1</i> , а максимальное значение числом <i>numeric_limits<EType>::max()/2-1</i> (например, в 32-разрядной среде эти значения становятся равными -2147483648/2+1 и 2147483647/2-1 соответственно). Для целей использования в библиотеке FROMA2 предельные значения избыточны и ресурсоемки. Поэтому минимальное и максимальное значение экспоненты существенно ограничены. В текущей реализации эти значения взяты равными -32768 и 32768.
const EType MaxEType	Максимальное значение экспоненты. Тип <i>EType</i>	
const size_t manDigits	Максимальное число знаков мантиссы. Тип <i>size_t</i>	При выполнении операций над числами и последующем использовании результатов расчетов в новых операциях, может происходить увеличение количества цифр мантиссы. Если таких операций много, число знаков мантиссы растет неконтролируемо, приводит к неоправданному потреблению ресурсов компьютера и увеличению времени вычислений. Введение ограничения на длину мантиссы является компромиссом между точностью с одной стороны и ресурсоемкостью с другой. В текущей реализации длина мантиссы ограничена 64 десятичными знаками. Это не ограничивает количество цифр мантиссы вводимых в операцию чисел, но ограничивает количество цифр мантиссы получаемого результата операции.
enum Scale{NANO=-9, MICRO=-6, MILL=-3, KILO = 3, MEGA=6, GIGA=9};	Именованные числа. Тип <i>int</i> .	Используются для удобства при масштабировании чисел в кило-, мега-, милли-, микро- и т.п. новые числа. Вместо указанных собственных имен можно использовать любые целые числа
class LongReal	Класс вещественного числа для арифметики длинных вещественных чисел	Экземпляр класса хранит одно число в виде <i>sign * 0.d₁d₂...d_n * 10^{exponent}</i> и методы, реализующие сервисные и арифметические операции с ним.
struct lstream	"Псевдопоток" для вывода в поток типа <i>ostream</i> чисел типа <i>LongReal</i> , <i>float</i> , <i>double</i> или <i>long double</i> с заданной точностью (количеством знаков после запятой) без необходимости менять код для	Используется для импорта и экспорта данных с мантиссой заданного размера

	пользовательского или встроенных типов вещественных чисел	
lstream lr_precision(const size_t w)	Манипулятор для чисел типа <i>LongReal</i> , <i>float</i> , <i>double</i> и <i>long double</i>	Устанавливает количество выводимых знаков после запятой
lstream operator <<(ostream& os, lstream&& m)	Оператор ввода объекта типа lstream в поток ostream	Позволяет подменить поток ostream потоком lstream и принимать в поток lstream последующие данные
ostream& lr_exit(ostream&)	Манипулятор для выхода из потока lstream и возвращения в поток ostream	Вместо манипулятора lr_exit могут использоваться манипуляторы std::endl, std::flush и другие output-манипуляторы из заголовочного файла <ostream>

Помимо указанных в таблице членов, в файле реализации *LongReal_lib.cpp* содержится ряд используемых исключительно в этом файле констант.

Исходный код программы для *тестирования логических и арифметических операций* над числами типа *LongReal* и *double* представлен в [репозитории](#) в папке [froma2/examples/LongReal_test](#); файл main.cpp.

КЛАСС LongReal

Возможности экземпляров класса *LongReal*, основные арифметические и логические операции над вещественными числами описаны ранее в разделе «Модуль LongReal» и покрывают все потребности библиотеки FROMA2.

ПОЛЯ КЛАССА LongReal

Таблица 19 Поля класса LongReal. Секция Private

Поле класса	Описание	Заметка
size_t NumCount	Количество цифр (десятичных разрядов) в вещественном числе. Тип <i>size_t</i> .	Максимальное значение ограничено константой manDigits
Stype sign	Знак числа (1 для отрицательных и 0 для положительных). Тип переменной определяется псевдонимом Stype .	Значение псевдонима, с которым тестировался класс, <i>bool</i> .
Dtype* digits	Указатель на динамический массив цифр длиной <i>NumCount</i> для хранения мантиссы числа. Тип элемента массива определяется псевдонимом Dtype .	Значение псевдонима, с которым тестировался класс, <i>short int</i> .
Etype exponent	Экспонента числа. Тип элемента массива определяется псевдонимом Etype	Значение псевдонима, с которым тестировался класс, <i>int</i> . Максимальное и минимальное значения ограничены константами MaxEtype и MinEtype соответственно.

Методы класса *LongReal* можно условно разделить на следующие группы:

- Сервисные функции;
- Конструкторы, деструктор, метод обмена;
- Перегрузка операторов присваивания;
- Get – методы;
- Методы масштабирования числа;
- Перегрузка логических и арифметических операторов;
- Перегрузка операторов ввода и вывода;
- Методы сериализации и десериализации;

СЕРВИСНЫЕ ФУНКЦИИ. СЕКЦИЯ PRIVATE

inline void setzero()

Метод вводит в экземпляр объекта число "ноль". Ноль может быть с любым знаком, но по умолчанию создается положительный ноль. Отрицательный ноль может возникнуть только в арифметических операциях, его присвоить нельзя.

Положительным нулем считается следующая комбинация значений полей класса:

```
sign = false,
NumCount = 0,
digits = nullptr,
exponent = MinEtype.
```

Отрицательным нулем считается комбинация, у которой *sign = true*.

inline void setposinf()

Метод устанавливает значение числа равное *положительной бесконечности*.

Положительной бесконечностью считается следующая комбинация значений полей класса:

```
sign = false,
NumCount = 0,
digits = nullptr,
exponent = MaxEtype.
```

inline void setneginf()

Метод устанавливает значение числа равное *отрицательной бесконечности*.

Отрицательной бесконечностью считается следующая комбинация значений полей класса:

```
sign = true,
NumCount = 0,
digits = nullptr,
exponent = MaxEtype.
```

inline void setNaN()

Метод устанавливает число в [NaN \(Not-A-Number\)](#). Значения NaN используются для идентификации неопределенных или непредставимых значений для элементов с плавающей запятой, например, таких как результат деления нуля на ноль.

NaN считается следующая комбинация значений полей класса:

```
sign = false,
NumCount = 1,
*digits = 256, (соответствует единице в девятом бите),
exponent = 0.
```

Если NaN установить не удалось (неудачное выделение памяти массиву [digits](#)), метод вызывает [setposinf](#) и возвращает положительную бесконечность.

void normalize()

Метод *нормализует* число, приводя его к виду $sign * 0.d_1d_2...d_n * 10^{exponent}$.

inline Dtype* cut(const Dtype _digits[], size_t Num)

Метод возвращает обрезанную копию произвольного массива, указатель на который передан в качестве параметра. В новый массив копируются только заданное число первых элементов исходного массива. Типы элементов исходного и нового массивов совпадают.

Параметры:

`_digits[]` – указатель на исходный массив; тип элементов массива определяется псевдонимом [Dtype](#);

`Num` – число первых элементов исходного массива, которые будут скопированы в новый массив; тип *size_t*.

inline void LongReal::Resize(const size_t _n)

Метод обрезаает внутренний массив [digits](#), оставляя в нем только заданное число первых элементов.

Параметры:

`_n` – новый размер массива `digits`; тип *size_t*.

void init(const string& s)

Метод инициализирует экземпляр класса *LongReal* числом, представленным строкою.

Параметры:

`&s` – ссылка на строку, состоящую из знака «+» или «-», цифр, одной точки (являющейся разделителем целой и дробной частей числа); тип *string*. Предполагается, что других символов в строке нет.

stringstream getstream() const

Метод возвращает хранимое число в поток типа [stringstream](#). Используется в публичных методах для вывода числа в переменные типа *string*, *long double*, *double*, *float*.

inline bool overflow(const Etype& E1, const Etype& E2) const

Метод осуществляет контроль переполнения: возвращает *true*, если при сложении экспонент двух чисел (например, в операции умножения) происходит выход за границы максимального или минимального значений для экспоненты, заданных константами [MaxEtype](#) или [MinEtype](#) соответственно.

Параметры:

`&E1` – ссылка на экспоненту первого числа; тип целого числа определяется псевдонимом [Etype](#);

`&E2` – ссылка на экспоненту второго числа; тип целого числа определяется псевдонимом [Etype](#).

LongReal incrE(const Etype& _E) const

Метод проверяет, что новое значение экспоненты, переданное в параметрах, больше, чем текущее. Только в этом случае метод создает копию текущего экземпляра, увеличивает в ней экспоненту до заданного большего значения, не изменяя самого числа (увеличивает экспоненту и одновременно добавляет слева нули в массив [digits](#) нового объекта) и возвращает это число. Метод используется в алгоритме выравнивания экспонент и мантисс у пары чисел для дальнейшего проведения с ними арифметических операций.

Параметры:

`&_E` – новое большее, чем текущее значение экспоненты; тип целого числа определяется псевдонимом [Etype](#).

void incrD(const size_t& _N)

Метод проверяет, что новое значение длины массива [digits](#), переданное в параметрах, больше, чем текущее. Только в этом случае метод увеличивает длину массива *digits* текущего экземпляра до нового большего значения за счет правых нулей. Число не меняется. Метод используется в алгоритме выравнивания экспонент и мантисс у пары чисел для дальнейшего проведения с ними арифметических операций.

inline LongReal _round(const size_t n) const

Метод возвращает копию числа типа *LongReal* с округлением до *_n* разрядов; значения в разрядах от *_n+1* до *NumCount* не меняются. Метод используется в функции [Get\(size_t\)](#). Метод не изменяет значение экземпляра класса, в котором вызывается.

Параметры:

n — количество разрядов для округления числа.

КОНСТРУКТОРЫ, ДЕСТРУКТОР, МЕТОД ОБМЕНА

LongReal()

Конструктор по умолчанию, без параметров. Создает экземпляр класса с числом *ноль*.

LongReal(const string& value)

Конструктор с инициализацией числа из строки.

Параметры:

&value — число, представленное строкою; тип *string*. При наличии в строке символов, отличных от символов «+» или «-», цифр и точки, лишние символы удаляются.

LongReal(const double& value)

Конструктор с инициализацией числа из вещественного числа типа *double*. Количество символов мантиссы получаемого числа типа *LongReal* в этом случае ограничено значением [std::numeric_limits<double>::max_digits10](#).

Параметры:

&value — вещественное число; тип *double*.

LongReal(const LongReal& obj)

Конструктор копирования.

Параметры:

&obj — копируемый объект типа *LongReal*.

LongReal(LongReal&& obj)

Конструктор перемещения.

Параметры:

&&obj — перемещаемый объект типа *LongReal*.

LongReal(Stype _sign, size_t _NumCount, Dtype* &_amp;_digits, Etype _exp)

Конструктор прямого создания числа из отдельных составляющих. Может быть полезен при создании констант и переменных локального назначения.

Параметры:

_sign — знак создаваемого числа (положительное или отрицательное); тип определяется псевдонимом [Stype](#);

_NumCount — количество цифр (десятичных разрядов) в вещественном числе. Тип *size_t*;

_digits — ссылка на указатель на динамический массив цифр длиной *NumCount* для хранения мантиссы числа; тип элемента массива определяется псевдонимом [Dtype](#);

_exp – экспонента числа; тип элемента массива определяется псевдонимом [Etype](#).

~LongReal()

Деструктор.

void swap(LongReal& obj) noexcept

Функция обмена значениями между объектами типа *LongReal*.

Параметры:

&obj – ссылка на обмениваемый экземпляр класса *LongReal*.

ПЕРЕГРУЗКА ОПЕРАТОРОВ ПРИСВАИВАНИЯ

LongReal& operator=(const LongReal &obj)

Перегрузка оператора присваивания копированием объекта типа *LongReal*.

Параметры:

&obj – присваиваемый объект типа *LongReal*.

LongReal& operator=(const double &value)

Перегрузка оператора присваивания копированием объекта типа *double*.

Параметры:

&value – присваиваемый объект типа *double*.

LongReal& operator=(const string &value)

Перегрузка оператора присваивания копированием объекта типа *string*.

Параметры:

&value – присваиваемый объект типа *string*.

LongReal& operator=(LongReal &&obj)

Перегрузка оператора присваивания перемещением объекта типа *LongReal*.

Параметры:

&obj – присваиваемый объект типа *LongReal*.

GET – МЕТОДЫ

size_t Size() const

Метод возвращает размер текущего экземпляра в байтах.

template<typename T> constexpr bool are_types_for_get_v = std::is_floating_point<T>::value || std::is_same<T, std::string>::value

Функция, определенная вне класса *LongReal*, выполняющаяся на этапе компиляции. Функция задает ограничение на используемые типы: только встроенные вещественные числа ([is_floating_point](#)) и объекты класса [std::string](#).

template<typename T, class=std::enable_if_t<are_types_for_get_v<T>>>

T Get() const

Шаблонный метод возвращает число в форме *string*, *long double*, *double* или *float*. Экземпляры шаблона:

- **template string Get<string>() const** – определяет возвращаемую величину типа *string*;
- **template long double Get<long double>() const** – определяет возвращаемую величину типа *long double*;
- **template double Get<double>() const** – определяет возвращаемую величину типа *double*;
- **template float Get<float>() const** – определяет возвращаемую величину типа *float*.

string Get(size_t _n) const

Метод возвращает число в виде строки *string* с заданным количеством знаков после разделителя (точки). Возвращаемое число округляется до n-разрядов.

Параметры:

_n – число знаков дробной части; тип *size_t*.

string EGet(size_t n) const

Метод возвращает число в виде строки *string* в научной форме: *.dddEn* (мантисса со степенью).

Параметры:

_n – число знаков дробной части; тип *size_t*.

explicit operator long double() const

Оператор преобразования типа. Объявлен *explicit* для предотвращения неявных (автоматических) преобразований типов¹⁶. Возвращает значение типа *long double*.

explicit operator long double() const

Оператор преобразования типа. Объявлен *explicit*. Возвращает значение типа *double*.

explicit operator float() const

Оператор преобразования типа. Объявлен *explicit*. Возвращает значение типа *float*.

МЕТОДЫ МАСШТАБИРОВАНИЯ ЧИСЛА**LongReal& Expchange(const Etype _exch)**

Метод возвращает объект, у которого экспонента изменена на заданную величину. Для удобства можно использовать вместо числа предопределенные константы из перечисления *Scale*: NANO=-9, MICRO=-6, MILL=-3, KILO = 3, MEGA=6, GIGA=9.

Метод позволяет использовать в качестве исходных данных числа с очень большими или очень малыми экспонентами, которые невозможно представить в виде чисел типа *long double*, *double* или *float*, а также неудобно представлять строкой из-за слишком большой ее длины. Достаточно ввести мантиссу числа любым доступным методом и затем изменить экспоненту до нужного размера.

Параметры:

¹⁶ Преобразование типов возможно только явно, например, с помощью оператора явного преобразования типа *static cast*. Например, присваивание двух переменных *double X* и *LongReal Y* вида *X=Y* будет отмечено компилятором, как ошибка. Чтобы ее избежать присваивание может быть следующим: *X=static_cast<double>(Y)*.

`_exch` – величина, на которую необходимо изменить экспоненту текущего числа; тип параметра определяется псевдонимом [Etype](#).

ПЕРЕГРУЗКА ЛОГИЧЕСКИХ И АРИФМЕТИЧЕСКИХ ОПЕРАТОРОВ

bool isZero() const

Метод возвращает *true*, если число равно *положительному* или *отрицательному нулю*.

bool isposInf() const

Метод возвращает *true*, если число равно [Infinity](#) (*положительная бесконечность*).

bool isnegInf() const

Метод возвращает *true*, если число равно *-Infinity* (*отрицательная бесконечность*).

bool isNaN() const

Метод возвращает *true*, если число *неопределено* (равно [NaN](#)).

bool operator==(const LongReal& x) const

Оператор *сравнения*. Сравнивает два числа и возвращает *true*, если числа равны. Плюс ноль и минус ноль равны друг другу. Если любой объект сравнения *NaN* то объекты не равны друг другу.

bool operator!=(const LongReal& x) const

Оператор неравенства. Обратный оператор предыдущему.

LongReal operator*(const LongReal& x) const

Оператор арифметического *умножения*. Возвращает результат умножения двух чисел типа *LongReal* в виде *нового* объекта типа *LongReal*.

Параметры:

`&x` – второй сомножитель для текущего числа; тип *LongReal*.

LongReal operator-() const

Оператор унарного минуса. Возвращает *текущий* объект со знаком минус.

bool operator>(const LongReal& x) const

Оператор сравнения «*больше*». Возвращает *true*, если текущее число больше сравниваемого.

Параметры:

`&x` – число для сравнения; тип *LongReal*.

bool operator<(const LongReal& x) const

Оператор сравнения «*меньше*». Возвращает *true*, если текущее число меньше сравниваемого.

Параметры:

`&x` – число для сравнения; тип *LongReal*.

bool operator>=(const LongReal& x) const

Оператор сравнения «*больше или равно*». Возвращает *true*, если текущее число больше сравниваемого или равно ему.

Параметры:

&x – число для сравнения; тип *LongReal*.

bool operator<=(const LongReal& x) const

Оператор сравнения «*меньше или равно*». Возвращает *true*, если текущее число меньше сравниваемого или равно ему

Параметры:

&x – число для сравнения; тип *LongReal*.

LongReal operator+(const LongReal& x) const

Оператор арифметического сложения. Возвращает результат сложения двух чисел типа *LongReal* в виде *нового* объекта типа *LongReal*.

Параметры:

&x – второе слагаемое для текущего числа; тип *LongReal*.

LongReal operator-(const LongReal& x) const

Оператор арифметического вычитания. Возвращает разность между двумя числами типа *LongReal* в виде *нового* объекта типа *LongReal*.

Параметры:

&x – вычитаемое число для текущего уменьшаемого числа; тип *LongReal*.

LongReal& operator--(const LongReal& x)

Оператор декремента. Возвращает *текущее* число типа *LongReal*, уменьшенное на заданную величину.

Параметры:

&x – вычитаемое число для текущего уменьшаемого числа; тип *LongReal*.

LongReal& operator+=(const LongReal& x)

Оператор инкремента. Возвращает *текущее* число типа *LongReal*, увеличенное на заданную величину.

Параметры:

&x – слагаемое для текущего увеличиваемого числа; тип *LongReal*.

LongReal inverse() const

Вычисление обратного числа текущему. Возвращает результат деления единицы на текущее число типа *LongReal* в виде *нового* объекта типа *LongReal*.

LongReal operator/(const LongReal& x) const

Оператор арифметического деления. Возвращает результат деления двух чисел типа *LongReal* в виде *нового* объекта типа *LongReal*.

Параметры:

&x – делитель для текущего делимого числа; тип *LongReal*.

friend LongReal fabs(const LongReal& x)

Возвращает модуль заданного числа типа *LongReal* в виде *нового* объекта типа *LongReal*. Функция не является членом класса *LongReal*, но объявлена дружественной ему ([friend](#)), чтобы иметь доступ к закрытым полям класса.

Параметры:

&x – число, модуль которого требуется найти; тип *LongReal*.

ПЕРЕГРУЗКА ОПЕРАТОРОВ ВВОДА И ВЫВОДА

friend ostream& operator<<(ostream& os, const LongReal& value)

Перегруженный оператор вывода вещественного числа типа *LongReal* в поток [ostream](#). Функция не является членом класса *LongReal*, но объявлена дружественной ему ([friend](#)), чтобы иметь доступ к закрытым полям класса.

Число типа *LongReal* конвертируется в число типа *double*, затем новое число типа *double* выводится в поток. Таким образом, длина мантиссы выводимого числа ограничивается длиной мантиссы числа типа *double*. Для вывода числа типа *LongReal* без промежуточной конвертации с длиной мантиссы, ограниченной типом *LongReal*, можно использовать поток типа [lstream](#) или манипулятор [lr_precision](#).

Параметры:

&os – поток типа *ostream*;

&value – вещественное число типа *LongReal*.

friend stringstream& operator>>(stringstream& ss, LongReal& value)

Перегруженный оператор ввода вещественного числа типа *LongReal* из потока [stringstream](#). Функция не является членом класса *LongReal*, но объявлена дружественной ему ([friend](#)), чтобы иметь доступ к закрытым полям класса.

Параметры:

&ss – поток типа *stringstream*;

&value – вещественное число типа *LongReal*.

friend istream& operator>>(istream& is, LongReal& value)

Перегруженный оператор ввода вещественного числа типа *LongReal* из потока [istream](#). Функция не является членом класса *LongReal*, но объявлена дружественной ему ([friend](#)), чтобы иметь доступ к закрытым полям класса.

Параметры:

&is – поток типа *istream*;

&value – вещественное число типа *LongReal*.

МЕТОДЫ СОХРАНЕНИЯ СОСТОЯНИЯ ОБЪЕКТА В ФАЙЛ И ВОССТАНОВЛЕНИЯ ИЗ ФАЙЛА

bool StF(ofstream &_outF) const

Метод имплементации записи в файловый поток текущего состояния экземпляра класса (запись в файловый поток, метод сериализации). Название метода является аббревиатурой, расшифровывается “StF” как “Save to File”. При удачной сериализации возвращает *true*, при ошибке записи возвращает *false*.

Параметры:

&_outF - экземпляр класса [ofstream](#) для записи данных. При инициализации переменной *_outF* необходимо установить флаг бинарного режима открытия потока (не текстового!) [ios::binary](#).

bool RfF(ifstream &_inF)

Метод имплементации чтения из файлового потока текущего состояния экземпляра класса (чтение из файлового потока, метод десериализации). Название метода является аббревиатурой, расшифровывается “RfF” как “Read from File”. При удачной десериализации возвращает *true*, при ошибке чтения возвращает *false*.

Параметры:

&_inF - экземпляр класса [ifstream](#) для чтения данных. При инициализации переменной *_inF* необходимо установить флаг бинарного режима открытия потока (не текстового!) [ios::binary](#).

bool SaveToFile(const string _filename) const

Метод записи текущего состояния экземпляра класса в файл. При удачной записи в файл, возвращает *true*, при ошибке записи возвращает *false*.

Параметры:

_filename – имя файла; тип *string*.

bool ReadFromFile(const string _filename)

Метод восстановления состояния экземпляра класса из файла. При удачном чтении из файла, возвращает *true*, при ошибке чтения возвращает *false*.

Параметры:

_filename – имя файла; тип *string*.

friend bool SEF(ofstream &_outF, LongReal& x)

Перегруженная функция SEF из модуля [SERIALIZATION](#) для записи в файловый поток вещественного числа типа *LongReal*. Функция не является членом класса *LongReal*, но объявлена дружественной ему ([friend](#)), чтобы иметь доступ к закрытым полям класса.

Параметры:

&_outF - экземпляр класса [ofstream](#) для записи данных;

&x – вещественное число типа *LongReal*.

В модуле [SERIALIZATION](#) одноименная функция записывает в файловый поток вещественное число типа *double*. Перегрузка этой функции для числа типа *LongReal* позволяет использовать методы сериализации более высокого порядка без оглядки на тип вещественного числа, представленного псевдонимом [decimal](#).

friend bool SEF(ofstream &_outF, LongReal x[], size_t Cnt)

Перегруженная функция SEF из модуля [SERIALIZATION](#) для записи в файловый поток массива вещественных чисел *LongReal*. Функция не является членом класса *LongReal*, но объявлена дружественной ему ([friend](#)), чтобы иметь доступ к закрытым полям класса.

Параметры:

&_outF - экземпляр класса [ofstream](#) для записи данных;

&x – указатель на массив вещественных чисел типа *LongReal*;

Cnt – размер массива вещественных чисел; тип *size_t*.

В модуле *SERIALIZATION* одноименная функция записывает в файловый поток массив вещественных чисел типа *double*. Перегрузка этой функции для числа типа *LongReal* позволяет использовать методы сериализации более высокого порядка без оглядки на тип элемента массива, представленного псевдонимом [decimal](#).

friend bool DSF(ifstream &_inF, LongReal& x)

Перегруженная функция DSF из модуля [SERIALIZATION](#) для чтения из файлового потока вещественного числа типа *LongReal*. Функция не является членом класса *LongReal*, но объявлена дружественной ему ([friend](#)), чтобы иметь доступ к закрытым полям класса.

Параметры:

&_inF – экземпляр класса [ifstream](#) для записи данных;

&x – вещественное число типа *LongReal*.

В модуле *SERIALIZATION* одноименная функция читает из файлового потока вещественное число типа *double*. Перегрузка этой функции для числа типа *LongReal* позволяет использовать методы десериализации более высокого порядка без оглядки на тип вещественного числа, представленного псевдонимом [decimal](#).

friend bool DSF(ifstream &_inF, LongReal x[], size_t Cnt)

Перегруженная функция DSF из модуля [SERIALIZATION](#) для чтения из файлового потока массива вещественных чисел *LongReal*. Функция не является членом класса *LongReal*, но объявлена дружественной ему ([friend](#)), чтобы иметь доступ к закрытым полям класса.

Параметры:

&_inF – экземпляр класса [ifstream](#) для записи данных;

&x – указатель на массив вещественных чисел типа *LongReal*;

Cnt – размер массива вещественных чисел; тип *size_t*.

В модуле *SERIALIZATION* одноименная функция читает из файлового потока массив вещественных чисел типа *double*. Перегрузка этой функции для числа типа *LongReal* позволяет использовать методы сериализации более высокого порядка без оглядки на тип элемента массива, представленного псевдонимом [decimal](#).

МЕТОДЫ ВИЗУАЛЬНОГО КОНТРОЛЯ

void View(ostream& os) const

Метод выводит в поток типа *ostream* вещественное число типа *LongReal* так, как оно хранится: отдельно знак числа, отдельно экспоненту, отдельно массив, содержащий мантиссу. Например, число 11.6508114157806 будет выведено так:

```
sign= 0
NumCount= 15
exponent= 2
digits are: 116508114157806
```

Параметры:

&os – экземпляр класса *ostream* для вывода числа.

void ViewM(ostream& os) const

Метод, аналогичный предыдущему, но массив с мантиссой числа выводится в столбик.

Параметры:

&os – экземпляр класса *ostream* для вывода числа.

СТРУКТУРА *lstream* И МЕТОДЫ ДЛЯ РАБОТЫ С НЕЙ

Структура *lstream* представляет собой тип, реализующий "псевдопоток" для вывода в поток типа *ostream* вещественных чисел типа *LongReal*, *float*, *double* или *long double* с заданной точностью (количеством знаков после запятой) без необходимости менять код для пользовательского или встроенных типов вещественных чисел.

Структура позволяет создать единый *манипулятор*, ограничивающий количество знаков после запятой у вещественных чисел указанных типов. Может применяться в методах, где используется псевдоним *decimal*, при этом манипулятор будет единым для всех указанных типов. Преимущество данного подхода в том, что не требуется промежуточная конвертация чисел *LongReal* в *double* и не теряется точность в передаваемых данных.

ПОЛЯ СТРУКТУРЫ *lstream*

Таблица 20 Поля класса *lstream*

Поле класса	Описание	Заметка
ostream *pos	Поток для реального вывода чисел и символов. Тип указатель на <i>ostream</i>	При создании экземпляра класса получает ссылку на <i>std::cout</i>
const size_t N	Количество знаков после запятой у вещественных чисел. Тип <i>size_t</i>	Размер мантиссы

МЕТОДЫ СТРУКТУРЫ *lstream*

lstream(const size_t w)

Конструктор с параметром. Инициализирует поле *N*, устанавливает *pos = &std::cout* и устанавливает точность для вывода встроенных вещественных чисел равную *N*.

Параметры:

w – число знаков после запятой (длина мантиссы); тип *size_t*.

lstream& operator<<(const LongReal& val)

Оператор вывода в поток *lstream* вещественных чисел типа *LongReal*.

Параметры:

val – ссылка на вещественное число типа *LongReal*.

lstream& operator<<(const string& val)

Оператор вывода в поток *lstream* строки типа *std::string*.

Параметры:

val – ссылка на строку типа *std::string*.

```
template<typename T> constexpr bool are_types_for_lstream_v = std::is_floating_point<T>::value ||
```

```
std::is_same<std::remove_const_t<std::remove_pointer_t<T>>, char>::value ||
std::is_same<std::remove_const_t<T>, size_t>::value;
```

Логическая функция, выполняющаяся на этапе компиляции и проверяющая ограничение шаблона. Функция возвращает *“true”* в случае, если используются встроенные типы вещественных чисел: *float*, *double* и *long double*¹⁷, а также тип символа *char*, указателя на массив символов *char** и тип целого беззнакового числа *size_t*. Иначе функция возвращает *“false”*. Выполняется на этапе компиляции (используется спецификатор [constexpr](#)).

```
template<typename T, class=std::enable_if_t<are_types_for_ostream_v<T>>>
ostream& operator<<(T val)
```

Оператор вывода в поток *ostream* вещественных чисел встроенных типов: *float*, *double* и *long double*, а также символов типа *char*, текста (указатель на массив *char**) и целочисленных значений *size_t*. Параметры передаются в функцию по значению. Это позволяет передавать в функцию как *lvalue*, так и *rvalue* параметры.

Параметры:

val – вещественное число или символ шаблонного типа *T*. В качестве допустимых типов используются встроенные типы: *float*, *double*, *long double*, *char*, *char** и *size_t*.

Шаблонный метод имеет специализации в файле *LongReal_lib.cpp* для следующих типов: *const float*, *const double*, *const long double*, *const char*, *const char**, *char**, *size_t*.

```
ostream& operator<<(ostream&(*f)(ostream&))
```

Оператор ввода в поток манипулятора для выхода из потока *istream* в поток *ostream*. В качестве аргумента может использоваться функция *lr_exit* или стандартные манипуляторы *std::ends*, *std::endl*, *std::flush* и другие манипуляторы из заголовочного файла [<ostream>](#).

Параметры:

F – указатель на функцию, принимающую в качестве аргумента и возвращающую в качестве результата ссылку на поток типа *ostream*.

МЕТОДЫ ДЛЯ РАБОТЫ СО СТРУКТУРОЙ *istream*

```
istream lr_precision(const size_t w)
```

Манипулятор для чисел типа *LongReal*, *float*, *double* и *long double*. Устанавливает длину мантиссы (количество выводимых знаков после запятой, точность числа). Возвращает поток типа *istream*.

Параметры:

w – число знаков после запятой; тип *size_t*.

```
istream operator <<(ostream& os, istream&& m)
```

Оператор ввода объекта типа *istream* в поток *ostream*. Позволяет подменить поток *ostream* потоком *istream* и принимать уже в поток *istream* последующие данные. Функция возвращает объект типа *istream*.

Параметры:

os – ссылка на поток типа *ostream*, принимающий параметр *m*;

m – ссылка на перемещаемый в поток *ostream* объект типа *istream*;

```
istream operator <<(istream& ios, istream&& m)
```

¹⁷ [Standard floating-point types](#)

Оператор ввода нового объекта типа *lstream* в поток *lstream*. Позволяет подменить прежний поток со старым значением длины мантиссы новым потоком с новым значением длины мантиссы.

Параметры:

los – ссылка на поток типа *lstream*, принимающий параметр *m*;

m – ссылка на перемещаемый в поток *lstream* объект типа *lstream*;

ostream& Ir_exit(ostream& os)

Манипулятор для выхода из потока *lstream* и возвращения в поток *ostream*. Вместо манипулятора *Ir_exit* могут использоваться манипуляторы *std::endl*, *std::flush* и другие манипуляторы из заголовочного файла *<ostream>*.

Параметры:

os – ссылка на поток типа *ostream*.

СЕРИАЛИЗАЦИЯ И ДЕСЕРИАЛИЗАЦИЯ ДАННЫХ

Как было сказано в разделе «[МОДУЛЬ SERIALIZATION](#)», основная цель функций, представленных в этом модуле, - обеспечить сохранение в файл и восстановление из файла данных в полях экземпляров основных классов библиотеки FROMA2 (сериализация и десериализация). Шаблоны и нешаблонные перегруженные функции для сериализации и десериализации данных типа *LongReal* были описаны ранее в параграфе «[МЕТОДЫ СОХРАНЕНИЯ СОСТОЯНИЯ ОБЪЕКТА В ФАЙЛ И ВОССТАНОВЛЕНИЯ ИЗ ФАЙЛА](#)» раздела «[АРИФМЕТИКА ДЛИННЫХ ВЕЩЕСТВЕННЫХ ЧИСЕЛ](#)». Шаблоны и нешаблонные перегруженные функции для сериализации и десериализации тривиально копируемых данных описаны ниже. Их сигнатура полностью совпадает с сигнатурой функций для данных типа *LongReal* для «бесшовного» переключения пользователем с использования данных базового типа *double* на использование данных типа *LongReal*.

template<typename T>

constexpr bool is_srlzd_v = std::is_trivially_copyable<T>::value

Функция с именем *is_srlzd_v* предназначена для проверки условия, является ли *тип T* тривиально копируемым типом. Возвращает *value = true*, если *тип T* является тривиально копируемым типом, *value = false* – в противном случае. Функция выполняется на *этапе компиляции* программного кода.

Параметры:

T – тип, используемый в шаблонах функций сериализации и десериализации.

template<typename T=string>

bool SEF(ofstream &_outF, string& str)

Нешаблонная функция сериализации строки в файловый поток.

Параметры:

&_outF – файловый поток типа *ofstream*;

&str – ссылка на строку, записываемую в поток; тип *string*.

template<typename T, class=std::enable_if_t<is_srlzd_v<T>>>

bool SEF(ofstream &_outF, T x[], size_t Cnt)

Шаблонная функция сериализации массива элементов типа *T* в файловый поток.

На этапе компиляции программного кода вызывается функция *is_srlzd_v*, которая проверяет *тип T* элементов массива на условие *тривиальной копируемости*. Если условие не выполняется, компилятор выдает ошибку (в случае, если тип *T* не является тривиально копируемым, но относится к классу *LongReal*, компилятор использует [одноименную функцию](#) с той же сигнатурой из модуля *LONGREAL*, и ошибка компиляции не

возникает). В случае, если элементы массива относятся к тривиально копируемому типу, компиляция завершается благополучно.

Параметры:

&_outF – файловый поток типа [ofstream](#);

x[] – указатель на массив с элементами *типа T*;

Cnt – размер массива; тип *size_t*.

```
template<typename T, class=std::enable_if_t<is_srlzd_v<T>>>
bool SEF(ofstream &_outF, T& x)
```

Шаблонная функция сериализации переменной типа T в файловый поток.

На этапе компиляции программного кода вызывается функция *is_srlzd_v*, которая проверяет *тип T* переменной на условие *тривиальной копируемости*. Если условие не выполняется, компилятор выдает ошибку (в случае, если тип *T* не является тривиально копируемым, но относится к классу [LongReal](#), компилятор использует [одноименную функцию](#) с той же сигнатурой из модуля [LONGREAL](#), и ошибка компиляции не возникает). В случае, если тип переменной относится к тривиально копируемому типу, компиляция завершается благополучно.

Параметры:

&_outF – файловый поток типа [ofstream](#);

&x – ссылка на переменную *типа T*.

```
template<typename T=string>
bool DSF(istream &_inF, string& str)
```

Нешаблонная функция десериализации строки из файлового потока. Файловый поток должен быть связан с файлом, в котором предварительно была записана строка с помощью метода [bool SEF\(ofstream&, string&\)](#).

Параметры:

&_inF – файловый поток типа [istream](#);

&str – ссылка на строку, читаемую из потока; тип *string*.

```
template<typename T, class=std::enable_if_t<is_srlzd_v<T>>>
bool DSF(istream &_inF, T x[], size_t Cnt)
```

Шаблонная функция для десериализации массива элементов типа T из файлового потока. Файловый поток должен быть связан с файлом, в котором предварительно был записан массив с помощью метода [bool SEF\(ofstream &_outF, T x\[\], size_t Cnt\)](#).

На этапе компиляции программного кода вызывается функция *is_srlzd_v*, которая проверяет *тип T* элементов массива на условие *тривиальной копируемости*. Если условие не выполняется, компилятор выдает ошибку (в случае, если тип *T* не является тривиально копируемым, но относится к классу [LongReal](#), компилятор использует [одноименную функцию](#) с той же сигнатурой из модуля [LONGREAL](#), и ошибка компиляции не возникает). В случае, если элементы массива относятся к тривиально копируемому типу, компиляция завершается благополучно.

Параметры:

&_inF – файловый поток типа [istream](#);

x[] – указатель на массив с элементами *типа T*;

Cnt – размер массива; тип *size_t*.

```
template<typename T, class=std::enable_if_t<is_srlzd_v<T>>>
bool DSF(istream &_inF, T& x)
```

Шаблонная функция для десериализации переменной типа *T* из файлового потока. Файловый поток должен быть связан с файлом, в котором предварительно был записан массив с помощью метода [bool SEF\(ofstream &_outF, T& x\)](#).

На этапе компиляции программного кода вызывается функция *is_srlzd_v*, которая проверяет *тип T* переменной на условие *тривиальной копируемости*. Если условие не выполняется, компилятор выдает ошибку (в случае, если тип *T* не является тривиально копируемым, но относится к классу [LongReal](#), компилятор использует [одноименную функцию](#) с той же сигнатурой из модуля [LONGREAL](#), и ошибка компиляции не возникает). В случае, если тип переменной относится к тривиально копируемому типу, компиляция завершается благополучно.

Параметры:

&_inF – файловый поток типа [ifstream](#);

&x – ссылка на переменную *типа T*.

РОДИТЕЛЬСКИЙ КЛАСС ДЛЯ МОДЕЛИ

На принципиальной схеме модели с одним центром затрат (Рисунок 10) *Склад ресурсов, Производство и Склад готовой продукции* обведены штрихпунктирной линией с названием «Объединяющий пользовательский класс “Предприятие”». Удобство такого объединения очевидно и для модели с одним центром затрат, и для модели с несколькими центрами затрат (Рисунок 11). Эти удобства были перечислены в разделе «[МОДУЛЬ BASEPROJECT](#)». Состав модуля приведен в таблице ниже.

Таблица 21 Состав модуля BaseProject

Состав модуля	Описание	Заметка
enum Tdev{nulldev=nmBPTypes::sZero, terminal, file}	Именованные числа. Тип данных перечисления <i>enum</i> ; состоит из конечного набора именованных констант типа <i>size_t</i> .	Назначение: выбор устройства для вывода отчета (пустое устройство, терминал, файл).
struct Descript { string *sComment; size_t sCount; }	Тип <i>Descript</i> включает в себя два поля: sComment – указатель на массив строк типа <i>string</i> с текстовым описанием проекта; sCount – количество элементов этого массива типа <i>size_t</i> .	Используется для объявления переменной с описанием проекта
class clsBaseProject	Класс базового проекта	Предоставляет удобство родительского класса (интерфейс) для объединения в себе полей типа clsStorage, clsManufactory и других полей, типы которых определены в библиотеке FROMA2

Остановимся подробнее на классе *clsBaseProject*, представляющем собой родительский класс (интерфейс) для такого объединения.

КЛАСС clsBaseProject

Состав класса *clsBaseProject* представлен в таблице ниже.

ПОЛЯ КЛАССА clsBaseProject

Таблица 22 Поля класса `clsBaseProject`. Секция `Private`

Поле класса	Описание	Заметка
string Title	Название проекта. Тип <i>string</i> .	Секция <code>private</code>
Descript About	Переменная с описанием проекта; тип <i>Descript</i> .	Секция <code>private</code>
string FName	Переменная с именем файла для чтения/записи состояния экземпляра класса на диск	Секция <code>protected</code>
string RName	Переменная с именем файла для вывода отчета в файл	Секция <code>protected</code>
Tdev Rdevice	Переменная с признаком устройства для вывода отчета	Секция <code>protected</code>

Методы класса *clsBaseProject* можно условно разделить на следующие группы:

- Конструкторы, деструктор, метод обмена, метод сброса состояния; Перегрузка операторов присваивания;
- Set – методы;
- Функции вывода отчета;
- Методы сериализации и десериализации.

КОНСТРУКТОРЫ, ДЕКТРУКТОР, МЕТОД ОБМЕНА, МЕТОД СБРОСА СОСТОЯНИЯ; ПЕРЕГРУЗКА ОПЕРАТОРОВ ПРИСВАИВАНИЯ

clsBaseProject()

Конструктор по умолчанию. Устанавливаем пустое название проекта, указатель на массив строк с описанием проекта на "пусто", нулевое число строк в описании проекта, пустую строку вместо имени файла для записи/чтения состояния в файл и имени файла отчета; устанавливает терминал в качестве устройства вывода отчета.

virtual ~clsBaseProject()

Виртуальный деструктор.

clsBaseProject(const clsBaseProject& other)

Конструктор копирования.

Параметры:

& other – ссылка на копируемый константный экземпляр класса *clsBaseProject*.

virtual void swap(clsBaseProject& other) noexcept

Функция обмена значениями между экземплярами класса.

Параметры:

& other – ссылка на обмениваемый экземпляр класса `clsBaseProject`.

clsBaseProject(clsBaseProject&& other)

Конструктор перемещения.

Параметры:

&& other – перемещаемый экземпляр класса `clsBaseProject`.

clsBaseProject& operator=(const clsBaseProject& other)

Перегрузка оператора присваивания копированием.

Параметры:

& other – ссылка на копируемый константный экземпляр класса clsBaseProject.

clsBaseProject& operator=(clsBaseProject&& other)

Перегрузка оператора присваивания перемещением

Параметры:

&& other – перемещаемый экземпляр класса clsBaseProject.

virtual void Reset()

Метод сброса всей информации в экземпляре класса до "пусто".

SET – МЕТОДЫ**void SetTitle(const string& _title)**

Метод ввода титула проекта. Устанавливает поле [Title](#).

Параметры:

&_title – ссылка на строку, содержащую титул проекта; тип ссылки string.

void SetComment(const string* _comment, const size_t& _count)

Метод ввода описания проекта. Устанавливает поле [About](#).

Параметры:

*_comment – указатель на массив строк с описанием проекта; тип указателя string;

&_count – размер массива; тип size_t.

void SetFName(const string _filename)

Метод ввода имени файла для сериализации и десериализации. Устанавливает поле [FName](#).

Параметры:

_filename – имя файла; тип *string*.

Метод удобно использовать для получения имени файла сериализации из полного имени исполняемого файла программы, находящегося в переменной *argv[0]* метода *main* консольного приложения C++. При подстановке переменной *argv[0]* вместо параметра *_filename*, функция ищет в имени расширение ".exe" и, если оно есть, то заменяет его расширением ".dat". Таким образом, например, из имени "C:/Users/.../ Business_06.exe" можно получить имя "C:/Users/.../ Business_06. dat".

Если строка символов _filename не содержит расширение ".exe", новое расширение ".dat" просто добавляется в конце строки.

void SetFName(const string _filename, const string _ext)

Метод ввода имени файла для сериализации и десериализации с явным указанием расширения для имени файла. Устанавливает поле [FName](#).

Параметры:

`_filename` – имя файла; тип `string`;

`_ext` – расширение, добавляемое к имени файла `_filename`; тип `string`.

Метод удобно использовать для получения имени файла сериализации с произвольным расширением.

bool SetRName(const string _filename)

Метод ввода имени для файла отчёта. Устанавливает поле `RName` и проверяет существование папки, в которую планируется вывести отчет. Если папка существует, то возвращает `true`, иначе возвращает `false`.

Параметры:

`_filename` – имя файла; тип `string`.

Метод удобно использовать для получения имени файла отчёта из полного имени исполняемого файла программы, находящегося в переменной `argv[0]` метода `main` консольного приложения C++. При подстановке переменной `argv[0]` вместо параметра `_filename`, функция ищет в имени расширение `".exe"` и, если оно есть, то заменяет его расширением `".txt"`. Таким образом, например, из имени `"C:/Users/.../ Business_06.exe"` можно получить имя `"C:/Users/.../ Business_06. txt"`.

Если строка символов `_filename` не содержит расширение `".exe"`, то новое расширение `".txt"` просто добавляется в конце строки.

void SetDevice(const Tdev& val)

Метод устанавливает выходное устройство для отчета. Устанавливает поле `Rdevice`.

Параметры:

`&val` – индекс устройства: `nulldev` - пустое устройство, `terminal` - вывод на экран, `file` - вывод в файл; тип перечисления целых чисел `Tdev`.

ФУНКЦИИ ВЫВОДА ОТЧЕТА. СЕКЦИЯ PROTECTED**virtual void reportstream(ostream& os) const**

Метод выводит отчет в заданный поток. В данном классе выводит в поток название проекта (содержимое поля `Title`) и описание (содержимое поля `About`). Предназначен для переопределения в классах-потомках.

Параметры:

`&os` – поток для вывода отчета. Тип `ostream`.

ФУНКЦИИ ВЫВОДА ОТЧЕТА. СЕКЦИЯ PUBLIC**void Report() const**

Метод вывода отчета на выбранное устройство. Устройство определяется содержимым поля `Rdevice` (`nulldev` или «0» – пустое устройство, `terminal` или «1» – экран, `file` или «2» – файл). Метод вызывает защищенный метод `reportstream`.

МЕТОДЫ СЕРИАЛИЗАЦИИ И ДЕСЕРИАЛИЗАЦИИ. СЕКЦИЯ PROTECTED**virtual bool StF(ofstream &_outF)**

Метод записывает состояние объекта типа *clsBaseProject* в файловый поток. Возвращает значение *true*, если операция прошла успешно, в противном случае возвращает *false*. Использует метод SEF из модуля [SERIALIZATION](#) или [LONGREAL](#) для каждого из полей. Предназначен для переопределения в классах-потомках.

Параметры:

&_outF – файловый поток, открытый для записи типа *ofstream*.

virtual bool RfF(ifstream &_inF)

Метод восстанавливает состояние объекта типа *clsBaseProject* из файлового потока. Возвращает значение *true*, если операция прошла успешно, в противном случае возвращает *false*. Использует метод DSF из модуля [SERIALIZATION](#) для каждого из полей. Предназначен для переопределения в классах-потомках.

Параметры:

&_inF – файловый поток, открытый для чтения типа *ifstream*.

МЕТОДЫ СЕРИАЛИЗАЦИИ И ДЕСЕРИАЛИЗАЦИИ. СЕКЦИЯ PUBLIC

bool SaveToFile(const string _filename)

Метод записи состояния текущего экземпляра класса в файл с заданным именем. Вызывает метод *StF*.

Параметры:

_filename – имя файла с расширением; тип *string*.

bool SaveToFile()

Метод записи состояния текущего экземпляра класса в файл с именем, сохраненным в поле [FName](#). Вызывает метод *StF*.

bool ReadFromFile(const string _filename)

Метод чтения состояния из файла с заданным именем и запись в текущий экземпляр класса. Вызывает метод *StF*.

Параметры:

_filename – имя файла с расширением; тип *string*.

bool ReadFromFile()

Метод чтения состояния из файла с именем, сохраненным в поле [FName](#) и запись в текущий экземпляр класса. Вызывает метод *StF*.

Параметры:

_filename – имя файла с расширением; тип *string*.

АГРЕГАТОР ДЛЯ МОДЕЛИ ПРЕДПРИЯТИЯ

Инструменты, представленные в данном модуле, позволяют создать модель предприятия, состоящую из последовательной цепочки объектов типа [clsStorage](#) и [clsManufactory](#), вне зависимости от их количества и порядка размещения, а также использовать общие методы для цепочки объектов в целом (см. краткое описание в разделе «[МОДУЛЬ AGREGATE](#)» выше). Пример приложения, реализующую данную технологию для построения модели склада с двумя центрами затрат, представлен в репозитории программой [Model 2](#).

Состав модуля представлен в таблице ниже.

Таблица 23 Состав модуля AGREGATE

Состав модуля	Описание	Заметка
enum Clc_type	Тип перечисление <i>enum</i> , состоящий из значений <i>seq</i> , <i>fut</i> , <i>thrd</i> .	Предназначен для выбора метода вычислений: последовательные, в асинхронных потоках или синхронных
const Clc_type clc	Константа с сепаратором выбора типа вычислений	
T_aggregate	Тип variant , принимающий значения <i>clsStorage</i> или <i>clsManufactory</i>	Тип для элемента контейнера, позволяющего организовывать цепочку объектов типа <i>clsStorage</i> и <i>clsManufactory</i>
Структуры - посетители для типа T_aggregate	Посетители используются в методе std::visit для вызова методов у объектов типа <i>variant</i>	Список посетителей приводится ниже .
nmAggregate	Пространство имен для модуля	Содержит константы для форматирования отчётов
clsAgregate	Класс для объединения объектов вариантного типа <i>T_aggregate</i> в контейнер типа std::vector . Является наследником класса clsBaseProject	Класс не предназначен для создания объектов (Конструктор по умолчанию <i>clsAgregate</i> перенесён в секцию <i>private</i> , чтобы его смог использовать только класс - наследник)

Работа с классом *clsAgregate* предполагает написание класса-наследника, создание публичного конструктора, переопределение виртуальных методов.

ПОСЕТИТЕЛИ ДЛЯ ВАРИАНТНОГО ТИПА T_aggregate

```
struct Getter_exporting_data
```

Структура с методами получения данных для экспорта в [CSV-файлы](#). При использовании в качестве посетителя объекта типа *T_aggregate* в функции *std::visit*, возвращает указатель на новый массив типа [strItem](#) с данными для последующего экспорта. Запрашиваемые данные определяются сепаратором [ChoiseData arr](#), который передаётся в качестве параметра в конструкторе структуры.

Getter_exporting_data(const ChoiseData _arr)

Конструктор с параметрами.

Параметры:

_arr – сепаратор выбора данных: *"purchase"* - поставки, *"balance"* - остатки/незавершенное производство, *"shipment"* – отгрузки; тип сепаратора *ChoiseData*.

strItem* operator() (const clsStorage& obj)

Оператор для вызова соответствующей функции ([GetPure](#), [GetBal](#) или [GetShip](#)) объекта типа *clsStorage*; тип возвращаемого объекта *strItem*.

strItem* operator() (const clsManufactory& obj)

Оператор для вызова соответствующей функции ([GetRMPurchPlan](#), [GetTotalBalance](#) или [GetTotalProduct](#)) объекта типа *clsManufactory*; тип возвращаемого объекта *strItem*.

```
struct Getter_exporting_names
```

Структура с методами получения наименований ресурсов для экспорта в CSV-файлы. При использовании в качестве посетителя объекта типа *T_aggregate* в функции *std::visit*, возвращает указатель на новый массив типа [strNameMeas](#) с наименованиями ресурсов и их единиц измерения для последующего экспорта.

Запрашиваемые данные определяются сепаратором [ChoiseData](#) *arr*, который передаётся в качестве параметра в конструкторе структуры.

Getter_exporting_names(const ChoiseData _arr)

Конструктор с параметрами.

Параметры:

_arr – сепаратор выбора данных: "purchase" - поставки, "balance" - остатки/незавершенное производство, "shipment" – отгрузки; тип сепаратора *ChoiseData*.

strNameMeas* operator() (const clsStorage& obj)

Оператор для вызова соответствующей функции ([GetNameMeas](#)) объекта типа *clsStorage*; тип возвращаемого объекта *strNameMeas*.

strNameMeas* operator() (const clsManufactory& obj)

Оператор для вызова соответствующей функции ([GetRawMatDescription](#) или [GetProductDescription](#)) объекта типа *clsManufactory*; тип возвращаемого объекта *strNameMeas*.

struct Getter_exporting_count

Структура с методами получения числа номенклатурных позиций в массиве с наименованиями ресурсов и их единиц измерения. Используется для экспорта в CSV-файлы. При использовании в качестве посетителя объекта типа *T_aggregate* в функции *std::visit*, возвращает число типа *size_t*. Запрашиваемое значение определяется сепаратором [ChoiseData](#) *arr*, который передаётся в качестве параметра в конструкторе структуры.

Getter_exporting_count(const ChoiseData _arr)

Конструктор с параметрами.

Параметры:

_arr – сепаратор выбора данных: "purchase" - поставки, "balance" - остатки/незавершенное производство, "shipment" – отгрузки; тип сепаратора *ChoiseData*.

size_t operator() (const clsStorage& obj)

Оператор для вызова соответствующей функции ([Size](#)) объекта типа *clsStorage*; тип возвращаемого значения *size_t*.

size_t operator() (const clsManufactory& obj)

Оператор для вызова соответствующей функции ([GetRMCount](#) или [GetProdCount](#)) объекта типа *clsManufactory*; тип возвращаемого значения *size_t*.

struct Calculater_back

Структура с методами расчёта потребности в ресурсах для классов [clsStorage](#) и [clsManufactory](#). Метод расчёта выбирается в зависимости от сепаратора [clc](#) на этапе компиляции.

TLack operator()(clsStorage& obj)

Оператор для вызова метода вычислений объекта типа *clsStorage* ([Calculate_future\(1\)](#), [Calculate_thread\(1\)](#) или [Calculate\(1\)](#), где "1" – параметр определяющий предмет вычисления); тип возвращаемого объекта [TLack](#).

TLack operator() (clsManufactory& obj)

Оператор для вызова метода вычислений объекта типа *clsManufactory* ([CalcRawMatPurchPlan future](#), [CalcRawMatPurchPlan thread](#) или [CalcRawMatPurchPlan](#)); вызываемые функции имеют тип *void*, оператор всегда возвращает объект типа *TLack* со значениями полей ноль и пустая строка.

```
struct Calculater_fwrd
```

Структура с методами расчёта стоимости для классов *clsStorage* и *clsManufactory*. Метод расчёта выбирается в зависимости от сепаратора [clc](#) на этапе компиляции.

TLack operator()(clsStorage& obj)

Оператор для вызова метода вычислений объекта типа *clsStorage* ([Calculate future\(2\)](#), [Calculate thread\(2\)](#) или [Calculate\(2\)](#)), где "2" – параметр определяющий предмет вычисления); тип возвращаемого объекта [TLack](#).

TLack operator() (clsManufactory& obj)

Оператор для вызова метода вычислений объекта типа *clsManufactory* ([Calculate future](#), [Calculate thread](#) или [Calculate](#)); вызываемые функции имеют тип *bool*: если результат работы функции равен *true*, то оператор возвращает объект типа *TLack* со значениями полей ноль и пустая строка, если же результат *false*, то оператор возвращает объект типа *TLack* со значениями полей "99999.99" и "false".

```
struct Setter_ship
```

Структура с методами ввода объемов отгрузок для классов *clsStorage* и *clsManufactory*. Объемы отгрузок в натуральном выражении вводятся в конструкторе.

Setter_ship(strItem* &_buffer)

Конструктор с параметрами.

Параметры:

_buffer – ссылка на указатель на массив с объемами отгрузок в натуральном выражении; массив одномерный, являющийся аналогом двумерной матрицы размером *количество_номенклатурных_позиций * число_периодов_проекта*; тип элемента массива [strItem](#). Массив, указанный в параметрах конструктора, перемещается внутрь структуры. После перемещения *_buffer* указывает на *nullptr*.

~Setter_ship()

Деструктор. Удаляет внутренний массив, если он существует.

bool operator() (clsStorage& obj)

Оператор для вызова соответствующей функции ([SetShipment](#)) объекта типа *clsStorage*; тип возвращаемого объекта *bool*.

bool operator() (clsManufactory& obj)

Оператор для вызова соответствующей функции ([SetProdPlan](#)) объекта типа *clsManufactory*; тип возвращаемого объекта *bool*.

```
struct Getter_ship
```

Структура с методами вывода объемов и стоимостных показателей отгрузок для классов *clsStorage* и *clsManufactory*.

strItem* operator() (const clsStorage& obj)

Оператор для вызова соответствующей функции ([GetShip](#)) объекта типа *clsStorage*; тип возвращаемого объекта *strItem*.

strItem* operator() (const clsManufactory& obj)

Оператор для вызова соответствующей функции ([GetTotalProduct](#)) объекта типа *clsManufactory*; тип возвращаемого объекта *strItem*.

struct Setter_purch

Структура с методами ввода цен и объемов поставок для классов *clsStorage* и *clsManufactory*. Цены вводятся в конструкторе.

Setter_purch(strItem* &_buffer)

Конструктор с параметрами.

Параметры:

_buffer – ссылка на указатель на массив с ценами; массив одномерный, являющийся аналогом двумерной матрицы размером *количество_номенклатурных_позиций * число_периодов_проекта*; тип элемента массива [strItem](#). Массив, указанный в параметрах конструктора, перемещается внутрь структуры. После перемещения *_buffer* указывает на *nullptr*.

~Setter_purch()

Деструктор. Удаляет внутренний массив, если он существует.

bool operator() (clsStorage& obj)

Оператор для вызова соответствующей функции ([SetPurchase](#)) объекта типа *clsStorage*; тип возвращаемого объекта *bool*.

bool operator() (clsManufactory& obj)

Оператор для вызова соответствующей функции ([SetRawMatPrice](#)) объекта типа *clsManufactory*; тип возвращаемого объекта *bool*.

struct Getter_purch

Структура с методами вывода объемов поставок в натуральном выражении для классов *clsStorage* и *clsManufactory*.

strItem* operator() (clsStorage& obj)

Оператор для вызова соответствующей функции ([GetPure](#)) объекта типа *clsStorage*; тип возвращаемого объекта *strItem*.

strItem* operator() (clsManufactory& obj)

Оператор для вызова соответствующей функции ([GetRMPurchPlan](#)) объекта типа *clsManufactory*; тип возвращаемого объекта *strItem*.

Getter_NameMeas

Структура с методами возврата указателя на вновь создаваемые массивы с наименованиями ресурсов и единицами их измерения для классов *clsStorage* и *clsManufactory*.

strNameMeas* operator() (const clsStorage& obj)

Оператор для вызова соответствующей функции ([GetNameMeas](#)) объекта типа *clsStorage*; тип возвращаемого объекта *strNameMeas*.

strNameMeas* operator() (const clsManufactory& obj)

Оператор для вызова соответствующей функции ([GetRawMatDescription](#)) объекта типа *clsManufactory*; тип возвращаемого объекта *strNameMeas*.

struct Setter_progress_message

Структура для установки сообщения индикатора прогресса для классов *clsStorage* и *clsManufactory*. Сообщение вводится при инициализации экземпляра структуры.

void operator() (clsStorage& obj)

Оператор для вызова соответствующей функции ([Set_progress_message](#)) объекта типа *clsStorage*.

void operator() (clsManufactory& obj)

Оператор для вызова соответствующей функции ([Set_progress_message](#)) объекта типа *clsManufactory*.

КЛАСС *clsAgregate*

Класс *clsAgregate* предназначен для объединения *неопределенного* числа объектов типа [clsStorage](#) и [clsManufactory](#) в последовательную цепочку элементов контейнера. В качестве контейнера используется [std::vector](#) с типом элемента [std::variant](#). Класс *clsAgregate* является наследником класса [clsBaseProject](#); к полям и методам базового класса, добавлены поля и методы для работы с последовательной цепочкой объектов *clsStorage* и *clsManufactory*. Поскольку количество таких объектов в данном классе не определено, работа с этим классом напрямую невозможна. Для работы с классом *clsAgregate* необходимо создать и использовать дочерний для него класс, в котором будет установлено такое количество. Именно поэтому конструктор класса *clsAgregate* находится в секции *protected*, чтобы предотвратить создание экземпляра класса. В дочернем классе должно быть установлено конкретное число элементов контейнера и прописан публичный конструктор.

Состав класса *clsAgregate* представлен в таблице ниже. В секции *private* нет полей. Приведенные в таблице поля относятся к секции *protected*.

ПОЛЯ КЛАССА *clsAgregate*

Таблица 24 Поля класса *clsAgregate*

Поле класса	Описание	Заметка
size_t PrCount	Количество периодов проекта	Секция <i>protected</i>
vector<T_aggregate> CostChain	Контейнер вариантного типа T_aggregate для объединения объектов типа <i>clsStorage</i> и <i>clsManufactory</i>	Секция <i>protected</i>
strItem* Purchase	Указатель на массив с ценами и объемами закупок; одномерный аналог матрицы размером <i>количество_номенклатурных_позиций</i> * <i>число_периодов_проекта</i> ; тип элемента массива strItem	Секция <i>protected</i>
strItem* Shipment	Указатель на буферный массив. До начала вычислений содержит объемы отгрузок; после всех вычислений - объемы и стоимостные данные; одномерный аналог матрицы размером <i>количество_номенклатурных_позиций</i> * <i>число_периодов_проекта</i> ; тип элемента массива <i>strItem</i>	Секция <i>protected</i>
size_t sh_size	Динамическое число строк массива <i>Shipment</i>	Секция <i>protected</i>

Методы класса *clsAgregate* можно условно разделить на следующие группы:

- Конструкторы, деструктор, метод обмена; Перегрузка операторов присваивания;
- Get – методы;
- Set – методы;
- Расчётные методы;
- Методы сериализации и десериализации;
- Методы визуального контроля (View- методы).

КОНСТРУКТОРЫ, ДЕКТРУКТОР, МЕТОД ОБМЕНА; ПЕРЕГРУЗКА ОПЕРАТОРОВ ПРИСВАИВАНИЯ

clsAgregate::clsAgregate()

Конструктор по умолчанию. Чтобы запретить создание экземпляров данного класса, конструктор перенесен в секцию *protected*. В наследуемых классах конструкторы необходимо объявлять в секции *public*.

Конструктор устанавливает количество периодов проекта *PrCount* и динамическое число строк массива *sh_size* равными нулю. Указатели на массивы *Purchase* и *Shipment* в *nullptr*.

Используется в методах сериализации и десериализации.

void clsAgregate::swap(clsAgregate& other) noexcept

Функция обмена значениями между объектами. Функция объявлена *noexcept* - не вызывающей исключения.

Параметры:

& other – ссылка на обмениваемый экземпляр класса *clsAgregate*.

clsAgregate::clsAgregate(const clsAgregate& other)

Конструктор копирования.

Параметры:

& other – ссылка на копируемый константный экземпляр класса *clsAgregate*.

clsAgregate::clsAgregate(clsAgregate&& other)

Конструктор перемещения.

Параметры:

&& other – перемещаемый экземпляр класса *clsAgregate*.

clsAgregate::~~clsAgregate()

Виртуальный деструктор.

clsAgregate& clsAgregate::operator=(const clsAgregate& rhs)

Перегрузка оператора присваивания копированием. Реализовано в идиоме КОПИРОВАНИЯ-И-ЗАМЕНЫ (*copy-and-swap idiom*).

Параметры:

& rhs – ссылка на копируемый константный экземпляр класса *clsAgregate*.

clsAgregate& clsAgregate::operator=(clsAgregate&& rhs)

Перегрузка оператора присваивания перемещением. Реализовано в идиоме ПЕРЕМЕЩЕНИЯ-И-ЗАМЕНЫ (*move-and-swap idiom*).

Параметры:

&& rhs – перемещаемый экземпляр класса *clsAgregate*.

GET – МЕТОДЫ**size_t clsAgregate::ProjectCount()**

Метод возвращает количество периодов проекта; тип возвращаемого значения *size_t*.

size_t clsAgregate::Counts(const ChoiseData _arr, const size_t _count)

Метод возвращает число ресурсов выбранного подразделения.

Параметры:

_arr – сепаратор выбора данных: "purchase" - поставки, "balance" - остатки/незавершенное производство, "shipment" – отгрузки; тип параметра [ChoiseData](#);

_count - индекс выбранного подразделения: 0 - первое подразделение, принимающее ресурсы (элемент контейнера [CostChain](#) с индексом ноль); подразделение, отгружающее готовую продукцию, будет иметь самый большой номер (элемент контейнера *CostChain* с максимальным индексом). Тип возвращаемого значения *size_t*.

strNameMeas* clsAgregate::GetNameMeas(const ChoiseData _arr, const size_t _count) const

Метод возвращает указатель на новый массив типа [strNameMeas](#) с названием ресурсов и единицами измерения. Число элементов массива определяется результатом работы предыдущей функции (*clsAgregate::Counts*).

Параметры:

_arr – сепаратор выбора данных: "purchase" - поставки, "balance" - остатки/незавершенное производство, "shipment" – отгрузки; тип параметра [ChoiseData](#);

_count - индекс выбранного подразделения: 0 - первое подразделение, принимающее ресурсы (элемент контейнера [CostChain](#) с индексом ноль); подразделение, отгружающее готовую продукцию, будет иметь самый большой номер (элемент контейнера *CostChain* с максимальным индексом). Тип возвращаемого значения *size_t*.

bool clsAgregate::Export_Data(const string filename, const size_t _dep, const ChoiseData _arr, const ReportData& flg) const

Метод записывает массив поставок, остатков или отгрузок выбранного подразделения в *csv*-файл с именем *filename*. Если экспорт удачен, функция возвращает *true*, иначе возвращает *false*.

Параметры:

Filename – имя файла для экспорта данных; тип *string*;

_dep - индекс подразделения в цепочке подразделений [CostChain](#); тип *size_t*;

_arr – сепаратор выбора данных: "purchase" - поставки, "balance" - остатки/незавершенное производство, "shipment" - отгрузки; тип [ChoiseData](#);

flg – сепаратор типа выводимой в файл информации: "volume" - в натуральном, "value" - в стоимостном, "price" - в ценовом измерении; тип [ReportData](#).

void clsAgregate::reportstream(ostream& os) const

Виртуальный метод из секции protected. Перегружает одноименный метод базового класса [clsBaseProject](#).

Параметры:

os — ссылка на выходной поток типа [ostream](#).

bool clsAgregate::Detail_report(const size_t _dep, const string _depName[], const ChoiseData _arr, const ReportData& flg) const

Функция выводит выбранный отчет. Отчет выводится в файл, заданный именем, содержащимся в поле [RName](#) базового класса. Если вывод отчёта удачен, функция возвращает *true*, иначе возвращает *false*.

Параметры:

_dep - индекс подразделения в цепочке подразделений [CostChain](#); тип *size_t*;

_depName - указатель на массив с названиями подразделений; может указывать на *nullptr*; в этом случае вместо названий подразделений в отчёт будут выведены их номера; тип элемента массива *string*;

_arr — сепаратор выбора данных: "purchase" - поставки, "balance" - остатки/незавершенное производство, "shipment" - отгрузки; тип [ChoiseData](#);

flg — сепаратор типа выводимой в файл информации: "volume" - в натуральном, "value" - в стоимостном, "price" - в ценовом измерении; тип [ReportData](#).

SET - МЕТОДЫ

void clsAgregate::Set_progress_shell(clsProgress_shell<type_progress>* _val)

Функция присваивает указателю *pshell* ([clsStorage::pshell](#) и [clsManufactory::pshell](#)) каждого объекта из контейнера *CostChain* адрес объекта *val* - [оболочки для индикатора прогресса](#).

Параметры:

_val — указатель на оболочку индикатора прогресса; тип оболочки [clsProgress_shell<type_progress>](#), где *type_progress* — псевдоним типа используемого класса индикатора прогресса (см. раздел «Отображение прогресса при выполнении расчетов»).

void clsAgregate::Set_share(const size_t index, const decimal _share[])

Функция устанавливает нормативы запасов на складе.

Параметры:

index - индекс элемента контейнера *CostChain*, в котором находится объект типа [clsStorage](#); тип *size_t*;

_share - указатель на массив с нормативами запасов ресурсов для каждой номенклатурной позиции склада. Запас выражается в долях от объемов отгрузки. Тип вещественного числа элемента массива определяется псевдонимом [decimal](#).

Если в контейнере под индексом *index* тип элемента отличается от *clsStorage*, метод не производит никаких действий.

void clsAgregate::Set_permission(const size_t index, const bool _perm[])

Функция устанавливает разрешение/запрет на отгрузку и закупку на складе в одном и том же периоде.

Параметры:

index - индекс элемента контейнера *CostChain*, в котором находится объект типа [clsStorage](#); тип *size_t*;

_perm — указатель на массив с флагами разрешения; тип *bool*.

Если в контейнере под индексом *index* тип элемента отличается от *clsStorage*, метод не производит никаких действий.

void clsAggregate::Set_autopurchase(const size_t index, const PurchaseCalc _pcalc[])

Параметры:

index - индекс элемента контейнера *CostChain*, в котором находится объект типа [clsStorage](#); тип *size_t*;

_pcalc - указатель на массив с флагами авторасчета. Тип элемента массива [PurchaseCalc](#).

void clsAggregate::Set_autopurchase(const size_t index, const size_t _pcalc[])

Аналогичен предыдущему методу. Отличие – в типе элементов массива *_pcalc*, передаваемого по указателю параметром в функцию.

Параметры:

index - индекс элемента контейнера *CostChain*, в котором находится объект типа [clsStorage](#); тип *size_t*;

_pcalc - указатель на массив с флагами авторасчета. Тип элемента массива *size_t*.

bool clsAggregate::Set_ship_volume(const strItem _Shipment[])

Метод ввода объемов отгрузки в подразделение с *самым большим* индексом. Метод возвращает *true* при удачном копировании входного массива во внутренний массив [Shipment](#). Иначе возвращает *false*.

Параметры:

_Shipment – указатель на вводимый массив с отгрузками. Тип элемента массива [strItem](#).

bool Set_pur_price(const strItem _Purchase[])

Метод ввода цен поставок в подразделение с *нулевым* индексом. Метод возвращает *true* при удачном копировании входного массива во внутренний массив [Purchase](#).

Параметры:

_Purchase – указатель на вводимый массив с поставками. Тип элемента массива *strItem*.

bool Set_pur_volume(const strItem _Pur_vol[])

Метод ввода объемов поставок в подразделение типа *clsStorage* с самым низким индексом. Используется в случае, когда хотя бы для одного SKU этого склада установлен флаг ручного ввода объемов поставок (флаг "[nocalc](#)" - без автоматического расчета). **ВНИМАНИЕ!!!** Метод необходимо вызвать **ДО** вызова функции [RCalculate](#). Метод возвращает *true* при удачном копировании входного массива во внутренний массив склада с самым низким индексом.

Параметры:

_Pur_vol – указатель на вводимый массив с поставками. Тип элемента массива *strItem*.

void clsAggregate::Reset()

Виртуальный метод сброса состояния объекта; перегружает одноименный метод базового класса [clsBaseProject](#).

МЕТОДЫ ВИЗУАЛЬНОГО КОНТРОЛЯ

void clsAggregate::ViewSettings() const

Функция выводит на экран информацию о настройках для расчета. В случае, если элемент контейнера имеет тип [clsStorage](#), функция вызывает метод [clsStorage::ViewSettings](#); если же элемент контейнера имеет тип [clsManufactory](#), вызывается метод [clsManufactory::ViewProjectParams](#).

РАСЧЁТНЫЕ МЕТОДЫ

bool clsAgregate::RCalculate(const string msg[])

Метод рассчитывает объем ресурсов в натуральном выражении, поступающих в подразделение с *нулевым* индексом. Если расчёты завершились удачно (метод вернул *true*), в массиве [Purchase](#) будут содержаться расчётные данные.

Параметры:

msg - указатель на массив с наименованиями индикаторов прогресса. Тип элемента массива *string*.

Функция вводит данные об отгрузке из массива *Shipment* в элемент контейнера с максимальным индексом, рассчитывает потребность в ресурсах для этого элемента, выгружает рассчитанные данные снова в массив *Shipment*. И так повторяет этот цикл для всех элементов контейнера в реверсивном направлении (от больших индексов к меньшим). Когда оказывается рассчитан элемент контейнера с нулевым индексом, массив *Shipment* копируется в массив *Purchase*.

bool clsAgregate::FCalculate(const string msg[])

Метод рассчитывает удельную и полную стоимости ресурсов, отгружаемых из подразделения с *максимальным* индексом. Если расчёты завершились удачно (метод вернул *true*), в массиве [Shipment](#) будут содержаться расчётные данные.

Параметры:

msg - указатель на массив с наименованиями индикаторов прогресса. Тип элемента массива *string*.

Функция вводит данные о поставке из массива *Shipment* в элемент контейнера с нулевым индексом, рассчитывает стоимостные показатели отгрузки из этого элемента, выгружает рассчитанные данные снова в массив *Shipment*. И так повторяет этот цикл для всех элементов контейнера в прямом направлении (от меньших индексов к большим). Когда оказывается рассчитан элемент контейнера с максимальным индексом, в массиве *Shipment* остается результат в виде стоимостных показателей отгрузки предприятия.

МЕТОДЫ СЕРИАЛИЗАЦИИ И ДЕСЕРИАЛИЗАЦИИ

Методы сериализации и десериализации являются *виртуальными* и перегружают одноименные методы базового класса [clsBaseProject](#). Методы находятся в секции *protected* и вызываются внутри методов [clsBaseProject::SaveToFile](#) и [clsBaseProject::ReadFromFile](#).

bool clsAgregate::StF(ofstream &_outF)

Метод имплементации записи в файловую переменную текущего экземпляра класса.

Параметры:

&_outF – ссылка на экземпляр класса [ofstream](#) для записи данных; тип *ofstream*.

bool clsAgregate::RfF(ifstream &_inF)

Метод имплементации чтения из файловой переменной экземпляра класса.

Параметры:

&_inF - ссылка на экземпляр класса [ifstream](#) для чтения данных; тип *ifstream*.

ОТОБРАЖЕНИЕ ПРОГРЕССА ПРИ ВЫПОЛНЕНИИ РАСЧЕТОВ

Как указывалось в разделе «[МОДУЛЬ PROGRESS](#)», присутствующие в нем инструменты позволяют визуализировать ход выполнения расчетов как в однопоточном, так и в многопоточном режимах, как в консольных, так и оконных приложениях. Состав модуля представлен в таблице ниже.

Таблица 25 Состав модуля PROGRESS

Состав модуля	Описание	Заметка
template <typename T> class clsProgress_shell	Оболочка для любого класса прогресс-бара типа <i>T</i> .	Данный класс представляет собой интерфейс, который используют объекты библиотеки FROMA2 для подключения сторонних произвольных индикаторов прогресса (например, wxProgressDialog из библиотеки wxWidgets).
class clsprogress_bar	Простой консольный индикатор прогресса	Один из возможных индикаторов прогресса для консольных приложений.

Исходный код программы для демонстрации представлен в [репозитории](#) в папке [froma2/examples/Progress_example](#); файл main.cpp.

КЛАСС clsProgress_shell

Данный класс является шаблонным и подразумевает явное указание типа класса индикатора, который будет в нем использоваться. Например, для использования прогресс-индикатора из этого же модуля, объявление переменной “x” будет выглядеть как `class clsProgress_shell<clsprogress_bar> x`.

ПОЛЯ КЛАССА clsProgress_shell

Таблица 26 Поля класса clsProgress_shell. Секция private

Поле класса	Описание	Заметка
T* pbar	Указатель шаблонного типа <i>T</i> на объект прогресс-индикатор	По умолчанию равен nullptr.
atomic<int> counter	Счетчик выполненных операций; тип atomic<int> .	При создании инициализируется нулем. Используется для подсчета выполненных операций в <i>многопоточных</i> вычислениях.
int maxcounter	Максимальное число операций; тип <i>int</i> .	По умолчанию равен 100.
int stepcount	Шаг, на который увеличивается значение счётчика при выполнении единичной итерации в цикле; тип <i>int</i> .	По умолчанию равен 1.

МЕТОДЫ КЛАССА clsProgress_shell

clsProgress_shell(T* _pbar, const int _maxcounter, const int _stepcount)

Конструктор с параметрами.

Параметры:

_pbar – указатель на объект прогресс-индикатора типа *T*; при *_pbar = nullptr* вывод индикатора не производится;

_maxcounter – максимальное число операций; тип *int*;

_stepcount – шаг, на который увеличивается значение счётчика при выполнении единичной итерации в цикле; тип *int*.

void Set_shell(T* _pbar, const int _maxcounter, const int _stepcount)

Метод устанавливает указатель на новый объект прогресс-индикатор и инициализирует поля заново.

Параметры:

`_pbar` – указатель на объект прогресс-индикатор типа *T*;

`_maxcounter` – максимальное число операций; тип *int*;

`_stepcount` – шаг, на который увеличивается значение счётчика при выполнении единичной итерации в цикле; тип *int*.

void Counter_inc()

Метод увеличивает значение поля счетчика `counter` на заданную величину `stepcount`. Метод предназначен для внедрения в каждый поток при многопоточном вычислении.

Логика индикации при многопоточных вычислениях следующая. В каждом потоке, где выполняется какая-либо задача, после ее благополучного завершения, вызывается метод `Counter_inc`, который увеличивает счетчик выполненных задач. В основном потоке другой метод (см. ниже метод `Progress_indicate`) считывает изменение счетчика и выводит индикатор на экран. Если в потоке выполняется цикл с вычислениями, то метод `Counter_inc` вызывается внутри тела этого цикла при каждой удачной итерации.

void Progress_indicate()

Метод считывает изменение счетчика выполненных в других потоках операций и выводит индикатор прогресса на экран. Используется для вывода индикатора прогресса в многопоточных вычислениях. При использовании экземпляров объекта типа `thread` для организации многопоточных вычислений, вызов метода `Progress_indicate` рекомендуется осуществлять в основном потоке после добавления задачи в поток и перед присоединением этого потока к главному (до вызова `join`).

При значении поля `pbar` равном `nullptr`, метод завершает работу без индикации прогресса.

void Counter_reset()

Метод устанавливает поле счетчика `counter` в ноль. Вызывается перед повторным использованием экземпляра класса `clsProgress_shell`.

void Counter_max()

Метод устанавливает поле счетчика `counter` в максимальное значение, равное `maxcounter`. Вызывается при досрочном завершении работы потока (прерывании цикла вычислений). Обеспечивает корректное завершение работы метода `Progress_indicate`.

void Update(int val)

Метод вывода индикатора прогресса на экран при однопоточных вычислениях. При однопоточных вычислениях счетчик `counter` не используется. Вызывается отрисовка индикатора в соответствии с переданным параметром. Вызов метода располагается в цикле с вычислениями сразу за благополучно завершенной операцией; в качестве параметра передается значение *счетчика цикла*.

Параметры:

`val` – номер благополучно завершенной операции; совпадает со значением *счетчика цикла* с вычислениями. Тип *Int*.

int Getmaxcounter()

Метод возвращает значение максимального числа операций `maxcounter`.

int Getstepcount()

Метод возвращает значение шага, на который увеличивается значение счётчика при выполнении единичной итерации в цикле [stepcount](#).

void SetMessage(string&& _message)

Метод устанавливает новое значение сообщения во время вывода индикатора. Вызывает метод [clsprogress_bar::Set_message](#).

Параметры:

_message – ссылка на сообщение, являющееся началом индикатора прогресса; тип указатель на *string*.

void Set_maxcount(const int _mx)

Метод устанавливает новое значение максимального числа итераций во время вывода индикатора. Вызывает метод [clsprogress_bar::Set_maxcount](#).

Параметры:

_mx – новое значение максимального числа итераций; тип *int*.

КЛАСС clsprogress_bar

Данный класс представляет собой вариант прогресс-индикатора для консольных приложений¹⁸. Вместо использования объекта данного класса для визуализации прогресса вычислений может быть использован экземпляр любого другого класса аналогичного назначения. Основное требование к такому классу – совпадение сигнатуры метода вывода индикатора *Update*.

ПОЛЯ КЛАССА clsprogress_bar

Таблица 27 Поля класса `clsprogress_bar`. Секция `private`

Поле класса	Описание	Заметка
<code>static const int overhead = sizeof"[100%]"</code>	Размер заданной константной части строки	
<code>ostream &os</code>	Выходной поток для вывода индикатора	
<code>const int bar_width</code>	Ширина индикатора в символах	
<code>string message</code>	Сообщение, являющееся началом индикатора	
<code>const string full_bar</code>	Строка, выбранная часть которой выводится как индикатор прогресса	
<code>int maxcount</code>	Максимальное число итераций/ операций, соответствующее 100% выполнению задачи	

МЕТОДЫ КЛАССА clsprogress_bar**void write(double fraction)**

Метод выводит в выходной поток индикатор прогресса, соответствующий *текущему состоянию* вычислительной задачи. Метод *private*.

¹⁸ Класс `clsprogress_bar` сделан с использованием кода Toby Speight “An alternative approach”, <https://codereview.stackexchange.com/questions/186535/progress-bar-in-c>

Параметры:

fraction – дробное число в диапазоне от 0 до 1, соответствующее текущему состоянию выполнения вычислительной задачи; тип *double*. Значение “0” соответствует состоянию перед началом выполнения задачи, значение “100” соответствует завершенной задаче.

clsprogress_bar(ostream &_os, int const line_width, string&& _message, const char symbol='.', const int _mx=100)

Конструктор с параметрами.

Параметры:

_os – выходной поток для отображения индикатора; тип *ostream*;

line_width – ширина (длина строки) индикатора в символах; тип *int*;

_message – ссылка на сообщение, являющееся началом индикатора прогресса; тип указатель на *string*;

symbol – символ заполнителя индикатора прогресса; тип *char*;

_mx – максимальное число итераций/ операций, соответствующее 100% выполнению задачи; тип *int*.

clsprogress_bar(const clsprogress_bar&) = delete

Конструктор копирования; копирование экземпляров класса запрещено.

clsprogress_bar& operator=(const clsprogress_bar&) = delete

Оператор присвоения копированием; присвоение экземпляров класса запрещено.

~clsprogress_bar()

Деструктор.

void Update(int value)

Метод выводит в выходной поток индикатор прогресса, соответствующий *текущему состоянию* вычислительной задачи. Метод *public*.

Параметры:

value – целое число в диапазоне от 0 до *maxcount*, соответствующее текущему состоянию выполнения вычислительной задачи; тип *double*. Значение “0” соответствует состоянию перед началом выполнения задачи, значение *maxcount* соответствует завершенной задаче. Тип параметра *int*.

Название и сигнатура метода совпадает с названием и сигнатурой метода аналогичного назначения класса [wxProgressDialog](#) библиотеки [wxWidgets](#).

void Set_maxcount(const int _mx)

Метод устанавливает новое значение максимального числа операций *maxcount*.

Параметры:

_mx – новое значение поля *maxcount*.

int Getmaxcount()

Метод возвращает значение максимального числа операций, соответствующее 100% выполнению задачи *maxcount*.

void Set_message(string&& _message)

Метод устанавливает новое значение сообщения во время вывода индикатора.

Параметры:

_message – ссылка на сообщение, являющееся началом индикатора прогресса; тип указатель на *string*.

КАК ВКЛЮЧИТЬ И ОТКЛЮЧИТЬ ИНДИКАЦИЮ ПРОГРЕССА

При желании включить индикацию прогресса при выполнении расчетов в классе *clsStorage*, необходимо выполнить следующие действия:

1. В модуле *COMMON VALUES* определить, какой прогресс бар будет использован в программе. Например, в качестве индикатора прогресса будем использовать класс *clsprogress_bar*. Установим для этого класса псевдоним *type_progress*:

```
typedef clsprogress_bar type_progress;
```

2. Создать экземпляр класса *индикатора прогресса* (например, с именем *progress*), используя псевдоним:

```
type_progress* progress = new type_progress(std::clog, 70u, "Working");
```

где *std::clog* – поток для вывода индикатора в консоль;

70u – количество символов индикатора (тип *unsigned int*);

"Working" – строка, являющаяся началом индикатора прогресса.

3. Создать экземпляр класса *оболочки* для индикатора прогресса и передать ему указатель на экземпляр класса индикатора прогресса:

```
clsProgress_shell<type_progress>* shell = new clsProgress_shell<type_progress>(progress, RMCount, 1);
```

где *progress* – указатель на ранее созданный экземпляр класса индикатора прогресса;

RMCount – число *SKU*¹⁹, по каждому из которых будет производится расчет;

1 – шаг шкалы индикатора прогресса.

4. Передать экземпляру класса *clsStorage* (например, с именем *Warehouse*) указатель на экземпляр класса оболочки для индикатора прогресса:

```
Warehouse->Set_progress_shell(shell);
```

5. По окончании работы с индикатором удалить динамические объекты (если оболочка и индикатор создавались, как динамические объекты) и установить указатель *clsStorage::pshell* на *nullptr* (во избежании указания на неопределённую область):

```
delete shell;
shell = nullptr;
delete progress;
progress = nullptr;
Warehouse->Set_progress_shell(nullptr);
```

¹⁹ *Stock Keeping Unit* - единица складского учета

Для отключения ранее включенной индикации можно выполнить следующие *альтернативные* действия:

Вариант 1	Вариант 2	Вариант 3	Вариант 4
Warehouse- >Set_progress_shell(nullptr);	shell-> Set_shell(nullptr);	delete shell; shell = nullptr; Warehouse- >Set_progress_shell(nullptr)	delete progress; progress = nullptr; shell-> Set_shell(nullptr);

Аналогично можно включить индикацию прогресса при выполнении расчетов в классе *clsManufactory*. Если в программе используются объекты обоих классов (*clsStorage* и *clsManufactory*), то создавать второй экземпляр класса *индикатора* прогресса и экземпляр класса *оболочки* для индикатора прогресса нет необходимости. В этом случае для включения индикации достаточно передать экземпляру класса *clsManufactory* (например, с именем *Manufactory*) указатель на *существующий* экземпляр класса оболочки для индикатора прогресса (не забыв перед этим установить указатель *pshell* использованного ранее класса в *nullptr*). Важно не забыть перед повторным использования индикатора прогресса сделать *сброс счетчика* с помощью метода *clsProgress_shell::Counter_reset*! Удаление динамических объектов оболочки и индикатора производится в самом конце программы после того, как они более не нужны.

```
Manufactory ->Set_progress_shell(shell);
```

ИНСТРУМЕНТЫ ДЛЯ ОТОБРАЖЕНИЯ РЕЗУЛЬТАТОВ РАСЧЕТОВ

Для отображения результатов расчётов в модуле *COMMON VALUES* предусмотрены два набора инструментов:

- классы и функции визуального контроля работоспособности методов из модулей *warehouse* и *manufact* (для удобства «отлова ошибок» при модернизации последних);
- классы и методы для вывода исходных данных и результатов расчетов на терминал и/или в текстовый файл.

КОНСТАНТЫ И КЛАССЫ ДЛЯ МЕТОДОВ ВИЗУАЛЬНОГО КОНТРОЛЯ И ОТЧЕТОВ

Константы и классы, используемые для методов визуального контроля, и для методов вывода отчетов находятся в пространстве имен *nmPrntSetup* модуля *COMMON VALUES*. Состав этого пространства имен приведен ниже.

Таблица 28 Состав пространства имен *nmPrntSetup*

Состав пространства имен <i>nmPrntSetup</i>	Описание	Заметка
const size_t limit = 9	Константа типа <i>size_t</i> . Предельное число столбцов одного фрагмента таблицы для вывода отчета; только для методов визуального контроля	
const size_t sPrec = 2	Константа типа <i>size_t</i> . Количество знаков после запятой в числовых данных (точность, разрядность)	
const char chFill = ' '	Константа типа <i>char</i> . Заполнитель (пробел)	
const string c_TableName = "Name", c_TableMeas = "Measure", c_ByVolume = "By volume", c_ByPrice = "By price", c_ByValue = "By value", c_HCurrency = "RUR";	Константы типа <i>string</i> . Наименования столбцов; только для методов визуального контроля	

class clsTextField	Класс для предустановки параметров печати текстовых полей ²⁰
class clsDataField	Класс для предустановки параметров печати полей с числовыми данными

Если требуемое количество столбцов таблицы больше, чем *limit*, то таблица разбивается на фрагменты и выводится один фрагмент под другим. В каждом фрагменте выводятся названия строк и столбцов.

КЛАСС CLASS clsTextField

clsTextField(size_t width)

Конструктор, устанавливающий ширину текстового поля при выводе данных в табличном виде.

Параметры:

width – ширина текстового поля в таблице; тип *size_t*.

friend std::ostream& operator<<(std::ostream& dest, clsTextField const& manip)

«Дружественный классу» перегруженный оператор вывода в поток. Обеспечивает форматированный вывод текстовой информации в поток.

Параметры:

&dest - выходной поток для вывода информации; тип [ostream](#);

&manip – предустановки форматированного вывода; тип *clsTextField*.

КЛАСС CLASS clsDataField

clsDataField(size_t width)

Конструктор, устанавливающий ширину поля с числовыми данными при их выводе в табличном виде.

Параметры:

width – ширина поля с числовыми данными в таблице; тип *size_t*.

clsDataField(size_t width, size_t prec)

Конструктор, устанавливающий ширину поля с числовыми данными и количества знаков после запятой у чисел при их выводе в табличном виде.

Параметры:

width – ширина поля с числовыми данными в таблице; тип *size_t*;

prec – количество знаков после запятой у выводимых в таблицу чисел; тип *size_t*.

friend std::ostream& operator<<(std::ostream& dest, clsDataField const& manip)

«Дружественный классу» перегруженный оператор вывода в поток. Обеспечивает форматированный вывод числовой информации в поток.

Параметры:

&dest - выходной поток для вывода информации; тип [ostream](#);

²⁰ В классах clsTextField и clsDataField использовалось решение James Kanze из <https://stackoverflow.com/questions/9206669/making-a-table-with-printf-in-c>

&manip – предустановки форматированного вывода; тип *clsDataField*.

МЕТОДЫ ВИЗУАЛЬНОГО КОНТРОЛЯ

Методы визуального контроля и необходимые для их работоспособности константы находятся в пространстве имен **nmPrntSrvs** модуля [COMMON VALUES](#). Состав этого пространства имен приведен ниже.

Таблица 29 Состав пространства имен nmPrntSrvs

Состав пространства имен nmPrntSrvs	Описание	Заметка
const size_t sNames = 15	Константа типа <i>size_t</i> . Установка формата вывода <i>имен</i> для использования в классе clsTextField .	Размеры полей выводимых таблиц, определяемые данными константами, являлись оптимальными для набора данных, использованных при проверке работоспособности библиотеки FROMA2 её автором. Они могут быть не оптимальными для других пользователей с другими наборами данных. Поэтому для вывода отчетов рекомендуются методы из следующего раздела.
const size_t sMeas = 12	Константа типа <i>size_t</i> . Установка формата вывода названий <i>единиц измерения</i> для использования в классе clsTextField .	
const size_t sNumb = 12	Константа типа <i>size_t</i> . Установка формата вывода <i>числовых</i> данных для использования в классе clsDataField .	
const string CurUnt = c_HCurrency + "/"	Константа типа <i>string</i> . Префикс для единицы денежного измерения удельных ценовых и стоимостных показателей.	
ArrPrint	Шаблонная функция отображения таблиц, состоящих из первого столбца наименований, второго столбца с единицами измерения и последующих столбцов с данными	

Ниже дано подробное описание указанных в таблице выше функций.

template<typename Tdata, typename TName>

void ArrPrint(const size_t ncount, const TName names[], const Tdata data[], const size_t dcount)

Шаблонная функция отображения таблиц, состоящих из первого столбца наименований, второго столбца с единицами измерения и последующих столбцов с данными. Ширина полей фиксированная и не изменяемая, определяется константами *sNames*, *sMeas* и *sNumb*. Количество столбцов одного фрагмента неизменяемо и равно [limit](#).

Параметры:

ncount – число строк, равное числу элементов массива *names[]*; тип *size_t*;

names[] – указатель на массив с наименованиями ресурсов и единицами их измерения; каждое название начинает новую строку таблицы; тип элементов массива определяется шаблоном типа *TName*. В данной версии библиотеки FROMA2 предполагается, что тип *TName* принимает значение единственного типа [strNameMeas](#);

data[] - одномерный массив, аналог двумерной матрицы размером *ncount*dcount* с данными; тип элементов массива определяется шаблоном типа *Tdata*. Предполагается подстановка любых арифметических типов и типа [LongReal](#);

`dcount` – число столбцов условной матрицы `data[]`; тип `size_t`;

template<typename Tdata, typename TName>

void ArrPrint(const size_t ncount, const TName names[], const Tdata data[], const size_t dcount, ReportData flg)

Функция отображения таблиц, аналогичная предыдущей. Отличается дополнительным параметром, позволяющим делать выбор между выводимыми полями, когда массив с данными представлен структурным типом.

Параметры:

`ncount` – число строк, равное числу элементов массива `names[]`; тип `size_t`;

`names[]` – указатель на массив с наименованиями ресурсов и единицами их измерения; каждое название начинает новую строку таблицы; тип элементов массива определяется шаблоном типа `TName`. В данной версии библиотеки FROMA2 предполагается, что тип `TName` принимает значение единственного типа `strNameMeas`;

`data[]` - одномерный массив, аналог двумерной матрицы размером `ncount*dcount` с данными; тип элементов массива определяется шаблоном типа `Tdata`. Предполагается подстановка структурного типа. В данной версии библиотеки FROMA2 предполагается, что тип `Tdata` принимает значение единственного типа `strItem`;

`dcount` – число столбцов условной матрицы `data[]`; тип `size_t`;

`flg` – флаг, определяющий, какое поле структурного типа `Tdata` будет выведено: `volume`, `price` или `value`.

template<typename Tdata, typename TName>

void ArrPrint(const size_t ncount, const TName names[], const Tdata data[], const size_t dcount, ReportData flg, const string& _hmcurl)

Функция отображения таблиц, аналогичная предыдущей. Отличается ещё одним дополнительным параметром - валютой, которую можно отобразить в выводимых таблицах вне зависимости от предустановки поля `c_HCurrency`. При этом перечень валют не ограничивается множеством перечисленных значений в массиве `CurrencyTXT`.

Параметры:

`ncount` – число строк, равное числу элементов массива `names[]`; тип `size_t`;

`names[]` – указатель на массив с наименованиями ресурсов и единицами их измерения; каждое название начинает новую строку таблицы; тип элементов массива определяется шаблоном типа `TName`. В данной версии библиотеки FROMA2 предполагается, что тип `TName` принимает значение единственного типа `strNameMeas`;

`data[]` - одномерный массив, аналог двумерной матрицы размером `ncount*dcount` с данными; тип элементов массива определяется шаблоном типа `Tdata`. Предполагается подстановка структурного типа. В данной версии библиотеки FROMA2 предполагается, что тип `Tdata` принимает значение единственного типа `strItem`;

`dcount` – число столбцов условной матрицы `data[]`; тип `size_t`;

`flg` – флаг, определяющий, какое поле структурного типа `Tdata` будет выведено: `volume`, `price` или `value`;

`&_hmcurl` – наименование валюты, выводимой в таблицах с денежными данными; тип – ссылка на строку с наименованием валюты.

КЛАССЫ И ФУНКЦИИ ДЛЯ ВЫВОДА ОТЧЕТА

Инструменты для вывода отчетов находятся в пространстве имен **nmRePrint**. Состав этого пространства имен приведен в таблице ниже.

Таблица 30 Состав пространства имен nmPrntSetup

Состав пространства имен nmRePrint	Описание	Заметка
const size_t c_n_min = 11	Константа типа <i>size_t</i> . Минимальный размер поля для названия продукта/ ресурса	
const size_t c_m_min = 12	Константа типа <i>size_t</i> . Минимальный размер поля для названия единицы измерения	
const size_t c_d_min = 3	Константа типа <i>size_t</i> . Минимальный размер поля для числовых данных	
const size_t c_n_max = 15	Константа типа <i>size_t</i> . Максимальный размер поля для названия продукта/ ресурса	
const size_t c_m_max = 15	Константа типа <i>size_t</i> . Максимальный размер поля для названия единицы измерения	
const size_t c_d_max = 16	Константа типа <i>size_t</i> . Максимальный размер поля для числовых данных	
const size_t smblcunt = 120	Константа типа <i>size_t</i> . Предельное число символов в строке;	Определяет «ширину» фрагмента таблицы, при разделении широкой таблицы на фрагменты
const char ch1 = ' '; const char ch2 = '*';	Константы типа <i>char</i> . Вспомогательные константы (пробел и звездочка)	
const size_t uOne = 1; const size_t uTwo = 2; const size_t uThree = 3;	Константы типа <i>size_t</i> . Вспомогательные константы	
CalcSingleDataLenth	Шаблонный метод расчета размера поля для форматированного вывода числа или строки	Определен для чисел и переопределен для строк
CalcArrayDataLenth	Шаблонный метод расчета максимального размера поля у элементов массива чисел или строк для форматированного вывода	
class clsRePrint	Основной шаблонный класс для форматированного вывода отчета на заданное устройство	
PrintHeader0	Метод для вывода заголовков отчета уровня 0	Вывод идет в задаваемый пользователем поток типа ostream
PrintHeader1	Метод для вывода заголовков отчета уровня 1	
PrintHeader2	Метод для вывода заголовков отчета уровня 2	
PrintHeader	Метод для вывода заголовков отчета	Вывод идет в стандартный поток cout
PrintUnderHeader	Метод для вывода подзаголовков отчета	

ВНЕКЛАССОВЫЕ ФУНКЦИИ

```
template<typename T, class=std::enable_if_t<is_real_v<T>>>
```

```
size_t CalcSingleDataLenth(const T& val)
```

Шаблонный метод расчёта размера поля для форматированного вывода числа типа T. Тип T может быть только [арифметическим типом \(is_arithmetic\)](#) или типом [LongReal](#).

Параметры:

& val – ссылка на число целого или вещественного типа, определяемого шаблоном T.

Метод возвращает рекомендуемую длину поля для корректного вывода заданного числа. Длина поля включает в себя число разрядов заданного числа, знак запятой, знаки дробной части после запятой и пустые разделительные поля; типа возвращаемого значения *size_t*.

**template<typename T=string>
size_t CalcSingleDataLenth(const string& str)**

Перегруженный шаблонный метод расчёта размера поля для форматированного вывода текстовой строки.

Параметры:

& val – ссылка на текстовую строку, типа *string*.

Метод возвращает рекомендуемую длину поля для корректного вывода текстовой строки. Длина поля включает в себя длину строки и пустые разделительные поля; типа возвращаемого значения *size_t*.

**template<typename T>
size_t CalcArrayDataLenth(const size_t _asize, const T _arr[], const size_t _minlimit, const size_t _maxlimit)**

Шаблонный метод расчета максимального размера поля у элементов массива чисел или строк для форматированного вывода этих чисел или строк. Метод вызывает функцию *CalcSingleDataLenth*, которая накладывает ограничения на тип T.

Параметры:

_asize – размер массива, в котором определяются поля; тип *size_t*;

_arr[] – указатель на массив чисел или строк в котором определяются поля; *арифметический* тип или тип *string*;

_minlimit – минимальный размер поля; тип *size_t*;

_maxlimit – максимальный размер поля; тип *size_t*.

void PrintHeader0(std::ostream& out, const size_t& Num, const string& val)

Метод выводит в отчёт заголовок нулевого уровня. Отчёт выводится в задаваемый пользователем поток.

Параметры:

&out – ссылка на поток для вывода отчёта; тип ссылки *ostream*;

&Num – ссылка на число, определяющее ширину заголовка в символах; тип ссылки *size_t*;

&val – ссылка на строку, содержащую текст заголовка; тип *string*.

void PrintHeader1(std::ostream& out, const size_t& Num, const string& val)

Метод выводит в отчёт заголовок первого уровня. Отчёт выводится в задаваемый пользователем поток.

Параметры:

&out – ссылка на поток для вывода отчёта; тип ссылки *ostream*;

&Num – ссылка на число, определяющее ширину заголовка в символах; тип ссылки *size_t*;

&val – ссылка на строку, содержащую текст заголовка; тип *string*.

void PrintHeader2(std::ostream& out, const size_t& Num, const string& val)

Метод выводит в отчёт заголовок второго уровня. Отчёт выводится в задаваемый пользователем поток.

Параметры:

&out – ссылка на поток для вывода отчёта; тип ссылки *ostream*;

&Num – ссылка на число, определяющее ширину заголовка в символах; тип ссылки *size_t*;

&val – ссылка на строку, содержащую текст заголовка; тип *string*.

void PrintHeader(const size_t& Num, const string& val)

Метод выводит в отчёт заголовок. Отчёт выводится в стандартный поток *cout*.

Параметры:

&Num – ссылка на число, определяющее ширину заголовка в символах; тип ссылки *size_t*;

&val – ссылка на строку, содержащую текст заголовка; тип *string*.

void PrintUnderHeader(const size_t& Num, const string& val)

Метод выводит в отчёт подзаголовок. Отчёт выводится в стандартный поток *cout*.

Параметры:

&Num – ссылка на число, определяющее ширину заголовка в символах; тип ссылки *size_t*;

&val – ссылка на строку, содержащую текст заголовка; тип *string*.

КЛАСС CLASS clsRePrint

Шаблон класса *clsRePrint* описывается так:

```
template<typename Tdata, class=std::enable_if_t<is_tble_v<Tdata>>,
        typename TName=strNameMeas, class=std::is_same<TName, strNameMeas>>
class clsRePrint
```

Шаблонный тип Tdata ограничен подстановкой арифметического типа, типа *LongReal* и структурного типа *strItem*. Шаблонный тип TName ограничен подстановкой структурного типа *strNameMeas*.

Состав шаблонного класса для форматированного вывода отчета на заданное устройство представлен в таблице ниже.

Таблица 31 Поля класса clsRePrint

Поле класса	Описание	Заметка
bool calcflag	Флаг автоматической/ручной установки размеров полей для форматированного вывода чисел и текста	При значении поля, равном <i>true</i> , размеры полей для вывода текстовой и числовой информации (полей <i>nwidth</i> , <i>mwidth</i> , <i>dwidth</i>) вычисляются автоматически;
size_t nwidth	Ширина колонки для поля с названием; тип <i>size_t</i>	
size_t mwidth	Ширина колонки для поля с ед.измерения; тип <i>size_t</i>	
size_t dwidth	Ширина колонки для числового поля; тип <i>size_t</i>	
size_t precis	Количество знаков после запятой у выводимых чисел; тип <i>size_t</i>	
size_t n_min	Минимальный размер поля для названия; тип <i>size_t</i>	
size_t m_min	Минимальный размер поля для ед.измерения; тип <i>size_t</i>	
size_t d_min	Минимальный размер для числового поля; тип <i>size_t</i>	

size_t n_max	Максимальный размер поля для названия; тип <i>size_t</i>	
size_t m_max	Максимальный размер поля для ед.измерения; тип <i>size_t</i>	
size_t d_max	Максимальный размер для числового поля; тип <i>size_t</i>	
clsTextField* tname	Формат вывода названий; тип clsTextField	
clsTextField* tmeas	Формат вывода единиц измерения; тип clsTextField	
clsDataField* tnumb	Формат вывода числовых данных; тип clsDataField	
string TableName	Заголовок первого столбца отчета; тип <i>string</i>	
string TableMeas	Заголовок второго столбца отчета; тип <i>string</i>	
string ByVolume	Заголовок таблицы для вывода натуральных показателей; тип <i>string</i>	
string ByPrice	Заголовок таблицы для вывода удельных стоимостных показателей; тип <i>string</i>	
string ByValue	Заголовок таблицы для вывода полных стоимостных показателей; тип <i>string</i>	
string HCurrency	Валюта проекта; тип <i>string</i>	
size_t rowcount	Количество строк отчета; тип <i>size_t</i>	
size_t colcount	Количество столбцов отчета; тип <i>size_t</i>	
const TName* rownames	Указатель на массив с названиями строк шаблонного типа <i>TName</i> ; размер массива <i>rowcount</i>	В данной версии библиотеки FROMA2 предполагается, что тип <i>TName</i> принимает значение единственного типа strNameMeas
const Tdata* Tcoldata	Указатель на массив с данными типа <i>Tdata</i> ; размер массива <i>rowcount*colcount</i>	Предполагается подстановка любых числовых типов, в том числе типа LongReal
streambuf* backup	Указатель на переменную типа буфер для сохранения состояния буфера вывода <i>cout</i> ; тип streambuf	
ofstream filestr	Переменная для записи в файловый поток; тип ofstream	
ostream* out	Поток для вывода отчета; тип ostream	

Методы класса `clsRePrint` приведены ниже.

`clsRePrint()`

Конструктор по умолчанию, с автоматической установкой ширины колонок отчета.

`clsRePrint(const size_t namewidth, const size_t measwidth, const size_t datawidth)`

Конструктор с ручной установкой ширины колонок отчета.

Параметры:

`namewidth` – ширина поля "Наименование"; тип *size_t*;

`measwidth` – ширина поля "Единицы измерения"; *size_t*;

`datawidth` – ширина колонок с данными; *size_t*.

clsRePrint(const string& filename, ios_base::openmode mode)

Конструктор с автоматической установкой ширины колонок отчета и перенаправлением вывода в файл.

Параметры:

&filename – имя файла отчета; тип *string*;

mode – флаг режима работы с файлом: “*app*” – данные добавляются в конец существующего файла, “*trunc*” – содержимое файла стирается перед записью; тип [ios_base::openmode](#).

clsRePrint(const size_t namewidth, const size_t measwidth, const size_t datawidth, const string& filename, ios_base::openmode mode)

Конструктор с ручной установкой ширины колонок отчета и перенаправлением вывода в файл.

Параметры:

namewidth – ширина поля "Наименование"; тип *size_t*;

measwidth – ширина поля "Единицы измерения"; *size_t*;

datawidth – ширина колонок с данными; *size_t*;

&filename – имя файла отчета; тип *string*;

mode – флаг режима работы с файлом: “*app*” – данные добавляются в конец существующего файла, “*trunc*” – содержимое файла стирается перед записью; тип [ios_base::openmode](#).

~clsRePrint()

Деструктор. Если класс был создан с помощью конструкторов для перенаправления вывода в файл, то вызов деструктора восстанавливает буфер *cout* в состояние, предшествующее созданию экземпляра класса *clsRePrint*.

void ResetReport()

Метод сбрасывает форматы вывода наименований, единиц измерений и числовых данных, очищает указатели на входные массивы, возвращает перенаправление вывода в буфер *cout* и закрывает файл, открытый для записи отчета.

Метод НЕ ИЗМЕНЯЕТ ранее введенные заголовки таблицы (поля [TableName](#), [TableMeas](#), [ByVolume](#), [ByPrice](#) и [ByValue](#)).

void SetStream(ostream& val)

Метод установки потока для вывода.

Параметры:

&val – ссылка на устанавливаемый поток для вывода; тип [ostream](#).

void SetLimits(size_t _nmin, size_t _mnin, size_t _dmin, size_t _nmax, size_t _mmax, size_t _dmax)

Метод устанавливает минимальные и максимальные пределы для ширины полей.

Параметры:

_nmin – минимальная ширина поля для названия; устанавливает значение поля [n_min](#); тип *size_t*;

_mnin – минимальная ширина поля для единиц измерения; устанавливает значение поля [m_min](#); тип *size_t*;

_dmin – минимальная ширина для числового поля; устанавливает значение поля [d_min](#); тип *size_t*;

_nmax – максимальная ширина поля для названия; устанавливает значение поля [n_max](#); тип [size_t](#);

_mmax – максимальная ширина поля для единиц измерения; устанавливает значение поля [m_max](#); тип [size_t](#);

_dmax – максимальная ширина для числового поля; устанавливает значение поля [d_max](#); тип [size_t](#).

void SetHeadings(const string& _tname, const string& _tmeasure, const string& _byvol, const string& _byprice, const string& _byvalue)

Метод устанавливает заголовки таблиц отчета.

Параметры:

_tname – ссылка на строку с заголовком первого столбца отчета; устанавливает значение поля [TableName](#); тип ссылка на [string](#);

_tmeasure – ссылка на строку с заголовком второго столбца отчета; устанавливает значение поля [TableMeas](#); тип ссылка на [string](#);

_byvol – ссылка на строку с заголовком таблицы для вывода натуральных показателей; устанавливает значение поля [ByVolume](#); тип ссылка на [string](#);

_byprice – ссылка на строку с заголовком таблицы для вывода удельных стоимостных показателей; устанавливает значение поля [ByPrice](#); тип ссылка на [string](#);

_byvalue – ссылка на строку с заголовком таблицы для вывода полных стоимостных показателей; устанавливает значение поля [ByValue](#); тип ссылка на [string](#).

void SetCurrency(Currency val)

Метод устанавливает наименование валюты, используемое в выводе данных из *price*- и *value*- полей массива данных типа [strItem](#).

Параметры:

val – индекс валюты; устанавливает значение поля [HCurrency](#); целочисленный тип из типа данных перечисления [Currency](#).

void ResetToAuto()

Метод устанавливает автоматический режим расчета ширины полей выводимых таблиц. Присваивает полю [calcflag](#) значение *true*.

void SetToManual(const size_t namewidth, const size_t measwidth, const size_t datawidth, size_t prec)

Метод устанавливает ручной режим расчета ширины полей выводимых таблиц. Присваивает полю [calcflag](#) значение *false*.

Параметры:

namewidth – ширина первого столбца таблицы; устанавливает значение поля [nwidth](#); тип [size_t](#);

measwidth – ширина второго столбца таблицы; устанавливает значение поля [mwidth](#); тип [size_t](#);

datawidth – ширина столбцов таблицы с числовыми данными; устанавливает значение поля [dwidth](#); тип [size_t](#);

prec – количество знаков после запятой для выводимых в таблицу вещественных чисел; устанавливает значение поля [precis](#); тип [size_t](#).

void SetPrecision(size_t _prec)

Метод устанавливает количество знаков после запятой при выводе в отчет вещественных чисел.

Параметры:

`_prec` – количество знаков после запятой для выводимых в таблицу вещественных чисел; устанавливает значение поля `precis`; тип `size_t`.

bool SetReport(const size_t ncount, const TName names[], const Tdata data[], const size_t dcount)

Метод вводит данные для отчета и, в случае установки флага автоматического расчета `calcflag`, рассчитывает ширину колонок выводимой таблицы отчета.

Параметры:

`ncount` – число элементов массива `names` и число строк отчета; тип `size_t`;

`names` – указатель на массив с наименованиями ресурсов и единицами их измерения; тип определяется шаблоном `TName`; В данной версии библиотеки FROMA2 предполагается, что шаблон типа `TName` принимает значение единственного типа `strNameMeas`;

`data` – массив с выводимыми в таблицу данными; в качестве шаблона типа `Tdata` может выступать любой арифметический тип (`is_arithmetic`) или тип `LongReal`; число элементов массива равно `ncount * dcount`;

`dcount` – число столбцов отчета; тип `size_t`.

bool SetReport(const size_t ncount, const TName names[], const Tdata data[], const size_t dcount, ReportData flg)

Метод, аналогичный предыдущему. Но используется для ввода числовых данных структурного типа `strItem`.

Параметры:

`ncount` – число элементов массива `names` и число строк отчета; тип `size_t`;

`names` – указатель на массив с наименованиями ресурсов и единицами их измерения; тип определяется шаблоном `TName`; В данной версии библиотеки FROMA2 предполагается, что шаблон типа `TName` принимает значение единственного типа `strNameMeas`;

`data` – массив с выводимыми в таблицу данными; в качестве шаблона типа `Tdata` может выступать только тип `strItem`; число элементов массива равно `ncount * dcount`;

`dcount` – число столбцов отчета; тип `size_t`;

`flg` – флаг выбора данных из структуры типа `strItem`; тип `ReportData`: «volume» или 0 – поле `volume`, «price» или 1 – поле `price` и «value» или 2 – поле `value` объекта типа `strItem`.

void Print()

Метод выводит отчёт в поток, установленный в поле `out`. Используется для вывода данных арифметического типа или типа `LongReal`.

void Print(ReportData flg)

Метод выводит отчёт в поток, установленный в поле `out`. Используется для вывода данных структурного типа `strItem`.

Параметры:

`flg` – флаг выбора данных из структуры типа `strItem`; тип `ReportData`: «volume» или 0 – поле `volume`, «price» или 1 – поле `price` и «value» или 2 – поле `value` объекта типа `strItem`.

void Header0(const string& val)

Метод выводит заголовок нулевого уровня. Ширина поля заголовка определяется константой `smblcunt` и равна значению этой константы, увеличенному на 3 символа.

Параметры:

`&val` – ссылка на строку, содержащую текст заголовка; тип `string`.

void Header1(const string& val)

Метод выводит заголовок первого уровня. Ширина поля заголовка определяется константой `smblcunt` и равна значению этой константы, увеличенному на 3 символа.

Параметры:

`&val` – ссылка на строку, содержащую текст заголовка; тип `string`.

void Header2(const string& val)

Метод выводит заголовок второго уровня. Ширина поля заголовка определяется константой `smblcunt` и равна значению этой константы, увеличенному на 3 символа.

Параметры:

`&val` – ссылка на строку, содержащую текст заголовка; тип `string`.

РЕПОЗИТАРИЙ

Библиотека FROMA2 предоставляется в виде доступа к репозитарию, в котором присутствуют файлы с исходными кодами, документация и примеры, позволяющие разобраться с инструментами библиотеки. Репозиторий располагается по адресу: <https://github.com/Alexandr-Pidkasisty/froma2>.

СОСТАВ И СТРУКТУРА РЕПОЗИТАРИЯ

Структура репозитория приведена на рисунке ниже.

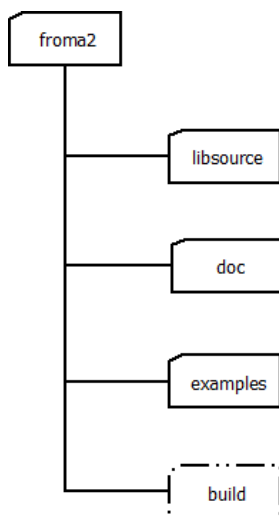


Рисунок 12 Структура репозитория библиотеки FROMA2

Корневой папкой репозитория является папка `froma2`. В ней располагаются остальные папки и файлы проекта. В таблице ниже дано описание файлов и папок проекта.

Таблица 32 Описание репозитория библиотеки FROMA2

Файл/ папка	Назначение
LICENSE	Файл с лицензией на библиотеку
readme.txt	Файл с текстом пояснения
.gitignore	Служебный файл репозитория со списком исключений для системы построения проектов CMake
changelog.log	Файл с историей коммитов системы построения проектов <i>CMake</i>
CMakeLists.txt	Конфигурационный файл системы построения проектов <i>CMake</i>
cmdbuild.cmd	Пример кроссплатформенного командного файла, автоматизирующего построение проекта библиотеки с примерами
winbuild.cmd	Пример командного файла для операционной системы <i>Windows 10</i> , автоматизирующего построение проекта библиотеки с примерами
libsource	Папка с <i>исходными кодами библиотеки FROMA2</i> и конфигурационным файлом системы <i>CMake</i> , обеспечивающим компилирование и сборку библиотеки; <i>основная папка</i> проекта
doc	Папка с <i>документацией библиотеки FROMA2</i> (в частности, данный документ хранится именно в этой папке)
examples	Папка с <i>примерами</i> , упрощающими знакомство с инструментами библиотеки FROMA2; каждый пример располагается в отдельной подпапке; в этой же подпапке располагается собственный конфигурационный файл системы <i>CMake</i> для этого примера
build	Папка <i>отсутствует</i> в репозитории, но <i>создается</i> на машине пользователя при запуске командного файла <i>cmdbuild.cmd</i> или <i>winbuild.cmd</i> из локальной копии репозитория; в подпапке <i>build/bin</i> создаются исполняемые файлы программ с примерами; в подпапке <i>build/lib</i> создается скомпилированный файл библиотеки FROMA2 с именем <i>"libfroma2.a"</i> . При сборке проекта без использования командного файла <i>cmdbuild.cmd</i> или <i>winbuild.cmd</i> рекомендуется придерживаться этой структуры папок

СБОРКА ПРОЕКТА С ПОМОЩЬЮ CMake

Перед сборкой библиотеки FROMA2 и примеров ее использования с помощью системы построения проектов [CMake](#), необходимо убедиться, что:

1. Система построения проектов *CMake* уже *установлена* на Вашем компьютере и путь до ее исполняемых файлов записан в переменную окружения *PATH*.
2. На компьютере уже *установлен* компилятор ([MinGW](#), [g++](#) и др.) и путь до его исполняемых файлов также записан в переменную окружения *PATH*.

Последовательность сборки тестовых программ следующая:

1. Скачиваем репозиторий и устанавливаем его в нужную папку. Желательно, чтобы путь к этой папке и её название не содержало пробелов, служебных символов, символов кириллицы и символов других алфавитов кроме латиницы. Переходим в папку *froma2* скачанной копии репозитория и создаем в ней подпапку *build*²¹:

```
cmake -E make_directory build
```

2. Переходим в созданную папку *build* и выполняем в ней команду по созданию *сценария сборки проекта*²²:

```
cmake -E chdir build cmake -DCMAKE_BUILD_TYPE=Release -G "MinGW Makefiles" ../
```

Команда *cmake -G "<Имя Генератора>"* указывает, что для создания сценария сборки проекта будет использован генератор конфигурации «Имя Генератора». В качестве генератора конфигурации в моей Windows 10 я использовал *MinGW Makefiles*. Список доступных у Вас имен генераторов можно получить командой *cmake -G*. Имя генератора должно соответствовать установленному на Вашем компьютере компилятору (например, для компилятора MinGW имя генератора будет *MinGW Makefiles*).

3. Запускаем *сборку* проекта командой²³:

```
cmake --build build
```

После выполнения этих команд в папке *build/lib* появится скомпилированный файл библиотеки *FROMA2* с именем *"libfroma2.a"*, а в папке *build/bin* - исполняемые файлы примеров.

КРАТКОЕ ОПИСАНИЕ ПРИМЕРОВ

Папка *examples* содержит папки с примерами. Исполняемые файлы для всех примеров создаются в папке *build/bin*, имя исполняемого файла совпадает с названием примера.

Входные данные для примеров (при наличии таковых) располагаются в папках с исходными кодами примеров *froma2/examples/<имя_примера>/input*.

Файлы с результатами расчётов (при наличии таковых) располагаются в папках *froma2/build/examples/<имя_примера>/output*.

Краткое описание примеров представлено в таблице ниже.

Таблица 33 Примеры программ, поставляемые в составе библиотеки

Папка (имя_примера)	Какой инструмент иллюстрирует	Исходные данные	Результаты расчётов	Что демонстриру ет пример
------------------------	-------------------------------------	-----------------	---------------------	---------------------------------

²¹ Используются кроссплатформенные команды системы построения проектов CMake. Для Windows можно использовать команду *mkdir build*.

²² Для Windows последовательность команд будет:

cd build

cmake -DCMAKE_BUILD_TYPE=Release -G "<Имя Генератора>" ../

²³ При использовании предыдущих команд Windows, Вы остались в папке *build*, поэтому команда сборки будет: *cmake --build .*

SKU_example	Моносклад ресурсов clsSKU	Присваиваются в исходном коде	Выводятся на экран	Рассчитывает закупки при заданных отгрузках и выводит закупки и отгрузки в натуральном, удельном и полном денежном выражении
Storage_example	Склад ресурсов clsStorage	Присваиваются в исходном коде	Выводятся на экран	Рассчитывает и выводит удельную учетную себестоимость отгружаемых ресурсов
Storage_stress_test	Склад ресурсов clsStorage	Присваиваются в исходном коде	Выводятся на экран	Тест производительности: выводит время вычислений выбранного метода расчета
ManufactItem_example	Рецептура продукта clsRecipeItem , Монопроизводство clsManufactItem	Присваиваются в исходном коде	Выводятся на экран	Рассчитывает объем, удельную и полную себестоимость незавершенного производства и готового единичного продукта
Manufactory_example	Производство ассортимента продукции clsManufactory	Присваиваются в исходном коде	Выводятся на экран	Рассчитывает объем, удельную и полную себестоимость незавершенного производства и готовой продукции всего ассортимента
Manufactory_stress_test	Производство ассортимента продукции clsManufactory	Присваиваются в исходном коде	Выводятся на экран	Тест производительности: выводит время вычислений выбранного набора методов расчета
Impex_example	Ввод данных из csv-файла, вывод в csv- и txt-файлы; clsImpex	./input/ship.csv	Файлы Array.txt, ship_T.csv, View_ship.txt и View_ship_T.txt в папке froma2/ build/ examples/ Impex_example/ output/	Импорт данных из csv-файла во внутреннее хранилище; транспонирование данных;

				вывод из внутреннего хранилища в csv-файлы
LongReal_test	Арифметика длинных вещественных чисел LongReal	Присваиваются в исходном коде	Результаты расчётов в файле Report.txt; сохраненное состояние вещественного числа в файле asf.dat в папке froma2/build/ examples/ LongReal_test/ output	Сравнение результатов выполнения логических и арифметических операций над числами типа <i>LongReal</i> и <i>double</i>
Progress_example	Оболочка для прогресс-бара clsProgress_she ll, прогресс-бар clsprogress_bar	Присваиваются в исходном коде	Выводятся на экран	Демонстрация работы индикатора прогресса в однопоточном и многопоточном режимах
Model_1	Программа для расчета полных и удельных переменных затрат <i>производственного предприятия</i>	build/examples/Model_1/config/config.cfg build/examples/Model_1/input data /*.csv build/examples/Model_1/input data /*.txt	Отчет build/examples/Model_1/reports/ Model_1_LR.txt Файлы экспорта build/examples/Model_1/outputdata/*.csv	Модель предприятия с одним центром затрат
CCStorage	Склад ресурсов с двумя центрами затрат	build/examples/CCStorage /inputdata /*.csv	Файлы экспорта build/examples/CCStorage /outputdata/*.csv	Демонстрация работы класса для описания предприятия с двумя центрами затрат
LR_stream	Поток для вещественных чисел типа <i>LongReal</i> , встроенных вещественных типов, символов и текста	Присваиваются в исходном коде	Выводятся на экран	Демонстрирует вывод в поток чисел с заданным количеством знаков после запятой, символов и текста
Model_2	Программа для расчета полных и удельных переменных затрат <i>склада</i>	build/examples/Model_2/config/config.cfg build/examples/Model_2/input data /*.csv build/examples/Model_2/input data /*.txt	Отчет build/examples/Model_2/reports/ Model_2.txt Файлы экспорта build/examples/Model_2/outputdata/*.csv	Модель предприятия с двумя центрами затрат

ПРОГРАММА ДЛЯ РАСЧЕТА ПОЛНЫХ И УДЕЛЬНЫХ ПЕРЕМЕННЫХ ЗАТРАТ ПРОИЗВОДСТВЕННОГО ПРЕДПРИЯТИЯ MODEL_1

Программа Model_1 описывает [модель предприятия с одним центром затрат](#) и демонстрирует совместную работу всех модулей и классов библиотеки FROMA2. Описание ситуации и соответствующая задача изложены в описании [Примера 1](#).

В отличие от других примеров, демонстрирующего использование какого-то одного инструмента библиотеки FROMA2 и представляющего интерес исключительно для программистов с точки зрения знакомства с возможностями этого инструмента, данный пример использует практически все элементы библиотеки, является *клиентоориентированным приложением* и может представлять интерес не только для моих коллег, но и для целевой аудитории, далекой от программирования.

ЦЕЛЕВАЯ АУДИТОРИЯ ДЛЯ ПРОГРАММЫ

Приложение консольное. Предназначено в помощь аналитикам небольших предприятий, реализующим управленческую отчетность и планирование с помощью *MS Excel, OpenOffice Calc* и других [электронных таблиц](#). Как правило, [ERP-системы](#) этих предприятий небольшие, имеют ограниченное число модулей. Обычно это бухгалтерский модуль и, в лучшем случае, специализированные модули, такие как модули склада и производства, функционирующие обособленно друг от друга. Работа аналитиков на таком предприятии заключается в выгрузке необходимых данных из каждого такого модуля и ручной обработкой этих данных в электронных таблицах.

Предлагаемая для этих сотрудников программа Model_1 читает исходные данные из файлов, выгруженных из ERP-системы предприятия, производит расчеты и экспортирует результаты в виде файлов, читабельных как людьми, так и ERP-системами. Формат входных и выходных файлов – самый распространенный, [CSV](#), совместимый с большинством ERP-систем и электронных таблиц. Это позволяет использовать программу для облегчения труда аналитиков как в работе над отчетом, так и в работе над планом.

ВОЗМОЖНОСТИ ПРОГРАММЫ

Программа позволяет рассчитать:

- объем производства продукции, необходимый для отгрузки потребителям и поддержания требуемого запаса на Складе готовой продукции;
- объем закупок сырья и материалов, необходимый для производства продукции и поддержания требуемого запаса на Складе ресурсов;
- учетную себестоимость сырья и материалов, направляемых в производство;
- учетную себестоимость остатков сырья и материалов на Складе ресурсов;
- материальную себестоимость незавершенного производства;
- материальную себестоимость продукции, направляемой на склад готовой продукции;
- материальную себестоимость готовой продукции, отгружаемой покупателям;
- материальную себестоимость остатков продукции на Складе готовой продукции.

Себестоимость сырья и материалов и готовой продукции рассчитывается исходя из выбранного принципа учета запасов: *FIFO* ("первым вошел - первым вышел"), *LIFO* ("последним вошел - первым вышел"), или *AVERAGE* ("по-среднему").

Программа позволяет учесть ситуацию, когда поступление товарно-материальных ценностей (ТМЦ) на склад и отгрузка этих ценностей со склада невозможна в одном периоде. Например, когда поступление ТМЦ осуществляется в конце периода, а отгрузка – в начале следующего.

Программа допускает длительные циклы производства и может быть полезной для учета и планирования в таких бизнесах, как строительство, сельское хозяйство и других с длительными производственными циклами.

ПАПКИ И ФАЙЛЫ

Программа поставляется в составе репозитория библиотеки FROMA2 в виде исходных текстов. Исходные тексты находятся в папке **froma2/examples/Model_1/**:

main.cpp

Файл реализации основного кода программы;

Include/clsEnterprise.h

Заголовочный файл для структур и классов программы;

Include/pathes.h

Заголовочный файл с путями для импорта и экспорта файлов; пути прописываются автоматически при сборке проекта с помощью *CMake*;

src/clsEnterprise.cpp

Файл реализации для структур и классов программы.

inputdata/about.txt

Пример текстового файла с кратким описанием программы;

inputdata/ship.csv

Пример csv-файла с объемами отгрузок в натуральном выражении со Склада готовой продукции (СГП);

inputdata/manufvol.csv

Пример csv-файла с объемами отгрузок в натуральном выражении из Производства на СГП;

inputdata/recipe_0.csv, inputdata/recipe_1.csv, inputdata/recipe_2.csv, inputdata/recipe_3.csv, inputdata/recipe_4.csv, inputdata/recipe_5.csv

Пример csv-файлов с рецептурами продуктов;

inputdata/rowmatprices.csv

Пример csv-файла с ценами ресурсов, закупаемых на Склад сырья и материалов (ССМ);

inputdata/ rowmatvolume.csv

Пример csv-файла с объемами закупок ресурсов на ССМ.

config/config.cfg, config/ config_empty.cfg, config_full_allowed.cfg

Пример конфигурационных файлов Программы;

В результате сборки проекта (см. раздел [СБОРКА ПРОЕКТА С ПОМОЩЬЮ CMake](#)), в целевой папке **froma2/build/examples/Model_1** получаем следующие папки и файлы, используемые непосредственно в ходе работы Программы:

config

В эту папку копируется содержимое папки *froma2/examples/Model_1/config*. Это рабочая папка скомпилированной программы, в которой изменяются, удаляются и создаются заново конфигурационные файлы в зависимости от действий пользователя. Имена файлов для импорта, строки и столбцы для чтения, символ разделителя данных, а также принцип учета запаса и валюта проекта задаются в конфигурационном файле с именем *config.cfg*. Программа читает конфигурационный файл, при его отсутствии предлагает ввести данные с экрана, записывает данные в новый конфигурационный файл. При желании пользователя, в программе можно отредактировать и сохранить существующий конфигурационный файл.

inputdata

В эту папку копируется содержимое папки *froma2/examples/Model_1/inputdata*. Это рабочая папка с файлами, которые пользователь хочет импортировать в Программу для проведения расчётов; Пользователь может заменять файлы с примерами собственными файлами для получения необходимых ему результатов;

reports

Эта папка используется Программой для вывода исходных и расчетных данных в читаемом табличном виде в текстовый файл. Отчет с текстом и таблицами выводится в файл с именем *Model_1_LR.txt* (или *Model_1_DB.txt*). Суффикс LR или DB показывает какой тип вещественных чисел использовался для расчётов (LongReal или double).

outputdata

Эта папка используется Программой для экспорта исходных и расчетных данных в csv-файлы. Имена файлов начинаются с символа 'f' (от названия библиотеки *froma2*), далее через нижнее подчеркивание следует символ, обозначающий подразделение:

w - Склад Готовой продукции (**w**arehouse);

r - Склад Сырья и Материалов (**r**owmatstock);

m - Производство (**m**anufacture).

Следующий символ обозначает вид выводимого в файл массива:

s - отгрузки (**s**hipment);

b - остатки (**b**alance);

p - поставки (**p**urchase).

Далее через нижнее подчеркивание следует тип выводимых данных:

volume - в натуральном измерении;

price - в удельном стоимостном измерении;

value - в полном стоимостном измерении.

Далее через нижнее подчеркивание следуют два символа, характеризующие тип вещественного числа, используемого программой:

LR - вещественное число типа *LongReal*;

DB - вещественное число типа *double*.

Например, файл с именем *f_wb_price_LR.csv* содержит данные об удельной стоимости остатков на Складе Готовой продукции. При расчетах использовался тип вещественных чисел *LongReal*.

Моделируемое предприятие состоит из трех основных подразделений и одного центра затрат:

- Склад ресурсов для хранения сырья и материалов описывается классом [clsStorage](#) из модуля [warehouse](#) библиотеки froma2;
- Производство описывается классом [clsManufactory](#) из модуля [manufact](#) библиотеки;
- Склад готовой продукции описывается классом [clsStorage](#) из модуля [warehouse](#) библиотеки.

Центром затрат является Производство, описываемое классом [clsManufactory](#) из модуля [manufact](#).

Объединяющей оболочкой для обоих складов и производства выступает класс [clsEnterprise](#), являющийся потомком класса [clsBaseProject](#) модуля [baseproject](#) библиотеки.

Логика работы Программы следующая:

0 этап.

Импорт данных из csv-файлов, определение числа позиций и названий продуктов, числа позиций и названий сырья и материалов, длительности проекта, единиц измерения продуктов и сырья. Задание принципа учета запасов, способа расчета получаемых на склады продукции и ресурсов (ручной или автоматический), разрешений/запретов на получение и отгрузку со склада в один и тот же период (например, когда поступление на склад осуществляется в конце периода, а отгрузка - только в следующем, возможно установить запрет).

1 этап.

Создание Склада готовой продукции на основе экземпляра класса [clsStorage](#), ввод данных о продажах продукции клиентам в натуральном измерении. Расчет производственной программы в натуральном измерении, обеспечивающей необходимый объем для отгрузки потребителям и требуемый запас продукции на складе.

2 этап.

Создание Производства на основе экземпляра класса [clsManufactory](#), ввод производственной программы и рецептур продуктов. Расчет потребности в сырье и материалах в натуральном измерении.

3 этап.

Создание Склада ресурсов на основе экземпляра класса [clsStorage](#), ввод данных о потребности в сырье и материалах в натуральном измерении для Производства, ввод цен на сырье и материалы, расчет объемов закупаемых сырья и материалов. Расчет учетной себестоимости сырья и материалов, направляемых в производство и образующих запас на складе.

4 этап.

Расчет материальной себестоимости продукции, направляемой на Склад готовой продукции из Производства.

5 этап.

Расчет материальной себестоимости продукции, отгружаемой покупателям со Склада готовой продукции и образующих запас на складе.

ПЕРВЫЕ ШАГИ РАБОТЫ С ПРОГРАММОЙ

Для практического применения Программы для своих нужд, пользователю необходимо сделать следующие шаги.

1 шаг.

Удалить из папки *froma2/build/examples/Model_1/inputdata* примеры с импортируемыми файлами и поместить в эту папку свои собственные файлы с исходными данными. Удалить из папки *froma2/build/examples/Model_1/config* примеры конфигурационных файлов и оставить папку пустой.

2 шаг.

Запустить на исполнение файл Программы *froma2/build/bin/Model_1.exe*²⁴.

```

C:\Users\Alexandr Pidkasty>cd "C:\git\froma2\build\bin"

C:\git\froma2\build\bin>dir
Volume in drive C is OS
Volume Serial Number is 7823-E55A

Directory of C:\git\froma2\build\bin

27.10.2025  16:53    <DIR>          .
27.10.2025  16:53    <DIR>          ..
27.10.2025  16:53             199 300 Impex_example.exe
27.10.2025  16:53             164 401 LongReal_test.exe
27.10.2025  16:53             333 885 ManufactItem_example.exe
27.10.2025  16:53             333 067 Manufacture_example.exe
27.10.2025  16:53             333 760 Manufacture_stress_test.exe
27.10.2025  15:16             603 969 Model_1.exe
27.10.2025  16:53             79 503 Progress_example.exe
27.10.2025  16:53             258 055 SKU_example.exe
27.10.2025  16:53             289 143 Storage_example.exe
27.10.2025  16:53             287 187 Storage_stress_test.exe
                10 File(s)                2 882 270 bytes
                2 Dir(s)  18 030 710 784 bytes free

C:\git\froma2\build\bin>Model_1.exe
  
```

Рисунок 13 Запуск программы Model_1

3 шаг.

После запуска программы появится диалог ввода конфигурации проекта, где отвечаем на вопросы.

```

C:\git\froma2\build\bin>C:\git\froma2\build\bin\Model_1.exe
*****
**                                                                 **
**              Программа для расчета полных и удельных переменных затрат Model_1              **
**                                                                 **
*****
ВВЕДИТЕ КОНФИГУРАЦИЮ ИМПОРТА
В []-скобках представлено значение по умолчанию. Для его ввода просто нажмите Enter
Имя файла с описанием проекта []: 
  
```

Рисунок 14 Приглашение к вводу информации

При вводе имен импортируемых файлов важно обратить внимание на следующее.

Все файлы должны быть в формате CSV.

²⁴ Все последующие скриншоты относятся к выполнению программы под операционной системой MS Windows. Для ОС семейства Linux действия будут аналогичными, однако внешний вид экрана может отличаться от приведенных в настоящем документе.

```

C:\git\froma2\build\bin>C:\git\froma2\build\bin\Model_1.exe
*****
**                                     **
**                                     **
**                                     **
**                                     **
*****
ВВЕДИТЕ КОНФИГУРАЦИЮ ИМПОРТА
В []-скобках представлено значение по умолчанию. Для его ввода просто нажмите Enter
имя файла с описанием проекта []: about.txt
имя файла с отгрузками из СГП в натуральном выражении []: ship.csv
имя файла с объемами производства, необязательно. Для пропуска введите nofile []:
имя файла с ценами закупок на ССМ []: rowmatprices.csv
имя файла с объемами закупок на ССМ, необязательно. Для пропуска введите nofile []:
префикс для имен файлов с рецептурами []: recipe
символ разделителя между полями в CSV-файлах [:]:
число столбцов с заголовками в CSV-файлах [2]:
число строк с заголовками в CSV-файлах [1]:
домашняя валюта проекта, RUR=1, USD=2, EUR=3, CNY=4 [1]:
принцип учета запасов, FIFO=0, LIFO=1, AVG=2 [0]: 2
запас продуктов на складе СГП, доля от отгрузок [0]: 0.5
запас ресурсов на складе ССМ, доля от отгрузок [0]: 0.5
флаг разрешения поступлений и отгрузок с СГП в одном периоде, PROHIBITED=0, ALLOWED=1 [1]:
флаг разрешения поступлений и отгрузок с ССМ в одном периоде, PROHIBITED=0, ALLOWED=1 [1]:

```

Рисунок 15 Ввод исходных данных

Если имя файла вводится без указания папки, где расположен этот файл, то программа ищет этот файл в папке *froma2/build/examples/Model_1/inputdata*. Если имя файла вводится с относительным или абсолютным путем (например, *inputdata/about.txt*), то программа ищет файл с указанным именем в указанной папке.

Файлов с рецептурами продуктов должно быть не менее, чем число продуктов, которое планируется ввести в программу. Каждое имя файла с рецептурой состоит из уникального префикса, задаваемого пользователем и суффикса вида *_i*. Если *i* – соответствует номеру вводимого продукта (например, для первого продукта, суффикс будет *_1*, для четвертого продукта, соответственно, *_4*), то программа работает быстрее. Каждое имя файла должно иметь расширение *.csv*. В данной версии программы число *i* ограничено диапазоном от 0 до 99. Это ограничение задается в файле *clsEnterprise.h* маской регулярного выражения *msks*.

Импортируемые файлы должны содержать строки и столбцы с заголовками. Например, для файла с объемами отгрузок покупателям и файла с объемами производства в натуральном измерении, первый и второй столбцы содержат названия продуктов и их единицы измерения. Для файла с ценами сырья и материалов и файла с объемами закупок сырья и материалов в натуральном измерении, первый и второй столбцы содержат названия ресурсов, из которых изготавливаются продукты и единицы измерения этих ресурсов. В этом случае, вводимое число столбцов с заголовками будет равно двум (Рисунок 16).

	A	B	C	D	E	F	G	H	I	J	K
	Name	Measure	0	1	2	3	4	5	6	7	8
2	Баклажаны	кг.	0.0000000	0.0000000	3242.4631	4415.7840	5830.5689	7254.6356	8671.9591	10084.966	11495.355
3	Говядина оковалок	кг.	0.0000000	0.0000000	990.75263	1349.2673	1781.5627	2216.6942	2649.7652	3081.5174	3512.4698
4	Горох	кг.	0.0000000	0.0000000	450.34210	613.30334	809.80123	1007.5882	1204.4387	1400.6897	1596.5771
5	Горошек зеленый с\м	кг.	0.0000000	0.0000000	900.68421	1226.6066	1619.6024	2015.1765	2408.8775	2801.3794	3193.1543
6	Итальянские травы	кг.	0.0000000	0.0000000	31.523947	42.931234	56.686086	70.531180	84.310713	98.048282	111.76040
7	Картофель	кг.	0.0000000	0.0000000	2296.7447	3127.8470	4129.9863	5138.7002	6142.6377	7143.5176	8142.5436
8	Корень сельдерея	кг.	0.0000000	0.0000000	90.068421	122.66066	161.96024	201.51765	240.88775	280.13794	319.31543
9	Крылья куриные	кг.	0.0000000	0.0000000	900.68421	1226.6066	1619.6024	2015.1765	2408.8775	2801.3794	3193.1543
10	Куриный бульон концентри	кг.	0.0000000	0.0000000	450.34210	613.30334	809.80123	1007.5882	1204.4387	1400.6897	1596.5771
11	Курица бедро	кг.	0.0000000	0.0000000	4053.0789	5519.7301	7288.2111	9068.2946	10839.948	12606.207	14369.194
12	Лавровый лист	кг.	0.0000000	0.0000000	13.510263	18.399100	24.294037	30.227648	36.133162	42.020692	47.897315
13	Лапша яичная	кг.	0.0000000	0.0000000	450.34210	613.30334	809.80123	1007.5882	1204.4387	1400.6897	1596.5771
14	Лук репка	кг.	0.0000000	0.0000000	2251.7105	3066.5167	4049.0061	5037.9414	6022.1938	7003.4487	7982.8859
15	Масло растительное	кг.	0.0000000	0.0000000	180.13684	245.32133	323.92049	403.03531	481.77550	560.27589	638.63087
16	Молоко	кг.	0.0000000	0.0000000	900.68421	1226.6066	1619.6024	2015.1765	2408.8775	2801.3794	3193.1543

Рисунок 16 Структура импортируемого файла

Если в импортируемых файлах присутствует строка с номерами временных периодов или датами, то вводимое число строк с заголовками будет равно единице.

Минимальное число строк с заголовками равно нулю. Минимальное число столбцов с заголовками равно двум. При количестве столбцов с заголовками более двух, столбцы с названиями номенклатурных позиций и единицами их измерения должны быть крайними правыми (при индексе самого левого столбца, равного ноль и общем числе столбцов с заголовками N, индекс столбцов с названиями и единицами измерения должны быть равны N-2 и N-1 соответственно).

Длина всех строк в импортируемом файле должна быть *одинаковой*. В противном случае поведение программы может стать непредсказуемым.

Запас продуктов или ресурсов на складе измеряется долями от объемов отгрузки. Например, при отгрузке за период 1000 килограмм и норме запаса 0.5, запас в натуральном измерении и составит 500 кг. При ответе на соответствующие вопросы о запасах на СГП и ССМ, для всех продуктов и ресурсов устанавливается введенный запас. Однако, в дальнейшем для номенклатурной позиции можно эти нормативы скорректировать. Поэтому имеет смысл вводить нормативы, применимые к большинству позиций, чтобы в дальнейшем сократить свои трудозатраты на корректировку нормативов у отдельных позиций.

Иногда поступления на склад осуществляются в конце отчетного периода, а отгрузки – не ранее начала следующего. Для такой ситуации можно установить *флаг разрешения поступлений и отгрузок в одном и том же периоде* «ЗАПРЕЩЕНО» (“PROHIBITED”). В случаях, когда и поступления, и отгрузки возможны в один и тот же отчетный период, флаг устанавливается в «РАЗРЕШЕНО» (“ALLOWED”). Опять же флаг выставляется для всех номенклатурных позиций и корректируется в дальнейшем для отдельных позиций.

4 шаг.

После ответа на последний вопрос, появится экран с вновь введенной конфигурацией и приглашение к изменению индивидуальных настроек расчета СГП.

```

ТЕКУЩАЯ КОНФИГУРАЦИЯ ИМПОРТА
имя файла с описанием проекта: about.txt
имя файла с отгрузками из СГП в натуральном выражении: ship.csv
имя файла с объемами производства:
имя файла с ценами закупок на ССМ: formatprices.csv
имя файла с объемами закупок на ССМ:
префикс для имен файлов с рецептурами: recipe
символ разделителя между полями в CSV_файлах: ;
число столбцов с заголовками в CSV_файлах: 2
число строк с заголовками в CSV_файлах: 1
домашняя валюта проекта: RUR
принцип учета запасов: AVERAGE
*****
**                               Индивидуальные настройки учета продуктов на СГП                               **
*****
№  Название                               Ед. измерения  Флаг периода  Авторасчет  Запас
0  Азу из говядины с картофелем шт.         ALLOWED       AUTO        0.50
1  Баклажаны с томатом Неапо шт.             ALLOWED       AUTO        0.50
2  Лапша куриная шт.                         ALLOWED       AUTO        0.50
3  Плов с курицей шт.                         ALLOWED       AUTO        0.50
4  Суп гороховый с колбасой шт.               ALLOWED       AUTO        0.50
5  Фрикасе из курицы с зеленью шт.            ALLOWED       AUTO        0.50
Введите номер номенклатурной позиции для редактирования [для выхода наберите "q" и нажмите "Enter"]:
```

Рисунок 17 Отображение текущей конфигурации

Сокращённые заголовки выводимой на экран таблицы «Индивидуальные настройки учёта...» обозначают:

- «Флаг периода» – флаг разрешения на отгрузки с СГП в том же периоде, что и поступления. “ALLOWED” – РАЗРЕШЕНО, “PROHIBITED” – ЗАПРЕЩЕНО;
- «Авторасчет» – флаг разрешающий/ запрещающий автоматический расчёт объемов поступления ресурсов/ продуктов на соответствующий склад “AUTO” – АВТОМАТИЧЕСКИЙ расчёт, “MANUAL” – РУЧНАЯ подстановка данных;
- «Запас» – норматив запаса ресурсов/ продуктов на соответствующем складе. Вещественное число. Доля от объемов отгрузки за период.

Для изменения настроек конкретного продукта, введите его номер и нажмите *Enter*. Для выхода из меню редактора индивидуальных настроек, введите символ *q* и нажмите *Enter*.

Предположим, что мы хотим поменять запас на СГП для продукта под номером 2 («Лапша куриная»), увеличив его долю с 0.5 до 1.0 от объемов отгрузки клиентам. Для этого вводим номер продукта 2 и нажимаем **Enter**. Появится диалог, как на рисунке ниже (Рисунок 18).

Для двух первых параметров принимаем значение по умолчанию (ничего не вводим, нажимаем «Enter»), а в ответ на приглашение ввести новый норматив запаса для этого продукта, вводим нужное число и нажимаем «Enter». Программа покажет наши новые настройки для этого продукта.

Поскольку, параметры у других продуктов мы менять не намерены, выходим из редактора индивидуальных настроек: вводим «q» и нажимаем «Enter». Если бы мы хотели поменять настройки у другого продукта, то вместо «q» ввели бы его номер и процесс редактирования повторился бы.

```

*****
**                                     Индивидуальные настройки учета продуктов на СГП                                     **
*****
№№ Название                     Ед.измерения  Флаг периода  Авторасчет  Запас
0 Азу из говядины с картофе шт.         ALLOWED      AUTO        0.50
1 Баклажаны с томатом Неапо шт.         ALLOWED      AUTO        0.50
2 Лапша куриная                шт.         ALLOWED      AUTO        0.50
3 Плов с курицей                шт.         ALLOWED      AUTO        0.50
4 Суп гороховый с копченост шт.         ALLOWED      AUTO        0.50
5 Фрикасе из курицы с зелен шт.         ALLOWED      AUTO        0.50
Введите номер номенклатурной позиции для редактирования [для выхода наберите "q" и нажмите "Enter"]: 2
ТЕКУЩИЕ ПАРАМЕТРЫ УЧЕТА ДЛЯ ПОЗИЦИИ № 2
наименование позиции:                Лапша куриная
единица измерения натурального объема: шт.
флаг разрешения поступлений и отгрузок в одном периоде: ALLOWED
флаг автоматического расчета поступлений на склад:      AUTO
норматив запаса номенклатурной позиции на складе:        0.50
ВВЕДИТЕ НОВЫЕ ПАРАМЕТРЫ УЧЕТА ДЛЯ ПОЗИЦИИ № 2
флаг разрешения поступлений и отгрузок в одном периоде [PROHIBITED=0, ALLOWED=1]: 1
флаг автоматического расчета поступлений на склад [AUTO=0, MANUAL=1]: 0
норматив запаса номенклатурной позиции на складе (доля от отгрузки) 1.0
НОВЫЕ ПАРАМЕТРЫ УЧЕТА ДЛЯ ПОЗИЦИИ № 2
наименование позиции:                Лапша куриная
единица измерения натурального объема: шт.
флаг разрешения поступлений и отгрузок в одном периоде: ALLOWED
флаг автоматического расчета поступлений на склад:      AUTO
норматив запаса номенклатурной позиции на складе:        1.00
Введите номер номенклатурной позиции для редактирования [для выхода наберите "q" и нажмите "Enter"]:
```

Рисунок 18 Редактирование индивидуальных настроек учета продуктов на СГП

После завершения редактирования настроек продуктов, программа создает СГП и рассчитывает объем производства продуктов в натуральном выражении (производственную программу): на экране появляется бегущий индикатор прогресса «Расчёт СГП».

```

НОВЫЕ ПАРАМЕТРЫ УЧЕТА ДЛЯ ПОЗИЦИИ № 2
наименование позиции:                Лапша куриная
единица измерения натурального объема: шт.
флаг разрешения поступлений и отгрузок в одном периоде: ALLOWED
флаг автоматического расчета поступлений на склад:      AUTO
норматив запаса номенклатурной позиции на складе:        1.00
Введите номер номенклатурной позиции для редактирования [для выхода наберите "q" и нажмите "Enter"]: q
Расчет СГП .....[100%]
Расчет поставок в производство .....[100%]
```

Рисунок 19 Расчет производственной программы и потребности в ресурсах

Далее программа создает Производство, вводит в него производственную программу, рецептуры и рассчитывает потребность в ресурсах для Производства: на экране появляется бегущий индикатор прогресса «Расчёт поставок в производство».

5 шаг.

Полученный массив с потребностью в ресурсах используется для создания ССМ. Как только будет создан склад, программа вводит в него рассчитанную потребность и выводит приглашение к изменению индивидуальных настроек учета сырья и материалов.

Номенклатурные позиции выводятся списком по 20 строк. Для продолжения вывода необходимо нажимать «Enter». Для изменения настроек конкретного ресурса, введите его номер и нажмите **Enter**. Для выхода из меню редактора индивидуальных настроек, введите символ **q** и нажмите **Enter**.

После завершения редактирования настроек ресурсов, программа рассчитывает потребность в сырье и материалах с учетом необходимого их запаса на складе и учетную себестоимость передаваемых в производство ресурсов и остатков на складе: на экране появляется бегущий индикатор прогресса «Расчёт ССМ» (Рисунок 20).

После того, как учетная себестоимость ресурсов будет введена в Производство, произойдет расчет производственной себестоимости готовой продукции и незавершенного производства: на экране появляется бегущий индикатор прогресса «Расчёт производственной себестоимости».

Далее произойдет передача данных о производственной себестоимости произведенной продукции на СГП, произойдет расчёт учетной себестоимости остатков на СГП и продаваемой продукции: на экране вновь появляется бегущий индикатор прогресса «Расчёт СГП».

```

Command Prompt - Model_1.exe
Расчет СГП .....[100%]
Расчет поставок в производство .....[100%]
*****
***** Индивидуальные настройки учета ресурсов на CSM *****
*****
№  Название                     Ед.измерения  Флаг периода  Авторасчет  Запас
0  Баклажаны                     кг.          ALLOWED       AUTO        0.50
1  Говядина оковалок             кг.          ALLOWED       AUTO        0.50
2  Горох                         кг.          ALLOWED       AUTO        0.50
3  Горошек зеленый с\м          кг.          ALLOWED       AUTO        0.50
4  Итальянские травы            кг.          ALLOWED       AUTO        0.50
5  Картофель                    кг.          ALLOWED       AUTO        0.50
6  Корень сельдерея             кг.          ALLOWED       AUTO        0.50
7  Крылья куриные               кг.          ALLOWED       AUTO        0.50
8  Куриный бульон концентрик    кг.          ALLOWED       AUTO        0.50
9  Курица бедра                 кг.          ALLOWED       AUTO        0.50
10 Лавровый лист                кг.          ALLOWED       AUTO        0.50
11 Лапша яичная                кг.          ALLOWED       AUTO        0.50
12 Лук репка                    кг.          ALLOWED       AUTO        0.50
13 Масло растительное           кг.          ALLOWED       AUTO        0.50
14 Молоко                       кг.          ALLOWED       AUTO        0.50
15 Морковь                     кг.          ALLOWED       AUTO        0.50
16 Мука пшеничная              кг.          ALLOWED       AUTO        0.50
17 Огурец соленый               кг.          ALLOWED       AUTO        0.50
18 Перец черный молотый         кг.          ALLOWED       AUTO        0.50
19 Рис                          кг.          ALLOWED       AUTO        0.50
20 Рулька                       кг.          ALLOWED       AUTO        0.50
Для продолжения вывода таблицы нажмите "Enter"
21 Соль                         кг.          ALLOWED       AUTO        0.50
22 Специи для плова             кг.          ALLOWED       AUTO        0.50
23 Томаты в собственном соку    кг.          ALLOWED       AUTO        0.50
24 Упаковка                     шт.          ALLOWED       AUTO        0.50
25 Чеснок                       кг.          ALLOWED       AUTO        0.50
Введите номер номенклатурной позиции для редактирования [для выхода наберите "q" и нажмите "Enter"] : q
Расчет CSM .....[100%]
Расчет производственной себестоимости .....[100%]
Расчет СГП .....[100%]

```

Рисунок 20 Редактирование индивидуальных настроек учета ресурсов на CSM

По окончании всех расчетов программа запросит подтверждение имени файла отчета. Имя файла задается без расширения, при выводе в файл программа сама добавляет расширение ".txt". По умолчанию отчет выводится в папку *froma2/build/examples/Model_1/reports* с именем *Model_1_LR.txt*. Папку и имя файла можно изменить. Если имя файла вводится с относительным или абсолютным путем, то программа записывает файл отчета с указанным именем в указанную папку. Если указанная папка не существует, отчет не выводится.

```

Расчет CSM .....[100%]
Расчет производственной себестоимости .....[100%]
Расчет СГП .....[100%]
Введите имя файла отчета без расширения [C:/git/froma2/build/examples/Model_1/reports/Model_1_LR]:
Формирую отчет...
Готово. Отчет выведен в файл C:/git/froma2/build/examples/Model_1/reports/Model_1_LR.txt
Введите папку для экспорта CSV-файлов [C:/git/froma2/build/examples/Model_1/outputdata/]:
Экспорт CSV-файлов в папку C:/git/froma2/build/examples/Model_1/outputdata/
Готово. Данные экспортированы
Copyright (c) 2025 Пидкасистый Александр Павлович

```

Рисунок 21 Формирование файла отчета и экспорт данных

Файл отчета содержит описание (добавляется из файла с описанием проекта, в нашем примере это файл с именем *about.txt*, Рисунок 15), таблицы с исходными и расчетными данными.

Помимо файла отчета, программа экспортирует исходные и расчетные данные в csv-файлы в папку по умолчанию *froma2/build/examples/Model_1/outputdata*. Папку можно изменить, можно указать ее название с относительным или абсолютным путем.

Имена экспортируемых файлов начинаются с символа 'f' (от названия библиотеки *FROMA2*), далее через нижнее подчеркивание следует символ, обозначающий подразделение:

- w - Склад Готовой продукции (warehouse);
- r - Склад Сырья и Материалов (rowmatstock);
- m - Производство (manufactory).

Следующий символ обозначает вид выводимого в файл массива:

- s - отгрузки (shipment);
- b - остатки (balance);
- p - поставки (purchase).

Далее через нижнее подчеркивание следует тип выводимых данных:

- volume - в натуральном измерении;
- price - в удельном стоимостном измерении;
- value - в полном стоимостном измерении.

Далее через нижнее подчеркивание следуют два символа, характеризующие тип вещественного числа, используемого программой:

- LR - вещественное число типа LongReal (встроенный в библиотеку тип для операций с длинными числами, модуль LongReal);
- DB - вещественное число типа double (стандарт C++).

Например, файл с именем *f_wb_price_LR.csv* содержит данные об удельной стоимости остатков на Складе Готовой продукции, при расчетах использовался тип вещественных чисел *LongReal*.

ПОВТОРНЫЙ ЗАПУСК ПРОГРАММЫ

При первом запуске в папке *froma2/build/examples/Model_1/config* программа создает конфигурационный файл *config.cfg*. Этот файл открывается при последующих запусках. Программа выводит на экран текущую конфигурацию и предлагает либо использовать, либо отредактировать её (Рисунок 22).

```

*****
**                                                                 **
**                                                                 **
**      Программа для расчета полных и удельных переменных затрат Model_1      **
**                                                                 **
*****
ТЕКУЩАЯ КОНФИГУРАЦИЯ ИМПОРТА
имя файла с описанием проекта: about.txt
имя файла с отгрузками из СГП в натуральном выражении: ship.csv
имя файла с объемами производства:
имя файла с ценами закупок на ССМ: rowmatprices.csv
имя файла с объемами закупок на ССМ:
префикс для имён файлов с рецептурами: recipe
символ разделителя между полями в CSV_файлах: ;
число столбцов с заголовками в CSV_файлах: 2
число строк с заголовками в CSV_файлах: 1
домашняя валюта проекта: RUR
принцип учета запасов: AVERAGE
Редактировать конфигурацию? [Y/N]_

```

Рисунок 22 Начало работы с текущим конфигурационным файлом

В случае отказа пользователя от редактирования общих настроек, программа выводит на экран индивидуальные настройки учета продуктов на СГП и предлагает отредактировать их (Рисунок 23).

```

*****
**                                                                 **
**              Программа для расчета полных и удельных переменных затрат Model_1              **
**                                                                 **
*****
ТЕКУЩАЯ КОНФИГУРАЦИЯ ИМПОРТА
имя файла с описанием проекта: about.txt
имя файла с отгрузками из СГП в натуральном выражении: ship.csv
имя файла с объемами производства:
имя файла с ценами закупок на ССМ: rowmatprices.csv
имя файла с объемами закупок на ССМ:
префикс для имён файлов с рецептурами: recipe
символ разделителя между полями в CSV_файлах: ;
число столбцов с заголовками в CSV_файлах: 2
число строк с заголовками в CSV_файлах: 1
домашняя валюта проекта: RUR
принцип учета запасов: AVERAGE
Редактировать конфигурацию? [Y/N]N
*****
**              Индивидуальные настройки учета продуктов на СГП              **
*****
№  Название                     Ед.измерения  Флаг периода  Авторасчет  Запас
0  Азу из говядины с картофе шт.             ALLOWED      AUTO        0.50
1  Баклажаны с томатом Неапо шт.             ALLOWED      AUTO        0.50
2  Лапша куриная                шт.             ALLOWED      AUTO        0.50
3  Плов с курицей                 шт.             ALLOWED      AUTO        0.50
4  Суп гороховый с копченост шт.             ALLOWED      AUTO        0.50
5  Фрикасе из курицы с зелен шт.             ALLOWED      AUTO        0.50
Введите номер номенклатурной позиции для редактирования [для выхода наберите "q" и нажмите "Enter"]:
```

Рисунок 23 Использование текущего конфигурационного файла

Далее работа с программой идентична работе при первом запуске, начиная с шага №4.

В случае решения пользователя отредактировать текущую конфигурацию, начинается диалог, похожий на диалог в шаге №3. Отличие заключается в том, что программа предлагает значения по умолчанию, взятые из текущего конфигурационного файла (Рисунок 24).

```

*****
**                                                                 **
**              Программа для расчета полных и удельных переменных затрат Model_1              **
**                                                                 **
*****
ТЕКУЩАЯ КОНФИГУРАЦИЯ ИМПОРТА
имя файла с описанием проекта: about.txt
имя файла с отгрузками из СГП в натуральном выражении: ship.csv
имя файла с объемами производства:
имя файла с ценами закупок на ССМ: rowmatprices.csv
имя файла с объемами закупок на ССМ:
префикс для имён файлов с рецептурами: recipe
символ разделителя между полями в CSV_файлах: ;
число столбцов с заголовками в CSV_файлах: 2
число строк с заголовками в CSV_файлах: 1
домашняя валюта проекта: RUR
принцип учета запасов: AVERAGE
Редактировать конфигурацию? [Y/N]Y
ВВЕДИТЕ КОНФИГУРАЦИЮ ИМПОРТА
В []-скобках представлено значение по умолчанию. Для его ввода просто нажмите Enter
имя файла с описанием проекта [about.txt]:
имя файла с отгрузками из СГП в натуральном выражении [ship.csv]:
имя файла с объемами производства, необязательно. Для пропуска введите nofile [nofile]:
имя файла с ценами закупок на ССМ [rowmatprices.csv]:
имя файла с объемами закупок на ССМ, необязательно. Для пропуска введите nofile [nofile]:
префикс для имён файлов с рецептурами [recipe]:
символ разделителя между полями в CSV_файлах [;]:
число столбцов с заголовками в CSV_файлах [2]:
число строк с заголовками в CSV_файлах [1]:
домашняя валюта проекта, RUR=1, USD=2, EUR=3, CNY=4 [1]:
принцип учета запасов, FIFO=0, LIFO=1, AVG=2 [2]:
ТЕКУЩАЯ КОНФИГУРАЦИЯ ИМПОРТА
имя файла с описанием проекта: about.txt
имя файла с отгрузками из СГП в натуральном выражении: ship.csv
имя файла с объемами производства:
имя файла с ценами закупок на ССМ: rowmatprices.csv
имя файла с объемами закупок на ССМ:
префикс для имён файлов с рецептурами: recipe
символ разделителя между полями в CSV_файлах: ;
число столбцов с заголовками в CSV_файлах: 2
число строк с заголовками в CSV_файлах: 1
домашняя валюта проекта: RUR
принцип учета запасов: AVERAGE
*****
**              Индивидуальные настройки учета продуктов на СГП              **
*****
№  Название                     Ед.измерения  Флаг периода  Авторасчет  Запас
0  Азу из говядины с картофе шт.             ALLOWED      AUTO        0.50
1  Баклажаны с томатом Неапо шт.             ALLOWED      AUTO        0.50
2  Лапша куриная                шт.             ALLOWED      AUTO        0.50
3  Плов с курицей                 шт.             ALLOWED      AUTO        0.50
4  Суп гороховый с копченост шт.             ALLOWED      AUTO        0.50
5  Фрикасе из курицы с зелен шт.             ALLOWED      AUTO        0.50
Введите номер номенклатурной позиции для редактирования [для выхода наберите "q" и нажмите "Enter"]:
```

Рисунок 24 Редактирование текущего конфигурационного файла

После окончания редактирования общих настроек, программа выводит на экран индивидуальные настройки учета продуктов на СГП и предлагает отредактировать их. Далее работа с программой идентична работе при первом запуске, начиная с шага №4.

Каждый запуск программы перезаписывает текущий конфигурационный файл. И при новом запуске пользователь видит на экране, может применить, а может изменить те настройки, которые он использовал в предыдущий раз.

При запуске программы без параметров (Рисунок 13), программа работает только с одним конфигурационным файлом *config.cfg*. Пользователь не может изменить имя или место расположения конфигурационного файла, но может запастись несколькими разными конфигурационными файлами, на разные случаи импорта.

```
12.11.2025 19:08 <DIR> .
12.11.2025 19:08 <DIR> ..
12.11.2025 18:30      1 324 config.cfg
25.10.2025 00:58      1 324 config_empty.cfg
25.10.2025 00:06      762 config_full_allowed.cfg
          3 File(s)          3 410 bytes
          2 Dir(s) 18 496 372 736 bytes free
```

Рисунок 25 Папка config: текущий и "запасные" конфигурационные файлы

И перед использованием программы создавать копию нужного в данный момент времени конфигурационного файла с именем *config.cfg* (Рисунок 25).

В зависимости от выбранного типа вещественных чисел, библиотека FROMA2 использует либо тип *double*, либо тип *LongReal*. Тип устанавливается на этапе компиляции библиотеки и не может быть изменен позже. Конфигурационные файлы, созданные программой, использующей библиотеку с установленным типом *double* **НЕСОВМЕСТИМЫ** с конфигурационными файлами, созданными программой, использующей библиотеку с установленным типом *LongReal*. Применение несовместимых конфигурационных файлов может привести к непредсказуемым последствиям.

ЗАПУСК ПРОГРАММЫ С ФАЙЛОМ КОНФИГУРАЦИИ В КАЧЕСТВЕ АРГУМЕНТА

Чтобы использовать файл конфигурации с произвольным именем и освободить себя от обязанности постоянно создавать копию нужного файла с именем *config.cfg*, программу требуется запустить, передав ей в качестве аргумента имя нужного файла конфигурации. Например, на рисунке ниже программе передается файл конфигурации с именем *other.cfg* (Рисунок 26).

```
C:\git\froma2\build\bin>Model_1 other.cfg
*****
**                                                                 **
**              Программа для расчета полных и удельных переменных затрат Model_1              **
**                                                                 **
*****
```

Рисунок 26 Запуск программы с файлом конфигурации в качестве аргумента

Если файла с таким именем нет, появится приглашение к вводу информации (Рисунок 14). Если файл с таким именем существует, то программа выведет на экран текущую конфигурацию и предложит ее отредактировать (Рисунок 23).

Имя файла конфигурации передается без пути к этому файлу. Файл конфигурации с произвольным именем создается в папке *froma2/build/examples/Model_1/config*. Использование других папок для файлов конфигурации не предусмотрено.

ПРОГРАММНЫЕ КОМПОНЕНТЫ

Программа состоит из основного файла реализации *main.cpp*, модуля *clsEnterprise*, включающего заголовочный файл *clsEnterprise.h* и соответствующий ему файл реализации *clsEnterprise.cpp* с классами и методами, реализующими функционал программы и заголовочного файла *pathes.h*, где прописаны пути для конфигурационного файла, файлов входных данных, файла отчета и файлов экспорта. Состав модуля *clsEnterprise* приведен в таблице ниже (Таблица 34).

Таблица 34 Состав модуля *clsEnterprise*

Состав модуля	Описание	Заметка
namespace nmEnterprise	Пространство имен с основными типами, константами	Таблица 35
template<typename T> constexpr bool is_ch_or_size_t_or_double_v	Функция, выполняемая на этапе компиляции и используемая в шаблонах для ограничения возможных типов	Ограничивает типы, применяемые в шаблонах следующим перечнем: <code>char</code> , <code>size_t</code> , <code>decimal</code>
const string confdir	Константа с путем к папке с файлами конфигурации	Устанавливается макросом <i>V_DIR_CONFIG</i> , определяемым в файле <i>pathes.h</i>
extern string cfg_file	Объявление глобальной переменной; содержит имя конфигурационного файла	Глобальная переменная определяется в файле <i>main.cpp</i>
const string NoFileName	Константа, определяющая строку, обозначающую отсутствие файла	По умолчанию равна строке "nofile"
const string indir	Константа с путем к папке с импортируемыми файлами	Устанавливается макросом <i>V_DIR_INPUTDATA</i> , определяемым в файле <i>pathes.h</i>
struct strImportConfig	Структурный тип для объекта, реализующего работу с конфигурационным файлом	Основной класс модели
class clsEnterprise	Класс для объекта, реализующего работу модели предприятия	Основной класс модели

Описание функций модуля *clsEnterprise* приведены ниже.

string DBLR_ind()

Функция без параметров, возвращает строку символов используемого типа вещественных чисел: "DB" - для вещественных чисел используется тип *double*, "LR" - для вещественных чисел используется тип *LongReal*. Тип возвращаемого значения *string*. Используется для добавления суффикса к имени выводимых файлов.

void strItemReplace(size_t count, strItem* dstn, strItem* src, ReportData flg)

Функция заменяет значения выбранных полей в заданном массиве значениями полей другого массива. Тип функции *void*. Используется в методах класса *clsEnterprise*.

Параметры:

count – размер массивов; тип *size_t*;

dstn – указатель на целевой массив, в котором меняются значения полей; тип элементов массива *strItem*;

src – указатель на массив-источник, откуда берутся новые значения для массива *dstn*; тип элементов массива *strItem*;

flg – флаг выбора поля элемента массива; тип *ReportData* («volume» или 0 – поле *volume*, «price» или 1 – поле *price* и «value» или 2 – поле *value*).

bool inData(string &_data, const string _defdata)

Функция вводит данные пользователя с устройства ввода (пользовательская альтернатива [std::cin](#)). Метод вводит строковые данные пользователя. При отсутствии данных, подставляются значения по умолчанию.

Параметры:

_data – ссылка на вводимую строку; тип *string*;

_defdata – строка по умолчанию; тип *string*.

Допускается в качестве параметров *_data* и *_defdata* использовать имя одной и той же переменной, например `inData(x, x)`, поскольку второй параметр передается в функцию как копия. Метод возвращает *true* в случае, если значение по умолчанию было отлично от [NoFileName](#) и новое значение равно *NoFileName*; в противном случае метод возвращает *false*. Используется в методе [strImportConfig::Entry](#).

template<typename T, class=std::enable_if_t<is_ch_or_size_t_or_double_v<T>>>**void inData(T &_data, const T _defdata)**

Шаблонная перегруженная функция вводит данные пользователя с устройства ввода (пользовательская альтернатива [std::cin](#)). Метод вводит данные типов, ограниченных функцией [is_ch_or_size_t_or_double_v](#).

Параметры:

_data – ссылка на вводимую строку; шаблонный тип *T*;

_defdata – строка по умолчанию; шаблонный тип *T*.

Допускается в качестве параметров *_data* и *_defdata* использовать имя одной и той же переменной, например `inData(x, x)`, поскольку второй параметр передается в функцию как копия. Метод *void*.

string FullFName(string _dir, string _fname)

Функция возвращает полное имя файла, komponуя его из строки с путем и строки с именем, являющихся входными параметрами. Если имя файла уже содержит путь, т.е. является полным, то путь не добавляется. Тип возвращаемого значения *string*.

Параметры:

_dir – путь к файлу; тип *string*;

_fname – имя файла; тип *string*.

Состав пространства имен *nmEnterprise* приведен в таблице ниже.

Таблица 35 Состав пространства имен *nmEnterprise*

Субъекты области имен <i>nmEnterprise</i>	Описание
struct strSettings	Структурный тип для описания индивидуальных настроек склада. Включает поля decimal share для хранения норматива товарного запаса, PurchaseCalc calc для хранения флага авто/ ручного расчета поступлений на склад, bool perm для хранения флага разрешения на поступления и отгрузки со склада в том же периоде; включает также методы сохранения состояния объекта в файл и восстановления из файла
const size_t w1, w2, w3	Константы, определяющие ширину столбцов выводимых в отчет таблиц; тип <i>size_t</i>
const size_t precis	Константа, определяющая количество знаков после запятой в выводимых в отчет вещественных числах; тип <i>size_t</i>

const size_t num_strings	Константа, определяющая количество строк, одновременно выводимых на экран при редактировании индивидуальных настроек склада; тип <i>size_t</i> . Используется в методе <i>showSKUsettings</i>
const unsigned int ProgressWide	Константа, определяющая ширину индикатора прогресса
enum SelectDivision	Тип данных перечисления <i>enum</i> ; состоит из конечного набора именованных констант типа <i>size_t</i> . Назначение: выбор нужного подразделения: « <i>warehouse</i> » или 0 – СГП, « <i>manufactory</i> » или 1 – Производство и « <i>rowmatstock</i> » или 2 – ССМ
enum ManufData	Тип данных перечисления <i>enum</i> ; состоит из конечного набора именованных констант типа <i>size_t</i> . Назначение: выбор нужных данных из Производства: « <i>manpurchase</i> » или 11 – потребность в сырье для производства, « <i>manbalance</i> » или 12 – незавершенное производство, « <i>manshipment</i> » или 13 – отгрузки произведенной продукции и « <i>recipe</i> » или 14 – рецептуры

Основными классами, реализующими функционал программы, являются класс *clsEnterprise* и структура *strImportConfig*.

Структура *strImportConfig* предназначена для работы с **конфигурационным файлом** программы (чтение, редактирование, запись). В ней сохраняются имена *csv*-файлов для импорта данных в модель, настройки импорта, общие и индивидуальные настройки учета номенклатурных позиций на складах. Состав структуры *strImportConfig* приведен ниже в таблице (Таблица 36).

Класс *clsEnterprise* представляет собой оболочку для трех объектов: *Склада готовой продукции* (объект типа *clsStorage*), *Производства* (объект типа *clsManufactory*) и *Склада ресурсов* (объект типа *clsStorage*). На схеме «Принципиальная схема модели с одним центром затрат» (Рисунок 10) класс *clsEnterprise* представляет собой сущность, обведенную штрихпунктирной линией черного цвета. Помимо функции оболочки, класс *clsEnterprise* представляет функцию интерфейса для взаимодействия указанных выше трех объектов между собой и внешней средой. Класс *clsEnterprise* наследуется от класса *clsBaseProject* библиотеки FROMA2. Структура класса *clsEnterprise* приведена в таблице ниже (Таблица 37).

Таблица 36 Состав структуры *strImportConfig*

Поле структуры	Описание	Заметка
string filename_About	Имя файла с описанием проекта. Тип <i>string</i>	См. описание в разделе Папки и файлы
string filename_Shipment	Имя файла с отгрузками с СГП в натуральном выражении. Тип <i>string</i>	
string filename_Production	Имя файла с объемами производства в натуральном выражении. Тип <i>string</i>	
string filename_Purchase	Имя файла с ценами закупок на ССМ. Тип <i>string</i>	
string filename_Purchase_V	Имя файла с объемами закупок на ССМ. Тип <i>string</i>	
string filenameprefix_Recipes	Префикс для имён файлов с рецептурами. Тип <i>string</i>	См. раздел Первые шаги работы с программой
char_ch	Символ разделителя между полями в <i>CSV</i> -файлах	
size_t HeadCols	Количество столбцов с заголовками в <i>CSV</i> -файлах; тип <i>size_t</i>	
size_t HeadRows	Количество строк с заголовками в <i>CSV</i> -файлах; тип <i>size_t</i>	
Currency_cur	Домашняя валюта проекта; тип	

AccountingMethod _amethod	Принцип учета запасов на складах; Тип AccountingMethod тип перечисление <i>enum</i> , состоящий из значений FIFO , LIFO , AVG
decimal P_Share	Запас ресурсов на складе ССМ, доля от отгрузок; начальная установка; тип вещественного числа определяется псевдонимом decimal
decimal S_Share	Запас продуктов на складе СГП, доля от отгрузок; начальная установка; тип вещественного числа определяется псевдонимом decimal
bool P_indr	Флаг разрешения на отгрузки с ССМ в том же периоде, что и поступления
bool S_indr	Флаг разрешения на отгрузки с СГП в том же периоде, что и поступления
strSettings* P_settings	Указатель на массив с индивидуальными настройками ССМ; тип элемента массива strSettings
strSettings* S_settings	Указатель на массив с индивидуальными настройками СГП; тип элемента массива strSettings
size_t PsetCount	Размер массива P_settings; тип size_t
size_t SsetCount	Размер массива S_settings; тип size_t

Ниже описываются методы структуры *strImportConfig*.

strImportConfig()

Конструктор по умолчанию (без параметров). Статическим полям класса присваивает значения по умолчанию. Динамические поля не создаются, указатели на них устанавливаются в *nullptr*.

strImportConfig(const strImportConfig& other)

Конструктор копирования.

Параметры:

& other – ссылка на копируемый константный экземпляр класса *strImportConfig*.

strImportConfig(strImportConfig&& other)

Конструктор перемещения.

Параметры:

&& other - перемещаемый экземпляр класса *strImportConfig*.

strImportConfig& operator=(const strImportConfig& obj)

Оператор присваивания копированием.

Параметры:

obj – ссылка копируемый константный экземпляр класса *strImportConfig*.

strImportConfig& operator=(strImportConfig&& obj)

Оператор присваивания перемещением.

Параметры:

obj – ссылка перемещаемый константный экземпляр класса *strImportConfig*.

~strImportConfig()

Деструктор.

bool SaveToFile(const string _filename)

Метод записи текущей конфигурации в файл.

Параметры:

_filename – имя файла для записи конфигурации. Тип *string*.

void swap(strImportConfig& obj) noexcept

Функция обмена значениями между объектами. Функция объявлена noexcept - не вызывающей исключения.

Параметры:

obj – ссылка на обмениваемый экземпляр класса *strImportConfig*.

bool ReadFromFile(const string _filename)

Метод восстановления конфигурации из файла в текущий объект.

Параметры:

_filename – имя файла для чтения конфигурации. Тип *string*.

void Entry()

Метод ввода конфигурационных данных с терминала. Ввод данных производится в диалоговом режиме.

void Show()

Метод отображает текущую конфигурацию на экране.

void Configure()

Метод читает конфигурацию импорта и, при необходимости редактирует её с последующим сохранением в конфигурационном файле с именем, содержащимся в переменной [cfg_file](#). Использует методы *Show*, *Entry*, *ReadFromFile* и *SaveToFile*.

size_t GetPsetCount() const

Метод возвращает размер массива [P_settings](#). Тип возвращаемого значения *size_t*.

size_t GetSsetCount() const

Метод возвращает размер массива [S_settings](#). Тип возвращаемого значения *size_t*.

strSettings* GetPurSettings() const

Метод возвращает указатель на вновь создаваемую копию массива *P_settings*. Тип возвращаемого значения *strSettings**.

strSettings* GetShpSettings() const

Метод возвращает указатель на вновь создаваемую копию массива *S_settings*. Тип возвращаемого значения *strSettings**.

bool SetPurSettings(const strSettings* _P_settings, const size_t pcount)

Метод копирует массив, заданный указателем *_P_settings* во внутренний массив с указателем *strImportConfig::P_settings*. При удачном копировании, метод возвращает *true*, иначе – *false*.

Параметры:

**_P_settings* – указатель на копируемый массив типа *strSettings*;

pcount – размер копируемого массива.

bool SetShpSettings(const strSettings* _S_settings, const size_t scount)

Метод копирует массив, заданный указателем *_S_settings* во внутренний массив с указателем *strImportConfig::S_settings*. При удачном копировании, метод возвращает *true*, иначе – *false*.

Параметры:

**_S_settings* – указатель на копируемый массив типа *strSettings*;

scount – размер копируемого массива.

void SetToAuto(const SelectDivision& _dep)

Метод устанавливает поля *calc* массива *P_settings* или *S_settings* в состояние *calc* (авторасчет поступлений). Выбор массива определяется флагом.

Параметры:

_dep – ссылка на флаг выбора массива: *rowmatstock* - для массива *P_settings*, *warehouse* - для массива *S_settings*; тип *SelectDivision*.

Состав класса *clsEnterprise* приведен ниже.

Таблица 37 Состав класса *clsEnterprise*

Поле класса	Описание	Заметка
clsStorage* Warehouse	Склад готовой продукции (СГП). Указатель на объект типа <i>clsStorage</i> .	Основные объекты класса <i>clsEnterprise</i> , определяющие структуру модели предприятия
clsManufactory* Manufactory	Производство. Указатель на объект типа <i>clsManufactory</i> .	
clsStorage* RawMatStock	Склад ресурсов (сырья и материалов, ССМ). Указатель на объект типа <i>clsStorage</i> .	
size_t ProdCount	Полное число выпускаемых продуктов. Тип <i>size_t</i> .	Элемент интерфейса к СГП и Производству. Выполняет роль буфера ввода данных.
strItem* Shipment	Указатель на массив отгрузок со склада СГП; размер массива <i>ProdCount*PrCount</i> . Тип элемента массива <i>strItem</i> .	Элемент интерфейса к СГП. Выполняет роль буфера обмена.
strItem* Production	Указатель на массив поступлений из производства на СГП; размер массива <i>ProdCount*PrCount</i> . Тип элемента массива <i>strItem</i> .	Элемент интерфейса к СГП и Производству. Выполняет роль буфера обмена.

strNameMeas* ProdNames	Указатель на массив названий и единиц измерения продуктов; размер массива <i>ProdCount</i> . Тип элемента массива <i>strNameMeas</i> .	Элемент интерфейса к СГП и Производству. Выполняет роль буфера ввода данных.
size_t RMCOUNT	Полное число наименований сырья и материалов. Тип <i>size_t</i> .	Элемент интерфейса к ССМ и Производству. Выполняет роль буфера ввода данных.
strItem* Purchase	Указатель на массив поступлений ресурсов на ССМ; размер массива <i>RMCOUNT*PrCount</i> . Тип элемента массива <i>strItem</i> .	Элемент интерфейса к ССМ. Выполняет роль буфера обмена.
strItem* Consumpt	Указатель на массив отгрузок ресурсов из ССМ в Производство; размер массива <i>RMCOUNT*PrCount</i> . Тип элемента массива <i>strItem</i> .	Элемент интерфейса к ССМ и Производству. Выполняет роль буфера обмена.
decimal* PriceBus	Указатель на массив стоимости ресурсов для производства; размер массива <i>RMCOUNT*PrCount</i> . Тип вещественного числа элемента массива определяется псевдонимом <i>decimal</i> .	Элемент интерфейса к ССМ и Производству. Выполняет роль буфера обмена.
strNameMeas* RMNames	Указатель на массив названий и ед. измерения ресурсов; размер массива <i>RMCOUNT</i> . Тип элемента массива <i>strNameMeas</i> .	Элемент интерфейса к ССМ и Производству. Выполняет роль буфера ввода данных.
vector<clsRecipeItem> Recipe	Вектор с рецептурами продуктов; размер вектора <i>ProdCount</i> . Тип элемента вектора <i>clsRecipeItem</i> .	Элемент интерфейса к Производству. Выполняет роль буфера ввода данных.
strSettings* P_settings	Указатель на массив с индивидуальными настройками ССМ размером <i>RMCOUNT</i> . Тип элемента массива <i>strSettings</i> .	Элемент интерфейса к ССМ. Выполняет роль буфера ввода данных.
strSettings* S_settings	Указатель на массив с индивидуальными настройками СГП размером <i>ProdCount</i> . Тип элемента массива <i>strSettings</i> .	Элемент интерфейса к СГП. Выполняет роль буфера ввода данных.
size_t PrCount	Количество периодов проекта, из них 0 - стартовый. Тип <i>size_t</i> .	Элемент интерфейса к СГП, Производству и ССМ. Выполняет роль буфера ввода данных.
Currency Cur	Домашняя валюта проекта. Тип <i>Currency</i> .	Элемент интерфейса к СГП, Производству и ССМ. Выполняет роль буфера ввода данных.
AccountingMethod Amethod	Принцип учета запасов. Тип <i>AccountingMethod</i> .	Элемент интерфейса к СГП и ССМ. Выполняет роль буфера ввода данных.
PurchaseCalc purcalc	Флаг разрешающий/ запрещающий автоматический расчёт объемов закупок ресурсов. Тип <i>PurchaseCalc</i> .	Элемент интерфейса к ССМ. Выполняет роль буфера ввода данных.
PurchaseCalc mancalc	Флаг разрешающий/ запрещающий автоматический расчёт объемов производства. Тип <i>PurchaseCalc</i> .	Элемент интерфейса к СГП. Выполняет роль буфера ввода данных.
decimal PurchShare	Запас ресурсов на складе ССМ, доля от отгрузок. Тип вещественного числа определяется псевдонимом <i>decimal</i> .	Элемент интерфейса к ССМ. Выполняет роль буфера ввода данных.
decimal ShipShare	Запас продуктов на складе СГП, доля от отгрузок. Тип вещественного числа определяется псевдонимом <i>decimal</i> .	Элемент интерфейса к СГП. Выполняет роль буфера ввода данных.
bool Purch_indr	Флаг разрешения на отгрузки с ССМ в том же периоде, что и поступления. Тип <i>bool</i> .	Элемент интерфейса к ССМ. Выполняет роль буфера ввода данных.
bool Ship_indr	Флаг разрешения на отгрузки с СГП в том же периоде, что и поступления. Тип <i>bool</i> .	Элемент интерфейса к СГП. Выполняет роль буфера ввода данных.

ПУБЛИЧНЫЕ МЕТОДЫ КЛАССА *clsEnterprise*

СЛУЖЕБНЫЕ МЕТОДЫ

clsEnterprise()

Конструктор по умолчанию (без параметров). Статическим полям класса присваивает значения по умолчанию. Динамические поля не создаются, указатели на них устанавливаются в *nullptr*.

void swap(clsEnterprise& other) noexcept

Функция обмена значениями между объектами.

Параметры:

& other – ссылка на обмениваемый экземпляр класса *clsEnterprise*.

clsEnterprise(const clsEnterprise& other)

Конструктор копирования.

Параметры:

& other – ссылка на копируемый константный экземпляр класса *clsEnterprise*.

clsEnterprise(clsEnterprise&& other)

Конструктор перемещения.

Параметры:

&& other - перемещаемый экземпляр класса *clsEnterprise*.

clsEnterprise& operator=(const clsEnterprise& other)

Перегрузка оператора присваивания копированием.

Параметры:

& other – ссылка на копируемый константный экземпляр класса *clsEnterprise*.

clsEnterprise& operator=(clsEnterprise&& other)

Перегрузка оператора присваивания перемещением

Параметры:

&& other - перемещаемый экземпляр класса *clsEnterprise*.

virtual ~clsEnterprise()

Виртуальный деструктор.

МЕТОДЫ ИМПОРТА

bool Import_Data()

Агрегированный метод импорта. Включает методы конфигурирования импорта, редактирования и сохранения конфигурации в файл; а также методы импорта данных. Основным «рабочим инструментом» метода является локальная переменная типа *strImportConfig*. Метод возвращает *true* в случае успешного импорта данных, в противном случае возвращает *false*.

СОЗДАНИЕ И РАСЧЕТ СКЛАДА ГОТОВОЙ ПРОДУКЦИИ (СГП)

bool SetWarehouse()

Метод создания склада готовой продукции (СГП) и ввода в него данных. Метод возвращает *true* в случае успешного создания склада и ввода в него данных, в противном случае возвращает *false*.

void StockEditSettings(SelectDivision stk)

Метод редактирует индивидуальные настройки учета номенклатурных единиц на выбранном складе (СГП или ССМ). Для каждого SKU возможно изменение данных: [флаг авторасчета закупок](#), [флаг разрешения на отгрузку и закупку в одном периоде](#) и [норматив запаса на складе](#). Настройки сохраняются в объекте "склад" и конфигурационном файле.

Параметры:

stk – признак выбранного склада: "0" или "warehouse" – СГП, любое другое значение – ССМ. Тип данных перечисления *enum*; состоит из конечного набора именованных констант типа *size_t*.

bool StockCalculate(const SelectDivision& _dep, size_t thr)

Метод рассчитывает требуемый объем закупок/поступлений на склад для тех *SKU*, у которых выставлен разрешающий такой расчет флаг.

Параметры:

&_dep – ссылка на признак выбранного склада: "0" или "warehouse" – СГП, любое другое значение – ССМ. Тип [SelectDivision](#) тип данных перечисления *enum*; состоит из конечного набора именованных констант типа *size_t*.

thr – флаг требуемого способа вычислений. Значением переменной *thr* устанавливается метод вычисления: "1" – [clsStorage::Calculate_future](#), "2" – [clsStorage::Calculate_thread](#), иначе [clsStorage::Calculate](#).

Метод возвращает *true* в случае удачного расчёта. В случае, если поступления на склад и отгрузка с него не сбалансированы (имеет место [дефицит](#)) или в случае отсутствия склада, метод возвращает *false*.

СОЗДАНИЕ И РАСЧЕТ ПРОИЗВОДСТВА

bool SetManufactory()

Метод создания Производства и ввода в него данных. Метод возвращает *true* в случае успешного создания Производства и ввода в него данных, в противном случае возвращает *false*.

void ManufCalculateIn(size_t thr)

Метод рассчитывает объем потребления сырья и материалов в производстве в натуральном выражении для всего плана выпуска всех продуктов. Значением переменной *thr* устанавливается метод вычисления: "1" – [clsManufactory::CalcRawMatPurchPlan_future](#), "2" – [clsManufactory::CalcRawMatPurchPlan_thread](#), иначе [clsManufactory::CalcRawMatPurchPlan](#).

Параметры:

thr – переменная для выбора способа расчета. Тип *size_t*.

bool ManufCalculateOut(size_t thr)

Метод рассчитывает объем, удельную и полную себестоимость незавершенного производства и готовой продукции для всех продуктов, выпускаемых на протяжении всего проекта. Значением переменной *thr* устанавливается метод вычисления: "1" – [clsManufactory::Calculate_future\(\)](#), "2" – [clsManufactory::Calculate_thread\(\)](#), иначе [clsManufactory::Calculate\(\)](#). Метод возвращает *true* в случае успешного расчета, в противном случае возвращает *false*.

Параметры:

thr – переменная для выбора способа расчета. Тип *size_t*.

bool MWCostTransmition()

Метод передает учетную себестоимость произведенной продукции из производства на склад готовой продукции. Метод возвращает *true* в случае успешной передачи данных, в противном случае возвращает *false*.

СОЗДАНИЕ И РАСЧЕТ СКЛАДА СЫРЬЯ И МАТЕРИАЛОВ (CCM)**bool SetRawMatStock()**

Метод создания склада сырья и материалов (CCM) и ввода в него данных. Метод возвращает *true* в случае успешного создания склада и ввода в него данных, в противном случае возвращает *false*.

bool RMCostTransmition()

Метод передает учетную себестоимость сырья и материалов со склада сырья и материалов в производство. Метод возвращает *true* в случае успешной передачи данных, в противном случае возвращает *false*.

МЕТОДЫ ДЛЯ ВИЗУАЛЬНОГО КОНТРОЛЯ И ОТЧЕТОВ**void ReportView(const SelectDivision& _rep, const int _arr, const ReportData flg) const**

Функция выводит выбранный отчет.

Параметры:

_rep - выбранное подразделение (*warehouse* - СГП, *manufactory* - производство, *rowmatstock* - CCM); тип [SelectDivision](#) тип данных перечисления *enum*, состоит из конечного набора именованных констант типа *size_t*;

arr - тип данных (*purchase* - массив поступлений, *balance* - массив остатков, *shipment* - массив отгрузок); тип *int*;

flg - тип выводимой информации: *volume* - в натуральном, *value* - в стоимостном, *price* - в ценовом измерении; тип [ReportData](#).

void StockSettingsView(const SelectDivision& _rep)

Функция выводит индивидуальные настройки склада.

Параметры:

_rep - выбранный склад (*warehouse* - СГП, *rowmatstock* или любой другой - CCM); тип [SelectDivision](#) тип данных перечисления *enum*, состоит из конечного набора именованных констант типа *size_t*.

МЕТОДЫ ЭКСПОРТА**bool Export_Data(string filename, const SelectDivision& _dep, const ChoiseData& _arr, const ReportData& flg) const;**

Метод экспортирует массив поставок, остатков или отгрузок со склада готовой продукции (СГП), склада сырья и материалов (CCM) или с Производства в *csv*-файл с именем *filename*.

Параметры:

_dep - флаг выбора подразделения; тип данных перечисления *enum*, состоит из конечного набора именованных констант типа *size_t* ("*warehouse*" - СГП, "*rowmatstock*" - CCM, "*manufactory*" - Производство); тип [SelectDivision](#);

`_arr` – флаг выбора данных; тип данных перечисления *enum*, состоит из конечного набора именованных констант типа `size_t` ("purchase" - поставки, "balance" - остатки/незавершенное производство, "shipment" - отгрузки); тип [ChoiseData](#);

`flg` - флаг выводимой в файл информации; тип данных перечисления *enum*, состоит из конечного набора именованных констант типа `size_t` (*volume* - в натуральном, *value* - в стоимостном, *price* - в ценовом измерении); тип [ReportData](#).

МЕТОДЫ СЕКЦИИ PRIVATE КЛАССА `clsEnterprise`

МЕТОДЫ ИМПОРТА

`bool Import_About(const string filename)`

Метод читает информацию из файла с описанием проекта и формирует поля *Title* и *Descript*, наследованные от класса [clsBaseProject](#). Импортируемый файл должен быть текстовым. Возвращает *true* при удачном импорте. Иначе возвращает *false*.

Параметры:

`filename` – имя файла для импорта; тип *string*.

МЕТОДЫ РЕДАКТИРОВАНИЯ

`bool SKUEdt(clsStorage* stock, const size_t num)`

Метод редактирования введенной ранее информации: редактирование номенклатурной позиции (*SKU*). Возвращает *true*, если на запрос о номере редактируемой позиции, указана существующая позиция. В противном случае возвращает *false*.

Параметры:

`stock` – указатель на конкретный склад ([Warehouse](#) или [RawMatStock](#)); тип [clsStorage](#);

`num` – номер редактируемой номенклатурной позиции; тип *size_t*.

МЕТОДЫ ДЛЯ ВИЗУАЛЬНОГО КОНТРОЛЯ И ОТЧЕТОВ

`void showSKUsettings(ostream& os, clsStorage* stock) const`

Метод вывода в выходной поток настроек учета на складе для каждого *SKU*: номер, название и ед. измерения *SKU*, флаг автоматического расчета закупок, флаг разрешения на отгрузку и закупку в одном периоде и норматив запаса на складе.

Параметры:

`os` – поток для вывода; тип *ostream*;

`stock` – указатель на конкретный склад ([Warehouse](#) или [RawMatStock](#)); тип [clsStorage](#).

`bool Report_Storage(clsStorage* obj, const int _arr, const ReportData flg) const`

Метод для вывода на экран отчетов по объекту типа *clsStorage* (*Warehouse* или *RawMatStock*). В зависимости от флага `_arr`, выводит на заданное устройство (терминал, файл или пустое устройство) массив поступлений, массив остатков или массив отгрузок со склада. В зависимости от флага *flg*, выводит данные в натуральном, полном стоимостном или удельном стоимостном (ценовом) выражении. Используется в методе *Report_Storage_to_dev*. В случае удачного вывода, возвращает *true*, иначе – *false*.

Параметры:

`obj` – указатель на `obj` – экземпляр класса *clsStorage* (*Warehouse* или *RawMatStock*); тип *clsStorage*;

`_arr` – флаг выбора данных для вывода, тип `int` (`_arr = purchase`, массив поступлений на склад; `_arr = balance`, массив остатков на складе; `_arr = shipment`, массив отгрузок со склада). Если значение `_arr` не относится к перечисляемому типу `ChoiseData`, то отчет не выводится;

`flg` – флаг, определяющий тип импортируемых данных: "volume" - объемы в натуральном выражении, "price" - цены, "value" – стоимость; тип `ReportData`.

bool Report_Storage_to_dev(clsStorage* obj, const int _arr, const ReportData flg) const

Метод направляет отчет, созданный методом `Report_Storage` на выбранное устройство: "пустое" устройство, экран или файл (определяется содержанием переменной `Rdevice`). Использует метод `Report_Storage`. В случае удачного вывода, возвращает `true`, иначе – `false`.

Параметры:

`obj` – указатель на `obj` – экземпляр класса `clsStorage` (`Warehouse` или `RawMatStock`); тип `clsStorage`;

`_arr` – флаг выбора данных для вывода, тип `int` (`_arr = purchase`, массив поступлений на склад; `_arr = balance`, массив остатков на складе; `_arr = shipment`, массив отгрузок со склада). Если значение `_arr` не относится к перечисляемому типу `ChoiseData`, то отчет не выводится;

`flg` – флаг, определяющий тип импортируемых данных: "volume" - объемы в натуральном выражении, "price" - цены, "value" – стоимость; тип `ReportData`.

void Report_Recipe() const

Метод для вывода в отчет рецептов всех продуктов. Используется в методе `Report_Manufactory`.

bool Report_Manufactory(const int _arr, const ReportData flg) const

Метод для вывода на экран отчетов по производству. В зависимости от флага `_arr`, выводит на заданное устройство (терминал, файл или пустое устройство) массив поступлений ресурсов, массив с незавершенным производством или массив отгрузок готовой продукции. В зависимости от флага `flg`, выводит данные в натуральном, полном стоимостном или удельном стоимостном (ценовом) выражении. Использует метод `Report_Recipe`. В случае удачного вывода, возвращает `true`, иначе – `false`.

Параметры:

`_arr` – флаг выбора данных для вывода, тип `int` (`_arr = manpurchase`, массив поступлений ресурсов в производство; `_arr = manbalance`, массив с балансами незавершенного производства для всех продуктов; `_arr = manshipment`, массив отгрузок готовой продукции; `_arr = recipe`, рецептуры продуктов). Если значение `_arr` не относится к перечисляемому типу `ManufData`, то отчет не выводится;

`flg` – флаг, определяющий тип импортируемых данных: "volume" - объемы в натуральном выражении, "price" - цены, "value" – стоимость; тип `ReportData`.

МЕТОДЫ СЕКЦИИ PROTECTED КЛАССА clsEnterprise

МЕТОДЫ ДЛЯ ВИЗУАЛЬНОГО КОНТРОЛЯ И ОТЧЕТОВ

virtual void reportstream(ostream& os) const

Метод выводит информацию о проекте в поток `os`. Метод переопределяет одноименный метод класса `clsBaseProject::reportstream`.

Параметры:

`os` – ссылка на поток для вывода отчета. Тип `ostream`.

КЛАСС `clsCostCenterStorage` - СКЛАД С ДВУМЯ ЦЕНТРАМИ ЗАТРАТ

Рисунок 11 представляет нам принципиальную схему модели предприятия с тремя центрами затрат. Внимательный читатель может заметить, что каждый склад (Склад ресурсов или Склад готовой продукции) имеет по одному центру затрат, который может начислять затраты на отгружаемые со склада ресурсы/продукцию. А как быть в ситуации, когда затраты должны начисляться на принимаемые складом ресурсы/продукты? Например, когда на складе возникают затраты по разгрузке поступающих на хранение товаров? Для типичного случая с затратами на разгрузку поступающих на склад ресурсов и погрузку, отгружаемых со склада ресурсов, эта схема оказывается недостаточной. Для этого случая склад должен иметь *два центра затрат*: Центр затрат *на приеме ресурсов* и центр затрат *на выдаче ресурсов*. Схема такого склада приведена на рисунке ниже.



Рисунок 27 Принципиальная схема модели склада с двумя центрами затрат

Приведенная выше схема реализована в классе `clsCostCenterStorage` в одноименном модуле. Класс `clsCostCenterStorage` наследуется от класса `clsStorage` из библиотеки *FROMA2*. Состоит из: ЦЗП - *центра учета затрат по поступлению* (PCC - Purchase Cost Centre), ЦУР - *центра учета ресурсов* (RAC - Resource Accounting Centre) и ЦЗО - *центра учета затрат по отгрузке* (SCC - Shipment Cost Centre). Роль ЦУР играет объект родительского класса `clsStorage`. Для демонстрации возможностей данного класса в репозиторий включена программа *CCStorage*.

Ресурсы, поступающие на такой склад, можно разделить на три группы (Рисунок 28):

- «Транзитные» ресурсы - ресурсы, которые поступают на склад, хранятся на нем и в дальнейшем отгружаются со склада;
- Ресурсы, потребляемые на складе при отгрузке транзитных ресурсов (ЦЗО);
- Ресурсы, потребляемые на складе при поступлении ресурсов (ЦЗП).

	A	B	C	D	E	F	G	H
1	Name	Measure						
2	Азу из говядины с картофелем	шт.						
3	Баклажаны с томатом Неаполь	шт.						
4	Лапша куриная	шт.						
5	Плов с курицей	шт.						
6	Суп гороховый с копченостями	шт.						
7	Фрикасе из курицы с зеленым горошком	шт.						
8	Погрузка	чел*час						
9	Разгрузка	чел*час						
10								

Рисунок 28 Пример ресурсов для склада с двумя центрами затрат

Пример *технологической карты* ресурса с конкретным наименованием для учета затрат на разгрузку в ЦЗП представлен на рисунке ниже (Рисунок 29).

Азу из говядины с картофелем	шт.	Расход
Азу из говядины с картофелем	шт.	1.0
Разгрузка	чел*час	0.001

Рисунок 29 Пример технологической карты для ЦЗП

Как мы видим, «сырьем» для «продукта» является сам продукт и те ресурсы, которые потребляются в ЦЗП при поступлении. В данном конкретном примере это *работа по разгрузке* транспорта на склад. Расход ресурса на самого себя в данном примере равен 1,0. Это означает, что потерь при разгрузке нет. Если бы потери имели место, то вместо единицы стояло бы число меньшее, - например, при потерях 5%, стояло бы число 0,95.

Пример технологической карты того же ресурса для учета затрат в ЦЗО представлен на рисунке ниже (Рисунок 30).

Азу из говядины с картофелем	шт.	Расход
Азу из говядины с картофелем	шт.	1.0
Погрузка	чел*час	0.001

Рисунок 30 Пример технологической карты для ЦЗО

Логика та же. «Сырьем» для «продукта» является сам продукт и те ресурсы, которые потребляются в ЦЗО при отгрузке. В данном конкретном примере это *работа по погрузке* со склада в транспорт. Потерь при отгрузке в примере нет.

Работа с классом *clsCostCenterStorage* состоит из следующих этапов.

1. Создаем склад, используя [конструктор с параметрами](#). Вводим в него: количество периодов проекта, домашнюю валюту, принцип учета запасов, число транзитных ресурсов, число ресурсов, являющихся транзитными в сумме с числом ресурсов, потребляемых в ЦЗО, общее число ресурсов (транзитных, потребляемых в ЦЗО и потребляемых в ЦЗП), а также наименования и единицы измерения всех ресурсов.
2. [Вводим в склад исходные данные](#) такие, как объем отгрузок транзитных ресурсов в натуральном выражении, цены на все поставляемые ресурсы (включая транзитные ресурсы, ресурсы для потребления в ЦЗО и ЦЗП), технологические карты для ЦЗО и ЦЗП и рассчитываем потребление ресурсов на входе склада²⁵.
3. При необходимости, корректируются индивидуальные настройки каждого наименования ресурсов (см. описание класса [clsStorage](#)).
4. [Рассчитываются](#) удельные и полные учетные себестоимости ресурсов.
5. [Выводим](#) или [экспортируем](#) результаты расчёта.

Так же как в моделях с одним (Рисунок 10) или с тремя центрами затрат (Рисунок 11), алгоритм расчета учетной себестоимости отгружаемой со Склада готовой продукции является *двухпроходным*: сначала *от клиентов к поставщикам* идет расчет *объемных* показателей в натуральном выражении, потом *от поставщиков к клиентам* идет расчет *денежных* показателей.

СОСТАВ МОДУЛЯ clsCostCenterStorage

Состав модуля clsCostCenterStorage приведен в таблице ниже.

Таблица 38 Состав модуля clsCostCenterStorage

Состав модуля	Описание	Заметка
---------------	----------	---------

²⁵ Класс *clsCostCenterStorage* наследуется от класса *clsStorage* и поэтому позволяет выбрать ручной режим ввода объемов потребляемых ресурсов. В этом случае также вводятся объемы приобретаемых ресурсов.

enum Clc_type	Тип перечисление <i>enum</i> , состоящий из значений seq, fut и thrd	Предназначен для выбора метода вычислений: последовательные, в асинхронных потоках или синхронных
enum count_choise	Тип перечисление <i>enum</i> , состоящий из значений inStock, midStock и outStock	Предназначен для выбора точки учёта: на входе в ЦЗП, в ЦУР, на выходе из ЦЗО
class clsCostCenterStorage : public clsStorage	Класс "Склад" с центром затрат. Наследник класса clsStorage из библиотеки FROMA2. Состоит из: ЦЗП - центра учета затрат по поступлению (PCC - Purchase Cost Centre), ЦУР - центр учета ресурсов (RAC - Resource Accounting Centre) и ЦЗО - центр учета затрат по отгрузке (SCC - Shipment Cost Centre).	Роль ЦУР играет объект родительского класса <i>clsStorage</i> .

ПОЛЯ КЛАССА clsCostCenterStorage

Таблица 39 Поля класса clsCostCenterStorage. Секция Private

Поле класса	Описание	Заметка
size_t InCount, MidCount, OutCount	Тип size_t . Количество номенклатурных позиций на входе, в середине и выходе склада	Значения полей отличаются друг от друга: часть поступающих на склад ресурсов после хранения отгружается потребителям, часть ресурсов потребляется в ЦЗО, часть – в ЦЗП.
TLack Lack	Размер дефицита и имя дефицитного ресурса. Тип TLack .	Поле хранит величину дефицита и название ресурса, где образовался этот дефицит
clsManufactory* OutCost	ЦЗО - центр учета затрат по отгрузке. Тип clsManufactory	Позволяет аллоцировать затраты на входящие в склад ресурсы
clsManufactory* InCost	ЦЗП - центр учета затрат по поступлению. Тип <i>clsManufactory</i> .	Позволяет аллоцировать затраты на исходящие со склада ресурсы
vector<clsRecipeItem> RecipeOut	Набор технологических карт для ЦЗО. Тип vector<clsRecipeItem>	Для корректной работы класса необходимо, чтобы размер контейнера равнялся значению поля <i>OutCount</i>
vector<clsRecipeItem> RecipeIn	Набор технологических карт для ЦЗП. Тип <i>vector<clsRecipeItem></i>	Для корректной работы класса необходимо, чтобы размер контейнера равнялся значению поля <i>MidCount</i>
strItem* Purchase	Данные о ценах и объемах закупок (в случае установки флага расчета PurchaseCalc в <i>nocalc</i>). Тип указатель на массив типа strItem	Массив является одномерным аналогом матрицы размером <i>InCount</i> * clsStorage::PrCount
strItem* Shipment	Данные о стоимости и объемах отгрузок. Тип указатель на массив типа <i>strItem</i>	Массив является одномерным аналогом матрицы размером <i>OutCount</i> * clsStorage::PrCount

Методы класса *clsCostCenterStorage* можно условно разделить на следующие группы:

- Конструкторы, деструктор, перегруженные операторы;
- Методы сохранения состояния объекта в файл и восстановления из файла;
- Set-методы;
- Расчетные методы;
- Get-методы;
- Методы экспорта;
- Удаляемые методы предка (методы не используются);

- Скрываемые методы предка.

Ниже подробно описываются все методы.

КОНСТРУКТОРЫ, ДЕСТРУКТОР, ПЕРЕГРУЖЕННЫЕ ОПЕРАТОРЫ

clsCostCenterStorage()

Конструктор по умолчанию (без параметров). Вызывает конструктор без параметров предка (конструктор [clsStorage](#)). Создает и инициализирует переменные: *InCount* = *MidCount* = *OutCount* = 0; *Lack* = {0, ""}; *OutCost* = *InCost* = *Purchase* = *Shipment* = *nullptr*; создает на стеке пустые контейнеры *RecipeOut* и *RecipeIn*.

clsCostCenterStorage(size_t _PrCount, Currency _cur, AccountingMethod _ac, size_t ProdCount, const size_t _MidCount, const size_t _RMCount, const strNameMeas _RMNames[])

Конструктор с вводом параметров. Вызывает [конструктор с параметрами класса-предка](#), передавая ему длительность проекта, валюту, принцип учета запасов и количество номенклатурных позиций, планируемых к добавлению в контейнер. Конструктор инициализирует поля *InCount*, *MidCount* и *OutCount* переданными значениями *_RMCount*, *_MidCount* и *ProdCount*, соответственно; устанавливает поле *Lack* = {0, ""}; выделяет память и создает центры затрат - ЦЗО и ЦЗП (объекты *OutCost*, *InCost*); указателям *Purchase* и *Shipment* присваивает *nullptr*.

Параметры:

_PrCount – число периодов проекта; тип *size_t*;

_cur – валюта проекта; тип [Currency](#);

_ac – принцип учета запасов (FIFO, LIFO, AVG); тип [AccountingMethod](#);

ProdCount – число номенклатурных позиций, отгружаемых со склада; тип *size_t*;

_MidCount – число позиций на выходе ЦЗП/ входе ЦЗО; тип *size_t*;

_RMCount – число номенклатурных позиций, поступающих на склад; тип *size_t*;

_RMNames – указатель на массив с именами и единицами измерения ресурсов, поступающих на склад, размером *_RMCount*; тип *size_t*;

~clsCostCenterStorage()

Деструктор. Виртуальный.

void swap(clsCostCenterStorage& other) noexcept

Функция обмена значениями между экземплярами класса.

Параметры:

&obj – ссылка на обмениваемый экземпляр класса *clsCostCenterStorage*.

clsCostCenterStorage(const clsCostCenterStorage& other)

Конструктор копирования. Вызывает [конструктор копирования предка](#), передавая ему параметр *other*.

Параметры:

& other – ссылка на копируемый константный экземпляр класса *clsCostCenterStorage*.

clsCostCenterStorage(clsCostCenterStorage&& other)

Конструктор перемещения. Вызывает конструктор без параметров *clsCostCenterStorage*, создает «пустой» объект и обменивается этим объектом с инициализирующим.

Параметры:

&& other - перемещаемый экземпляр класса *clsCostCenterStorage*.

clsCostCenterStorage& operator=(const clsCostCenterStorage& other)

Перегрузка оператора присваивания копированием.

Параметры:

& other – ссылка на копируемый константный экземпляр класса *clsCostCenterStorage*.

clsCostCenterStorage& operator=(clsCostCenterStorage&& other)

Перегрузка оператора присваивания перемещением

Параметры:

&& other - перемещаемый экземпляр класса *clsCostCenterStorage*.

МЕТОДЫ СЕРИАЛИЗАЦИИ И ДЕСЕРИАЛИЗАЦИИ

bool StF(ofstream &_outF)

Метод имплементации записи в файловую переменную текущего экземпляра класса (запись в файл, метод сериализации).

Параметры:

&_outF – ссылка на экземпляр класса *ofstream* для записи данных. При инициализации переменной *_outF* необходимо установить флаг бинарного режима открытия потока (не текстового!) [*ios::binary*](#).

bool RfF(ifstream &_inF)

Метод имплементации чтения из файловой переменной текущего экземпляра класса (чтение из файла, метод десериализации).

Параметры:

&_inF - ссылка на экземпляр класса *ifstream* для чтения данных. При инициализации переменной *_inF* необходимо установить флаг бинарного режима открытия потока (не текстового!) [*ios::binary*](#).

bool SaveToFile(const string _filename)

Метод записи текущего состояния экземпляра класса в файл. При удачной записи в файл, возвращает *true*, при ошибке записи возвращает *false*.

Параметры:

_filename – имя файла; тип *string*.

bool ReadFromFile(const string _filename)

Метод восстановления состояния экземпляра класса из файла. При удачном чтении из файла, возвращает *true*, при ошибке чтения возвращает *false*.

_filename – имя файла; тип *string*.

SET-МЕТОДЫ

bool SetStorage(strItem _ShipPlan[], strItem _Purchase[], const clsRecipeItem _RecipeOut[], const clsRecipeItem _RecipeIn[])

Метод ввода исходных данных.

Параметры:

_ShipPlan – указатель на массив отгрузок со склада размером *OutCount* * *_PrCount*, где *PrCount* - количество периодов проекта, введенное ранее в [конструкторе с параметрами](#); тип элемента массива [strItem](#);

_Purchase – указатель на массив с ценами поступающих на склад ресурсов и (при отсутствии [флага авторасчёта](#)) с объемами поступлений размером *InCount* * *_PrCount*; тип элемента массива [strItem](#);

_RecipeOut – константный указатель на массив с технологическими картами для ЦЗО; тип элемента массива [clsRecipeItem](#);

_RecipeIn – константный указатель на массив с технологическими картами для ЦЗП; тип элемента массива [clsRecipeItem](#).

Данные из массивов *_RecipeOut* и *_RecipeIn* копируются.

bool SetStorage(strItem _ShipPlan[], strItem _Purchase[], vector<clsRecipeItem>&& _RecipeOut, vector<clsRecipeItem>&& _RecipeIn)

Метод ввода исходных данных, аналогичный предыдущему.

Параметры:

_ShipPlan – указатель на массив отгрузок со склада размером *OutCount* * *_PrCount*, где *PrCount* - количество периодов проекта, введенное ранее в [конструкторе с параметрами](#); тип элемента массива [strItem](#);

_Purchase – указатель на массив с ценами поступающих на склад ресурсов и (при отсутствии [флага авторасчёта](#)) с объемами поступлений размером *InCount* * *_PrCount*; тип элемента массива [strItem](#);

_RecipeOut – [вектор](#) с технологическими картами для ЦЗО; тип элемента вектора [clsRecipeItem](#);

_RecipeIn – [вектор](#) с технологическими картами для ЦЗП; тип элемента вектора [clsRecipeItem](#).

Данные из векторов *RecipeOut* и *_RecipeIn* перемещаются.

void SetCurrency(const Currency& _cur)

Метод устанавливает основную валюту проекта.

Параметры:

& _cur – ссылка на основную валюту проекта. Тип [Currency](#).

void Set_progress_shell(clsProgress_shell<type_progress>* val)

Метод присваивает указателям [this->pshell](#), [OutCost->pshell](#) и [InCost->pshell](#) адрес объекта оболочки для прогресс-бара.

Параметры:

val – указатель на объект [оболочки для прогресс-бара](#), тип которого определен псевдонимом [type_progress](#).

РАСЧЕТНЫЕ МЕТОДЫ

bool Calculate()

Метод рассчитывает объем, удельную и полную *себестоимость ресурсов*, поступающих на склад и отгружаемых со склада. Если расчёты завершились удачно (метод вернул *true*), в массивах [Purchase](#) и [Shipment](#) будут содержаться расчётные данные: поступающих на склад и отгружаемых со склада ресурсов соответственно. Если при расчетах будет обнаружен дефицит ресурсов, метод вернет *false*, а размер дефицита и имя дефицитного ресурса будут содержаться в поле [clsCostCenterStorage::Lack](#).

Выбор метода вычислений (последовательные, в асинхронных потоках или синхронных) устанавливается в *данном конкретном примере* макросом *cmeth*, определенным в заголовочном файле *clsCostCenterStorage.h*. Макрос принимает значение из списка перечислений [Clc_type](#): seq – последовательные вычисления, fut и thrd – вычисления в потоках.

В зависимости от выбранного типа вычислений, для ЦУР вызывается метод [Calculate](#), [Calculate_future](#) или [Calculate_thread](#), а для ЦЗП и ЦЗО – методы [CalcRawMatPurchPlan](#) и [Calculate_CalcRawMatPurchPlan_future](#) и [Calculate_future](#) или [CalcRawMatPurchPlan_thread](#) и [Calculate_thread](#).

GET – МЕТОДЫ**TLack GetLack()**

Метод возвращает размер дефицита и имя ресурса, где он обнаружен. Используется после возвращения *false* методом [Calculate](#). Тип возвращаемой информации [TLack](#).

size_t Counts(count_choise _count)

Метод возвращает число ресурсов в выбранной точке учёта склада: на входе ЦЗП, в ЦУР или на выходе ЦЗО. Тип возвращаемого значения [size_t](#).

Параметры:

_count – флаг выбранной точки учёта: *inStock* - на входе в ЦЗП, *midStock* - в ЦУР, *outStock* - на выходе из ЦЗО.

strNameMeas* GetNameMeas(count_choise _count)

Метод возвращает указатель на вновь создаваемый массив типа [strNameMeas](#) с названием ресурсов и единицами измерения этих ресурсов для выбранной точки учёта. Тип элемента массива *strNameMeas*.

Параметры:

_count – флаг выбранной точки учёта: *inStock* - на входе в ЦЗП, *midStock* - в ЦУР, *outStock* - на выходе из ЦЗО.

Метод удобен тогда, когда есть необходимость корректировки (например, сокращения длинных названий при выводе списка ресурсов в отчет или на экран.

const strNameMeas* GetNameMeas()

Метод возвращает const- указатель на внутренний массив типа *strNameMeas* с названием ресурсов и единицами измерения. Общее число элементов массива равно *clsCostCenterStorage::Counts(inStock)*, включая отгружаемые со склада и потребляемые складом ресурсы. Из них ресурсы под номерами от нуля до *clsCostCenterStorage::Counts(outStock)-1* поступают и отгружаются со склада, ресурсы под номерами от *clsCostCenterStorage::Counts(outStock)* до *clsCostCenterStorage::Counts(midStock)-1* потребляются на выходе со склада (в ЦЗО), а ресурсы под номерами от *clsCostCenterStorage::Counts(midStock)* до *clsCostCenterStorage::Counts(inStock)-1* потребляются на входе склада (в ЦЗП). Тип элемента массива *strNameMeas*.

const strItem* GetShip() const

Метод возвращает const- указатель на внутренний массив типа [strItem](#) с объемами, удельной и полной себестоимостью ресурсов, отгружаемых со склада. Массив является одномерным аналогом двумерной

матрицы размером `clsCostCenterStorage::Counts(outStock)*clsCostCenterStorage::ProjectCount()`. Вещественный тип элемента массива определяется псевдонимом `decimal`.

const strItem* GetPure() const

Метод возвращает const- указатель на внутренний массив типа `strItem` с объемами, удельной и полной себестоимостью ресурсов, поступающих на склад. Массив является одномерным аналогом двумерной матрицы размером `clsCostCenterStorage::Counts(inStock)*clsCostCenterStorage::ProjectCount()`. Вещественный тип элемента массива определяется псевдонимом `decimal`.

strItem* GetInRAC() const

Метод возвращает указатель на вновь создаваемый массив типа `strItem` с объемами, удельной и полной себестоимостью ресурсов, поступаемых из ЦЗП (РСС) в ЦУР (РАС). Массив является одномерным аналогом двумерной матрицы размером `clsCostCenterStorage::Counts(MidStock)*clsCostCenterStorage::ProjectCount()`. Вещественный тип элемента массива определяется псевдонимом `decimal`.

После использования полученного массива, во избежание утечки памяти он требует явного удаления с помощью оператора `delete[]`.

strItem* GetOutRAC() const

Метод возвращает указатель на вновь создаваемый массив типа `strItem` с объемами, удельной и полной себестоимостью ресурсов, поступаемых из ЦУР (РАС) в ЦЗО (SCC). Массив является одномерным аналогом двумерной матрицы размером `clsCostCenterStorage::Counts(MidStock)*clsCostCenterStorage::ProjectCount()`. Вещественный тип элемента массива определяется псевдонимом `decimal`.

После использования полученного массива, во избежание утечки памяти он требует явного удаления с помощью оператора `delete[]`.

МЕТОДЫ ЭКСПОРТА

bool Export_Data(string filename, count_choise _dep, ChoiseData _arr, const ReportData& flg) const

Метод экспортирует данные о поставках, остатках или отгрузках ЦЗП, ЦУР или ЦЗО в [CSV-файл](#). Возвращает `true` в случае успешной операции, иначе возвращает `false`.

Параметры:

`filename` – имя файла для экспорта данных; тип `string`;

`_dep` – флаг выбора точки учёта: `inStock` - ЦЗП, `midStock` - ЦУР, `outStock` - ЦЗО; тип `count_choise`;

`_arr` – флаг выбора типа данных: `purchase` - поставки, `balance` - остатки/незавершенное производство, `shipment` - отгрузки; тип `ChoiseData`;

`flg` – флаг выводимой в файл информации: `volume` - в натуральном, `value` - в стоимостном, `price` - в ценовом измерении; тип `ReportData`.

В качестве разделителя между полями *CSV-файла* используется символ ';' (точка с запятой).

УДАЛЯЕМЫЕ МЕТОДЫ ПРЕДКА

Класс `clsCostCenterStorage` наследуется от класса `clsStorage`. Некоторые методы предка не используются, поэтому удалены:

Name;

Measure;

GetShipPrice;
GetDataItem;
SetCount;
SetName;
SetMeasure;
SetDataItem;
SetSKU;
SetData;
SetData;
EraseSKU;
View;

СКРЫВАЕМЫЕ МЕТОДЫ ПРЕДКА

Часть методов предка *clsStorage* скрыта для предотвращения непреднамеренного использования:

Size;
GetNameMeas;
GetShip;
GetPure;
SetCurrency;
SetPurchase;
SetPurPrice;
CheckPurchase;
SetStorage;
Calculate_future;
Calculate_thread;
StF;
RfF

ОПИСАНИЕ ДЕМОНСТРАЦИОННОЙ ПРОГРАММЫ ДЛЯ КЛАССА CLSCOSTCENTERSTORAGE

ФАЙЛЫ С ПРОГРАММНЫМИ КОДАМИ

Тестовая программа поставляется в составе репозитория библиотеки FROMA2 в виде исходных текстов. Исходные тексты находятся в папке **froma2/examples/CCStorage/**:

main.cpp

Файл реализации основного кода программы;

Include/clsCostCenterStorage.h

Заголовочный файл для класса *clsCostCenterStorage*;

Include/pathes.h

Заголовочный файл с путями для импорта и экспорта файлов; пути прописываются автоматически при сборке проекта с помощью *CMake*;

Include/clsCCStorageTest.h

Заголовочный файл для класса *clsCCStorageTest*, реализующего чтение исходных данных из CSV-файлов и их подготовку для ввода в класс *clsCostCenterStorage*;

src/clsCostCenterStorage.cpp

Файл реализации для класса *clsCostCenterStorage*;

src/clsCCStorageTest.cpp

Файл реализации для класса *clsCCStorageTest*;

ФАЙЛЫ С ТЕСТОВЫМИ ДАННЫМИ

Для демонстрации работы класса *clsCostCenterStorage* используются CSV-файлы с тестовыми данными:

inputdata/ship.csv

Пример csv-файла с объемами отгрузок ресурсов в натуральном выражении со Склада;

inputdata/pur_price.csv

Пример csv-файла с ценами ресурсов, поставляемых на Склад;

inputdata/recipe_in_0.csv, inputdata/recipe_in_1.csv, inputdata/recipe_in_2.csv, inputdata/recipe_in_3.csv, inputdata/recipe_in_4.csv, inputdata/recipe_in_5.csv, inputdata/recipe_in_6.csv

Пример csv-файлов с технологическими картами для ЦЗП.

inputdata/recipe_out_0.csv, inputdata/recipe_out_1.csv, inputdata/recipe_out_2.csv, inputdata/recipe_out_3.csv, inputdata/recipe_out_4.csv, inputdata/recipe_out_5.csv

Пример csv-файлов с технологическими картами для ЦЗО.

В результате сборки проекта (см. раздел [СБОРКА ПРОЕКТА С ПОМОЩЬЮ CMake](#)), в целевой папке **froma2/build/examples/CCStorage** получаем следующие папки и файлы, используемые непосредственно в ходе работы программы:

inputdata

В эту папку копируется содержимое папки *froma2/examples/CCStorage/inputdata*. Это рабочая папка с демонстрационными файлами;

outputdata

Эта папка используется Программой для экспорта исходных и расчетных данных в csv-файлы. Имена файлов начинаются с символа 'f' (от названия библиотеки froma2), далее через нижнее подчеркивание следует символ, обозначающий подразделение:

w - Склад в целом;

PCC – ЦЗП, центр учета затрат по поступлению (PCC - Purchase Cost Centre);

RAC – ЦУР, центр учета ресурсов (RAC - Resource Accounting Centre);

SCC – ЦЗО, центра учета затрат по отгрузке (SCC - Shipment Cost Centre).

Следующий символ обозначает вид выводимого в файл массива:

s - отгрузки (**s**hipment);

p - поставки (**p**urchase).

Далее через нижнее подчеркивание следует тип выводимых данных:

volume - в натуральном измерении;

price - в удельном стоимостном измерении;

value - в полном стоимостном измерении.

Например, файл с именем *f_SCCp_value.csv* содержит результаты расчетов с полной стоимостью поставляемых на ЦЗО ресурсов.

temp

Папка для сохранения в файл и восстановления из файла состояния объекта типа *clsCostCenterStorage*; файл *clsCostCenterStorage.dat*.

ВСПОМОГАТЕЛЬНЫЙ КЛАСС ДЛЯ ПРЕОБРАЗОВАНИЯ ДАННЫХ

Указанные выше файлы обрабатываются экземпляром вспомогательного класса *clsCCStorageTest*. Объект типа *clsCCStorageTest* импортирует данные из файлов и преобразует их к форматам, пригодным для ввода в объект типа *clsCostCenterStorage*. Описание методов класса *clsCCStorageTest* приведено ниже.

clsCCStorageTest()

Конструктор по умолчанию.

~clsCCStorageTest()

Деструктор.

clsCCStorageTest(const clsCCStorageTest&) = delete

clsCCStorageTest(clsCCStorageTest&&) = delete

Конструкторы копирования и перемещения удалены. Их использование не предполагается.

bool Import_Data(Currency _cur, AccountingMethod _ac)

Метод читает информацию из файлов с исходными данными и вводит основные параметры.

Параметры:

_cur – валюта проекта; тип [Currency](#);

_ac – принцип учета запасов; тип [AccountingMethod](#).

Имена файлов (приведены выше) в данном примере определяются макросами *filename_shipment*, *filename_purprice*, *filename_recipe_in*, *filename_recipe_out* и маской *msks*. Макросы определены в заголовочном файле *clsCCStorageTest.h*.

size_t GetProjectCount() const

Метод возвращает число периодов проекта. Тип возвращаемого значения *size_t*.

Currency GetCurrency() const

Метод возвращает валюту проекта. Тип возвращаемого значения *Currency*.

AccountingMethod GetAccounting() const

Метод возвращает принцип учёта запасов. Тип возвращаемого значения *AccountingMethod*.

size_t GetOutCount() const

Метод возвращает число отгружаемых со склада номенклатурных позиций. Тип возвращаемого значения *size_t*.

size_t GetMidCount() const

Метод возвращает число номенклатурных позиций на входе в ЦЗО. Тип возвращаемого значения *size_t*.

size_t GetInCount() const

Метод возвращает число номенклатурных позиций поступающих на склад (на входе в ЦЗП). Тип возвращаемого значения *size_t*.

strNameMeas* GetNames()

Метод возвращает указатель на внутренний массив с перечнем номенклатурных позиций и их единицами измерения. Тип элемента массива [strNameMeas](#).

strItem* GetPurch()

Метод возвращает указатель на внутренний массив поставленных на склад ресурсов. Тип элемента массива [strItem](#).

strItem* GetShip()

Метод возвращает указатель на внутренний массив отгружаемых со склада ресурсов. Тип элемента массива *strItem*.

vector<clsRecipeItem> GetInRecipe()

Метод возвращает контейнер с технологическими картами для ЦЗП путем перемещения внутреннего контейнера. Тип элемента контейнера [clsRecipeItem](#).

vector<clsRecipeItem> GetOutRecipe()

Метод возвращает контейнер с технологическими картами для ЦЗО путем перемещения внутреннего контейнера. Тип элемента контейнера *clsRecipeItem*.

ЛОГИКА РАБОТЫ ДЕМОНСТРАЦИОННОЙ ПРОГРАММЫ

Логика работы демонстрационной программы следующая.

- Создаём экземпляр класса [clsCCStorageTest](#).
- Вызываем метод [clsCCStorageTest::Import_Data](#) для импорта тестовых данных из заданных CSV-файлов.
- Вызываем конструктор класса [clsCostCenterStorage](#) с параметрами. В качестве параметров ему передаём валюту проекта, принцип учёта запасов и величины, возвращаемые методами

clsCCStorageTest::GetProjectCount, clsCCStorageTest::GetOutCount, clsCCStorageTest::GetMidCount, clsCCStorageTest::GetInCount, clsCCStorageTest::GetNames.

- Создаём индикатор прогресса, оболочка индикатора прогресса. С помощью наследуемого метода clsStorage::Set progress shell в созданном ранее экземпляре класса *clsCostCenterStorage* устанавливаем индикатор прогресса.
- Вызываем метод clsCostCenterStorage::SetStorage для создания склада. В качестве параметров ему передаём указатель на массив с отгрузками *clsCCStorageTest::GetShip*, указатель на массив поставленных на склад ресурсов *clsCCStorageTest::GetPurch*, контейнеры с технологическими картами для ЦЗО *clsCCStorageTest::GetOutRecipe* и ЦЗП *clsCCStorageTest::GetInRecipe*.
- Рассчитываем склад. Для этого вызываем метод clsCostCenterStorage::Calculate.
- Экспортируем результаты расчетов в CSV-файлы. Для этого последовательно вызываем метод clsCostCenterStorage::Export Data, подставляя в него необходимые имена файлов и флаги.
- Удаляем динамически созданные объекты.

ПРОГРАММА ДЛЯ РАСЧЁТА ПОЛНЫХ И УДЕЛЬНЫХ ПЕРЕМЕННЫХ ЗАТРАТ СКЛАДА MODEL_2

Программа *Model_2* так же, как и программа CCStorage на базе класса clsCostCenterStorage, описывает модель предприятия с двумя центрами затрат, представленную на рисунке Рисунок 28. Однако технологически она построена иначе, - с использованием агрегатора для модели предприятия - модуля aggregate. Что позволило сделать программу проще и понятнее.

Для построения модели создан класс *clsCompany*, являющийся наследником класса clsAggregate. Цепочка объектов типа clsStorage и clsManufactory, наследуемая от родителя, ограничена *тремя* звеньями в следующей последовательности: ЦЗП - *центр учета затрат по поступлению* (PCC - Purchase Cost Centre, на базе объекта типа *clsManufactory*), ЦУР - *центр учета ресурсов* (RAC - Resource Accounting Centre, на базе объекта типа *clsStorage*) и ЦЗО - *центр учета затрат по отгрузке* (SCC - Shipment Cost Centre, на базе объекта типа *clsManufactory*).

В результате использования класса *clsAggregate* в качестве родительского класса, схема предприятия и процедура создания модели упростилась (Рисунок 31). Ввод и вывод данных организован по отношению к контейнеру в целом и не требует взаимодействия с отдельными его элементами.

Поскольку конструктор родительского класса *clsAggregate* находится в секции *protected*, для создания класса предприятия *clsCompany* программисту необходимо написать конструктор по умолчанию и конструктор с параметрами для ввода данных, перечисленных на рисунке *черным шрифтом*. Методы для ввода исходных данных, представленных на рисунке *синим и красными шрифтами*, уже *входят* в базовый класс *clsAggregate*, их переписывать не требуется.

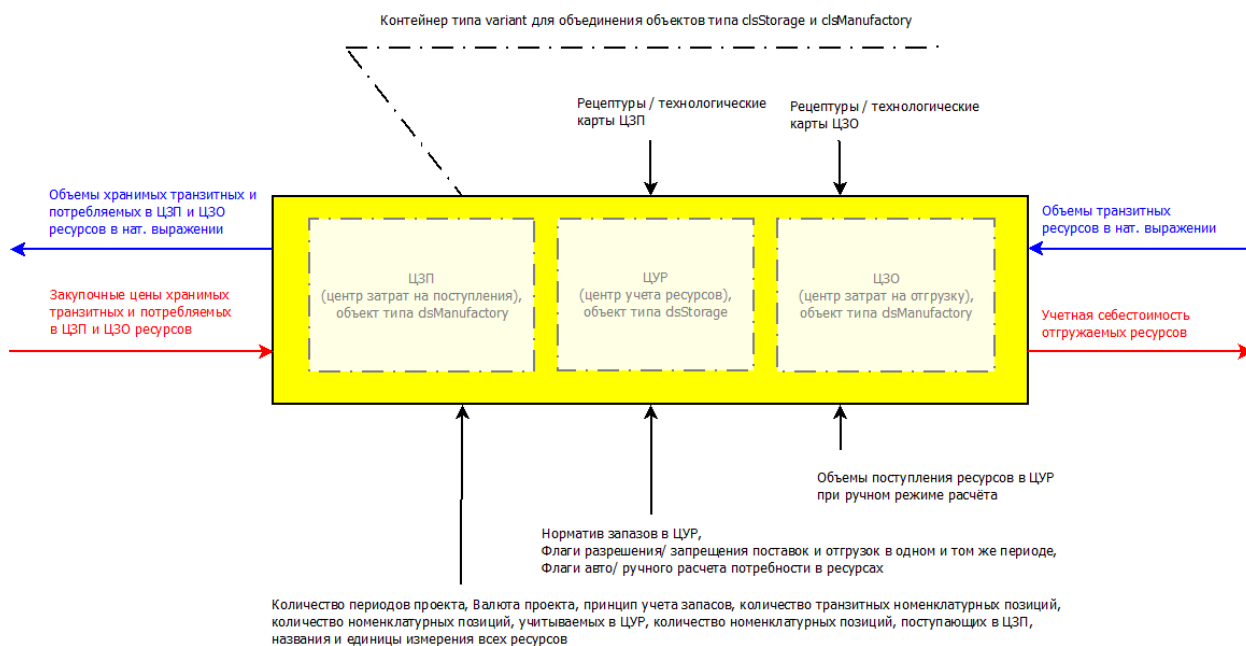


Рисунок 31 Принципиальная схема модели склада с двумя центрами затрат на базе класса `clsAggregate`

Для удобства работы в класс `clsCompany` добавлены также некоторые другие функции, которые будут описаны ниже.

ЦЕЛЕВАЯ АУДИТОРИЯ ДЛЯ ПРОГРАММЫ

Целевая аудитория для программы *Model_2* совпадает с целевой аудиторией программы Model_1. Также как *Model_1*, приложение *Model_2* консольное. Предназначено в помощь аналитикам небольших предприятий, реализующим управленческую отчетность и планирование с помощью *MS Excel*, *OpenOffice Calc* и других электронных таблиц.

Предлагаемая программа *Model_2* читает исходные данные из файлов, выгруженных из ERP-системы предприятия, производит расчеты и экспортирует результаты в виде файлов, читабельных как людьми, так и ERP-системами. Формат входных и выходных файлов – самый распространенный, CSV, совместимый с большинством ERP-систем и электронных таблиц. Это также позволяет использовать программу для облегчения труда аналитиков как в работе над отчетом, так и в работе над планом.

В отличие от программы CCStorage, представляющей интерес исключительно для программистов с точки зрения знакомства с возможностями инструмента, данный пример, так же, как и программа Model_1, является клиентоориентированным приложением и может представлять интерес не только для моих коллег, но и для целевой аудитории, далекой от программирования.

ВОЗМОЖНОСТИ ПРОГРАММЫ

Программа позволяет рассчитать:

- объем поступлений транзитных ресурсов, необходимый для отгрузки потребителям и поддержания требуемого запаса на складе;
- объем поступлений ресурсов для внутреннего потребления в ЦЗП и ЦЗО и поддержания требуемого запаса этих ресурсов в ЦУР;
- учетную себестоимость транзитных ресурсов и ресурсов для внутреннего потребления на выходе ЦЗП;
- учетную себестоимость остатков транзитных ресурсов и ресурсов для внутреннего потребления в ЦУР;

- учетную себестоимость транзитных ресурсов и ресурсов для внутреннего потребления на входе ЦЗО;
- учетную себестоимость транзитных ресурсов на выходе ЦЗО.

Себестоимость ресурсов рассчитывается исходя из выбранного принципа учета запасов: *FIFO* ("первым вошел - первым вышел"), *LIFO* ("последним вошел - первым вышел"), или *AVERAGE* ("по-среднему").

Программа позволяет учесть ситуацию, когда поступление товарно-материальных ценностей (ТМЦ) на склад и отгрузка этих ценностей со склада невозможна в одном периоде. Например, когда поступление ТМЦ осуществляется в конце периода, а отгрузка – в начале следующего.

ПАПКИ И ФАЙЛЫ

Программа поставляется в составе репозитория библиотеки FROMA2 в виде исходных текстов. Исходные тексты находятся в папке **froma2/examples/Model_2/**:

main.cpp

Файл реализации основного кода программы;

Include/clsCompany.h

Заголовочный файл для класса *clsCompany*;

Include/clsImportConfig.hpp

Файл с кодом для импорта данных из csv-файлов и приведению их к формату, пригодному для ввода в объект типа *clsCompany*;

Include/pathes.h

Заголовочный файл с путями для импорта и экспорта файлов; пути прописываются автоматически при сборке проекта с помощью *CMake*;

src/clsCompany.cpp

Файл реализации для класса *clsCompany*;

inputdata/about.txt

Пример текстового файла с кратким описанием программы;

inputdata/f_RACp_volume.csv

Пример файла с объемами поступлений ресурсов в ЦУР при выбранном флаге ручного расчета ЦУР;

inputdata/pcalc.csv

Пример файла с флагами автоматического расчета ЦУР;

inputdata/pcalc_man.csv

Пример файла с флагами ручного расчета ЦУР;

inputdata/permit.csv

Пример файла с флагами разрешения/ запрещения поступлений и отгрузок в одном и том же периоде;

inputdata/pur_price.csv

Пример файла с ценами ресурсов на входе ЦЗП;

inputdata/recipe_in_0.csv, inputdata/recipe_in_1.csv, inputdata/recipe_in_2.csv, inputdata/recipe_in_3.csv, inputdata/recipe_in_4.csv, inputdata/recipe_in_5.csv, inputdata/recipe_in_6.csv

Пример csv-файлов с технологическими картами для ЦЗП;

inputdata/recipe_out_0.csv, inputdata/recipe_out_1.csv, inputdata/recipe_out_2.csv, inputdata/recipe_out_3.csv, inputdata/recipe_out_4.csv, inputdata/recipe_out_5.csv

Пример csv-файлов с технологическими картами для ЦЗО;

inputdata/shares.csv

Пример файла с нормативами запасов каждого из ресурса в ЦУР;

inputdata/ship.csv

Пример csv-файла с объемами отгрузок в натуральном выражении с ЦЗО;

config/config.cfg, config_base.cfg

Пример конфигурационных файлов Программы;

В результате сборки проекта (см. раздел [СБОРКА ПРОЕКТА С ПОМОЩЬЮ CMake](#)), в целевой папке **froma2/build/examples/Model_2** получаем следующие папки и файлы, используемые непосредственно в ходе работы Программы:

config

В эту папку копируется содержимое папки *froma2/examples/Model_2/config*. Это рабочая папка скомпилированной программы, в которой изменяются, удаляются и создаются заново конфигурационные файлы в зависимости от действий пользователя. Имена файлов для импорта, строки и столбцы для чтения, символ разделителя данных, а также принцип учета запаса и валюта проекта задаются в конфигурационном файле с именем *config.cfg* (или любым другим именем, заданным пользователем в качестве аргумента при запуске Программы). Программа читает конфигурационный файл, при его отсутствии предлагает ввести данные с экрана, записывает данные в новый конфигурационный файл. При желании пользователя, в программе можно отредактировать и сохранить существующий конфигурационный файл.

inputdata

В эту папку копируется содержимое папки *froma2/examples/Model_2/inputdata*. Это рабочая папка с файлами, которые пользователь хочет импортировать в Программу для проведения расчетов; Пользователь может заменять файлы с примерами собственными файлами для получения необходимых ему результатов;

reports

Эта папка используется Программой для вывода исходных и расчетных данных в читаемом табличном виде в текстовый файл. Отчет с текстом и таблицами выводится в файл с именем *Model_2.txt*.

outputdata

Эта папка используется Программой для экспорта исходных и расчетных данных в csv-файлы. Имена файлов заданы макросами в начале файла *main.cpp*. использованы следующие обозначения:

Таблица 40 Имена файлов для экспорта программы Model_2

Макрос	Имя файла	Назначение
f_ws_volume	f_ws_volume.csv	Объемы отгрузок со склада в натуральном выражении

f_ws_price	f_ws_price.csv	Цены отгрузок
f_ws_value	f_ws_value.csv	Стоимость отгрузок
f_wp_volume	f_wp_volume.csv	Объемы закупок на склад в натуральном выражении
f_wp_price	f_wp_price.csv	Цены закупок
f_wp_value	f_wp_value.csv	Стоимость закупок
f_PCCs_volume	f_PCCs_volume.csv	Объем отгрузок из ЦЗП в ЦУР в натуральном выражении
f_PCCs_price	f_PCCs_price.csv	Цены отгрузок из ЦЗП в ЦУР
f_PCCs_value	f_PCCs_value.csv	Стоимость отгрузок из ЦЗП в ЦУР
f_RACp_volume	f_RACp_volume.csv	Объем поставок в ЦУР из ЦЗП в натуральном выражении
f_RACp_price	f_RACp_price.csv	Цены поставок в ЦУР из ЦЗП
f_RACp_value	f_RACp_value.csv	Стоимость поставок в ЦУР из ЦЗП
f_RACs_volume	f_RACs_volume.csv	Объем отгрузок из ЦУР в ЦЗО в натуральном выражении
f_RACs_price	f_RACs_price.csv	Цены отгрузок из ЦУР в ЦЗО
f_RACs_value	f_RACs_value.csv	Стоимость отгрузок из ЦУР в ЦЗО
f_SCCp_volume	f_SCCp_volume.csv	Объем поставок в ЦЗО из ЦУР в натуральном выражении
f_SCCp_price	f_SCCp_price.csv	Цены поставок в ЦЗО из ЦУР
f_SCCp_value	f_SCCp_value.csv	Стоимость поставок в ЦЗО из ЦУР

СТРУКТУРА МОДЕЛИ И ЛОГИКА РАБОТЫ

Моделируемое предприятие состоит из трех основных подразделений и двух центров затрат:

- Центр затрат по поступлению (ЦЗП), - описывается классом [clsManufactory](#) из модуля [manufact](#) библиотеки;
- Центр учета ресурсов (ЦУР), - описывается классом [clsStorage](#) из модуля [warehouse](#) библиотеки;
- Центр затрат по отгрузке (ЦЗО), - описывается классом [clsManufactory](#) из модуля [manufact](#).

Все три подразделения последовательно добавляются в контейнер [CostChain](#), родительского класса [clsAgregate](#), наследуемый в потомке [clsCompany](#).

Логика работы Программы следующая:

0 этап.

Импортируем данные из csv-файлов, готовим их к формату, пригодному для ввода в качестве параметров в методы класса [clsCompany](#), [clsAgregate](#), [clsBaseProject](#).

1 этап.

Создаем склад, используя конструктор с параметрами. Вводим в него: количество периодов проекта, домашнюю валюту, принцип учета запасов, число транзитных ресурсов, число ресурсов, являющихся транзитными в сумме с числом ресурсов, потребляемых в ЦЗО, общее число ресурсов (транзитных, потребляемых в ЦЗО и потребляемых в ЦЗП), наименования и единицы измерения всех ресурсов, а также технологические карты для ЦЗО и ЦЗП.

2 этап.

Вводим в склад исходные данные такие, как нормативы запасов на складе, флаги разрешения/ запрещения поставок и отгрузок в одном и том же периоде, флаги автоматического/ ручного расчета объемов потребляемых ресурсов, объем отгрузок транзитных ресурсов в натуральном выражении и, если для каких-то

позиций установлены флаги ручного расчета, вводим для этих позиций объемы поступлений ресурсов на склад.

3 этап.

Вводим цены на все поставляемые ресурсы (включая транзитные ресурсы, ресурсы для потребления в ЦЗО и ЦЗП).

4 этап.

Рассчитываем потребность в ресурсах в натуральном выражении на входе склада²⁶.

5 этап.

Рассчитываем стоимостные показатели отгружаемых со склада ресурсов.

6 этап.

Выводим отчет и/или экспортируем результаты расчета.

Во всех моделях алгоритм расчета учетной себестоимости отгружаемой со Склада готовой продукции является *двухпроходным*: сначала *от клиентов к поставщикам* идет расчет *объемных* показателей в натуральном выражении, потом *от поставщиков к клиентам* идет расчет *денежных* показателей.

ПЕРВЫЕ ШАГИ РАБОТЫ С ПРОГРАММОЙ

Первые шаги работы с программой *Model_2* во многом совпадают с [первыми шагами работы с программой Model_1](#).

1 шаг.

Удалить из папки *froma2/build/examples/Model_2/inputdata* примеры с импортируемыми файлами и поместить в эту папку свои собственные файлы с исходными данными. Удалить из папки *froma2/build/examples/Model_2/config* примеры конфигурационных файлов и оставить папку пустой.

2 шаг.

Запустить на исполнение файл Программы *froma2/build/bin/Model_2.exe*²⁷.

3 шаг.

После запуска программы появится диалог ввода конфигурации проекта, где в диалоговом режиме нужно ввести следующие данные:

- имя файла с описанием проекта (например, *about.txt*);
- имя файла с отгрузками из СГП в натуральном выражении (например, *ship.csv*);
- имя файла с ценами поставок (например, *pur_price.csv*);
- имя файла с объемами поставок в ЦУР (в *nocalc*-режиме, например, *f_RACp_volume.csv*; в режиме автоматического расчета (*calc*) следует просто нажать “Enter”);
- префикс для файлов с технологическими картами ЦЗП (например, *recipe_in*);
- префикс для файлов с технологическими картами ЦЗО (например, *recipe_out*);
- маска *regex* для поиска файлов технологических карт (маска *регулярного выражения*, например, *_d{1,3}\.csv*);
- имя файла с *нормативами запасов на складе* (например, *shares.csv*);

²⁶ Этапы 3 и 4 можно менять местами.

²⁷ Все последующие скриншоты относятся к выполнению программы под операционной системой MS Windows. Для ОС семейства Linux действия будут аналогичные, однако внешний вид экрана может отличаться от приведенных в настоящем документе.

- имя файла с флагами разрешения на отгрузку и закупку в одном и том же периоде (например, *permit.csv*);
- имя файла с флагами автоматического/ ручного расчёта потребности в ресурсах (например, *pcalc_man.csv*);
- символ разделителя между полями в CSV-файлах (по умолчанию используется символ ';' – точка с запятой);
- число столбцов с заголовками в CSV-файлах (по умолчанию равно 2: первый столбец для названия ресурсов, второй – для единиц измерения их натурального количества);
- число строк с заголовками в CSV-файлах (по умолчанию равно 1);
- домашняя валюта проекта (выбирается из доступных: "RUR", "USD", "EUR", "CNY");
- принцип учета запасов (FIFO, LIFO или AVG).

Так же, как и в программе *Model_1*, важно обратить внимание на следующее.

Все файлы должны быть в формате CSV.

Если имя файла вводится без указания папки, где расположен этот файл, то программа ищет этот файл в папке *froma2/build/examples/Model_2/inputdata*. Если имя файла вводится с относительным или абсолютным путем (например, *inputdata/about.txt*), то программа ищет файл с указанным именем в указанной папке.

Файлов с технологическими картами для ЦЗП и ЦЗО должно быть ровно столько, сколько ресурсов планируется ввести в соответствующее подразделение. Каждое имя файла с рецептурой состоит из уникального префикса, задаваемого пользователем и суффикса вида *_i*. Если *i* – соответствует номеру вводимого продукта (например, для первого продукта, суффикс будет *_1*, для четвертого продукта, соответственно, *_4*), то программа работает быстрее. Каждое имя файла должно иметь расширение *.csv*. Число *i* ограничено диапазоном, задаваемом маской регулярного выражения (например, для маски *_d{1,3}\.csv* максимальное число 999).

	A	B	C	D
1	Азу из гов. шт.		Расход	
2	Азу из гов. шт.		1.0	
3	Разгрузка чел*час		0.001	
4	Ресурсы на входе ЦЗП			

	A	B	C	D
1	Азу из гов. шт.		Расход	
2	Азу из гов. шт.		1.0	
3	Погрузка чел*час		0.001	
4	Ресурсы на входе ЦЗО			

Рисунок 32 Примеры технологических карт для ЦЗП и ЦЗО

На рисунке (Рисунок 32) представлены технологические карты для ресурса с названием «Азу из говядины с картофелем». Для того, чтобы из ЦЗП в ЦУР поступил на хранение ресурс *Азу из говядины с картофелем*, на входе ЦЗП требуется поступление ресурсов: *Азу из говядины с картофелем* и *Разгрузка*. Ресурс *Азу из говядины с картофелем* является транзитным для ЦЗП, а ресурс *Разгрузка* потребляется в ЦЗП.

Аналогично для ЦЗО. Чтобы отгрузить потребителям *Азу из говядины с картофелем*, на вход ЦЗО из ЦУР должны поступить ресурсы: *Азу из говядины с картофелем* и *Погрузка*. Ресурс *Погрузка* потребляется в ЦЗО.

На рисунке ниже (Рисунок 33) представлена технологическая карта для ЦЗП ресурса, который транзитом проходит ЦЗП и потребляется в ЦЗО.

	A	B	C	D
1	Погрузка	чел*час	Расход	
2	Погрузка	чел*час		1
3				

Рисунок 33 Пример технологической карты для ЦЗП ресурса, потребляемого в ЦЗО

Импортируемые файлы должны содержать строки и столбцы с заголовками. Например, для файла с объемами отгрузок покупателям и файла с объемами производства в натуральном измерении, первый и второй столбцы содержат названия продуктов и их единицы измерения. Для файла с ценами сырья и материалов и файла с объемами закупок сырья и материалов в натуральном измерении, первый и второй столбцы содержат названия ресурсов, из которых изготавливаются продукты и единицы измерения этих ресурсов. В этом случае, вводимое число столбцов с заголовками будет равно двум (Рисунок 16).

Если в импортируемых файлах присутствует строка с номерами временных периодов или датами, то вводимое число строк с заголовками будет равно единице.

Минимальное число строк с заголовками равно нулю. Минимальное число столбцов с заголовками равно двум. При количестве столбцов с заголовками более двух, столбцы с названиями номенклатурных позиций и единицами их измерения должны быть крайними правыми (при индексе самого левого столбца, равного ноль и общем числе столбцов с заголовками N, индекс столбцов с названиями и единицами измерения должны быть равны N-2 и N-1 соответственно).

Длина всех строк в импортируемом файле должна быть *одинаковой*. В противном случае поведение программы может стать непредсказуемым.

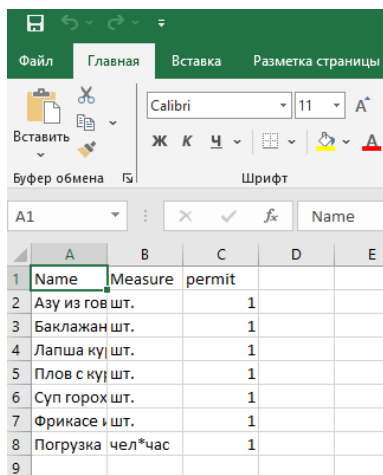
Запас продуктов или ресурсов на складе измеряется долями от объемов отгрузки. Например, при отгрузке за период 1000 килограмм и норме запаса 0.5, запас в натуральном измерении и составит 500 кг. Нормативы задаются в файле *shares.csv* (Рисунок 34).

	A	B	C	D	E
1	Name	Measure	share		
2	Азу из гов шт.		0.5		
3	Баклажан шт.		0.5		
4	Лапша ку шт.		0.5		
5	Плов с ку шт.		0.5		
6	Суп горох шт.		0.5		
7	Фрикасе и шт.		0.5		
8	Погрузка	чел*час		0	
9					

Рисунок 34 Структура файла с нормативами запасов

Иногда поступления на склад осуществляются в конце отчетного периода, а отгрузки – не ранее начала следующего. Для такой ситуации можно установить *флаг разрешения поступлений и отгрузок в одном и том же периоде «ЗАПРЕЩЕНО»* ("PROHIBITED"). В случаях, когда и поступления, и отгрузки возможны в один и тот

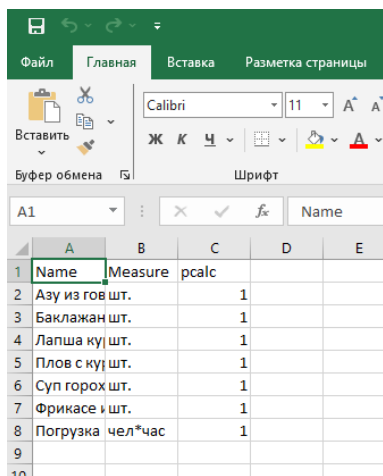
же отчетный период, флаг устанавливается в «РАЗРЕШЕНО» (“ALLOWED”). Для флага «РАЗРЕШЕНО» в файле устанавливается “1” (единица), для флага «ЗАПРЕЩЕНО» в файле устанавливается “0” (ноль). На рисунке ниже (Рисунок 35) для всех ресурсов установлены флаги «РАЗРЕШЕНО».



	A	B	C	D	E
1	Name	Measure	permit		
2	Азу из гов шт.		1		
3	Баклажан шт.		1		
4	Лапша ку шт.		1		
5	Плов с ку шт.		1		
6	Суп горох шт.		1		
7	Фрикасе и шт.		1		
8	Погрузка чел*час		1		
9					

Рисунок 35 Структура файла с разрешениями отгрузок и закупок в одном периоде

Для указания программе, как проводить расчёт потребности в ресурсах: в ручном или автоматическом режиме, в программу вводится файл с флагами расчёта. Для указания автоматического расчёта какого-либо ресурса, у этого ресурса устанавливается “0” (ноль). Пример файла для автоматического расчета поступлений у всех ресурсов представлен на рисунке ниже (Рисунок 37). Для указания ручного ввода объемов поступлений для какого-то ресурса, у этого ресурса устанавливается “1” (единица). Пример файла для ручного расчёта представлен также на рисунке (Рисунок 36).



	A	B	C	D	E
1	Name	Measure	pcalc		
2	Азу из гов шт.		1		
3	Баклажан шт.		1		
4	Лапша ку шт.		1		
5	Плов с ку шт.		1		
6	Суп горох шт.		1		
7	Фрикасе и шт.		1		
8	Погрузка чел*час		1		
9					

Рисунок 36 Структура файла с флагами ручного расчёта поступления ресурсов

	A	B	C	D	E	F
1	Name	Measure	pcalc			
2	Азу из гов шт.		0			
3	Баклажан шт.		0			
4	Лепша кушт.		0			
5	Плов с кушт.		0			
6	Суп горох шт.		0			
7	Фрикасе шт.		0			
8	Погрузка чел*час		0			
9						
10						
11						
12						

Рисунок 37 Структура файла с флагами автоматического ввода объемов поступления ресурсов

В отличие от программы *Model_1*, в программе *Model_2* редактирование нормативов запасов на складе и флагов производится путем изменения данных в csv-файлах *до запуска* программы.

4 шаг.

Когда импортирование данных завершено, программа выводит на экран основные настройки и производит расчёты.

Основные настройки выводятся в следующем порядке. Сначала выводится *общая информация о проекте*, включая число периодов проекта, число ресурсов на входе склада (входе ЦЗП), число ресурсов на хранении (в ЦУР), число отгружаемых со склада ресурсов (из ЦЗО). Ниже следует перечень ресурсов с разбивкой по указанным выше группам.

Далее на экран выводятся *настройки каждого элемента* смоделированной цепочки подразделений: ЦЗО, ЦУР, ЦЗП (Рисунок 38, *красная, желтая и синяя* обводки).

Информация о ЦЗП и ЦЗО содержит данные о количестве периодов проекта, числе транзитных ресурсов («число позиций продуктов») и общем числе транзитных и потребляемых в центре затрат ресурсов («общее число позиций сырья и материалов»), а также о домашней валюте проекта.

Информация о ЦУР содержит данные о принципе учета запасов (заголовок столбца “Accounting”), флаге разрешающем/ запрещающем отгрузку в одном и том же периоде (заголовок столбца “Permission”) и флаге автоматического/ ручного расчёта поступлений ресурсов на склад (заголовок столбца “AutuPurchase”).

```

Количество периодов проекта: 60
Число ресурсов на входе склада (включая отгружаемые и потребляемые) 8
Число ресурсов на хранении 7
Число отгружаемых со склада ресурсов 6
*****
** Ресурсы на складе (включая отгружаемые и потребляемые) **
*****
***** Ресурсы поступающие на склад и отгружаемые со склада *****
Азу из говядины с картофелем шт.
Баклажаны с томатом Неаполь шт.
Лапша куриная шт.
Плов с курицей шт.
Суп гороховый с копченостями шт.
Фрикасе из курицы с зеленым горошком шт.
***** Ресурсы, потребляемые в ЦЗО *****
Погрузка чел*час
***** Ресурсы, потребляемые в ЦЗП *****
Разгрузка чел*час
*****
** Настройки расчёта **
*****
*** Основные параметры проекта ***
Количество периодов проекта 60
Общее число позиций продуктов 6 ЦЗО
Общее число позиций сырья и материалов 7
Валюта проекта RUR
*****
Name Measure Accounting Permission AutoPurchase Share
Азу из говядины шт. AVERAGE ALLOWED MANUAL 0.5
Баклажаны с томш. AVERAGE ALLOWED MANUAL 0.5
Лапша куриная шт. AVERAGE ALLOWED MANUAL 0.5 ЦУР
Плов с курицей шт. AVERAGE ALLOWED MANUAL 0.5
Суп гороховый шт. AVERAGE ALLOWED MANUAL 0.5
Фрикасе из куриш. AVERAGE ALLOWED MANUAL 0.5
Погрузка чел*час AVERAGE ALLOWED MANUAL 0
*****
*** Основные параметры проекта ***
Количество периодов проекта 60
Общее число позиций продуктов 7 ЦЗП
Общее число позиций сырья и материалов 8
Валюта проекта RUR

```

Рисунок 38 Вывод программы Model_2 на экран после введения всех исходных данных

Во время *расчётов* на экране появляются бегущие индикаторы прогресса: расчета потребности в ресурсах для ЦЗО, ЦУР и ЦЗП и расчёта стоимостных показателей уже в обратном порядке: ЦЗП, ЦУР, ЦЗО (Рисунок 39).

```

ЦЗО.....[100%]
ЦУР.....[100%]
ЦЗП.....[100%]

ЦЗП.....[100%]
ЦУР.....[100%]
ЦЗО.....[100%]

Введите имя файла отчета без расширения [reports/Model_2]:
Готово. Отчет выведен в файл reports/Model_2
Введите папку для экспорта CSV-файлов [outputdata/]:
Экспорт CSV-файлов в папку outputdata/
Готово. Данные экспортированы
Copyright (c) 2026 Пидкасистый Александр Павлович
Process returned 0 (0x0) execution time : 12.149 s
Press any key to continue.

```

Рисунок 39 Отображение индикаторов прогресса при расчётах и диалоги по выводу отчетов и экспорту файлов

По окончании всех расчетов программа запросит подтверждение имени файла отчета. Имя файла задается без расширения, при выводе в файл программа сама добавляет расширение “.txt”. По умолчанию отчет выводится в папку *froma2/build/examples/Model_2/reports* с именем *Model_2.txt*. Папку и имя файла можно изменить. Если имя файла вводится с относительным или абсолютным путем, то программа записывает файл отчета с указанным именем в указанную папку. Если указанная папка не существует, отчет не выводится (Рисунок 39).

Помимо файла отчета, программа экспортирует исходные и расчетные данные в csv-файлы в папку по умолчанию *froma2/build/examples/Model_2/outputdata*. Папку можно изменить, можно указать ее название с относительным или абсолютным путем. Имена экспортируемых файлов представлены в таблице выше (Таблица 40).

На этом работа программы заканчивается.

ПОВТОРНЫЙ ЗАПУСК ПРОГРАММЫ

По аналогии с программой *Model_1*, при первом запуске в папке *froma2/build/examples/Model_2/config* программа создает конфигурационный файл *config.cfg*. Этот файл открывается при последующих запусках, когда программа запускается без параметров. При запуске программы с указанием имени и полного пути к

конфигурационному файлу (см. Рисунок 26 для программы *Model_1*), программа открывает именно его (например, *config_base.cfg*).

Если файла с таким именем нет, появится приглашение к вводу информации. Если файл с таким именем существует, то программа выведет на экран текущую конфигурацию и предложит либо использовать, либо отредактировать её (Рисунок 40).

```

*****
**
**
**
**
*****
ТЕКУЩАЯ КОНФИГУРАЦИЯ ИМПОРТА (config_base.cfg)
имя файла с описанием проекта: about.txt
имя файла с отгрузками из СГП в натуральном выражении: ship.csv
имя файла с ценами поставок: pur_price.csv
имя файла с объемами поставок в ЦУР (nocalc-режим): f_RACp_volume.csv
префикс для файлов с техкартами ЦЭП: recipe_in
префикс для файлов с техкартами ЦЭО: recipe_out
маска regex для поиска файлов техкарт: \d{1,3}\.csv
имя файла с нормативами запасов на складе: shares.csv
имя файла с флагами разрешения на отгрузку и закупку в одном и том же периоде: permit.csv
имя файла с флагами авторасчета: pcalc_man.csv
символ разделителя между полями в CSV_файлах: ;
число столбцов с заголовками в CSV_файлах: 2
число строк с заголовками в CSV_файлах: 1
домашняя валюта проекта: RUR
принцип учета запасов: AVERAGE
Редактировать конфигурацию? [Y/N]_

```

Рисунок 40 Вывод программы Model_2 при запуске с указанием конфигурационного файла

Имя файла конфигурации передается без пути к этому файлу. Файл конфигурации с произвольным именем создается в папке *froma2/build/examples/Model_2/config*. Использование других папок для файлов конфигурации не предусмотрено.

В зависимости от выбранного типа вещественных чисел, библиотека FROMA2 использует либо тип *double*, либо тип *LongReal*. Тип устанавливается на этапе компиляции библиотеки и не может быть изменен позже. Конфигурационные файлы, созданные программой, использующей библиотеку с установленным типом *double* **НЕСОВМЕСТИМЫ** с конфигурационными файлами, созданными программой, использующей библиотеку с установленным типом *LongReal*. Применение несовместимых конфигурационных файлов может привести к непредсказуемым последствиям.

После отказа от редактирования или после окончания редактирования конфигурации, программа производит расчёт и запрашивает имя файла для отчёта и папку для экспорта в *csv*-файлы (Рисунок 39).

На этом работа программы заканчивается.

ПРОГРАММНЫЕ КОМПОНЕНТЫ

Программа состоит из основного файла реализации *main.cpp*, модуля *clsCompany*, включающего заголовочный файл *clsCompany.h* и соответствующий ему файл реализации *clsCompany.cpp* с классами и методами, реализующими функционал программы, модуля *clsImportConfig* (в виде единственного файла *clsImportConfig.hpp*), реализующего работу с файлами конфигурации и данных и заголовочного файла *pathes.h*, где прописаны пути для конфигурационного файла, файлов входных данных, файла отчета и файлов экспорта.

Состав модуля *clsCompany* приведен в таблице ниже (Таблица 41).

Таблица 41 Состав модуля clsCompany

Состав модуля	Описание	Заметка
<code>enum count_choise</code>	Тип перечисление <i>enum</i> , состоящий из значений <i>inStock</i> , <i>midStock</i> и <i>outStock</i>	Сепаратор выбора подразделения

const std::string Message[3]	Массив типа <i>string</i> из трех элементов, состоящий из названий подразделений: "ЦЗП", "ЦУР", "ЦЗО"	Используется для вывода заголовков индикаторов прогресса и в заголовках отчетов
clsCompany	Класс потомок класса clsAgregate	Класс конкретной реализации предприятия из трех подразделений: <i>ЦЗП</i> (центр учета затрат по поступлению, <i>PCC</i> - Purchase Cost Centre), <i>ЦУР</i> (центр учета ресурсов, <i>RAC</i> - Resource Accounting Centre) и <i>ЦЗО</i> (центр учета затрат по отгрузке, <i>SCC</i> - Shipment Cost Centre). Является наследником класса <i>clsAgregate</i>

Состав модуля *clsImportConfig* приведен в таблице ниже (Таблица 42).

Таблица 42 Состав модуля *clsImportConfig*

Состав модуля	Описание	Заметка
const string confdir	Константа с путем к файлам конфигурации; тип <i>string</i>	Присвоено значение макроса <i>V_DIR_CONFIG</i> , определенного в файле <i>pathes.h</i>
const string NoFileName	Константа с именем отсутствующего файла; тип <i>string</i>	По умолчанию равна " <i>nofile</i> "
constexpr bool is_ch_or_size_t_or_double_v	Функция, выполняемая на этапе компиляции, ограничивающая перечень возможных типов для метода <i>inData</i>	Допустимые значения типов: <i>char</i> , <i>size_t</i> и decimal .
bool inData(string &_data, const string _defdata)	Внеклассовый метод вводит строковые данные пользователя. При отсутствии данных, подставляются значения по умолчанию.	Пользовательская альтернатива std::cin
template<typename T, class=std::enable_if_t<is_ch_or_size_t_or_double_v<T>>> void inData(T &_data, const T _defdata)	Внеклассовый метод вводит данные пользователя, тип которых определен функцией <i>is_ch_or_size_t_or_double_v</i> . При отсутствии данных, подставляются значения по умолчанию.	
clsImportConfig	Класс, отвечающий за импорт данных из csv-файлов и подготовку их для ввода в класс <i>clsCompany</i>	

КЛАСС *clsCompany*

Класс *clsCompany* предназначен для моделирования склада, состоящего из трех подразделений: Центра учета затрат по поступлению (сокращенно *ЦЗП*), Центра учета ресурсов (*ЦУР*) и Центра учета затрат по

отгрузке (ЦЗО). В ЦЗП формируется добавленная стоимость благодаря дополнительным операциям, осуществляемым при поступлении ресурсов на склад. В ЦУР происходит учёт разных по объёму и стоимости партий ресурсов, поступающих в разное время на склад и формирование учётной стоимости отгружаемых со склада ресурсов и складских остатков. В ЦЗО формируется добавленная стоимость благодаря операциям, осуществляемым при отгрузке ресурсов со склада.

Основные *задачи*, которые можно решить применением класса *clsCompany*, приведены выше в подразделе «Возможности программы».

Состав класса *clsCompany* приведен в таблице ниже.

Таблица 43 Состав класса *clsCompany*

Поле класса	Описание	Заметка
<code>size_t InCount, MidCount, OutCount</code>	Количество позиций на входе ЦЗП, в ЦУР и выходе ЦЗО. Тип <code>size_t</code> .	Задаются в конструкторе

Класс *clsCompany* наследуется от класса *clsAggregate*. Для реализации описанной выше модели из трех подразделений классу *clsCompany* к полям и методам родительского класса вполне достаточно добавить поля из таблицы выше (Таблица 43) и всего несколько методов, описанных ниже.

МЕТОДЫ КЛАССА *clsCompany*

***clsCompany*(const `size_t` _PrCount, const `Currency` _cur, const `AccountingMethod` _ac, const `size_t` ProdCount, const `size_t` _MidCount, const `size_t` _RMCount, const `strNameMeas` _RMNames[], const `clsRecipeItem` _RecipeOut[], const `clsRecipeItem` _RecipeIn[])**

Конструктор с параметрами. Инициализирует поля родительского и своего класса.

Параметры:

_PrCount – количество периодов проекта; тип `size_t`;

_cur – валюта проекта; тип `Currency`;

_ac – принцип учета запасов; тип `AccountingMethod`;

ProdCount – количество количество номенклатурных позиций отгружаемых из ЦЗО продуктов; тип `size_t`;

_MidCount – количество номенклатурных позиций, хранимых в ЦУР; тип `size_t`;

_RMCount – количество номенклатурных позиций, поставляемых в ЦЗП; тип `size_t`;

_RMNames – указатель на массив с наименованиями ресурсов и единицами их измерения; тип элемента массива `strNameMeas`;

_RecipeOut – указатель на массив с рецептурами для ЦЗО; тип элемента массива `clsRecipeItem`;

_RecipeIn – указатель на массив с рецептурами для ЦЗП; тип элемента массива `clsRecipeItem`.

В текущей версии программы *Model_2* конструктор копирования, конструктор перемещения, оператор присваивания копированием и оператор присваивания перемещением удалены за ненадобностью.

virtual void reportstream(ostream& os) const override

Перегрузка виртуального метода вывода информации в отчет. Метод из секции *protected*. Используется не виртуальным публичным методом *clsBaseProject::Report*.

Параметры:

os – поток для записи типа [ostream](#).

Собственно все. Благодаря родительскому классу [clsAgregate](#), реализация модели свелась к написанию всего двух методов.

КЛАСС `clsImportConfig`

Класс `clsImportConfig` предназначен для работы с [конфигурационным файлом](#) программы (чтение, редактирование, запись). В создаваемом экземпляре этого класса сохраняются имена `csv`-файлов для импорта данных в модель, настройки импорта, общие настройки учета номенклатурных позиций на складе. Методы класса `clsImportConfig` импортируют из `csv`-файлов и подготавливают данные к вводу в экземпляр класса `clsCompany`. Состав класса `clsImportConfig` приведен ниже в таблице ().

Поле класса	Описание	Заметка
<code>string filename_cfg</code>	Имя файла конфигурации. Тип <code>string</code>	См. описание в разделе Папки и файлы
<code>string filename_About</code>	Имя файла с описанием проекта. Тип <code>string</code>	
<code>string filename_shipment</code>	Имя файла с отгрузками. Тип <code>string</code>	
<code>string filename_purprice</code>	Имя файла с ценами поставок. Тип <code>string</code>	
<code>string filename_purvolume</code>	Имя файла с объемами поставок в ЦУР (posalc-режим). Тип <code>string</code>	
<code>string filename_recipe_in</code>	Префикс для файлов с техкартами ЦЗП. Тип <code>string</code>	
<code>string filename_recipe_out</code>	Префикс для файлов с техкартами ЦЗО. Тип <code>string</code>	Задается в коде программы, в секции <code>private</code> класса; по умолчанию равна <code>"_\\d{1,3}\\ .csv"</code>
<code>string msk</code>	Маска регулярного выражения для поиска файлов технологических карт в заданной папке. Тип <code>string</code>	
<code>string filename_shares</code>	Имя файла с нормативами запасов на складе. Тип <code>string</code>	
<code>string filename_permission</code>	Имя файла с флагами разрешения на отгрузку и закупку в одном и том же периоде. Тип <code>string</code>	Рисунок 36, Рисунок 37
<code>string filename_purchasercalc</code>	Имя файла с флагами авторасчета. Тип <code>string</code>	